

SOLUZIONI ESERCIZI DE IL NUOVO JAVA

(www.nuovojava.it)

[Versione 6.0 12/2024]



EDITORE ULRICO HOEPLI MILANO

Indice delle soluzioni degli esercizi

Soluzioni degli esercizi del capitolo 1	6
Introduzione a Java.....	6
Soluzioni degli esercizi del capitolo 2	15
Componenti fondamentali di un programma Java.....	15
Soluzioni degli esercizi del capitolo 3	32
Stile di codifica, tipi di dati ed array	32
Soluzioni degli esercizi del capitolo 4	43
Operatori e Gestione del flusso di esecuzione	43
Soluzioni degli esercizi del capitolo 5	64
Incapsulamento e visibilità	64
Soluzioni degli esercizi del capitolo 6	103
Ereditarietà ed interfacce.....	103
Soluzioni degli esercizi del capitolo 7	122
Polimorfismo	122
Soluzioni degli esercizi del capitolo 8	142
Eccezioni ed asserzioni	142
Soluzioni degli esercizi riassuntivi Parte II	153
Un esempio guidato alla programmazione ad oggetti	153
Soluzioni degli esercizi del capitolo 9	191
Tipi Enumerazioni e Tipi Record	191
Soluzioni esercizi del capitolo 10.....	206
Tipi Generici e Collezioni	206
Soluzioni degli esercizi del capitolo 11 e dell'appendice E.....	223
Tipi Annotazioni, Reflection e la variabile d'ambiente CLASSPATH	223
Soluzioni degli esercizi del capitolo 12	243
Gestione dei thread.....	243

Soluzioni degli esercizi del capitolo 13 e dell'appendice F	259
Espressioni Lambda e package java.lang.....	259
Soluzioni degli Esercizi del capitolo 14 e dell'appendice H	281
Stream API e Framework Collections	281
Soluzioni degli esercizi del capitolo 15 e dell'appendice I.....	293
Input-Output e Date-Time API.....	293
Soluzioni degli esercizi del capitolo 16	349
Moduli	349
Soluzioni degli esercizi del capitolo 17 e dell'appendice G	367
Java Database Connectivity e package java.util	367
Soluzioni degli esercizi del capitolo 18 e dell'appendice M	382
Java e il mondo XML e JAXB	382
Soluzioni degli esercizi del capitolo 19	383
Introduzione alle Interfacce grafiche (GUI)	383
Soluzioni degli esercizi del capitolo 20	384
Introduzione a JavaFX.....	384

Soluzioni degli esercizi del capitolo 1

Introduzione a Java

Le soluzioni di codice proposte per questo e tutti gli altri capitoli, non rappresentano le uniche soluzioni esistenti in assoluto. Per risolvere i nostri esercizi, esistono spesso centinaia di soluzioni diverse e tutte valide. Ognuna di essa ha dei pro e dei contro. Quindi non bisogna prendere la soluzione proposta per un esercizio come l'unica esistente. Questo concetto vale anche per tutti gli altri capitoli. Questo non si applica a soluzioni non costituite da codice (per esempio esercizi "Vero o Falso").

Soluzione 1.a)

Il listato è simile al seguente:

```
public class HelloWorld {  
    public static void main(String args[]) {  
        System.out.println("Hello World!");  
    }  
}
```

Per le operazioni da eseguire dalla prompt dei comandi per compilare ed eseguire il codice consultare il paragrafo 1.5.

Soluzione 1.b) *Concetti base di informatica, Vero o Falso:*

1. **Vero.**
2. **Falso**, il sistema operativo è un software.
3. **Vero.**
4. **Vero.**
5. **Vero.**
6. **Falso**, ogni processore conosce il suo codice sorgente.
7. **Vero.**
8. **Vero.**
9. **Falso**, perché un linguaggio interpretato deve alternare alla fase di esecuzione la fase di traduzione, quindi è solitamente più lento.
10. **Falso**, un programma eseguibile è composto dai suoi file binari.

Soluzione 1.c) *Caratteristiche di Java, Vero o Falso:*

1. **Vero.**
2. **Falso.**
3. **Falso.**
4. **Vero.**
5. **Vero.**
6. **Vero.**
7. **Falso.**
8. **Falso.**
9. **Falso.**

10. **Vero.**

Soluzione 1.d) Codice Java, Vero o Falso:

1. **Falso**, manca il tipo di ritorno (`void`).
2. **Falso**, l'identificatore dovrebbe iniziare con lettera minuscola (`main`).
3. **Vero.**
4. **Vero.**
5. **Falso**, manca l'identificatore.
6. **Falso**, la parola chiave si scrive con lettera iniziale minuscola (`class`).
7. **Vero.**
8. **Falso.**
9. **Falso**, le parentesi sono graffe.
10. **Falso**, le parentesi sono graffe.

Soluzione 1.e) Ambiente e processo di sviluppo, Vero o Falso:

1. **Vero.**
2. **Vero.**
3. **Falso**, bisogna prima compilarlo per poi mandarlo in esecuzione.
4. **Vero.**
5. **Falso**, bisogna anche tenere conto delle lettere maiuscole e minuscole.
6. **Vero.**
7. **Falso**, un browser è sufficiente solo per eseguire applet (che oggi non sono più supportate).
8. **Vero.**
9. **Falso**, il comando giusto è `java Pippo`.
10. **Falso**, il comando giusto è `java Pippo`.

Soluzione 1.f)

Il listato dovrebbe essere simile al seguente:

```
public class HelloWorld {  
    public void main(String args[]) {  
        System.out.println("Hello World!");  
    }  
}
```

Il file compila tranquillamente, ma al momento del lancio ci avvertirà che abbiamo definito un metodo `main` che non è un valido per poter far partire l'applicazione, proprio perché non è stato dichiarato `static`. A seconda dell'errore, del vostro sistema operativo (o della virtual machine installata), i messaggi del compilatore potrebbero essere in italiano o in inglese. Se il messaggio è in italiano dovrebbe essere il seguente:

```
Errore: il metodo principale non è static nella classe HelloWorld.  
Definire il metodo principale come:  
public static void main(String[] args)
```

Nel caso i messaggi fossero in inglese si tenga presente che nel campo informatico un minimo di inglese tecnico lo si deve conoscere.

Soluzione 1.g)

Il listato dovrebbe essere simile al seguente:

```
public class HelloWorld
{
    public static void main(String args[]) {
        System.out.println("Hello World!");
    }
}
```

Il messaggio di errore del compilatore (questa volta in inglese, ma di facile comprensione) è il seguente:

```
error: '{' expected
public class HelloWorld
                        ^
1 error
```

Soluzione 1.h)

Il listato dovrebbe essere simile al seguente:

```
public class HelloWorld {
    public static void main(String args[]) {
        System.out.println("Hello World!");
    }
}
```

Il messaggio di errore del compilatore in inglese è il seguente:

```
error: reached end of file while parsing
    }
    ^
1 error
```

che ci dice che è stata raggiunta la fine del file e manca qualcosa.

Soluzione 1.i)

Il listato dovrebbe essere simile al seguente:

```
public class HelloWorld {
    public static void main(String args[]) {
        System.out.println("Hello World!")
    }
}
```

Il messaggio di errore del compilatore in inglese è il seguente:

```
error: ';' expected
        System.out.println("Hello World!")
                                ^
1 error
```

Dove il compilatore ci avverte che manca un punto e virgola.

Soluzione 1.j)

Il messaggio di errore è il seguente:

```
HelloWorld.java:5: error: class, interface, or enum expected
}
^
1 error
```


Il messaggio è lo stesso che abbiamo già visto nell'approfondimento 1.5, dove avevamo definito una classe utilizzando erroneamente la lettera maiuscola per la parola chiave `class`. In quel caso, come sappiamo un file sorgente Java deve obbligatoriamente definire al suo interno una classe. Il compilatore siccome non aveva trovato una definizione valida, reclamava la definizione di una classe (o un'interfaccia o una enumerazione, ma questi ultimi due concetti non sono stati ancora definiti). In questo caso invece, il compilatore si aspetta che al posto della parentesi graffa superflua, sia definita un'altra classe (o un'interfaccia o una enumerazione). Infatti, come vedremo più avanti è possibile definire anche altre classi all'interno di un unico file sorgente.

Soluzione 1.k)

Il file viene compilato ed eseguito come se la coppia di parentesi non esistesse. Infatti, è possibile utilizzare coppie di parentesi graffe all'interno dei nostri metodi, magari che circondano altre istruzioni. Per esempio potremmo anche scrivere:

```
public class HelloWorld {  
    public static void main(String args[]) {  
        {  
            System.out.println("Hello World!");  
        }  
    }  
}
```

In questo caso, le parentesi sono indubbiamente superflue, ma esistono rari casi in cui la parentesi possono essere utilizzate per isolare dei pezzi di codice, dal resto. Per ora ci dobbiamo solo sapere che le parentesi graffe, se usate in coppia, possono essere utilizzate all'interno dei nostri file sorgente.

Soluzione 1.l)

Il listato dovrebbe essere simile al seguente:

```
public class HelloWorld {  
    public static void main(String args[]) {  
        System.out.println("Hello World!");  
    }  
}
```

Il file però viene compilato ed eseguito senza errori. Infatti il simbolo `;` superfluo, viene considerato dal compilatore come una terminazione (legale) di uno statement vuoto. Potremmo scriverlo anche al rigo successivo (visto che come abbiamo letto nel capitolo 1 non cambia nulla) per rendere l'idea un po' più chiaramente:

```
public class HelloWorld {  
    public static void main(String args[]) {  
        System.out.println("Hello World!");  
        ;  
    }  
}
```

Soluzione 1.m)

Il listato dovrebbe essere simile al seguente:

```
public  
class  
HelloWorld  
{
```

```
public
static
void
main
(
String
args
[
]
)
{
System
.
out
.
println
(
"Hello World!"
)
;
}
}
```

Il file però viene compilato ed eseguito senza errori. Infatti, come vedremo nei prossimi capitoli, Java è un linguaggio a schema libero. Il problema è che diventa molto complicato da leggere.

Soluzione 1.n)

Il listato potrebbe essere simile al seguente:

```
public class HelloWorld {
    public static void main(String args[]) {
        System.out.println("Una frase a piacere!");
    }
}
```

Soluzione 1.o)

Il listato potrebbe essere simile al seguente:

```
public class HelloWorld {
    public static void main(String args[]) {
        System.out.println("8");
    }
}
```

Ma abbiamo stampato il numero come stringa (le stringhe saranno argomento del terzo capitolo), infatti l'abbiamo rinchiuso tra due virgolette. Potremmo scrivere anche direttamente:

```
public class HelloWorld {
    public static void main(String args[]) {
        System.out.println(8);
    }
}
```

Questa volta stiamo stampando un tipo di dato diverso (non ci sono virgolette). Nel prossimo esercizio inizieremo a capire meglio la situazione.

Soluzione 1.p)

Il listato potrebbe essere simile al seguente:

```
public class HelloWorld {  
    public static void main(String args[]) {  
        System.out.println("25+7");  
    }  
}
```

L'output quindi non stamperà una somma ma semplicemente:

```
25+7
```

Potremmo scrivere anche questo direttamente:

```
public class HelloWorld {  
    public static void main(String args[]) {  
        System.out.println(25+7);  
    }  
}
```

In questo caso l'output sarà quello desiderato:

```
32
```

Infatti i tipi di dati numerici (che vengono scritti senza virgolette come vedremo nel capitolo 3) ci permettono di eseguire operazioni aritmetiche.

Soluzione 1.q)

Il programma non stampa niente perché manca l'istruzione di stampa (`System.out.println`).

Soluzione 1.r)

Il programma non stampa niente perché all'istruzione di stampa non viene passato nulla da stampare. Si può notare però che il cursore è sceso alla riga successiva, per effetto del fatto che l'istruzione `System.out.println` va sempre a capo dopo aver stampato (ma anche dopo non aver stampato). Infatti `println`, sta per “print line” (in italiano “stampa riga”).

Soluzione 1.s)

In questo caso l'output sarà simile al seguente:

```
[Ljava.lang.String;@5679c6c6
```

È stato “stampato un oggetto” (chiamato `args`) e capiremo nei prossimi capitoli il perché abbia una rappresentazione sotto forma di stringa così misteriosa.

Soluzione 1.t)

Il listato dovrebbe essere simile al seguente:

```
public class Elenco {  
    public static void main(String args[]) {  
        System.out.println("Pane");  
        System.out.println("caffè");  
        System.out.println("tè");  
        System.out.println("frutta");  
    }  
}
```

E l'output è il seguente:

```
Pane  
caffè  
tè  
frutta
```

Soluzione 1.u)

Il listato dovrebbe essere simile al seguente:

```
public class ElencoCompatto {  
    public static void main(String args[]) {  
        System.out.println("Pane, caffè, tè, frutta");  
    }  
}
```

E l'output è il seguente:

```
Pane, caffè, tè, frutta
```

Soluzione 1.v)

Il listato potrebbe essere simile al seguente:

```
public class SayJava {  
    public static void main(String args[]) {  
        System.out.println("-----");  
        System.out.println("|  JAVA  |");  
        System.out.println("-----");  
    }  
}
```

Soluzione 1.w)

Il listato richiesto dovrebbe essere il seguente:

```
public class Frecce {  
    public static void main(String args[]) {  
        System.out.println("<----<<<");  
        System.out.println("");  
        System.out.println(">>>---->");  
    }  
}
```

Per stampare una riga vuota, la soluzione naturale è stata quella di usare l'istruzione

`System.out.println("")`, che passa come parametro al metodo `println` una stringa vuota `""`.

In realtà è possibile anche utilizzare in alternativa l'istruzione `System.out.println()`, senza passare nessun parametro al metodo.

Soluzione 1.x)

Il codice richiesto potrebbe essere simile al seguente:

```
public class StampaContatti {  
    public static void main(String args[]) {  
        System.out.println("Elenco Contatti");  
        System.out.println();  
        System.out.println("Claudio De Sio Cesari");  
        System.out.println("Via Java 15");  
        System.out.println("151515151515");  
        System.out.println();  
        System.out.println("Stevie Wonder");  
        System.out.println("Via Musica 10");  
        System.out.println("1010101010");  
        System.out.println();  
        System.out.println("Gennaro Capuozzo");  
    }  
}
```

```

        System.out.println("Piazza Quattro Giornate 1");
        System.out.println("1111111111");
    }
}

```

Notare che per stampare le righe vuote, abbiamo usato il metodo `System.out.println` senza passare nessun parametro. La soluzione sarebbe stata valida anche se avessimo usato una stringa vuota in questo modo: `System.out.println("")`.

Soluzione 1.y)

Il codice richiesto dovrebbe essere il seguente:

```

public class StampaClasseRigaVuota {
    public static void main(String args[]) {
        System.out.println("public class RigaVuota {");
        System.out.println("    public static void main(String args[]) {");
        System.out.println("        System.out.println();");
        System.out.println("    }");
        System.out.println("}");
    }
}

```

Notare che non abbiamo fatto stampare nulla al metodo `System.out.println`, perché altrimenti avremmo avuto problemi con le virgolette. Infatti, se avessimo scritto:

```
System.out.println("        System.out.println("");");
```

avremmo ottenuto il seguente errore:

```

StampaClasseRigaVuota.java:5: error: ')' expected
    System.out.println("        System.out.println("");");
                                ^
1 error

```

Questo perché il compilatore non può capire che la seconda virgoletta che incontra deve essere considerata una virgoletta da stampare e non una virgoletta che chiude una stringa. La seconda virgoletta quindi, viene considerata la virgoletta di chiusura della stringa evidenziata in grassetto:

```
System.out.println("        System.out.println("");");
```

Quindi la successiva virgoletta non viene accettata, perché il compilatore si aspetta che venga chiusa la parentesi del metodo `println`. Vedremo nei prossimi capitoli come risolvere questo problema.

Soluzione 1.z)

Il listato potrebbe essere simile al seguente:

```

public class SayMyName {
    public static void main(String args[]) {
        System.out.println("***** * * ** * *****");
        System.out.println("* * * * * * *");
        System.out.println("** * ***** * * * *");
        System.out.println("* * * * * * * * *");
        System.out.println("***** ***** * * ***** *** * *****");
    }
}

```

Soluzione 1.aa)

Potremmo codificare la classe `SommaTreInteri` nel seguente modo:

```
import java.util.Scanner;
```

```
public class SommaTreInteri {  
    public static void main(String args[]) {  
        System.out.println("Ciao, so sommare tre numeri interi.");  
        System.out.println("Scrivi il primo valore e batti invio");  
        Scanner keyboardScanner = new Scanner(System.in);  
        int operatore1 = keyboardScanner.nextInt();  
        System.out.println("Scrivi il secondo valore e batti invio");  
        int operatore2 = keyboardScanner.nextInt();  
        System.out.println("Scrivi il terzo valore e batti invio");  
        int operatore3 = keyboardScanner.nextInt();  
        int risultato = operatore1 + operatore2 + operatore3;  
        System.out.println("Il risultato è:");  
        System.out.println(risultato);  
    }  
}
```

Soluzioni degli esercizi del capitolo 2

Componenti fondamentali di un programma Java

Soluzione 2.a)

Di seguito viene listata una classe che aderisce ai requisiti richiesti:

```
public class ClasseRichiesta {  
    public static void main (String args []) {  
        NumeroIntero uno = new NumeroIntero();  
        NumeroIntero due = new NumeroIntero();  
        uno.numeroIntero = 1;  
        due.numeroIntero = 2;  
        uno.stampaNumero();  
        due.stampaNumero();  
    }  
}
```

Inoltre un costruttore per la classe `NumeroIntero` potrebbe impostare l'unica variabile d'istanza `numeroIntero`:

```
public class NumeroIntero {  
    public int numeroIntero;  
    public NumeroIntero(int n) {  
        numeroIntero = n;  
    }  
    public void stampaNumero() {  
        System.out.println(numeroIntero);  
    }  
}
```

In tal caso, però, per istanziare oggetti dalla classe `NumeroIntero`, non sarà più possibile utilizzare il costruttore di default (che non sarà più inserito dal compilatore). Quindi la seguente istruzione produrrebbe un errore in compilazione:

```
NumeroIntero uno = new NumeroIntero();
```

Bisogna invece creare oggetti passando al costruttore direttamente il valore della variabile da impostare, per esempio:

```
NumeroIntero uno = new NumeroIntero(1);
```

Risposte alle ultime due domande:

4. Trattasi di una variabile d'istanza, perché dichiarata all'interno di una classe, al di fuori di metodi.
5. Il valore sarà zero, ovvero il valore nullo per una variabile intera. Infatti, quando si istanzia un oggetto, le variabili d'istanza vengono inizializzate ai valori nulli se non esplicitamente inizializzate ad altri valori.

Soluzione 2.b) Concetti sui componenti fondamentali. Vero o Falso:

1. **Falso**, una variabile locale deve essere per forza inizializzata dal programmatore.
2. **Falso**, una variabile d'istanza condivide il ciclo di vita con l'oggetto a cui appartiene.
3. **Vero**.
4. **Falso**, una variabile d'istanza appartiene ad un oggetto istanziato dalla classe in cui è dichiarata.

5. **Vero.**
6. **Vero.**
7. **Falso**, un costruttore è un metodo che non restituisce mai niente, infatti non ha tipo di ritorno.
8. **Falso**, un costruttore viene detto di default se viene inserito dal compilatore. Inoltre non ha parametri, ma non tutti i costruttori di default non hanno parametri.
9. **Falso**, un costruttore è un metodo speciale che ha la caratteristica di essere invocato una ed una sola volta nel momento in cui si istanzia un oggetto.
10. **Vero.**

Soluzione 2.c) Sintassi dei componenti fondamentali. Vero o Falso:

1. **Vero.**
2. **Falso**, tenta di restituire un valore intero ma possiede tipo di ritorno `void`.
3. **Falso**, il metodo dovrebbe restituire un valore intero.
4. **Vero.**
5. **Falso**, l'operatore dot deve essere applicato all'oggetto e non alla classe:

```
Punto p1 = new Punto();  
p1.x = 10;
```
6. **Falso**, l'operatore dot deve essere applicato all'oggetto e non alla classe, inoltre la classe non "contiene" l'oggetto.
7. **Falso**, l'operatore dot deve essere applicato all'oggetto. Il compilatore non troverebbe infatti la dichiarazione della variabile `x`.
8. **Falso**, un costruttore è un metodo speciale che ha la caratteristica di essere invocato una ed una sola volta nel momento in cui si istanzia un oggetto.
9. **Falso**, il costruttore non dichiara tipo di ritorno e deve avere nome coincidente con la classe.
10. **Falso**, il costruttore deve avere nome coincidente con la classe.

Soluzione 2.d)

Il listato dovrebbe essere simile al seguente:

```
public class Quadrato {  
    public int lato;  
  
    public int perimetro() {  
        int perimetro = lato * 4;  
        return perimetro;  
    }  
  
    public int area() {  
        int area = lato * lato;  
        return area;  
    }  
}
```

Soluzione 2.e)

Il listato dovrebbe essere simile al seguente:


```
public class TestQuadrato {
    public static void main(String args[]) {
        Quadrato quadrato = new Quadrato();
        quadrato.lato = 5;
        int perimetro = quadrato.perimetro();
        System.out.println(perimetro);
        int area = quadrato.area();
        System.out.println(area);
    }
}
```

Si noti che abbiamo creato le variabili locali `perimetro` e `area` con lo stesso nome del metodo, e questo non è un problema. Infatti il nome di un metodo si differenzia sempre dal nome di una variabile perché dichiarato con parentesi tonde. Avremmo anche potuto chiamare in modo diverso le variabili, ma è buona norma che i nomi siano auto-esplicativi. Tuttavia potevamo anche evitare completamente l'uso di queste variabili se avessimo scritto la classe in questo modo:

```
public class TestQuadrato {
    public static void main(String args[]) {
        Quadrato quadrato = new Quadrato();
        quadrato.lato = 5;
        System.out.println(quadrato.perimetro());
        System.out.println(quadrato.area());
    }
}
```

Il codice è più compatto, ma almeno all'inizio conviene utilizzare le variabili per meglio memorizzare le definizioni.

Soluzione 2.f)

Il listato della classe `Quadrato` dovrebbe essere simile al seguente:

```
public class Quadrato {
    public int lato;

    public Quadrato(int l) {
        lato = l;
    }

    public int perimetro() {
        int perimetro = lato * 4;
        return perimetro;
    }

    public int area() {
        int area = lato * lato;
        return area;
    }
}
```

Il listato della classe `TestQuadrato` modificato dovrebbe invece essere:

```
public class TestQuadrato {
    public static void main(String args[]) {
        Quadrato quadrato = new Quadrato(5);
        int perimetro = quadrato.perimetro();
        System.out.println(perimetro);
        int area = quadrato.area();
        System.out.println(area);
    }
}
```

Soluzione 2.g)

Il listato potrebbe essere simile a:

```
public class Quadrato {
    public final int NUMERO_LATI = 4;

    public int lato;

    public int perimetro() {
        int perimetro = lato * NUMERO_LATI;
        return perimetro;
    }

    public int area() {
        int area = lato * lato;
        return area;
    }
}
```

Soluzione 2.h)

Il listato per la classe Rettangolo potrebbe essere il seguente:

```
public class Rettangolo {
    public final int NUMERO_LATI_UGUALI = 2;
    public int base;
    public int altezza;

    public Rettangolo(int b, int h) {
        base = b;
        altezza = h;
    }

    public int perimetro() {
        int perimetro = (base + altezza) * NUMERO_LATI_UGUALI;
        return perimetro;
    }

    public int area() {
        int area = base * altezza;
        return area;
    }
}
```

Soluzione 2.i)

Il listato per la classe TestRettangolo potrebbe essere il seguente:

```
public class TestRettangolo {
    public static void main(String args[]) {
        Rettangolo rettangolo1 = new Rettangolo(5,6);
        Rettangolo rettangolo2 = new Rettangolo(8,9);
        System.out.println("Perimetro del rettangolo 1 = ")
    }
}
```

```
        + rettangolo1.perimetro());
    System.out.println("Area del rettangolo 1 = "
        + rettangolo1.area());
    System.out.println("Perimetro del rettangolo 2 = "
        + rettangolo2.perimetro());
    System.out.println("Area del rettangolo 2 = "
        + rettangolo2.area());
    }
}
```

Soluzione 2.j)

Il codice della classe `Quadrato` potrebbe essere il seguente (in grassetto il nuovo metodo richiesto):

```
public class Quadrato {
    public final int NUMERO_LATI = 4;
    public int lato;

    public Quadrato(int l) {
        lato = l;
    }

    public int perimetro() {
        int perimetro = lato * NUMERO_LATI;
        return perimetro;
    }

    public int area() {
        int area = lato * lato;
        return area;
    }

    public void stampaDettagli(){
        System.out.println("Questo quadrato, ha lato = " + lato +
            ", perimetro = " + perimetro() + ", area = " + area());
    }
}
```

Il codice della classe `Rettangolo` potrebbe essere il seguente (in grassetto il nuovo metodo richiesto):

```
public class Rettangolo {
    public final int NUMERO_LATI_UGUALI = 2;
    public int base;
    public int altezza;

    public Rettangolo(int b, int h) {
        base = b;
        altezza = h;
    }

    public int perimetro() {
        int perimetro = (base + altezza) * NUMERO_LATI_UGUALI;
        return perimetro;
    }

    public int area() {
        int area = base * altezza;
        return area;
    }
}
```

```
    public void stampaDettagli(){
        System.out.println("Questo rettangolo, ha base = " + base +
            ", altezza = " + altezza + ", perimetro = " + perimetro()+
            ", area = " + area());
    }
}
```

Il codice della classe `TestQuadrato` potrebbe essere il seguente (notare come il codice sia più semplice, breve e leggibile):

```
public class TestQuadrato {
    public static void main(String args[]) {
        Quadrato quadrato = new Quadrato(5);
        quadrato.stampaDettagli();
    }
}
```

Stesso discorso per la classe `TestRettangolo`, il cui codice potrebbe essere il seguente:

```
public class TestRettangolo {
    public static void main(String args[]) {
        Rettangolo rettangolo1 = new Rettangolo(5,6);
        Rettangolo rettangolo2 = new Rettangolo(8,9);
        rettangolo1.stampaDettagli();
        rettangolo2.stampaDettagli();
    }
}
```

Soluzione 2.k)

Il codice della classe `Quadrato` potrebbe essere il seguente (in grassetto il codice modificato):

```
public class Quadrato {
    public final int NUMERO_LATI = 4;
    public int lato;

    public Quadrato(int l) {
        lato = l;
    }

    public int perimetro() {
        int perimetro = lato * NUMERO_LATI;
        return perimetro;
    }

    public int area() {
        int area = lato * lato;
        return area;
    }

    public void stampaDettagli(){
        System.out.println(dammiDettagli());
    }

    public String dammiDettagli(){
        return "Questo quadrato, ha lato di lunghezza = " + lato +
        " , perimetro = " + perimetro()+ " , area = " + area();
    }
}
```

Con lo stesso criterio potremmo modificare la classe `Rettangolo` con il seguente codice (in grassetto il codice modificato):

```
public class Rettangolo {
    public final int NUMERO_LATI_UGUALI = 2;
    public int base;
    public int altezza;

    public Rettangolo(int b, int h) {
        base = b;
        altezza = h;
    }

    public int perimetro() {
        int perimetro = (base + altezza ) * NUMERO_LATI_UGUALI;
        return perimetro;
    }

    public int area() {
        int area = base * altezza;
        return area;
    }

    public void stampaDettagli(){
        System.out.println(dammiDettagli());
    }

    public String dammiDettagli(){
        return "Questo rettangolo, ha base = " + base + ", altezza =
            " + altezza + ", perimetro = " + perimetro()+ ", area = " + area();
    }
}
```

Il codice della classe `TestQuadrilateri` potrebbe essere il seguente:

```
public class TestQuadrilateri {
    public static void main(String args[]) {
        Quadrato quadrato = new Quadrato(5);
        quadrato.stampaDettagli();
        Rettangolo rettangolo = new Rettangolo(5,6);
        rettangolo.stampaDettagli();
    }
}
```

Soluzione 2.I)

Il listato per la classe `Nazione` potrebbe essere il seguente:

```
public class Nazione {
    public String nome;
    public String capitale;
    public int popolazione;

    public Nazione(String n, String c, int p) {
        nome = n;
        capitale = c;
        popolazione = p;
    }
}
```

L'astrazione, pur essendo molto generica sembra corretta. L'unico costruttore definito implica la specificazione in fase di istanza dei tre parametri in input, che abbiamo quindi imposto essere obbligatori. Infatti quando istanziamo un oggetto `Nazione`, dobbiamo specificare i parametri di input del costruttore:

```
Nazione italia = new Nazione("Italia", "Roma", "60000000");
```

Non si doveva obbligatoriamente creare una classe con le stesse variabili, l'importante è avere una propria astrazione corretta.

Soluzione 2.m)

È possibile che qualcuno sia riuscito a creare dei metodi all'interno di questa classe. Nell'esercizio precedente è stata richiesta solo in modo generico un'astrazione della classe `Nazione`, senza specificare il contesto o il programma in cui tale classe dovrà avere un ruolo. È per questo che ci risulta difficile creare dei metodi, visto che attualmente ignoriamo il programma in cui `Nazione` sarà utilizzato. Potremmo utilizzare questa classe in un programma che conserva i dati fisici delle nazioni, ma potremmo anche utilizzarlo in un videogioco che simula il famoso gioco da tavola Risiko. I metodi (ma anche le variabili d'istanza) da definire, potrebbero cambiare drasticamente da contesto a contesto. Nel primo caso infatti definiremmo variabili d'istanza come `fiumi`, `laghi`, `montagne`, `superficie` etc., e metodi come `produce`, `esporta`, `importa`. Nel secondo caso invece potremmo definire i `confini`, e il metodo `difendi`.

Concludendo, abbiamo reso estremamente generica la definizione della classe `Nazione`, proprio perché non avevamo vincoli da poter sfruttare.

Soluzione 2.n)

La soluzione è la risposta numero 1, ovvero il codice può essere compilato senza errori. Non ci si faccia trarre in inganno dai nomi delle variabili (vedi altre possibili risposte) che potrebbero portare (volontariamente) a fare confusione. Inoltre non è un problema l'ordine in cui sono specificati i modificatori.

Soluzione 2.o)

L'errore è che manca il `;` accanto allo statement del metodo `toString`:

```
return "Esercizio20"
```

che dovrebbe essere corretto così:

```
return "Esercizio20";
```

Gli altri metodi sono tutti corretti.

Soluzione 2.p)

L'errore è che non si può dichiarare `public` una variabile locale all'interno di un metodo. Infatti `public` definisce la visibilità al di fuori della classe di una variabile d'istanza, non al di fuori di un metodo.

Abbiamo definito `argz` il parametro del `main`, in luogo dello standard `args`. Ciò è legale perché si tratta solo del nome di un parametro.

Soluzione 2.q)

Il primo errore è che mancano le parentesi quadre per il parametro `args` del `main`. Il secondo è perché manca il `;` accanto all'unico statement del metodo `main`. Il terzo è dovuto da una parentesi graffa di chiusura in più. Corretti questi errori, la classe compila e può anche essere eseguita visto che contiene un metodo `main`. La classe corretta sarà la seguente:

```
public class Esercizio2Q_OK {  
  
    public static void main(String args[]) {  
        System.out.println("Quelo");  
    }  
}
```

Soluzione 2.r)

Il programma non compilerà per via dello statement:

```
System.out.println("Esercizio 2.r");
```

che non appartiene a nessun blocco di codice di metodo. Ma abbiamo visto che all'interno di una classe sono definiti solo variabili e metodi, non statement. Eliminando quello statement, il programma:

```
public class Esercizio2R_OK {  
  
    public int var1;  
    public int var2;  
  
    public Esercizio2R_OK() {  
  
    }  
  
    public Esercizio2R_OK(int a , int b) {  
        var1 = b;  
        var2 = a;  
    }  
  
    public static void main(String args[]) {  
        Esercizio2R_OK esercizio2R = new Esercizio2R_OK(4,7);  
        System.out.println(esercizio2R.var1);  
        System.out.println(esercizio2R.var2);  
    }  
}
```

compilerà e stamperà al runtime:

```
7  
4
```

Soluzione 2.s)

La risposta corretta è la 3. Sarebbe possibile anche compilare solo la classe `Esercizio2S`, visto che utilizzando la classe `Corso`, essa obbligherà il compilatore a compilare anche quest'ultima. Quindi possiamo anche eseguire questa sequenza:

```
javac Esercizio2S.java
java Esercizio2S
```

Soluzione 2.t)

La risposta corretta è la 3.

Soluzione 2.u)

La risposta corretta è la 1. Infatti la variabile locale `c` non è stata inizializzata e provocherà quest'errore:

```
Esercizio2U.java:6: error: variable c might not have been initialized
    System.out.println(a+b+c+d);
                        ^
1 error
```

Come asserito in questo capitolo, la variabile d'istanza omonima non c'entra nulla con la variabile locale. In qualsiasi caso, una volta inizializzata a 3:

```
public class Esercizio2u_OK {
    int c = 3;
    public static void main(String args[]) {
        int a = 1;
        int b = 2, c = 3, d = 4;
        System.out.println(a+b+c+d);
    }
}
```

stamperà:

```
10
```

Soluzione 2.v)

Il listato per la classe `Esercizio2V` potrebbe essere il seguente:

```
public class Esercizio2V {
    public int somma2Int(int a, int b) {
        return a+b;
    }

    public int somma5Int(int a, int b, int c, int d, int e) {
        return a+b+c+d+e;
    }

    public int somma10Int(int a, int b, int c, int d, int e,
                          int f, int g, int h, int i, int l) {
        return a+b+c+d+e+f+g+h+i+l;
    }
}
```



```
//Giusto per testare
public static void main(String args[]) {
    Esercizio2V es = new Esercizio2V();
    System.out.println(es.somma2Int(1,1));
    System.out.println(es.somma5Int(1,1,1,1,1));
    System.out.println(es.somma10Int(1,1,1,1,1,1,1,1,1,1));
}
}
```

Soluzione 2.w)

Il codice della classe Contatto potrebbe essere il seguente:

```
public class Contatto {

    public String nome;

    public String indirizzo;

    public String numeroDiTelefono;

    public Contatto (String nom, String num){
        nome = nom;
        numeroDiTelefono = num;
    }

    public Contatto (String nom, String ind, String num){
        nome = nom;
        indirizzo = ind;
        numeroDiTelefono = num;
    }
}
```

Notare che abbiamo deciso di definire due costruttori, uno che prende in input i valori delle tre variabili, a l'altro che fa a meno dell'indirizzo. Non abbiamo introdotto altri costruttori visto che reputiamo inutile che un contatto sia creato senza specificare almeno un nome e il numero di telefono.

Soluzione 2.x)

Il codice della classe StampaContatti potrebbe essere il seguente:

```
public class StampaContatti {
    public static void main(String args[]) {
        System.out.println("Elenco Contatti");
        System.out.println();
        Contatto contatto1 = new Contatto("Claudio De Sio Cesari",
            "Via Java 13", "131313131313");
        Contatto contatto2 = new Contatto("Stevie Wonder", "Via Musica 10",
            "1010101010");
        Contatto contatto3 = new Contatto("Gennaro Capuozzo",
            "Piazza Quattro Giornate 1", "1111111111");
        System.out.println(contatto1.nome);
        System.out.println(contatto1.indirizzo);
        System.out.println(contatto1.numeroDiTelefono);
        System.out.println();
    }
}
```

```
        System.out.println(contatto2.nome);
        System.out.println(contatto2.indirizzo);
        System.out.println(contatto2.numeroDiTelefono);
        System.out.println();
        System.out.println(contatto3.nome);
        System.out.println(contatto3.indirizzo);
        System.out.println(contatto3.numeroDiTelefono);
    }
}
```

Il codice però risulta piuttosto verboso.

Come consigliato, aggiungiamo il metodo `stampaDettagli` nella classe `Contatto` (vedi codice in grassetto):

```
public class Contatto {
    public String nome;

    public String indirizzo;

    public String numeroDiTelefono;

    public Contatto (String nom, String num) {
        nome = nom;
        numeroDiTelefono = num;
    }

    public Contatto (String nom, String ind, String num) {
        nome = nom;
        indirizzo = ind;
        numeroDiTelefono = num;
    }

    //usato dalla versione 2
    public void stampaDettagli() {
        System.out.println(nome);
        System.out.println(indirizzo);
        System.out.println(numeroDiTelefono);
        System.out.println();
    }
}
```

Possiamo riscrivere la classe `StampaContatti` più semplicemente (modifiche in grassetto):

```
public class StampaContatti {
    public static void main(String args[]) {
        System.out.println("Elenco Contatti");
        System.out.println();
        Contatto contatto1 = new Contatto("Claudio De Sio Cesari",
            "Via Java 13", "131313131313");
        Contatto contatto2 = new Contatto("Stevie Wonder",
            "Via Musica 10", "1010101010");
        Contatto contatto3 = new Contatto("Gennaro Capuozzo",
            "Piazza Quattro Giornate 1", "1111111111");
        contatto1.stampaDettagli();
        contatto2.stampaDettagli();
        contatto3.stampaDettagli();
    }
}
```

Soluzione 2.y)

Il codice della classe `Rubrica` potrebbe essere il seguente:

```
public class Rubrica {
    public Contatto contatto1;
    public Contatto contatto2;
    public Contatto contatto3;
    public Rubrica () {
        contatto1 = new Contatto("Claudio De Sio Cesari",
            "Via Java 13", "1313131313");
        contatto2 = new Contatto("Stevie Wonder", "Via Musica 10",
            "1010101010");
        contatto3 = new Contatto("Gennaro Capuozzo",
            "Piazza Quattro Giornate 1", "1111111111");
    }
}
```

Di conseguenza la classe `StampaContatti` potrà essere modificata nel seguente modo (modifiche in grassetto):

```
public class StampaContatti {
    public static void main(String args[]) {
        System.out.println("Elenco Contatti");
        System.out.println();
        Rubrica rubrica = new Rubrica();
        rubrica.contatto1.stampaDettagli();
        rubrica.contatto2.stampaDettagli();
        rubrica.contatto3.stampaDettagli();
    }
}
```

Soluzione 2.z)

Una soluzione potrebbe essere costituita dalla codifica delle seguenti classi:

```
public class Citta {
    public String nome;

    public Citta (String n) {
        nome = n;
    }
}

public class Nazione {
    public String nome;
    public Citta capitale;
    public int popolazione;

    public Nazione (String n, Citta c, int p) {
        nome = n;
        capitale = c;
        popolazione = p;
    }
}

public class Esercizio2z {
    public static void main(String args[]) {
        Citta capitale = new Citta("Roma");
        Nazione italia = new Nazione("Italia", capitale, 60_000_000);
        System.out.println("L'" + italia.nome
```

```
        + " ha come capitale " + italia.capitale.nome);  
    }  
}
```

Soluzione 2.aa)

Una soluzione potrebbe essere costituita dalle seguenti classi. Iniziamo dalla classe `Libro`:

```
public class Libro {  
    public String titolo;  
    public String autore;  
    public String editore;  
  
    public Libro (String t, String a, String e) {  
        titolo = t;  
        autore = a;  
        editore = e;  
    }  
  
    public void stampaDettagli() {  
        System.out.println(titolo);  
        System.out.println(autore);  
        System.out.println(editore);  
        System.out.println();  
    }  
}
```

Nonostante il codice sia abbastanza leggibile ed interpretabile, facciamo prima qualche osservazione sulla classe `Libro`, per non avere dubbi su quello che abbiamo scritto.

Questa classe definisce tre variabili chiamate `titolo`, `autore` e `editore`. Tali variabili sono definite usando il modificatore `public`, che come vedremo ci permetterà di utilizzarle anche in altre classi. Le variabili sono dichiarate di tipo `String`, ovvero possiamo assegnare a queste variabili delle **stringhe**, ovvero sequenze di caratteri comprese tra virgolette (abbiamo già visto diverse stringhe nei programmi fatti nel capitolo 1).

Queste rappresentano per noi le caratteristiche essenziali per astrarre una classe `Libro`. Potevamo dichiarare anche la variabile `sottotitolo`, `numeroPagine`, `prezzo` e così via, ma quando si decide di creare una classe, si deve astrarla con le caratteristiche necessarie alla situazione. Siccome la nostra necessità è solo quella di fare un esempio, non abbiamo ritenuto utile aggiungere altre caratteristiche.

Come si può notare, le variabili della classe non hanno dei valori, perché non stiamo definendo un particolare libro.

Con il seguente codice creiamo quindi oggetti di tipo `Libro`:

```
public class TestLibro {  
    public static void main(String args[]) {  
        Libro nuovoJava = new Libro("Il nuovo Java", "Claudio De Sio Cesari",  
                                     "Hoepli");  
  
        nuovoJava.stampaDettagli();  
  
        Libro giornoDellaCivetta = new Libro("Il giorno della civetta",  
                                              "Leonardo Sciascia", "Adelphi");  
  
        giornoDellaCivetta.stampaDettagli();  
    }  
}
```

Soluzione 2.bb)

La classe richiesta potrebbe essere implementata nel seguente modo:

```
public class CalcolaPerimetroQuadrato {  
    public static void main(String args[]) {  
        int lunghezzaLatoQuadrato = 17;  
        int numeroLati = 4;  
        int perimetroQuadrato = lunghezzaLatoQuadrato * numeroLati;  
        System.out.println("Il perimetro del quadrato è:");  
        System.out.println(perimetroQuadrato);  
    }  
}
```

L'output sarà:

```
Il perimetro del quadrato è:  
68
```

Questa classe calcola il perimetro di un quadrato facendo uso di tre variabili di tipo `int`. In particolare, possiamo notare come alla variabile `perimetroQuadrato` venga assegnato il valore calcolato dalla moltiplicazione del valore della variabile `lunghezzaLatoQuadrato` e il valore della variabile `numeroLati`. Notare che le variabili hanno aggiunto leggibilità al codice. Infatti, avremmo potuto ottenere lo stesso risultato e il medesimo output con il seguente codice che non fa uso di variabili:

```
public class CalcolaPerimetroQuadratoSenzaVariabili {  
    public static void main(String args[]) {  
        System.out.println("Il perimetro del quadrato è:");  
        System.out.println(17 * 4);  
    }  
}
```

Ma probabilmente avremmo avuto qualche dubbio in più rispetto alla comprensione del codice. Scegliere un identificatore per una variabile, potrebbe sembrare una banalità, ma non lo è, e non deve mai esserlo! Per esempio, potremmo riscrivere la classe `CalcolaPerimetroQuadrato` senza impegnarci a trovare dei nomi significativi per le variabili nel seguente modo:

```
public class CalcolaPerimetroQuadratoABC {  
    public static void main(String args[]) {  
        int a = 17;  
        int b = 4;  
        int c = a * b;  
        System.out.println("Il perimetro del quadrato è:");  
        System.out.println(c);  
    }  
}
```

Non è più difficile da leggere? Inoltre, scrivere un programma, significa ritornare a leggere e modificare continuamente il codice affinché esso funzioni. Non sempre il codice funziona al primo tentativo, ed inoltre il software ha bisogno di evolversi in futuro, aggiungendo nuove funzionalità e modificando quelle esistenti. Una variabile con un identificatore adatto, non solo ci permetterà di leggere più facilmente il vostro codice anche dopo tanto tempo, ma a volte ci indirizzerà verso la soluzione più corretta.

Soluzione 2.cc)

Notiamo che il nome del metodo (`somma`) non descrive completamente la funzionalità che vuole definire. L'identificatore `sommaDueInteri`, sarebbe più "parlante", e non lascerebbe dubbi sulla

implementazione del metodo. Tutto sommato però, anche il nome `somma` sembra accettabile. Infatti, questo metodo dichiara come lista di parametri due variabili di tipo `int`, e i parametri di un metodo rappresentano l'input del metodo. Quando questo metodo verrà chiamato, bisognerà passargli in input due valori di tipo intero. Non potremo passare a questo metodo valori non interi come `26.1`, oppure `abc`. L'identificatore `somma` quindi, può essere considerato un buon identificatore, perché non ci sono dubbi sul fatto che il metodo possa fare la somma solamente di due numeri interi. D'altronde, “un metodo è determinato dalla coppia costituita dall'identificatore e l'eventuale lista dei parametri” (*firma del metodo*).

Come nome dei parametri abbiamo scelto `a` e `b`. Sebbene, siano identificatori coincisi, che ci permettono di risparmiare qualche digitazione, rimangono nomi che non astraggono bene i concetti che vogliono rappresentare. Ne comprendiamo l'utilità, solo perché stiamo definendo un metodo molto semplice e conosciamo bene cosa sia una addizione. Tuttavia, non sarà sempre così. Scegliere identificatori “parlanti”, è fondamentale per la leggibilità del nostro codice, e quindi per semplificare la nostra interazione con il codice stesso. Sarebbe preferibile quindi, utilizzare identificatori come `addendo1` e `addendo2`, oppure i più generici `operando1` ed `operando2`. Per quanto riguarda il tipo di ritorno, per la prima volta non troviamo la parola chiave `void`, ma `int`, che rappresenta un numero intero. Ciò significa che quando questo metodo sarà invocato, esso restituirà in output un valore di tipo intero. Infatti, nell'ultima istruzione del metodo `sommaDueInteri`, i due parametri saranno sommati tramite l'operatore `+`, e il risultato della loro somma verrà restituito come output tramite il comando `return`:

```
return (a + b);
```

Il risultato che viene restituito da questo metodo è la somma di due interi, che è ovviamente un intero. Ecco perché il tipo di ritorno è dichiarato come `int`.

Nell'unico statement contenuto nel blocco di codice quindi, ci sono due istruzioni: la somma dei due parametri ed il comando `return`, che provoca la terminazione del metodo, ritornando il valore successivamente specificato.

Anche in questo caso, sottolineiamo che il blocco di codice del metodo, anche se tecnicamente corretto, potrebbe essere astratto in maniera diversa. Condensando tutta la logica in un unico statement, abbiamo indubbiamente sacrificato la leggibilità del codice alla sinteticità. Per un esempio così semplice questo è accettabile, ma dopo le nostre considerazioni possiamo creare una nuova versione della classe `Aritmetica`, usando l'astrazione per rendere il programma più leggibile:

```
public class AritmeticaLeggibile {  
    public int sommaDueInteri(int addendo1, int addendo2) {  
        int risultato = addendo1 + addendo2;  
        return risultato;  
    }  
}
```

Soluzione 2.dd)

Un giusto compromesso, e questo dipende dalla nostra capacità di astrazione, la nostra pazienza, dalla nostra forma mentis, dalla nostra esperienza di programmazione ed altri fattori. Per questo esempio, potrebbe essere un compromesso accettabile la seguente implementazione:

```
public class AritmeticaCompromesso {  
    public int somma(int addendo1, int addendo2) {  
        return addendo1 + addendo2;  
    }  
}
```

Tuttavia, essendo un parere soggettivo potreste scegliere di implementare la classe in maniera diversa.

Soluzioni degli esercizi del capitolo 3

Stile di codifica, tipi di dati ed array

Soluzione 3.a)

```
public class Esercizio3A {
    public static void main(String args[]) {
        int a = 5, b = 3;
        double r1 = (double)a/b;
        System.out.println("r1 = " + r1);
        char c = 'a';
        short s = 5000;
        int r2 = c*s;
        System.out.println("r2 = " + r2);
        int i = 6;
        float f = 3.14F;
        float r3 = i + f;
        System.out.println("r3 = " + r3);
        double r4 = r1 - r2 - r3;
        System.out.println("r4 = " + r4);
    }
}
```

Soluzione 3.b)

```
public class Persona {
    public String nome;
    public String cognome;
    public int eta;
    public String dettagli() {
        return nome + " " + cognome + " anni " + eta;
    }
}

public class Principale {
    public static void main(String args []) {
        Persona personal = new Persona();
        Persona persona2 = new Persona();
        personal.nome = "Mario";
        personal.cognome = "Rossi";
        personal.eta = 30;
        System.out.println("personal "+personal.dettagli());
        persona2.nome = "Giuseppe";
        persona2.cognome = "Verdi";
        persona2.eta = 40;
        System.out.println("persona2 "+persona2.dettagli());
        Persona persona3 = personal;
        System.out.println("persona3 "+persona3.dettagli());
    }
}
```


Soluzione 3.c) Array, Vero o Falso:

1. **Vero.**
2. **Vero.**
3. **Falso**, la variabile `length` restituisce il numero degli elementi di un array.
4. **Vero.**
5. **Falso.**
6. **Vero**, un `byte` (che occupa solo 8 bit) può essere immagazzinato in una variabile `int` (che occupa 32 bit).
7. **Vero**, un `char` (che occupa 16 bit) può essere immagazzinato in una variabile `int` (che occupa 32 bit).
8. **Falso**, un `char` è un tipo di dato primitivo e `String` è una classe. I due tipi di dati non sono compatibili.
9. **Falso**, in Java la stringa è un oggetto istanziato dalla classe `String` e non un array di caratteri (anche se internamente alla classe, viene usato un array di caratteri).
10. **Falso**, tutte le affermazioni sono giuste tranne `arr[1][2] = 3;` perché questo elemento non esiste.

Soluzione 3.d)

Il listato dovrebbe essere simile al seguente:

```
public class StampaMioNome {
    public static void main(String args[]) {
        char [] nome = {'C', 'l', 'a', 'u', 'd', 'i', 'o'};
        System.out.println(nome);
    }
}
```

Soluzione 3.e)

Il listato della classe `Risultato` potrebbe essere il seguente:

```
public class Risultato {
    public float risultato;

    public Risultato(float ris) {
        risultato = ris;
    }

    public void stampa() {
        System.out.println(risultato);
    }
}
```

Si noti che abbiamo creato un costruttore e un metodo di stampa di comodità.

Il listato della classe `CambiaRisultato` potrebbe essere il seguente:

```
public class CambiaRisultato {
    public void cambiaRisultato(Risultato risultato) {
        risultato.risultato += 1;
    }
}
```

Il listato della classe `TestRisultato` potrebbe essere il seguente:

```
public class TestRisultato {
```

```
public static void main(String args[]) {  
    Risultato risultato = new Risultato(5.0F);  
    risultato.stampa();  
    CambiaRisultato cr = new CambiaRisultato();  
    cr.cambiaRisultato(risultato);  
    risultato.stampa();  
}
```

L'output del precedente codice sarà:

```
5.0  
6.0
```

Soluzione 3.f)

Il listato della classe `CambiaRisultato` dovrebbe cambiare come segue:

```
public class CambiaRisultato {  
    public void cambiaRisultato(Risultato risultato) {  
        risultato.risultato += 1;  
    }  
  
    public float cambiaRisultato(float risultato) {  
        risultato += 1;  
        return risultato;  
    }  
}
```

Si noti che questa volta il metodo deve ritornare il nuovo valore della variabile trattandosi di una variabile di tipo primitivo (cfr. paragrafo 3.3).

Il listato della classe `TestRisultatoFloat` potrebbe essere il seguente:

```
public class TestRisultatoFloat {  
    public static void main(String args[]) {  
        float risultato = 5.0F;  
        System.out.println(risultato);  
        CambiaRisultato cr = new CambiaRisultato();  
        risultato = cr.cambiaRisultato(risultato);  
        System.out.println(risultato);  
    }  
}
```

Si noti che abbiamo dovuto riassegnare il valore della variabile `risultato` dopo la computazione del metodo `cambiaRisultato`.

Soluzione 3.g)

Il listato dovrebbe essere simile a:

```
public class TestArgs {  
    public static void main(String args[]) {  
        System.out.println(args[0]);  
    }  
}
```

Si noti che se non passate un argomento quando eseguite l'applicazione otterrete un'eccezione al runtime:

```
java TestArgs  
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 0
```

```
at TestArgs.main(TestArgs.java:3)
```

Le eccezioni sono trattate nel capitolo 8.

Soluzione 3.h)

Solo gli ultimi due identificatori non sono validi, infatti:

- L'identificatore `Break` è differente da `break` (tutte le parole chiave non hanno lettere maiuscole).
- L'identificatore `String` coincide con il nome della classe `String`, ma non essendo una parola chiave, è possibile utilizzarla come identificatore. Ciononostante si tratta di una cattiva pratica.
- L'identificatore `character` non è una parola chiave (lo è invece `char`).
- L'identificatore `bit` non è una parola chiave (lo è invece `byte`).
- L'identificatore `continues` non è una parola chiave (lo è invece `continue`).
- L'identificatore `exports` è una parola a utilizzo limitato, e sarebbe inutilizzabile all'interno della dichiarazione di un modulo, ma in questo contesto non crea problemi.
- L'identificatore `Class` non è una parola chiave (lo è invece `class`, vedi punto 1).
- L'identificatore `imports` non è una parola chiave (lo è invece `import`).
- L'identificatore `_AAA_` non è una parola chiave.
- L'identificatore `_@_` non è legale perché non si può usare il simbolo di chiocciola negli identificatori.
- L'identificatore `_` è una parola riservata a partire da Java 9.

È possibile commentare i due identificatori non validi nel seguente modo:

```
public class Esercizio3H {  
    public String Break,  
        String,  
        character,  
        bit,  
        continues,  
        exports,  
        Class,  
        imports,  
        _AAA_/*,  
        _@_,  
        _*/;  
}
```

Abbiamo utilizzato un commento su più righe per commentare solo ciò che va commentato.

Chiaramente questo approccio va a discapito della leggibilità del codice. Il modo più opportuno per commentare richiede il cambio del simbolo di “,” con il simbolo “;” subito dopo la dichiarazione dell'identificatore `_AAA_`, e l'utilizzo del commento a singola riga:

```
public class Esercizio3H {  
    public String Break,  
        String,  
        character,  
        bit,  
        continues,  
        exports,  
        Class,  
        imports,  
        _AAA_;  
    // _@_,  
}
```

```
// _;
```

Soluzione 3.i)

L'unica convenzione non utilizzata correttamente è quella per la costante (si ricorda che le convenzioni non influiscono sulla compilabilità del codice).

Il codice andrebbe corretto nel seguente modo:

```
package com.claudiodesio.manuale.esercizi;  
  
public class Esercizio3ISoluzione {  
    public final String NOME_LINGUAGGIO = "Java 9";  
    public int intero;  
    public void stampaStringa() {  
        System.out.println(NOME_LINGUAGGIO);  
    }  
}
```

Soluzione 3.j)

Il problema sorge per la stampa delle virgolette definite all'interno del metodo `println`. La soluzione consiste nell'utilizzare i caratteri di escape (in grassetto):

```
public class StampaClasseRigaVuota {
    public static void main(String args[]) {
        System.out.println("public class RigaVuota {");
        System.out.println("    public static void main(String args[]) {");
        System.out.println("        System.out.println(\"\\n\\n\");");
        System.out.println("    }");
        System.out.println("}");
    }
}
```

Soluzione 3.k)

Il codice compila, viene eseguito senza errori e stampa:

999999997952

ovvero, un numero imprecisato. Infatti i tipi `float`, per causa delle limitazioni dello standard IEEE-754, superate le 9 cifre decimali usa dei numeri approssimati (vedi paragrafo 3.3.2.1).

La risposta giusta è quindi al numero 2.

Soluzione 3.1)

I tipi `bit` e `integer` non esistono (semmai esistono `byte` e `int`). Tutti i valori dichiarati rientrano nei rispettivi intervalli di rappresentazione, compreso il valore `0x11B` che vale 283.0 per il sistema decimale e che, essendo immagazzinato in un `double`, è assolutamente compatibile.

Soluzione 3.m)

La risposta giusta è la numero 2. Infatti un literal booleano verrà stampato esattamente con il suo valore literal (`true` in questo caso). Invece `c+1` viene promosso a numero intero, e da 73 diventa 74. Per potergli far stampare il valore carattere relativo (`␣`) bisognerebbe fare un cast su tutta l'operazione in questo modo:

```
System.out.println((char) (c+1));
```

Soluzione 3.n)

Avremo un errore in compilazione perché il valore assegnato al carattere `c`, è una stringa (si notino le virgolette al posto degli apici).

Soluzione 3.o)

Le risposte corrette sono la 1 e la 2, tutte le altre sono scorrette a livello sintattico.

Soluzione 3.p)

La risposta corretta è la 3 perché sono state utilizzate nel codice entrambe le classi `Auto` e `Autista`.

Soluzione 3.q)

Sì, il ragionamento è corretto.

Soluzione 3.r)

Il listato dovrebbe essere simile a:

```
public class Esercizio3R {  
    public static void main(String args[]) {  
        String[] array = new String[5];  
        array[2] = args[0];  
    }  
}
```

Soluzione 3.s)

Il codice richiesto dovrebbe essere simile al seguente:

```
public class Nave {  
  
    int indice = 0;  
    public Auto[] autoArray;  
  
    public Nave() {
```

```
        autoArray = new Auto[100];
    }

    public void caricaAuto(Auto auto) {
        autoArray[indice] = auto;
        System.out.println("Auto: " + auto.tipo + " caricata");
        indice++;
    }
}
```

Dove abbiamo usato un indice per tenere traccia delle posizioni già occupate nella nave. Questo viene incrementato ogni volta che viene caricata un'auto, e poi sfruttato per caricare la prossima. La seguente classe di test soddisfa la richiesta:

```
public class Esercizio3S {
    public static void main(String args[]) {
        Nave nave = new Nave();
        Auto auto1 = new Auto("Renault");
        Auto auto2 = new Auto("Volkswagen");
        Auto auto3 = new Auto("Nissan");
        nave.caricaAuto(auto1);
        nave.caricaAuto(auto2);
        nave.caricaAuto(auto3);
    }
}
```

Soluzione 3.t)

Il listato richiesto dovrebbe essere simile al seguente:

```
public class Esercizio3T {
    public static void main(String args[]) {
        int dimensioneArray = Integer.parseInt(args[0]);
        int [] array = new int[dimensioneArray];
        System.out.println("L'array ha dimensione " + array.length);
    }
}
```

Si noti che il metodo `parseInt` è statico (argomento non ancora affrontato) e può essere utilizzato con la sintassi: `nomeClasse.parseInt`, ma è possibile anche istanziare un oggetto e invocarlo come un normale metodo (ma è inutile istanziare l'oggetto).

Soluzione 3.u)

Il listato richiesto dovrebbe essere simile al seguente:

```
import java.util.Arrays;

public class Esercizio3U {
    public static void main(String args[]) {
        char[] array = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
            'i', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'z'};
        System.out.println(Arrays.toString(array));
    }
}
```

Il metodo `toString` della classe `Arrays` era il metodo richiesto. Per utilizzarlo è necessario importare la classe `Arrays`.

Soluzione 3.v)

Il listato richiesto dovrebbe essere simile al seguente:

```
import java.util.Random;

public class Esercizio3V {
    public static void main(String args[]) {
        Random random = new Random();
        System.out.println(random.nextInt());
    }
}
```

Soluzione 3.w)

La classe `Esercizio3W` verrà compilata correttamente e quando eseguita stamperà:

```
var i = 8;
```

Soluzione 3.x)

Possiamo sostituire le variabili d'istanza della classe `Rubrica` con un array, utilizzando il seguente codice:

```
public class Rubrica {
    public Contatto[] contatti;
    public Rubrica () {
        contatti = new Contatto[]{
            new Contatto("Claudio De Sio Cesari", "Via Java 13", "131313131313"),
            new Contatto("Stevie Wonder", "Via Musica 10", "1010101010"),
            new Contatto("Gennaro Capuozzo", "Piazza Quattro Giornate 1",
                "1111111111");
        }
    }
}
```

Che è equivalente ad utilizzare la sintassi ordinaria dell'array:

```
public class Rubrica {
    public Contatto[] contatti;
    public Rubrica () {
        contatti = new Contatto[3];
        contatti[0] = new Contatto("Claudio De Sio Cesari", "Via Java 13",
            "131313131313");
        contatti[1] = new Contatto("Stevie Wonder", "Via Musica 10",
            "1010101010");
        contatti[2] = new Contatto("Gennaro Capuozzo",
            "Piazza Quattro Giornate 1", "1111111111");
    }
}
```

Per quanto riguarda l'introduzione della parola `var`, nelle classi `Contatto` e `Rubrica`, non essendo definita alcuna variabile locale, non è possibile utilizzare la parola `var`, quindi tali classi non vanno modificate. La classe `StampaContatti` invece, può essere modificata in questo modo (in grassetto le modifiche):

```
public class StampaContatti {
    public static void main(String args[]) {
        System.out.println("Elenco Contatti");
    }
}
```

```
        System.out.println();
        var rubrica = new Rubrica();
        rubrica.contatti[0].stampaDettagli();
        rubrica.contatti[1].stampaDettagli();
        rubrica.contatti[2].stampaDettagli();
    }
}
```

Soluzione 3.y)

Nessuno snippet è corretto. I numeri 1 e 2 riportano classi con identificatore `var`, ma non è possibile utilizzare la parola `var` come identificatore per i tipi (vedi paragrafo 3.7.2). Inoltre nel caso dello snippet numero 2, non è neanche possibile utilizzare la parola chiave `private` per definire una classe (lo vedremo meglio nei prossimi capitoli). Nello snippet 3, c'è una sintassi che prova a definire una classe usando la parola chiave `var` al posto della parola chiave `class`, ma non è questa la funzione per cui è stata creata la parola `var`. Nel quarto snippet si prova ad utilizzare la parola `var` come identificatore di un array, ma sempre nel paragrafo 3.7.2, viene specificato chiaramente che questo è illegale. Infine nello snippet numero 5, possiamo dedurre dalla presenza del modificatore `public`, che si tratta di una definizione di una variabile d'istanza e non di una variabile locale, e quindi l'utilizzo della parola `var` non è consentito.

Soluzione 3.z)

Decidiamo di creare un'astrazione per la classe `Studente`:

```
public class Studente {
    public String nome;
    public String cognome;
    public String classe;

    public Studente(String nom, String cog, String cla) {
        nome = nom;
        cognome = cog;
        classe = cla;
    }
}
```

Abbiamo dichiarato le informazioni essenziali (richieste dalla traccia dell'esercizio) e abbiamo creato un costruttore per impostare queste informazioni. Abbiamo anche creato un metodo `toString` che restituisce una stringa descrittiva dell'oggetto.

Vedremo più avanti che questo metodo sarà utilizzato spessissimo nella programmazione Java, perché già presente in ogni classe.

Poi creiamo una classe `Pagella` che astrae il concetto di tabella:

```
import java.util.Arrays;

public class Pagella {
    public Studente studente;
    public String[][] tabellaVoti;

    public Pagella(Studente stu, String [][] tab) {
        studente = stu;
    }
}
```



```
        tabellaVoti = tab;
    }

    public void stampaPagella() {
        System.out.println(studente.toString());
        System.out.println(Arrays.toString(tabellaVoti[0]));
        System.out.println(Arrays.toString(tabellaVoti[1]));
        System.out.println(Arrays.toString(tabellaVoti[2]));
        System.out.println(Arrays.toString(tabellaVoti[3]));
        System.out.println(Arrays.toString(tabellaVoti[4]));
        System.out.println(Arrays.toString(tabellaVoti[5]));
        System.out.println(Arrays.toString(tabellaVoti[6]));
    }
}
```

Si noti che questa classe dichiara un oggetto `studente` e un array bidimensionale `tabellaVoti`, entrambi da impostare quando si istanzia l'oggetto con il costruttore fornito. Inoltre dichiara il metodo `stampaPagella` che usa il metodo statico `toString` della classe `java.util.Arrays` per formattare il contenuto di ogni "riga" dell'array bidimensionale `tabellaVoti`.

Infine con la seguente classe, andiamo a stampare due pagelle:

```
public class Esercizio3Z {
    public static void main(String args[]) {
        Studente studente1 = new Studente("Giovanni", "Battista", "5A");
        String [][] tabellaVoti1 = {
            {"Italiano", "7", "Non si impegna troppo."},
            {"Matematica", "9", "È molto portato per questa materia."},
            {"Storia", "7", "Potrebbe fare di più."},
            {"Geografia", "8", "Appassionato."},
            {"Inglese", "9", "Capace di sostenere dialoghi."},
            {"Scienze Motorie", "6", "Voto d'incoraggiamento."},
            {"Musica", "7", "Ha una certa passione per la materia."}
        };
        Pagella pagella1 = new Pagella(studente1, tabellaVoti1);

        Studente studente2 = new Studente("Daniele", "Sapore", "2A");
        String [][] tabellaVoti2 = {
            {"Italiano", "8", "Manifesta entusiasmo per la materia."},
            {"Matematica", "5", "Per niente interessato."},
            {"Storia", "6", "Interessato, ma si impegna poco."},
            {"Geografia", "6", "Potrebbe fare di più."},
            {"Inglese", "8", "Ottima pronuncia."},
            {"Scienze Motorie", "7", "Un po' pigro."},
            {"Musica", "9", "Suona diversi strumenti ed ha un'ottima voce."}
        };
        Pagella pagella2 = new Pagella(studente2, tabellaVoti2);

        pagella1.stampaPagella();
        pagella2.stampaPagella();
    }
}
```

L'output è il seguente:

```
Studente: Giovanni Battista
Classe 5A
[Italiano, 7, Non si impegna troppo.]
[Matematica, 9, È molto portato per questa materia.]
[Storia, 7, Potrebbe fare di più.]
[Geografia, 8, Appassionato.]
[Inglese, 9, Capace di sostenere dialoghi.]
[Scienze Motorie, 6, Voto d'incoraggiamento.]
```

```
[Musica, 7, Ha una certa passione per la materia.]
Studente: Daniele Sapore
Classe 2A
[Italiano, 8, Manifesta entusiasmo per la materia.]
[Matematica, 5, Per niente interessato.]
[Storia, 6, Interessato, ma si impegna poco.]
[Geografia, 6, Potrebbe fare di più.]
[Inglese, 8, Ottima pronuncia.]
[Scienze Motorie, 7, Un po' pigro.]
[Musica, 9, Suona diversi strumenti ed ha un'ottima voce.]
```

Soluzione 3.aa)

Una soluzione potrebbe essere la seguente:

```
public class CalcolaAreaTriangolo {
    public static void main(String args[]) {
        double base = 3.8;
        double altezza = 5.4;
        double areaTriangolo = (base * altezza)/2;
        System.out.println("L'area del triangolo è:");
        System.out.println(areaTriangolo);
    }
}
```

Il cui output sarà:

```
L'area del triangolo è:
10.26
```

Notare che le variabili `double`, possono avere come valore un numero decimale (notare che al posto della virgola viene utilizzato un punto, perché questo è lo standard negli Stati Uniti).

Non è possibile però assegnare un numero decimale ad una variabile di tipo `int`, per esempio l'istruzione:

```
int base = 3.8;
```

provocherebbe il seguente errore in compilazione:

```
error: incompatible types: possible lossy conversion from double to int
    int base = 3.8;
                ^
1 error
```

Soluzioni degli esercizi del capitolo 4

Operatori e Gestione del flusso di esecuzione

Soluzione 4.a)

```
public class TestPari {  
    public static void main(String args[]) {  
        int i = 0;  
        while (true) {  
            i++;  
            if (i > 10)  
                break;  
            if ((i % 2) != 0)  
                continue;  
            System.out.println(i);  
        }  
    }  
}
```

Nel metodo `main` dichiariamo prima una variabile `i` che fa da indice e che inizializziamo a 0. Poi dichiariamo il ciclo infinito `while`, la cui condizione è sempre `true`. All'interno del ciclo incrementiamo subito di un'unità il valore della variabile `i`. Poi controlliamo se il valore della suddetta variabile è maggiore di 10, se la risposta è sì, il costrutto `break` seguente ci farà uscire dal ciclo che doveva essere infinito, e di conseguenza il programma terminerà subito dopo. Nel caso invece `i` valesse meno di 10, utilizzando l'operatore modulo (%) si controlla se il resto della divisione tra `i` e 2 è diverso da 0. Ma tale resto sarà diverso da 0 se e solo se `i` è un numero dispari. Se quindi il numero è dispari, con il costrutto `continue` passeremo alla prossima iterazione partendo dalla prima istruzione del ciclo `while` (`i++`). Se invece `i` è un numero pari, allora esso verrà stampato.

L'output del precedente programma è il seguente:

```
2  
4  
6  
8  
10
```

Soluzione 4.b)

```
public class TestArray {  
    public static void main(String args[]) {  
        for (int i = 0; i < 26; ++i) {  
            char c = (char) ('a' + i);  
            System.out.println(c);  
        }  
    }  
}
```

Si esegue un ciclo con l'indice `i` che varia da 0 a 25. Aggiungendo al carattere 'a' il valore dell'indice `i` (che ad ogni iterazione si incrementa di un'unità), otterremo gli altri caratteri

dell'alfabeto. Il cast a `char` è necessario perché la somma tra un carattere ed un intero viene promosso ad ""intero.

Soluzione 4.c)

Il listato potrebbe essere il seguente:

```
public class Tabelline {
    public static void main(String args[]) {
        for (int i = 1; i <= 10; ++i) {
            for (int j = 1; j <= 10; ++j) {
                System.out.print(i*j + "\t");
            }
            System.out.println();
        }
    }
}
```

Con questo doppio ciclo innestato ed utilizzando il carattere di escape di tabulazione (`\t`), riusciamo a stampare la tavola pitagorica con poche righe di codice.

Soluzione 4.d) Operatori e flusso di esecuzione, Vero o Falso:

1. **Vero.**
2. **Vero.**
3. **Falso**, la variabile booleana `flag` avrà valore `true`. Le espressioni *atomiche* valgono rispettivamente `true`-`false`-`true`, sussistendo le seguenti uguaglianze: `i = 6`, `j = 7`, `k = 5`, `h = 6`. Infatti `(i != j)` vale `true` e inoltre `(i <= h)` vale `true`. L'espressione `(j <= k) || (i <= h)` vale `true`, sussistendo l'operatore OR. Infine l'operatore AND fa sì che la variabile `flag` valga `true`.
4. **Vero.**
5. **Falso**, `switch` può testare solo una variabile intera (o compatibile) confrontandone l'uguaglianza con costanti (in realtà dalla versione 5 si possono utilizzare come variabili di test anche le enumerazioni e il tipo `Integer`, e dalla versione 7 anche le `stringhe`). Il costrutto `if` permette di svolgere controlli incrociati sfruttando oggetti, espressioni booleane etc.
6. **Falso**, l'operatore ternario è equivalente ad un'espressione che restituisce un valore. In particolare produce sempre un valore, e questo deve essere per forza assegnato o utilizzato in qualche modo (assegnandolo ad una variabile, passandolo come argomento ad un metodo, etc.). Per esempio, se `i` e `j` sono due interi, la seguente espressione: `i < j ? i : j`; provocherebbe un errore in compilazione (oltre a non avere senso), visto che il risultato non viene sfruttato.
7. **Vero.**
8. **Falso**, il `do` in qualsiasi caso garantisce l'esecuzione della prima iterazione sul codice. Il `while` potrebbe prescindere da questa soluzione.
9. **Falso**, lo `switch` è una condizione non un ciclo.
10. **Falso**, il `continue` non si può utilizzare nello `switch` ma solo nei cicli.

Soluzione 4.e)

Il listato potrebbe essere il seguente:

```
public class TestArgs {  
    public static void main(String args[]) {  
        if (args.length == 1) {  
            System.out.println(args[0]);  
        } else {  
            System.out.println("Specificare un valore da riga di comando");  
        }  
    }  
}
```

Soluzione 4.f)

Questo perché non sappiamo a priori come si evolverà il nostro programma, e quindi, anche se nel momento in cui si scrive il programma potrebbe non essere necessario la clausola `default`, modificando il programma potrebbe nascere una nuova condizione non prevista, creando un baco nella nostra applicazione. Infatti il nuovo caso non sarà contemplato e il flusso di esecuzione non entrerà in nessuna clausola del costrutto `switch`. Anche usare il costrutto `default` solo per stampare una frase “Caso non previsto” potrebbe essere una buona abitudine.

Soluzione 4.g)

La risposta è identica alla precedente. La clausola `else` per un `if` è equivalente a una clausola `default` per lo `switch`.

Soluzione 4.h)

Il listato richiesto potrebbe essere il seguente:

```
public class Esercizio4H {  
    public static void main(String args[]) {  
        for (int i = 1, j = 1; j <= 10; i++) {  
            if (i % 3 == 0) {  
                System.out.println("Numero multiplo di 3 = " + i);  
                j++;  
            }  
        }  
    }  
}
```

Soluzione 4.i)

Il listato richiesto potrebbe essere il seguente:

```
public class Esercizio4I {  
    public static void main(String args[]) {  
        int i = 1, j = 1;  
        while (j <= 10) {
```

```
        if (i % 3 == 0) {
            System.out.println("Numero multiplo di 3 = " + i);
            j++;
        }
        i++;
    }
}
```

Soluzione 4.j)

Il listato richiesto potrebbe essere il seguente:

```
public class Esercizio4J {
    public static void main(String args[]) {
        int i = 1, j = 1;
        do {
            if (i % 3 == 0) {
                System.out.println("Numero multiplo di 3 = " + i);
                j++;
            }
            i++;
        } while(j <= 10);
    }
}
```

Soluzione 4.k)

Il codice della classe PariODispari richiesto potrebbe essere il seguente:

```
import java.util.*;

public class PariODispari {
    public String dammiPariODispari() {
        Random random = new Random();
        return random.nextInt() % 2 == 0 ? "Pari" : "Dispari";
    }
}
```

Notare che abbiamo usato un operatore ternario perché più adatto alla situazione, ma avremmo potuto utilizzare anche un semplice costrutto `if`.

Mentre il codice della classe che TestPariODispari potrebbe essere il seguente:

```
public class PariODispariTest {
    public static void main(String args[]) {
        PariODispari pariODispari = new PariODispari();
        System.out.println(pariODispari.dammiPariODispari());
    }
}
```

Soluzione 4.l)

Il codice della classe TestaOCroce richiesto potrebbe essere il seguente:

```
public class TestaOCroce {
    public String dammiTestaOCroce() {
        PariODispari pariODispari = new PariODispari();
        String pariODispariString = pariODispari.dammiPariODispari();
        String testaOCroce = switch (pariODispariString) {
            case "Pari" -> "Testa";
            case "Dispari" -> "Croce";
            default -> "C'è un Bug!!!";
        };
        return testaOCroce;
    }
}
```

mentre il codice della classe che TestaOCroceTest potrebbe essere il seguente:

```
public class TestaOCroceTest {
    public static void main(String args[]) {
        TestaOCroce testaOCroce = new TestaOCroce();
        System.out.println(testaOCroce.dammiTestaOCroce());
    }
}
```

Soluzione 4.m)

Il listato richiesto potrebbe essere il seguente:

```
import java.util.Scanner;

public class Moderatore {

    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        String stringa = "";
        System.out.println("Digita qualcosa e batti enter, oppure scrivi "
            + "\"fine\" per terminare il programma");
        while (!(stringa = scanner.next()).equals("fine")) {
            stringa = moderaStringa(stringa);
            System.out.println("Hai digitato " + stringa.toUpperCase() + "!");
        }
        System.out.println("Fine programma!");
    }

    private static String moderaStringa(String stringa) {
        switch (stringa) {
            case "accipicchiolina":
            case "perbacco":
            case "stupidino":
            case "giulivo":
            case "giocondo":
            case "perdindirindina":
                stringa = "CENSURATA!";
                break;
            default:
                break;
        }
        return stringa;
    }
}
```

È solo una delle soluzioni (non di certo la più elegante).

Soluzione 4.n)

Elenchiamo tutti i casi:

- `+` se inteso come operatore di addizione no, ma come operatore di concatenazione di stringhe permette di concatenare una stringa a un booleano.
- `%` no.
- `++` no.
- `/=` no.
- `&` sì, per esempio `(true & false) = true`.
- `=` sì, è un operatore di assegnazione, applicabile a qualsiasi tipo.
- `!!` non è un operatore.
- `>>>` no.

Soluzione 4.o)

La risposta corretta è: 99.

Infatti `i` inizialmente vale 99. Poi nella condizione del primo `if`, viene usato un operatore di post-incremento, il quale avendo minore priorità rispetto all'operatore `>=` viene eseguito dopo di esso. Questo implica che `i` vale ancora 99 quando viene testato se è `>= 100`, e solo dopo il test viene incrementato a 100. Quindi la condizione dell'`if` è `false`, e non viene eseguito il relativo blocco di codice. Quindi viene eseguito il blocco di codice della clausola `else`. Qui viene stampato il risultato di un operatore ternario. Infatti viene decrementato il valore di `i` da 100 a 99, e quindi l'operatore ternario ritorna il primo valore, ovvero `i++`. Anche in questo caso si tratta di un operatore di post-incremento (a bassa priorità), e quindi viene prima stampato il valore di `i` (99) e poi incrementata la variabile (ma tanto il programma termina subito dopo).

Soluzione 4.p)

La risposta corretta è: 32 e al rigo successivo 2, ovvero l'output è il seguente:

```
32
2
```

Infatti inizialmente `i` vale 22, e `j` vale quanto il resto di 22 (e non 23 perché il post-incremento viene applicato dopo l'operatore modulo `%`) diviso 3, ovvero 1. Dopodiché ad `i` viene assegnato il valore di ritorno dell'operatore ternario che controlla se `j != 0` (e lo è). Quindi viene ritornato il valore di `j` che è 1. Nel costrutto `switch` si entra nel `case 1` dove viene stampato `8<<2`, che equivale a 8 moltiplicato per 2 alla seconda, ovvero 32. Inoltre viene eseguito anche il `case 2`, visto che non c'è un `break` che fa uscire dal costrutto. Quindi viene stampato `8>>2` che equivale a 8 diviso 2 alla seconda, ovvero 2.

Soluzione 4.q)

Il listato potrebbe essere il seguente:

```
import java.util.Scanner;
```



```
public class Esercizio4Q {  
  
    public static void main(String args[]) {  
        Scanner scanner = new Scanner(System.in);  
        System.out.println("Digita il numero di giorni passati "  
            + "dall'ultima vacanza");  
        int giorni = scanner.nextInt();  
        System.out.println("Hai digitato " + giorni + " giorni!");  
        int ore = giorni*24;  
        int minuti = ore*60;  
        System.out.println("Quindi sono passati " + minuti + " minuti!");  
    }  
}
```

Soluzione 4.r)

La soluzione potrebbe essere la seguente:

```
public class Esercizio4R {  
  
    private static int matrice[][] = {  
        {1, 7, 3, 9, 5, 3},  
        {6, 2, 3},  
        {7, 5, 1, 4, 0},  
        {1, 0, 2, 9, 6, 3, 7, 8, 4}  
    };  
  
    public static void main(String args[]) {  
        int numeroDaTrovare = Integer.parseInt(args[0]);  
        PRIMA_LABEL:  
        for (int i = 0; i < matrice.length; i++) {  
            int[] riga = matrice[i];  
            for (int j = 0; j < riga.length; j++) {  
                if (riga[j] == numeroDaTrovare) {  
                    System.out.println(numeroDaTrovare + " trovato a riga = "  
                        + ++i + ", colonna = " + ++j);  
                    break PRIMA_LABEL;  
                }  
            }  
        }  
        System.out.println("Ricerca terminata");  
    }  
}
```

Abbiamo dapprima dovuto convertire `args[0]` tramite il metodo statico della classe `Integer` `parseInt` (cfr. esercizio 3.t) immagazzinandolo in una variabile `numeroDaTrovare`. Poi abbiamo utilizzato un doppio ciclo innestato per navigare all'interno delle celle della matrice, sfruttando gli indici `i` (per le righe) e `j` (per le colonne). Si noti l'utilizzo della label che abbiamo chiamato `PRIMA_LABEL`, che marca il ciclo esterno. Quando viene trovata la prima occorrenza del numero da trovare, l'istruzione:

```
break PRIMA_LABEL;
```

fa terminare anche il ciclo esterno, e il programma continua stampando il messaggio `Ricerca terminata`, e poi termina.

Soluzione 4.s)

Il listato potrebbe essere il seguente:

```
public class Esercizio4S {

    private static int matrice[][] = {
        {1, 7, 3, 9, 5, 3},
        {6, 2, 3},
        {7, 5, 1, 4, 0},
        {1, 0, 2, 9, 6, 3, 7, 8, 4}
    };

    public static void main(String args[]) {
        int numeroDaTrovare = controllaArgomento(args);
        if (numeroDaTrovare == -1) {
            System.out.println("Specificare un numero intero compreso tra"
                + " 0 e 9");
            return;
        }
        PRIMA_LABEL:
        for (int i = 0; i < matrice.length; i++) {
            int[] riga = matrice[i];
            for (int j = 0; j < riga.length; j++) {
                if (riga[j] == numeroDaTrovare) {
                    System.out.println(numeroDaTrovare + " trovato a riga = "
                        + ++i + ", colonna = " + ++j);
                    break PRIMA_LABEL;
                }
            }
        }
        System.out.println("Ricerca terminata");
    }

    private static int controllaArgomento(String[] args) {
        if (args.length == 1) {
            if (args[0].length() == 1) {
                for (int i = 0; i < 10; i++) {
                    if (args[0].equals("" + i)) {
                        return Integer.parseInt(args[0]);
                    }
                }
            }
        }
        return -1;
    }
}
```

Si noti che abbiamo delegato al metodo `controllaArgomento` la correttezza dell'input specificato dall'utente. Tale metodo ritorna il valore specificato, oppure, nel caso non sia corretto, il valore `-1`. Come è possibile vedere dal codice del metodo `main`, nel caso il valore ritornato dal metodo `controllaArgomento` sia `-1`, viene stampato un messaggio di istruzioni per l'utente, e con l'istruzione `return`, si esce dal metodo. Si noti che il metodo `main` ritorna `void`, quindi per uscire dal metodo si usa l'istruzione `return` senza specificare cosa ritornare.

Analizziamo ora il metodo `controllaArgomento`. Con il primo `if` abbiamo dapprima controllato che la lunghezza dell'array `args` sia 1, ovvero che sia stato specificato un unico argomento, utilizzando la variabile `length` dell'array (cfr. paragrafo 3.6.5). Con il secondo `if` abbiamo controllato che la lunghezza della stringa `args[0]` sia esattamente 1. Abbiamo sfruttato la chiamata al metodo `length` della classe `String` (da non confondersi con la variabile `length` dell'array). Il

ciclo `for` seguente esegue un ciclo su valori che vanno da 0 a 9, e controlla che `args[0]` coincida con uno dei valori. Nel momento in cui trova una corrispondenza, il valore corrente viene ritornato dopo averlo convertito a intero mediante la chiamata al metodo statico della classe `Integer.parseInt` (cfr. esercizio 3.t). Se invece nel ciclo `for` non si trovano corrispondenze, per esempio perché è stata specificata una lettera o un simbolo (quindi non un intero compreso tra 0 e 9), allora il ciclo terminerà e sarà ritornato il valore -1.

Soluzione 4.t)

La classe `CalcolatriceSemplificata` richiesta potrebbe essere simile alla seguente:

```
public class CalcolatriceSemplificata {

    public double somma(double d1, double d2) {
        return d1 + d2;
    }

    public double sottrai(double d1, double d2) {
        return d1 - d2;
    }

    public double moltiplica(double d1, double d2) {
        return d1 * d2;
    }

    public double dividi(double d1, double d2) {
        return d1 / d2;
    }

    public double restituisciResto(double d1, double d2) {
        return d1 % d2;
    }

    public double massimo(double d1, double d2) {
        return d1 > d2 ? d1 : d2;
    }

    public double minimo(double d1, double d2) {
        return d1 > d2 ? d2 : d1;
    }

    public double media(double d1, double d2) {
        return (d1 + d2)/2;
    }

}
```

Mentre la classe per il suo test potrebbe essere:

```
public class Esercizio4T {

    public static void main(String args[]) {
        CalcolatriceSemplificata calcolatriceSemplificata =
            new CalcolatriceSemplificata();
        System.out.println("42.7 + 47.8 = " +
            calcolatriceSemplificata.somma(42.7, 47.8));
        System.out.println("42.7 - 47.8 = " +
            calcolatriceSemplificata.sottrai(42.7, 47.8));
        System.out.println("42.7 x 47.8 = " +
            calcolatriceSemplificata.moltiplica(42.7, 47.8));
    }
}
```

```
        System.out.println("42.7 : 47.8 = " +
            calcolatriceSemplificata.dividi(42.7, 47.8));
        System.out.println("il resto della divisione tra 42.7 e 47.8 è " +
            calcolatriceSemplificata.restituisceResto(42.7, 47.8));
        System.out.println("Il massimo tra 42.7 e 47.8 è " +
            calcolatriceSemplificata.massimo(42.7, 47.8));
        System.out.println("Il minimo tra 42.7 e 47.8 è " +
            calcolatriceSemplificata.minimo(42.7, 47.8));
        System.out.println("La media tra 42.7 e 47.8 è " +
            calcolatriceSemplificata.media(42.7, 47.8));
    }
}
```

Soluzione 4.u)

Il listato richiesto potrebbe essere il seguente:

```
import java.util.*;

public class Esercizio4U {

    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Digita il primo operando e batti enter.");
        double primoOperando = Double.parseDouble(scanner.nextLine());
        System.out.println(
            "Ora scegli l'operazione da eseguire e batti enter:");
        String operazione = scanner.nextLine();
        System.out.println("Ora scegli il secondo operando e batti enter ");
        double secondoOperando = Double.parseDouble(scanner.nextLine());
        double risultato =
            eseguiOperazione(primoOperando, secondoOperando, operazione);
        System.out.println("Risultato = " + risultato);
    }

    private static double eseguiOperazione(double primoOperando,
        double secondoOperando, String operazione) {
        CalcolatriceSemplificata calcolatriceSemplificata =
            new CalcolatriceSemplificata();
        switch (operazione) {
            case "+":
                return calcolatriceSemplificata.somma(
                    primoOperando, secondoOperando);
            case "-":
                return calcolatriceSemplificata.sottrai(
                    primoOperando, secondoOperando);
            case "x":
                return calcolatriceSemplificata.moltiplica(
                    primoOperando, secondoOperando);
            case "d":
                return calcolatriceSemplificata.dividi(
                    primoOperando, secondoOperando);
            case "r":
                return calcolatriceSemplificata.restituisceResto(
                    primoOperando, secondoOperando);
            case "u":
                return calcolatriceSemplificata.massimo(
```

```
        primoOperando, secondoOperando);
    case "m":
        return calcolatriceSemplificata.minimo(
            primoOperando, secondoOperando);
    default:
        System.out.println("Operazione specificata " + operazione +
            " non valida");
        System.exit(1);
        return Double.NaN;
    }
}

private static void stampaTabellaOperazioni() {
    System.out.println("'+' : somma");
    System.out.println("'-' : sottrai");
    System.out.println("'x' : moltiplica");
    System.out.println("'d' : dividi");
    System.out.println("'r' : restituisci resto della divisione");
    System.out.println("'u' : massimo");
    System.out.println("'m' : minimo");
}
}
```

La traccia dell'esercizio aveva già delineato come implementare l'interazione tra l'utente e il programma. Per il resto il codice è abbastanza chiaro, e faremo giusto delle osservazioni sui punti più oscuri.

Si noti che per eseguire i calcoli abbiamo usato il tipo di dato numerico più largo: il `double`. Si noti anche l'utilizzo del metodo statico `parseDouble` della classe `Double`, che converte una stringa in `double` (cfr. documentazione ufficiale), così come il metodo `parseInt` della classe `Integer` converte una stringa in `int` (cfr. esercizio 3.t).

Notare però che l'istruzione `Double.parseDouble(scanner.nextLine())` che abbiamo usato per ottenere il contenuto dei due operandi potrebbe essere tranquillamente sostituita dalla più semplice istruzione `scanner.nextDouble()`. Infatti la classe `Scanner` definisce metodi per il recupero di tutti i tipi di dati primitivi.

In realtà, invece di passare il risultato del metodo `nextLine` al metodo `parseDouble` avremmo potuto anche utilizzare direttamente il metodo `parseDouble` della classe `Scanner`. Non abbiamo gestito (non era richiesto) l'eventuale scorretto utilizzo del programma da parte dell'utente, perché troppo impegnativo per le nozioni che abbiamo studiato sin qui. Con la gestione delle eccezioni che studieremo nel capitolo 8 invece si troveranno delle soluzioni semplici. Purtroppo il programma in alcune operazioni perde di precisione. Potete verificarlo per esempio eseguendo una operazione di resto dalla divisione tra 2.3 e 2, che dà per risultato un valore non preciso come si può notare da questo esempio di output:

```
Digita il primo operando e batti enter.
2.3
Ora scegli l'operazione da eseguire e batti enter:
'+' : somma
'-' : sottrai
'x' : moltiplica
'd' : dividi
'r' : restituisci resto della divisione
'u' : massimo
```

```
'm' : minimo  
Ora scegli il secondo operando e batti enter  
2  
Risultato = 0.2999999999999998
```

Questo è dovuto dal modo in cui viene rappresentato il tipo `double` in memoria, e lo avevamo anche accennato nel paragrafo 3.3.2.1 (bisognerebbe utilizzare il tipo `BigDecimal`, cfr. documentazione ufficiale), ed è un problema comune a tutti i linguaggi di programmazione moderni che utilizzano lo stesso metodo di rappresentazione in memoria (Standard IEEE-754). Il metodo `System.exit` fa terminare il programma istantaneamente. C'è da notare inoltre che il programma propone all'utente di scegliere delle lettere specifiche (vedi metodo `eseguiOperazione`) per eseguire la scelta dell'operazione da eseguire.

Soluzione 4.v)

La classe `StranaCalcolatrice` potrebbe essere la seguente:

```
public class StranaCalcolatrice {  
  
    public double somma(double[] doubles) {  
        double risultato = 0;  
        for (double unDouble : doubles) {  
            risultato += unDouble;  
        }  
        return risultato;  
    }  
  
    public double moltiplica(double[] doubles) {  
        double risultato = doubles[0];  
        for (int i = 1; i < doubles.length; i++) {  
            risultato *= doubles[i];  
        }  
        return risultato;  
    }  
  
    public double massimo(double[] doubles) {  
        double max = doubles[0];  
        for (int i = 1; i < doubles.length; i++) {  
            double unDouble = doubles[i];  
            if (unDouble > max) {  
                max = unDouble;  
            }  
        }  
        return max;  
    }  
  
    public double minimo(double[] doubles) {  
        double min = doubles[0];  
        for (int i = 1; i < doubles.length; i++) {  
            double unDouble = doubles[i];  
            if (unDouble < min) {  
                min = unDouble;  
            }  
        }  
        return min;  
    }  
}
```

Il metodo `somma` è molto semplice e viene usato un semplice ciclo `foreach` per eseguire la somma. Negli altri tre casi, abbiamo dovuto prima recuperare il primo elemento, per poi svolgere le operazioni con il resto degli elementi dell'array passato in input.

Mentre la classe di test potrebbe essere la seguente:

```
public class Esercizio4V {  
  
    public static void main(String args[]) {  
        StranaCalcolatrice stranaCalcolatrice = new StranaCalcolatrice();  
        double [] a1 = {42.7, 47.8};  
        System.out.println("42.7 + 47.8 = " + stranaCalcolatrice.somma(a1));  
        double [] a2 = {42.7, 47.8, 2};  
        System.out.println("42.7 x 47.8 x 2= " +  
            stranaCalcolatrice.moltiplica(a2));  
        double [] a3 = {42.7, 47.8, 50, 2, 8, 89};  
        System.out.println("Il massimo tra 42.7, 47.8, 50, 2, 8, 89 è " +  
            stranaCalcolatrice.massimo(a3));  
        double [] a4 = {42.7, 47.8, 50, 2, 8, 89};  
        System.out.println("Il minimo tra 42.7, 47.8, 50, 2, 8, 89 è " +  
            stranaCalcolatrice.minimo(a4));  
    }  
}
```

Tuttavia il dover creare prima degli array per poter invocare questi metodi risulta piuttosto scomodo. C'è una soluzione più funzionale che viene spiegata solo nel paragrafo 7.2.2. Infatti, invece di dichiarare array come parametri dei metodi, possiamo usare dei **varargs**, ovvero dei parametri che accettano un numero indefinito di elementi. Basta riscrivere i nostri parametri usando un'ellissi (ovvero dei puntini sospensivi "..."), al posto delle parentesi quadre. In pratica basta riscrivere la classe `StranaCalcolatrice` in questo modo:

```
public class StranaCalcolatrice {  
  
    public double somma(double... doubles) {  
        double risultato = 0;  
        for (double unDouble : doubles) {  
            risultato += unDouble;  
        }  
        return risultato;  
    }  
  
    public double moltiplica(double... doubles) {  
        double risultato = doubles[0];  
        for (int i = 1; i < doubles.length; i++) {  
            risultato *= doubles[i];  
        }  
        return risultato;  
    }  
  
    public double massimo(double... doubles) {  
        double max = doubles[0];  
        for (int i = 1; i < doubles.length; i++) {  
            double unDouble = doubles[i];  
            if (unDouble > max) {  
                max = unDouble;  
            }  
        }  
        return max;  
    }  
  
    public double minimo(double... doubles) {  
        double min = doubles[0];
```

```
        for (int i = 1; i < doubles.length; i++) {
            double unDouble = doubles[i];
            if (unDouble < min) {
                min = unDouble;
            }
        }
        return min;
    }
}
```

È possibile notare che il codice dei metodi non è cambiato affatto. Infatti, all'interno del metodo, un `varargs` è solamente un *array che usa una sintassi diversa*. Il vantaggio di utilizzare un `varargs` al posto di un array come parametro di un metodo, è quella che non è necessario creare prima un array per passarlo come parametro, ma possiamo direttamente passare gli elementi che avremmo dovuto mettere all'interno dell'array. Quindi possiamo riscrivere la classe di test nel seguente modo:

```
public class Esercizio4V {

    public static void main(String args[]) {
        StranaCalcolatrice stranaCalcolatrice = new StranaCalcolatrice();
        System.out.println("42.7 + 47.8 = " +
            stranaCalcolatrice.somma(42.7, 47.8));
        System.out.println("42.7 x 47.8 x 2 = " +
            stranaCalcolatrice.moltiplica(42.7, 47.8, 2));
        System.out.println("Il massimo tra 42.7, 47.8, 50, 2, 8, 89 è " +
            stranaCalcolatrice.massimo(42.7, 47.8, 50, 2, 8, 89));
        System.out.println("Il minimo tra 42.7, 47.8, 50, 2, 8, 89 è " +
            stranaCalcolatrice.minimo(42.7, 47.8, 50, 2, 8, 89));
    }
}
```

semplificando molto il nostro sforzo implementativo. Per tutti i dettagli potete approfondire il discorso studiando il paragrafo 7.2.2, anche se per ora non è necessario.

Soluzione 4.w)

Il listato richiesto potrebbe essere il seguente:

```
import java.util.*;

public class Esercizio4Z {

    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        System.out.println(
            "Digita un operando e batti enter, tutte le volte che vuoi." +
            "\nQuando hai finito scegli l'operazione da eseguire e batti enter.");
        stampaTabellaOperazioni();
        String temp;
        String operandiString = "";
        while (isNotOperation(temp = scanner.nextLine())) {
            operandiString += temp + "-";
        }
        if (isNotOperation(temp)) {
            System.out.println("Errore codice operazione");
        }
        String[] operandiArray = operandiString.split("-");
        double[] operandi = new double[operandiArray.length];
        for (int i = 0; i < operandiArray.length; i++) {
            operandi[i] = Double.parseDouble(operandiArray[i]);
        }
        double risultato = eseguiOperazione(operandi, temp);
    }
}
```



```
        System.out.println("Risultato = " + risultato);
    }

    private static boolean isNotOperation(String line) {
        if (line.equals("+") || line.equals("x") || line.equals("u") ||
            line.equals("m")) {
            return false;
        }
        return true;
    }

    private static double eseguiOperazione(double[] operandi,
        String operazione) {
        StranaCalcolatrice stranaCalcolatrice = new StranaCalcolatrice();
        switch (operazione) {
            case "+":
                return stranaCalcolatrice.somma(operandi);
            case "x":
                return stranaCalcolatrice.moltiplica(operandi);
            case "u":
                return stranaCalcolatrice.massimo(operandi);
            case "m":
                return stranaCalcolatrice.minimo(operandi);
            default:
                System.out.println("Operazione specificata " + operazione
                    + " non valida");
                System.exit(1);
                return Double.NaN;
        }
    }

    private static void stampaTabellaOperazioni() {
        System.out.println("'+' : somma");
        System.out.println("'x' : moltiplica");
        System.out.println("'u' : massimo");
        System.out.println("'m' : minimo");
    }
}
```

Abbiamo deciso di far specificare all'utente tutti gli operandi, per poi eseguire l'operazione nel momento in cui viene specificato un possibile operatore. Progettare una soluzione al problema non era affatto banale.

Anche in questo caso non abbiamo gestito (non era richiesto) l'eventuale scorretto utilizzo del programma da parte dell'utente, perché troppo impegnativo per le nozioni che abbiamo studiato sin qui. Con la gestione delle eccezioni che studieremo nel capitolo 8 invece si troveranno delle soluzioni semplici.

Il punto critico del codice riguarda la gestione dell'input, che deve essere analizzato e trasformato in tipi di dati che servono per compiere le nostre operazioni (`double`). Purtroppo ci manca un argomento molto importante che non abbiamo ancora studiato come le **collection** (che approfondiremo solo nell'appendice H, ma introdurremo a partire dal capitolo 10). Senza questo argomento, per immagazzinare i vari operandi specificati dall'utente, ci siamo dovuti arrangiare con una stringa (`operandiString`) che conteneva i vari operandi separati dal simbolo di trattino. Poi con il metodo `split` (vedi documentazione della classe `String`), abbiamo ottenuto un array di operandi sotto forma di stringa (`operandiArray`). Poi abbiamo istanziato un array di `double` `operandi` della stessa dimensione di `operandiArray`, e lo abbiamo riempito con gli operandi di tipo `double` dopo averli convertiti tramite il metodo statico `parseDouble` della classe `Double`. Una soluzione piuttosto artificiosa, che però ha funzionato.

Ecco un esempio di esecuzione dell'applicazione:

```
Digita un operando e batti enter, tutte le volte che vuoi.
Quando hai finito scegli l'operazione da eseguire e batti enter.
'+' : somma
'x' : moltiplica
'u' : massimo
'm' : minimo
2
3
4
5
6
7.28
x
Risultato = 5241.6
```

Soluzione 4.x)

Il codice richiesto potrebbe essere il seguente:

```
import java.util.*;

public class GiocoTestaOCroce {
    public static void main(String args[]) {
        System.out.println("Cosa scegli testa o croce?");
        Scanner scanner = new Scanner(System.in);
        String scelta = scanner.nextLine();
        if ("testa".equals(scelta)) {
            System.out.println("Ok, lancio...");
            TestaOCroce testaOCroce = new TestaOCroce();
            String risultatoLancio = testaOCroce.dammiTestaOCroce();
            System.out.print("È uscito " + risultatoLancio + "...");
            System.out.println("testa".equalsIgnoreCase(risultatoLancio) ?
                "hai vinto!" : "hai perso!");
        } else if ("croce".equals(scelta)) {
            System.out.println("Ok, lancio...");
            TestaOCroce testaOCroce = new TestaOCroce();
            String risultatoLancio = testaOCroce.dammiTestaOCroce();
            System.out.print("È uscito " + risultatoLancio + "...");
            System.out.println("croce".equalsIgnoreCase(risultatoLancio) ?
                "hai vinto!" : "hai perso!");
        } else {
            System.out.println(
                "Mi dispiace, puoi scrivere solo testa o croce, riprova...");
            System.out.println("Programmata terminato... ciao!");
        }
    }
}
```

L'algoritmo implementato sembra funzionare, purtroppo lo stesso codice è ripetuto più volte. Per ora va bene così perché abbiamo raggiunto il nostro scopo, ma nei prossimi capitoli cercheremo di migliorare la qualità del nostro codice.

Soluzione 4.y)

Il codice richiesto potrebbe essere il seguente:

```
import java.util.*;

public class GiocoTestaOCroce {
    public static void main(String args[]) {
        System.out.println(
            "Giochiamo a testa o croce, quante lanci vuoi fare?");
        Scanner scanner = new Scanner(System.in);
        String numeroLanciString = scanner.nextLine();
        int numeroLanci = Integer.parseInt(numeroLanciString);
        int contatoreLanci = 1;
        var numeroVittorie = 0;
        var numeroSconfitte = 0;
        String messaggio = "";
        System.out.println("Hai scelto di fare " + numeroLanci + "...iniziamo!");
        while (contatoreLanci <= numeroLanci) {
            System.out.println("Lancio numero " + contatoreLanci);
            System.out.println("Cosa scegli testa o croce?");
            String scelta = scanner.nextLine();
            if ("testa".equals(scelta)) {
                System.out.println("Ok, lancio...");
                TestaOCroce testaOCroce = new TestaOCroce();
                String risultatoLancio = testaOCroce.dammiTestaOCroce();
                contatoreLanci++;
                System.out.print("È uscito " + risultatoLancio + "...");
                if ("testa".equalsIgnoreCase(risultatoLancio)) {
                    messaggio = "hai vinto!";
                    numeroVittorie++;
                } else {
                    messaggio = "hai perso!";
                    numeroSconfitte++;
                }
            } else if ("croce".equals(scelta)) {
                System.out.println("Ok, lancio...");
                TestaOCroce testaOCroce = new TestaOCroce();
                String risultatoLancio = testaOCroce.dammiTestaOCroce();
                contatoreLanci++;
                System.out.print("È uscito " + risultatoLancio + "...");
                if ("croce".equalsIgnoreCase(risultatoLancio)) {
                    messaggio = "hai vinto!";
                    numeroVittorie++;
                } else {
                    messaggio = "hai perso!";
                    numeroSconfitte++;
                }
            } else {
                messaggio = "Mi dispiace, puoi scrivere solo testa o croce,"
                    + " riprova...";
            }
            System.out.println(messaggio);
        }
        messaggio = "Hai vinto " + numeroVittorie + " volte, e perso " +
            numeroSconfitte + " volte, quindi...";
        if (numeroVittorie > numeroSconfitte) {
            messaggio += "hai vinto la partita! Complimenti!";
        } else if (numeroVittorie < numeroSconfitte) {
            messaggio += "hai perso la partita! Ah ah!";
        } else {

```

```
        messaggio += "hai pareggiato partita! Riprova!";
    }
    System.out.println(messaggio);
}
}
```

Il codice è abbastanza comprensibile, ed il lettore dovrebbe essere in grado di interpretarlo da solo. Notiamo che abbiamo usato il metodo `equalsIgnoreCase` della classe `String` per fare i confronti senza tenere conto delle lettere maiuscole e minuscole. Inoltre possiamo notare la gestione della variabile `contatoreLanci`, che viene incrementato solo quando effettivamente il lancio viene eseguito, ma non quando il lancio non viene eseguito nel caso l'input dell'utente non sia compatibile con la logica del programma.

Soluzione 4.z)

Inseriamo nella classe `Rubrica` il metodo richiesto (in grassetto):

```
public class Rubrica {
    public Contatto[] contatti;
    public Rubrica () {
        contatti = new Contatto[]{
            new Contatto("Claudio De Sio Cesari", "Via Java 13",
                "131313131313"),
            new Contatto("Stevie Wonder", "Via Musica 10", "1010101010"),
            new Contatto("Gennaro Capuozzo", "Piazza Quattro Giornate 1",
                "1111111111")
        };
    }

    public Contatto[] cercaContattiPerNome(String nome) {
        Contatto []contattiTrovati = new Contatto[contatti.length];
        for (int i = 0, j = 0; i < contattiTrovati.length; i++) {
            if (contatti[i].nome.toUpperCase().contains(nome.toUpperCase())) {
                contattiTrovati[j] = contatti[i];
                j++;
            }
        }
        return contattiTrovati;
    }
}
```

L'algoritmo non è banale, ed usa anche un ciclo `for` che sfrutta due indici (`i` e `j`), che vengono usati per i due array coinvolti. Notare che l'array restituito (`contattiTrovati`) avrà la stessa dimensione dell'array `contatti`, anche se magari i suoi elementi non saranno inizializzati. Notare anche che prima di usare il metodo `contains` della classe `String` per controllare se un nome di un contatto contiene la parametro di metodo `nome`, abbiamo usato il metodo `toUpperCase` per fare un confronto che non dipende dalla maiuscole e dalle minuscole.

La classe `RicercaContatti` richiesta, la potremmo implementare nel seguente modo:

```
import java.util.Scanner;

public class RicercaContatti {
    public static void main(String args[]) {
        System.out.println("Ricerca Contatti");
        System.out.println();
        var rubrica = new Rubrica();
        System.out.println("Inserisci nome o parte del nome da ricercare");
        Scanner scanner = new Scanner(System.in);
    }
}
```

```
String input = scanner.nextLine();
Contatto[] contattiTrovati = rubrica.cercaContattiPerNome(input);
System.out.println("Contatti trovati con nome contenente \"" +
    input + "\"");
for (Contatto contatto : contattiTrovati) {
    if (contatto != null) {
        contatto.stampaDettagli();
    }
}
}
```

Notare che vengono stampati gli elementi dell'array ritornato, solo se diversi da `null`.

Soluzione 4.aa) JShell, Vero o Falso:

1. **Vero.**
2. **Falso.**
3. **Vero.**
4. **Vero.**
5. **Vero.**
6. **Vero**, le variabili non inizializzate vengono inizializzate ai propri valori nulli. Una stringa, essendo un oggetto, ha come valore nullo proprio `null`.
7. **Falso.**
8. **Falso.**
9. **Falso**, è possibile dichiarare un metodo `main`, ma non avrà lo stesso ruolo di “metodo iniziale” come in un ordinario programma Java.
10. **Vero.**

Soluzione 4.bb)

1. **Falso**, bisogna digitare il comando `exit`.
2. **Falso**, tutti i comandi JShell devono avere come prefisso `/`.
3. **Vero.**
4. **Falso**, è vero che il comando `history` mostra tutti gli snippet e tutti i comandi lanciati dall'utente nella sessione corrente. Ma non è vero che accanto agli snippet è presente uno snippet id.
5. **Falso**, il comando `list` mostra tutti gli snippet digitati dall'utente in questa sessione, con accanto i rispettivi snippet id.
6. **False**, il comando `/types -all` elencherà tutte i tipi (classi, interfacce, enumerazioni, annotazioni) dichiarate nella sessione corrente. Piuttosto il comando `/variables -all` elencherà tutte le variabili dichiarate nella sessione corrente.
7. **Vero.**
8. **Vero.**
9. **Falso**, è vero che importeremo in questa sessione, tutte i package di Java Standard Edition, ma non nelle future sessioni (avremmo dovuto esplicitare anche l'opzione `-retain`).
10. **Vero.**

Soluzione 4.cc)

L'output del comando `list` sarà vuoto. Infatti il comando `reset` avrà resettati tutti gli snippet immessi.

Soluzione 4.dd)

L'output del comando `history` sarà il seguente:

```
public int a;  
private String a  
/reset  
/list  
/history
```

Soluzione 4.ee)

Il comando corretto è:

```
/open HelloWorld.java
```

Soluzione 4.ff)

Possiamo solo invocare il metodo `main` nel seguente modo:

```
jshell> HelloWorld hw = new HelloWorld();  
hw ==> HelloWorld@52a86356  
  
jshell> hw.main(null);  
Hello World!
```

Notare che siccome l'array `args` non viene utilizzato all'interno del metodo `main`, allora abbiamo potuto passargli `null`.

Anche se non l'abbiamo ancora studiato seriamente, il modificatore `static` ci permette di evitare di istanziare l'oggetto `hw`, e lanciare direttamente il comando usando il nome della classe:

```
jshell> HelloWorld.main(null)  
Hello World!
```

Soluzione 4.gg)

Il comando corretto è:

```
/save C:/cartella/HelloWorld.java
```

Soluzione 4.hh)

1. **Vero**, in questo caso si parla di variabile implicita, e JShell ne dedurrà automaticamente il tipo.

2. **Vero**, in questo caso si parla di forwarding reference, e JShell creerà il reference, ma non lo renderà disponibile sino a quando non dichiareremo anche il suo tipo. A quel punto il reference verrà sostituito e inizializzato a `null`.
3. **Falso**, bisogna invece premere contemporaneamente i tasti SHIFT e TAB, rilasciarli e dopo premere il tasto `v` (che sta per “variable”). JShell dedurrà il tipo della variabile, la dichiarerà e posizionerà il cursore subito dopo per permetterci di definire il reference.
4. **Falso**, bisogna invece premere contemporaneamente i tasti SHIFT e TAB, rilasciarli e dopo premere il tasto `i` (che sta per “input”).
5. **Falso**, verrà aperto il programma JShell Edit Pad, a meno che prima non si sia impostato come editor di default Notepad++ mediante il comando:
`/set editor C:\Program Files (x86)\Notepad++\notepad++.exe`
6. **Falso**, la pressione del tasto TAB provoca l’auto-completamento del codice che si è iniziato a scrivere, o nel caso di più opzioni disponibili, la scelta tra esse.
7. **Vero**.
8. **Vero**.
9. **Vero**.
10. **Vero**.

Soluzione 4.ii)

Il codice richiesto potrebbe essere il seguente:

```
public class AlfabetoArray {
    public static void main(String args[]) {
        char alfabeto [] = new char[26];
        int i = 0;
        for (char c = 'a'; c <= 'z'; c++, i++) {
            alfabeto[i] = (char)c;
        }
        System.out.println(alfabeto);
    }
}
```

Soluzione 4.jj)

Meglio usare un ciclo `for` migliorato:

```
public class AlfabetoArray {
    public static void main(String args[]) {
        char alfabeto [] = new char[26];
        int i = 0;
        for (char c = 'a'; c <= 'z'; c++, i++) {
            alfabeto[i] = (char)c;
        }
        for (char lettera : alfabeto) {
            System.out.println(lettera);
        }
    }
}
```

Soluzioni degli esercizi del capitolo 5

Incapsulamento e visibilità

Soluzione 5.a) *Object Orientation in generale (teoria), Vero o Falso:*

1. **Falso**, esiste dagli anni '60.
2. **Vero**.
3. **Falso**, ogni linguaggio fornisce supporto ai vari paradigmi in maniera diversa.
4. **Vero**.
5. **Vero**.
6. **Vero**.
7. **Vero**.
8. **Vero**.
9. **Vero**.
10. **Falso**, bisogna conoscere l'interfaccia pubblica e non l'implementazione interna.

Soluzione 5.b)

Il listato potrebbe essere simile al seguente:

```
public class Pilota {
    private String nome;

    public Pilota(String nome) {
        setNome(nome);
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}

public class Auto {
    private String scuderia;
    private Pilota pilota;

    public Auto(String scuderia, Pilota pilota) {
        setScuderia(scuderia);
        setPilota(pilota);
    }

    public void setScuderia(String scuderia) {
        this.scuderia = scuderia;
    }

    public String getScuderia() {
        return scuderia;
    }
}
```



```
}

public void setPilota(Pilota pilota) {
    this.pilota = pilota;
}

public Pilota getPilota() {
    return pilota;
}

public String dammiDettagli() {
    return getPilota().getNome() + " su " + getScuderia();
}
}
```

Soluzione 5.c) *Modificatori e package, Vero o Falso:*

1. **Falso**, `private` non si può utilizzare con la dichiarazione di una classe.
2. **Vero**, `static` non si può utilizzare con la dichiarazione di una classe.
3. **Vero**, `protected` non è applicabile a classi.
4. **Vero**, `static` deve essere posizionato prima del `void`.
5. **Vero**.
6. **Falso**, `static` non è un modificatore d'accesso.
7. **Falso**, `static` non è applicabile a classi.
8. **Vero**.
9. **Vero**.
10. **Vero**.

Soluzione 5.d) *Object Orientation in Java (pratica), Vero o Falso:*

1. **Falso**.
2. **Falso**, non si tratta di parole chiave ma solo di una convenzione.
3. **Falso**, possono essere private ed essere utilizzate tramite i metodi accessor e mutator.
4. **Vero**.
5. **Vero**.
6. **Vero**.
7. **Vero**.
8. **Falso**, nel caso ci sia ambiguità tra nomi di variabili d'istanza e locali, la parola chiave `this` è fondamentale (cfr. paragrafo 5.2.2).
9. **Falso**, solo un altro costruttore della stessa classe può usare quella sintassi.
10. **Vero**.

Soluzione 5.e)

Il listato potrebbe essere simile al seguente:

```
/**
 * Questa classe astrae il concetto di Moneta (esercizio 5.e)
 */
```

```
* @author Claudio De Sio Cesari
*/
public class Moneta {

    /**
     * La valuta è una costante impostata al valore EURO.
     */
    public final static String VALUTA = "EURO";

    /**
     * Rappresenta il valore della moneta in centesimi.
     */
    private int valore;

    /**
     * Costruttore che prende in input il valore della moneta.
     *
     * @param valore il valore della moneta.
     */
    public Moneta(int valore) {
        this.valore = valore;
        System.out.println("Creata una moneta da " + valore + " centesimi");
    }

    /**
     * Imposta la variabile d'istanza valore.
     *
     * @param valore contiene il valore a cui deve essere impostato il valore
     * della variabile d'istanza valore.
     */
    public void setValore(int valore) {
        this.valore = valore;
    }

    /**
     * Restituisce il valore della variabile d'istanza valore.
     *
     * @return
     * il valore della variabile d'istanza valore.
     */
    public int getValore() {
        return valore;
    }
}
```

È sufficiente un costruttore per specificare il vincolo richiesto. Inoltre la valuta, essendo fissa per tutta le monete, è stata dichiarata come costante statica.

Soluzione 5.f)

Nell'attuale situazione, dove le specifiche hanno richiesto solo di creare una classe che abbia il vincolo di essere istanziata sempre con un valore, la domanda potrebbe risultare ambigua. Tuttavia non avendo altri vincoli espliciti, è ragionevole pensare di avere i vincoli che esistono nel mondo reale. Una moneta che ha valore specificato (supponiamo 5 centesimi) non potrà mai cambiare il proprio valore. Quindi sembra superfluo quanto meno il metodo `setValore`. Quindi sarebbe corretto rimuoverlo. È anche consigliabile rendere la variabile `final` per rinforzare il concetto di immutabilità. Di seguito il listato modificato:

```
/**
```

```
* Questa classe astrae il concetto di Moneta.
*
* @author Claudio De Sio Cesari
*/
public class Moneta {

    /**
     * La valuta è una costante impostata al valore EURO.
     */
    public final static String VALUTA = "EURO";

    /**
     * Rappresenta il valore della moneta in centesimi.
     */
    private final int valore;

    /**
     * Costruttore che prende in input il valore della moneta.
     *
     * @param valore il valore della moneta.
     */
    public Moneta(int valore) {
        this.valore = valore;
        System.out.println("Creata una moneta da " + valore + " centesimi di "
            + VALUTA);
    }

    /**
     * Restituisce il valore della variabile d'istanza valore.
     *
     * @return
     *         il valore della variabile d'istanza valore.
     */
    public int getValore() {
        return valore;
    }
}
```

Il codice è più compatto, ma forse almeno per l’inizio conviene utilizzare le variabili per meglio memorizzare le definizioni.

Soluzione 5.g)

Il listato della classe `TestMonete` potrebbe essere il seguente:

```
public class TestMonete {
    public static void main(String args[]) {
        Moneta monetaDaVentiCentesimi = new Moneta(20);
        Moneta monetaDaUnCentesimo = new Moneta(1);
    }
}
```

Eseguendo quest'applicazione l’output sarà:

```
Creata una moneta da 20 centesimi di EURO
Creata una moneta da 1 centesimi di EURO
```

Ma sarebbe più giusto che nella seconda riga la parola “centesimi” sia al singolare.

Per risolvere questo problema potremmo modificare la classe `Moneta` nel seguente modo (riportiamo solo il costruttore responsabile della stampa e un metodo di utilità):

```
public Moneta(int valore) {
    this.valore = valore;
```

```
        System.out.println("Crea una moneta da " +  
            formattaUnitaDiMisura(valore) + VALUTA);  
    }  
  
    private static String formattaUnitaDiMisura(int valore) {  
        return valore + (valore == 1 ? " centesimo di " : " centesimi di ");  
    }  
}
```

Abbiamo delegato ad un nuovo metodo d'utilità privato la formattazione di un pezzo della frase da stampare usando un semplice operatore ternario (cfr. paragrafo 4.3.4), ed abbiamo risolto il nostro baco. Ora rieseguendo la classe `TestMonete` otterremo il seguente output:

```
Crea una moneta da 20 centesimi di EURO  
Crea una moneta da 1 centesimo di EURO
```

Soluzione 5.h)

Il listato della classe `TestMonete` dovrebbe solo essere arricchito da un'istruzione di questo tipo:

```
Moneta monetaDaUnEuro = new Moneta(100);
```

L'esecuzione di questa applicazione genererà:

```
Crea una moneta da 20 centesimi di EURO  
Crea una moneta da 1 centesimo di EURO  
Crea una moneta da 100 centesimi di EURO
```

Ma sarebbe più giusto che nella terza riga "100 centesimi di EURO" fosse "1 EURO".

Per risolvere questo problema potremmo modificare la classe `Moneta` nel seguente modo

(riportiamo solo come cambiare il metodo di utilità):

```
    private static String formattaStringaDescrittiva (int valore) {  
        String stringaFormattata = " centesimi di ";  
        if (valore == 1) {  
            stringaFormattata = " centesimo di ";  
        } else if (valore > 99){  
            stringaFormattata = " ";  
            valore /= 100;  
        }  
        return valore + stringaFormattata;  
    }  
}
```

Abbiamo modificato il metodo d'utilità privato introdotto nell'esercizio precedente. Non è stato possibile usare l'operatore ternario, quindi abbiamo usato un costrutto `if`, ed abbiamo risolto il nostro baco. Ora rieseguendo la classe `TestMonete` otterremo il seguente output:

```
Crea una moneta da 20 centesimi di EURO  
Crea una moneta da 1 centesimo di EURO  
Crea una moneta da 1 EURO
```

Il metodo `getDescrizione` quindi potrebbe essere codificato così:

```
/**  
 * Ritorna una descrizione della moneta corrente.  
 *  
 * @return  
 *         una descrizione della moneta corrente.  
 */  
public String getDescrizione() {  
    String descrizione = "moneta da " + formattaStringaDescrittiva(valore)  
        + VALUTA;  
    return descrizione;  
}
```

E quindi anche il costruttore potrebbe riusare questo metodo nel seguente modo:

```
public Moneta(int valore) {
```

```
        this.valore = valore;
        System.out.println("Creata una " + getDescrizione());
    }
```

Soluzione 5.i)

Il listato della classe `PortaMonete` potrebbe essere il seguente:

```
/**
 * Astrae il concetto di portamonete che può contenere un numero limitato di
 * monete.
 *
 * @author Claudio De Sio Cesari
 */
public class PortaMonete {

    /**
     * Un array che contiene un numero limitato di monete.
     */
    private final Moneta[] monete = new Moneta[10];

    /**
     * Crea un oggetto portamonete contenente monete i cui valori sono
     * specificati dal varargs valori.
     *
     * @param valori
     *         un varargs di valori di monete.
     */
    public PortaMonete(int... valori) {
        int numeroMonete = valori.length;
        for (int i = 0; i < numeroMonete; i++) {
            if (i >= 10) {
                System.out.println(
                    "Sono state inserite solo le prime 10 monete!");
                break;
            }
            monete[i] = new Moneta(valori[i]);
        }
    }
}
```

Si noti che abbiamo usato un array di 10 oggetti `Moneta` dichiarato `final`, che farà da contenitore delle nostre monete. Inoltre abbiamo usato un varargs `valori` per impostare il contenuto del `PortaMonete`. Questo ci sarà comodo quando effettivamente creeremo oggetti di tipo `PortaMonete`.

Abbiamo già introdotto il concetto di varargs nella soluzione del paragrafo 4.v e sarà spiegato in dettaglio nel libro nel paragrafo 7.2.2.

Nel caso siano passati più di dieci valori al costruttore, questo imposterà solo i primi dieci e stamperà un messaggio di avvertimento.

Soluzione 5.j)

Il listato della classe `TestMonete` modificato dovrebbe essere qualcosa di simile al seguente:

```
/**
 * Classe di test per le classi Moneta e PortaMonete.
 *
 * @author Claudio De Sio Cesari
 */
public class TestMonete {

    public static void main(String args[]) {
        Moneta monetaDaVentiCentesimi = new Moneta(20);
        Moneta monetaDaUnCentesimo = new Moneta(1);
        Moneta monetaDaUnEuro = new Moneta(100);
        // Creiamo un portamonete con 8 monete
        PortaMonete portaMonete =
            new PortaMonete(new int[]{2, 5, 2, 5, 100, 10, 50, 10, 100, 200});
        // Creiamo un portamonete con 11 monete
        PortaMonete portaMoneteInsufficiente =
            new PortaMonete(new int[]{2, 5, 100, 10, 50, 10, 100, 200, 10,
                5, 2});
    }
}
```

L'output dovrebbe essere il seguente:

```
Creata una moneta da 20 centesimi di EURO
Creata una moneta da 1 centesimo di EURO
Creata una moneta da 1 EURO
Creata una moneta da 2 centesimi di EURO
Creata una moneta da 5 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 50 centesimi di EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 2 EURO
Creata una moneta da 2 centesimi di EURO
Creata una moneta da 5 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 50 centesimi di EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 2 EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 5 centesimi di EURO
Sono state inserite solo le prime 10 monete!
```

Soluzione 5.k)

Il codice risolutore proposto crea come soluzione anche un metodo privato d'utilità che restituisce il primo indice dell'array libero per contenere la nuova moneta:

```
/**
 * Aggiunge una moneta al portamonete. Se questo è pieno la moneta non è
 * aggiunta e viene stampato un errore significativo.
 */
```

```
* @param moneta
*         la moneta da aggiungere.
*/
public void aggiungi(Moneta moneta) {
    System.out.println("Proviamo ad aggiungere una " +
        moneta.getDescrizione());
    int indiceLibero = primoIndiceLibero();
    if (indiceLibero == -1) {
        System.out.println("Portamonete pieno! La moneta " +
            moneta.getDescrizione() + " non è stata aggiunta...");
    } else {
        monete[indiceLibero] = moneta;
        System.out.println("E' stata aggiunta una " +
            moneta.getDescrizione());
    }
}

/**
 * Restituisce il primo indice libero nell'array delle monete o -1 se il
 * portamonete è pieno.
 */
* @return
 *         il primo indice libero nell'array delle monete o -1 se il
 *         portamonete è pieno.
*/
private int primoIndiceLibero() {
    int indice = -1;
    for (int i = 0; i < 10; i++) {
        if (monete[i] == null) {
            indice = i;
            break;
        }
    }
    return indice;
}
```

Soluzione 5.1)

Il listato per il metodo stato potrebbe essere il seguente:

```
/**
 * Stampa il contenuto del portamonete.
 */
public void stato() {
    System.out.println("Il portamonete contiene:");
    for (Moneta moneta : monete) {
        if (moneta == null) {
            break;
        }
        System.out.println("Una " + moneta.getDescrizione());
    }
}
```

Soluzione 5.m)

Il listato per il metodo `preleva` potrebbe essere il seguente (anche in questo caso abbiamo creato un metodo privato di utilità):

```
/**
 * Esegue un prelievo della moneta specificata dal portamonete corrente. Nel
 * caso non sia presente la moneta specificata, un errore significativo
 * verrà stampato e null verrà ritornato.
 *
 * @param moneta
 *         la moneta da prelevare.
 * @return
 *         la moneta trovata o null se non trovata.
 */
public Moneta preleva(Moneta moneta) {
    System.out.println("Proviamo a prelevare una " +
        moneta.getDescrizione());
    Moneta monetaTrovata = null;
    int indiceMonetaTrovata = indiceMonetaTrovata(moneta);
    if (indiceMonetaTrovata == -1) {
        System.out.println("Moneta non trovata!");
    } else {
        monetaTrovata = moneta;
        monete[indiceMonetaTrovata] = null;
        System.out.println("Una " + moneta.getDescrizione() + " prelevata");
    }
    return monetaTrovata;
}

private int indiceMonetaTrovata(Moneta moneta) {
    int indiceMonetaTrovata = -1;
    for (int i = 0; i < 10; i++) {
        if (monete[i] == null) {
            continue;
        }
        int valoreMonetaNelPortaMoneta = monete[i].getValore();
        int valore = moneta.getValore();
        if (valore == valoreMonetaNelPortaMoneta) {
            indiceMonetaTrovata = i;
            break;
        }
    }
    return indiceMonetaTrovata;
}
```

Soluzione 5.n)

Come soluzione proponiamo un codice che cerca di testare anche le situazioni di errore:

```
/**
 * Classe di test per la classe Moneta.
 *
 * @author Claudio De Sio Cesari
 */
public class TestMonete {
    public static void main(String args[]) {
        Moneta monetaDaVentiCentesimi = new Moneta(20);
        Moneta monetaDaUnCentesimo = new Moneta(1);
    }
}
```

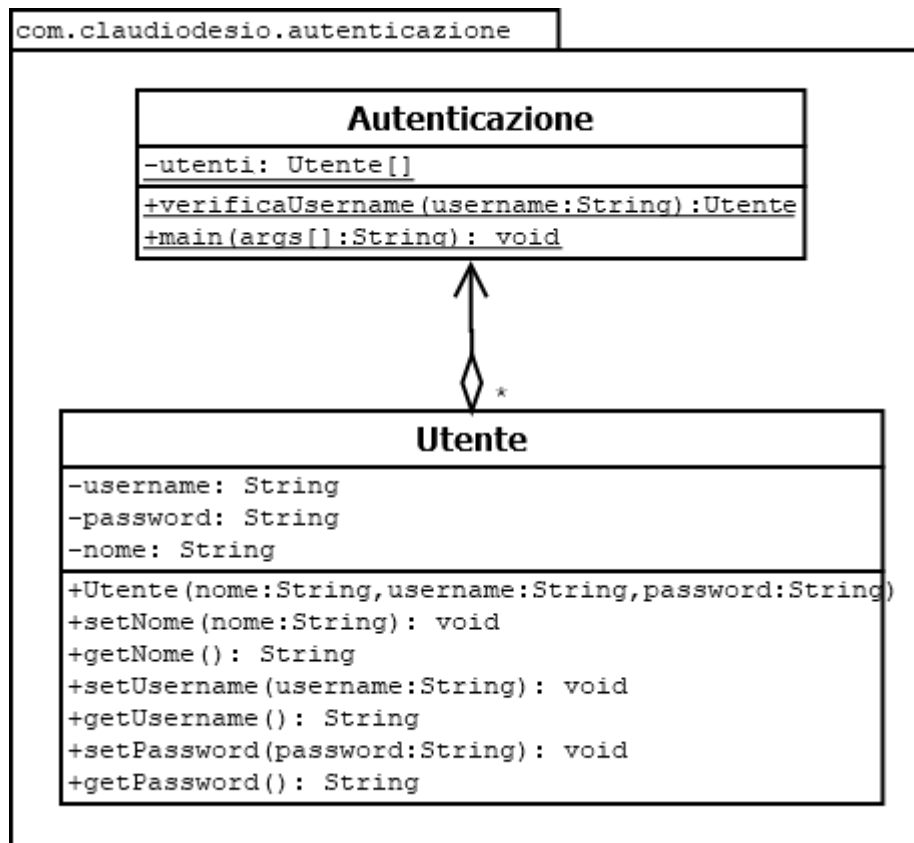


```
Moneta monetaDaUnEuro = new Moneta(100);
// Creiamo un portamonete con 11 monete
PortaMonete portaMoneteInsufficiente =
    new PortaMonete(2, 5, 100, 10, 50, 10, 100, 200, 10, 5, 2);
// Creiamo un portamonete con 8 monete
PortaMonete portaMonete =
    new PortaMonete(2, 5, 100, 10, 50, 10, 100, 200);
portaMonete.stato();
// Aggiungiamo una moneta da 20 centesimi
portaMonete.aggiungi(monetaDaVentiCentesimi);
// Aggiungiamo la decima moneta da 1 centesimo.
portaMonete.aggiungi(monetaDaUnCentesimo);
// Aggiungiamo l'undicesima moneta (dovremmo ottenere un errore e la
// moneta non sarà aggiunta)
portaMonete.aggiungi(monetaDaUnEuro);
// Valutiamo lo stato del portamonete.
portaMonete.stato();
// preleviamo 20 centesimi
portaMonete.preleva(monetaDaVentiCentesimi);
// Aggiungiamo l'undicesima moneta (dovremmo ottenere un errore e la
// moneta non sarà aggiunta)
portaMonete.aggiungi(monetaDaUnEuro);
portaMonete.stato();
//Cerchiamo una moneta non esistente (dovremmo ottenere una stampa di
// errore)
portaMonete.preleva(new Moneta(7));
}
}
```

L'output dovrebbe essere il seguente:

```
Creata una moneta da 20 centesimi di EURO
Creata una moneta da 1 centesimo di EURO
Creata una moneta da 1 EURO
Creata una moneta da 2 centesimi di EURO
Creata una moneta da 5 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 50 centesimi di EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 2 EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 5 centesimi di EURO
Sono state inserite solo le prime 10 monete!
Creata una moneta da 2 centesimi di EURO
Creata una moneta da 5 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 50 centesimi di EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 2 EURO
Il portamonete contiene:
Una moneta da 2 centesimi di EURO
Una moneta da 5 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 10 centesimi di EURO
Una moneta da 50 centesimi di EURO
```

```
Una moneta da 10 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 2 EURO
Proviamo ad aggiungere una moneta da 20 centesimi di EURO
E' stata aggiunta una moneta da 20 centesimi di EURO
Proviamo ad aggiungere una moneta da 1 centesimo di EURO
E' stata aggiunta una moneta da 1 centesimo di EURO
Proviamo ad aggiungere una moneta da 1 EURO
Portamonete pieno! La moneta moneta da 1 EURO non è stata aggiunta...
Il portamonete contiene:
Una moneta da 2 centesimi di EURO
Una moneta da 5 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 10 centesimi di EURO
Una moneta da 50 centesimi di EURO
Una moneta da 10 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 2 EURO
Una moneta da 20 centesimi di EURO
Una moneta da 1 centesimo di EURO
Proviamo a prelevare una moneta da 20 centesimi di EURO
Una moneta da 20 centesimi di EURO prelevata
Proviamo ad aggiungere una moneta da 1 EURO
E' stata aggiunta una moneta da 1 EURO
Il portamonete contiene:
Una moneta da 2 centesimi di EURO
Una moneta da 5 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 10 centesimi di EURO
Una moneta da 50 centesimi di EURO
Una moneta da 10 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 2 EURO
Una moneta da 1 EURO
Una moneta da 1 centesimo di EURO
Creata una moneta da 7 centesimi di EURO
Proviamo a prelevare una moneta da 7 centesimi di EURO
Moneta non trovata!
```

Soluzione 5.o)

Il diagramma in figura 5.o.1

potrebbe rappresentare il processo presentato.

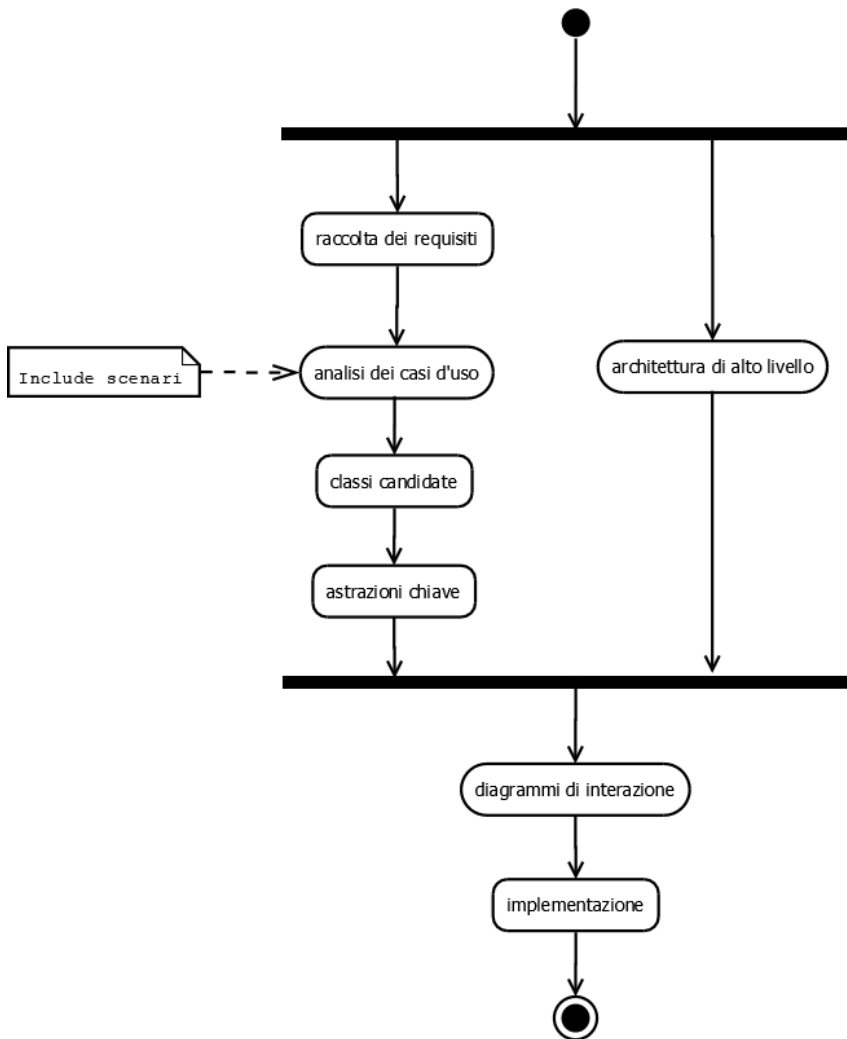


Figura 5.o.1 - Activity diagram che modella il processo.

Si possono notare le barre di sincronizzazione che ci sono servite per mostrare il possibile parallelismo tra le attività di analisi/progettazione e l'architettura.

Soluzione 5.p)

In realtà esiste un unico caso d'uso, che abbiamo chiamato "Login" (vedi figura 5.r.1). Infatti le interazioni che l'utente avrà con il sistema, si limitano alle attività relative all'autenticazione, come l'inserimento della username e della password. Anche pensando a diverse tipologie di flussi che possono realizzare l'autenticazione, il compito finale è unico quello di autenticarsi.

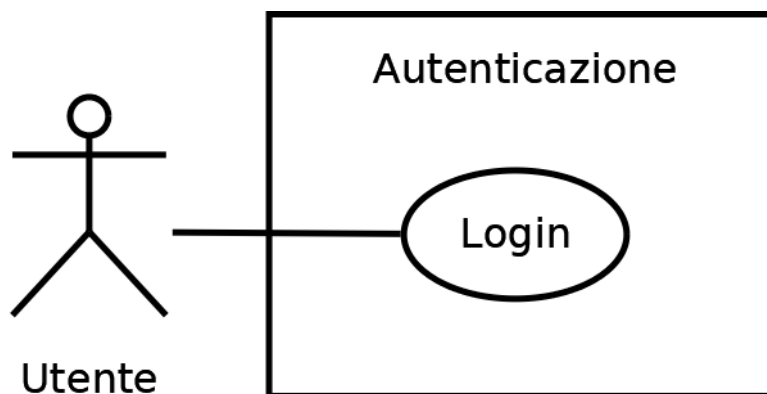


Figura 5.p.1 – Use case diagram.

Nel nostro caso, abbiamo trovato un unico caso d’uso, ma questo non significa che sarà sempre così, e che quindi bisogna evitare l’analisi degli use case. A nostro giudizio essa è indispensabile anche per i programmi più banali.

Soluzione 5.q)

L’analisi degli scenari è molto soggettiva. Nel momento in cui la scriviamo, stiamo decidendo “cosa” deve fare l’applicazione, un qualcosa tutt’altro che scontata.

Partiamo dal prerequisito che non abbiamo ancora studiato le interfacce grafiche (a cui sono dedicati due ultimi capitoli del libro), e quindi abbiamo pensato di realizzare un programma che funzioni solo da riga di comando.

Un altro prerequisito è che il sistema ha precaricato staticamente alcune coppie di username e password valide.

Scenario principale:

1. Il sistema chiede all’utente di inserire la username.
2. L’utente inserisce la username.
3. Il sistema verifica che la username sia valida, e chiede di inserire la password.
4. L’utente inserisce la password.
5. Il sistema controlla che la password sia valida e risponde con un messaggio di conferma di avvenuta autenticazione, utilizzando il vero nome dell’utente che si è autenticato.

Come già detto questa è solo una delle possibili soluzioni. Potremmo anche pensare di specificare insieme la username e la password, convalidare l’autenticazione con un codice captcha, avvertire l’utente se il tasto CAPS LOCK (BLOCCO MAIUSCOLO) è inserito, mascherare i caratteri della password con asterischi, richiedere all’utente se vuole memorizzare la username per il prossimo accesso e così via. Abbiamo scelto un’interazione semplice.

Scenario 2:

1. Il sistema chiede all’utente di inserire la username.
2. L’utente inserisce la username.
3. Il sistema non riconosce la username inserita, stampa un messaggio e torna al punto 1.

Scenario 3:

1. Il sistema chiede all’utente di inserire la username.

2. L'utente inserisce la username.
3. Il sistema verifica che la username sia valida, e chiede di inserire la password.
4. L'utente inserisce la password.
5. Il sistema non riconosce la password inserita, stampa un messaggio e torna al punto 1.

Definendo questi tre banali scenari, ora abbiamo ben chiaro cosa fare.

Soluzione 5.r)

Con il diagramma della figura 5.r.1, evidenziamo come creeremo un componente software che contiene le classi che realizzano l'autenticazione separato dalla classe del `main`. L'unico strumento che attualmente conosciamo per separare le classi sono i package, quindi potremo usare package diversi.

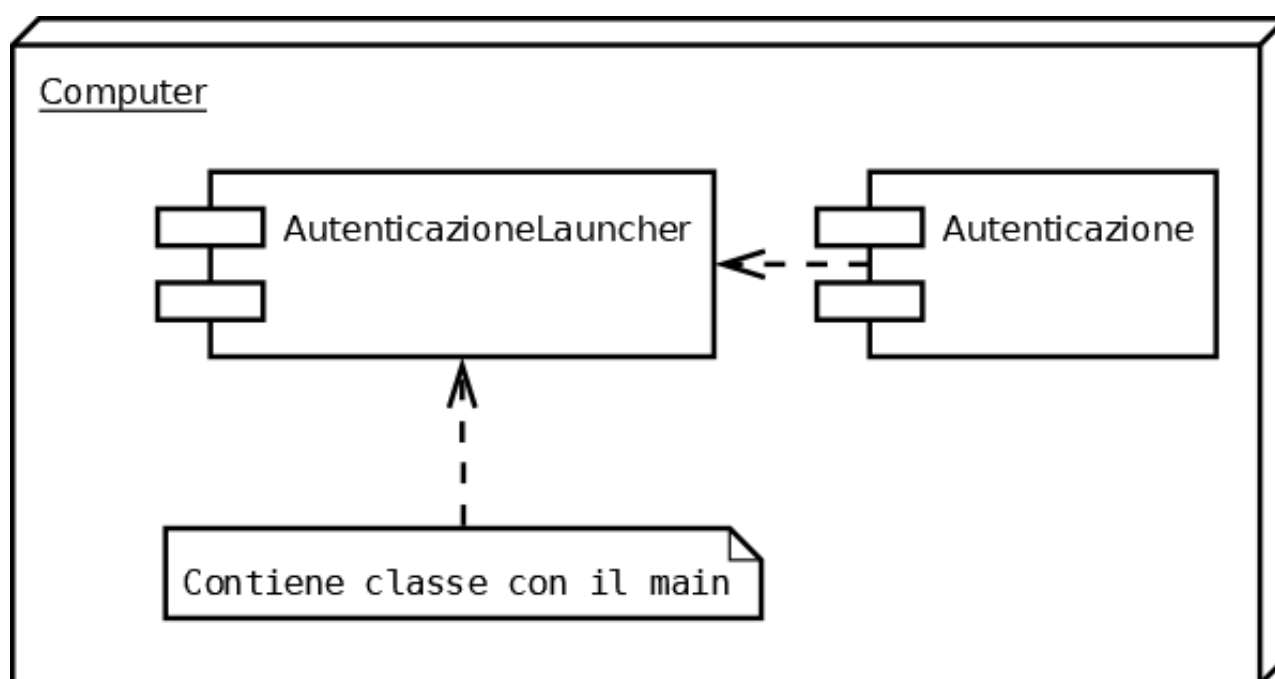


Figura 5.r.1 – Architettura dell'applicazione espressa con un high level deployment diagram.

Consultando l'appendice E potete anche imparare a creare il file JAR contenente le classi del componente dell'autenticazione. In questo modo sarà più semplice poterlo riutilizzare in un secondo momento trattandosi di un unico file. In realtà la traccia dell'esercizio E.a relativo a quell'appendice, richiede proprio che sia creato il file JAR con le classi di questo esercizio.

Soluzione 5.s)

Seguendo il processo descritto nell'approfondimento 5.7, per poter trovare la lista delle *key abstraction*, dobbiamo dapprima stilare la lista delle *candidate classes*. Segue la nostra lista con relativi commenti:

- **Autenticazione:** potrebbe essere una classe a cui diamo la responsabilità di gestire la funzionalità principale dell'applicazione.

- Login: sembra più il nome del metodo principale dell'applicazione. Potrebbe essere un metodo all'interno di `Autenticazione`, o di un oggetto dichiarato all'interno di `Autenticazione`.
- **Utente**: potrebbe essere l'entità che contiene le informazioni per l'autenticazione.
- Sistema: troppo generico, non sembra essere il nome giusto per una classe.
- Username: potrebbe essere una proprietà della classe `Utente`.
- Password: potrebbe essere una proprietà della classe `Utente`.
- Nome: potrebbe essere una proprietà della classe `Utente`.
- Verifica: potrebbe essere un metodo di `Autenticazione`, o di un oggetto dichiarato all'interno di `Autenticazione`.
- Inserimento: potrebbe definire un metodo, ma non sembra un'astrazione chiave.
- Messaggio: per ora sembra solo una stringa, non di certo un'astrazione chiave. Questo non esclude che possa diventare una classe in seguito.
- Sostituzione: potrebbe definire un metodo, ma non sembra un'astrazione chiave.

Quindi la lista delle astrazioni chiave, per ora si limita solo alle uniche due classi della lista in grassetto: `Autenticazione` e `Utente`. A queste aggiungiamo una classe `AutenticazioneLauncher` che contiene solo il metodo `main`, e che ha la sola responsabilità di avviare l'applicazione istanziando i giusti oggetti e chiamando i giusti metodi.

Ricordiamo al lettore che la scelta di queste classi, come i precedenti passi, dipendono dell'esperienza, delle intenzioni, dal tempo a disposizione, dalla predisposizione, dalla concretezza e dalla forma mentis della persona che svolge l'analisi. Esistono migliaia di soluzioni che possono portare al successo lo sviluppo, ognuna delle quali ha dei pro e dei contro. Il consiglio è quello di orientarsi (soprattutto nei primi tempi) sulla soluzione più semplice possibile.

Soluzione 5.t)

Seguendo il processo descritto nell'approfondimento 5.7, dobbiamo ora verificare la definizione superficiale delle key abstraction, con diagrammi di interazione (cfr. approfondimento 4.6), ovvero collaboration o sequence diagram. Questi due diagrammi sono equivalenti, tanto che molti tool UML, permettono di trasformare un collaboration diagram in un sequence diagram e viceversa con la pressione di un tasto. In particolare un sequence diagram mostra le interazioni tra gli oggetti in un determinato lasso di tempo, enfatizzando la sequenza dei messaggi che le entità si scambiano. Un collaboration diagram invece, come il sequence diagram mostra le interazioni tra gli oggetti in un determinato lasso di tempo, ma enfatizza l'organizzazione strutturale delle entità che interagiscono. Siccome in questo caso ci sembra più interessante enfatizzare la sequenza di messaggi scambiati tra gli oggetti, useremo sequence diagram per descrivere gli scenari descritti nella soluzione dell'esercizio 5.q, usando gli oggetti descritti nell'esercizio 5.s. Dato che questi ultimi oggetti sono solo astrazioni chiave, in questo momento possiamo anche decidere se dobbiamo creare nuove classi, aggiungere, modificare o spostare metodi, rinominare le classi esistenti e così via. Il diagramma, con la sua "visione dell'alto", favorisce l'individuazione di eventuali situazioni errate, migliorabili o fallaci.

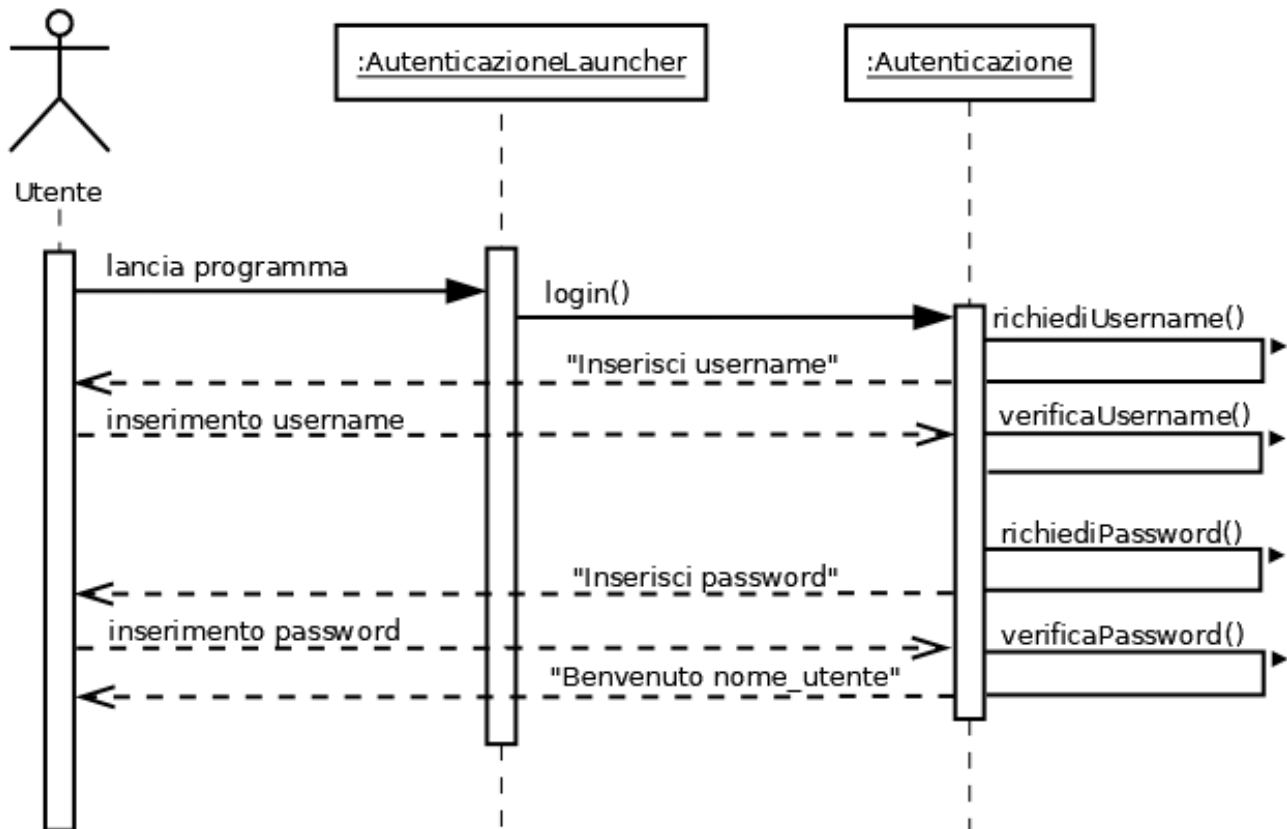


Figura 5.t.1 – Sequence diagram che rappresenta lo scenario principale.

Nella figura 5.t.1 ci siamo limitati a trovare un flusso coerente con quello descritto nello scenario principale, utilizzando solo le key abstraction individuate nell'esercizio precedente. La situazione vista così, sembra funzionare.

L'utente dell'applicazione lancia l'applicazione tramite la classe `AutenticazioneLauncher`.

Questa chiama un metodo che si chiama `login` sulla classe `Autenticazione`. Da qui in poi questo metodo eseguirà delle operazioni. Prima richiama il metodo `richiediUsername`, che richiede la username all'utente stampando un messaggio, e aspetta l'input dell'utente.

Questo metodo è interno, cioè definito nella stessa classe. Lo si capisce dal fatto che la freccia che lo definisce parte e termina nella stessa classe. In pratica, in UML la chiamata di un metodo è indicata dalla partenza di una "freccia" (che come elemento UML viene chiamato messaggio) da un certo oggetto. La freccia punta all'oggetto dove risiede il metodo chiamato. Quindi una freccia che ritorna sullo stesso oggetto sta ad indicare la chiamata ad un metodo interno.

Poi una volta che l'utente inserisce la username, la classe `Autenticazione` chiamerà un metodo interno chiamato `verificaUsername`. Tale metodo recupererà l'oggetto utente dalla collezione di utenti che è stata definita per rappresentare la lista di utenti (che implementeremo con un array).

Questa parte non è stata rappresentata nel diagramma, per non complicarlo troppo. Avremmo potuto aggiungere un elemento nota per specificare le nostre intenzioni, sarebbe stato più corretto, ma abbiamo evitato visto che dovevamo spiegare con queste righe il diagramma.

Quindi anche se l'oggetto `Utente`, non compare sul diagramma, è in qualche modo coinvolto.

Questo perché nella nostra mente, un utente dovrebbe definire le variabili `username`, `password` e `nome`, e quindi per verificare se esiste una certa username, ma anche per vedere se una password è associata ad una certa username, bisogna utilizzare un oggetto `Utente`.

Il resto del flusso è molto semplice da interpretare. Viene chiamato il metodo interno `richiediPassword`, che richiede la password all'utente stampando un messaggio e aspetta l'input dell'utente. Poi una volta che l'utente inserisce la password, la classe `Autenticazione` chiamerà un metodo interno chiamato `verificaPassword`. (ed anche in questo caso la verifica verrà fatta utilizzando un oggetto `Utente`). Infine viene stampato il messaggio "Benvenuto" specificando il nome dell'utente che si è loggato (che si prende dall'oggetto utente che è stato usato per convalidare l'autenticazione).

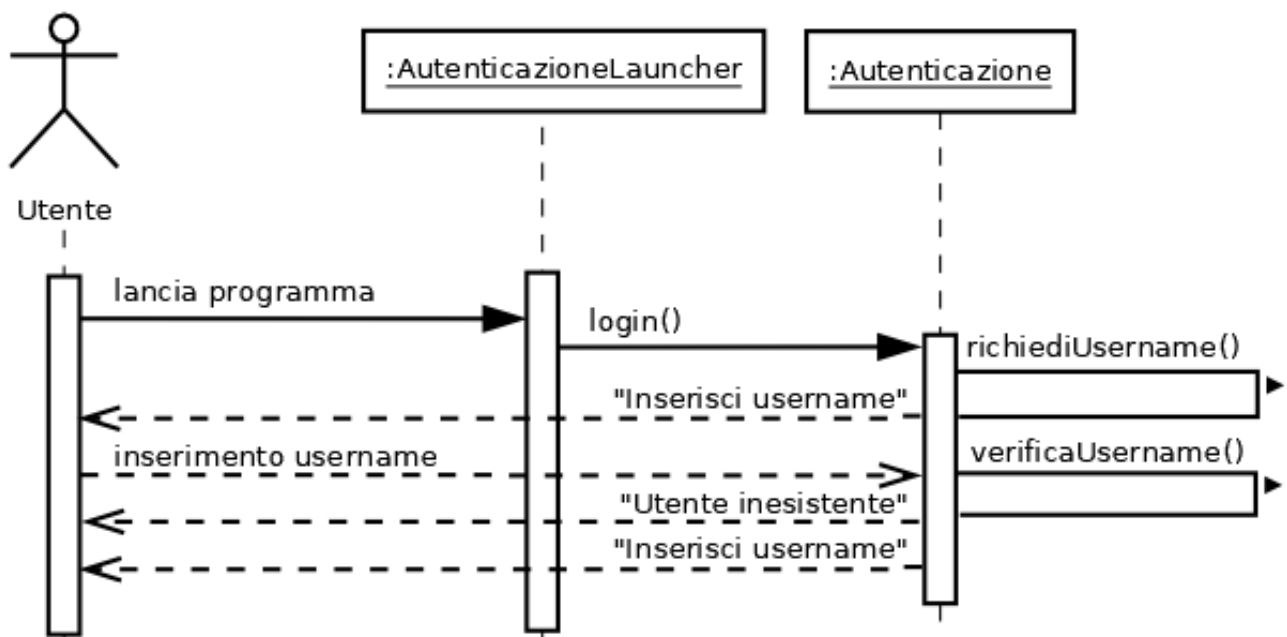


Figura 5.t.2 – Sequence diagram che rappresenta il secondo scenario.

Possiamo notare che per quello che abbiamo progettato con il diagramma in figura 5.t.2, e nello scenario relativo, la username verrà richiesta sin quando non verrà immessa una username valida. Stesso discorso per la password come è possibile osservare nella figura 5.t.3 che mostra un sequence diagram relativo al terzo scenario individuato.

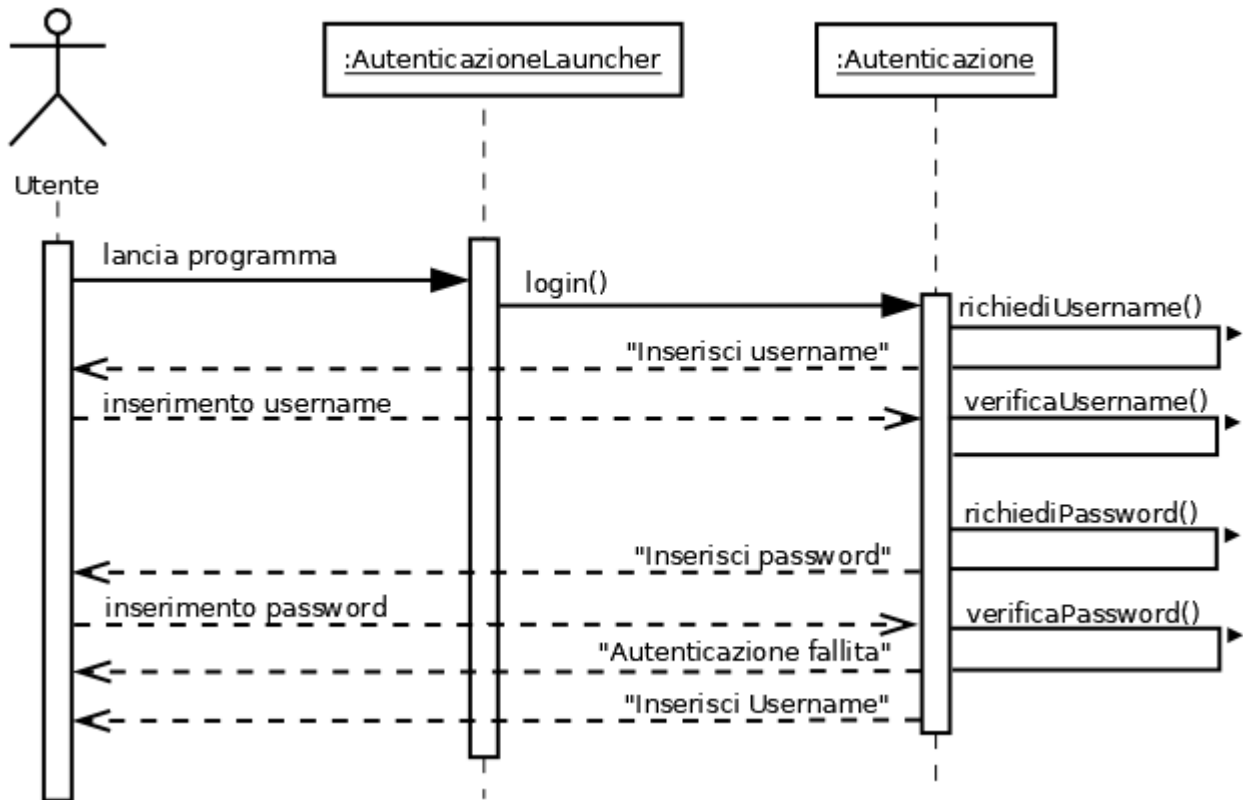


Figura 5.t.3 – Sequence diagram che rappresenta il terzo scenario.

Soluzione 5.u)

Il listato che è scaturito dalla nostra analisi, non è proprio quello che ci aspettavamo.

La classe `Utente` potrebbe essere definita così:

```
package com.claudiodesio.autenticazione;

public class Utente {
    public String nome;
    public String username;
    public String password;

    public Utente(String n, String u, String p) {
        this.nome = n;
        this.username = u;
        this.password = p;
    }
}
```

Mentre questa è la classe `Autenticazione`:

```
package com.claudiodesio.autenticazione;

import java.util.*;

public class Autenticazione {

    private static final Utente[] utenti = {
        new Utente("Daniele", "dansap", "musica"),
        new Utente("Giovanni", "giobat", "scienze"),
        new Utente("Ligeia", "ligder", "arte")
    }
}
```

```
};

public static void main(String args[]) {
    Scanner scanner = new Scanner(System.in);
    while (true) {
        System.out.println("Inserisci username.");
        String username = scanner.nextLine();
        Utente utente = verificaUsername(username);
        if (utente == null) {
            System.out.println("Utente non trovato!");
            continue;
        }
        System.out.println("Inserisci password");
        String password = scanner.nextLine();
        if (password != null && password.equals(utente.password)) {
            System.out.println("Benvenuto " + utente.nome);
            break;
        } else {
            System.out.println("Autenticazione fallita");
        }
    }
}

private static Utente verificaUsername(String username) {
    if (username != null) {
        for (Utente utente : utenti) {
            if (username.equals(utente.username)) {
                return utente;
            }
        }
    }
    return null;
}
}
```

Nel momento in cui abbiamo scritto il codice, abbiamo trovato alcune difficoltà. La prima è stata implementare l'algoritmo descritto negli scenari: abbiamo dovuto introdurre un ciclo infinito, e i comandi `break` e `continue`, non proprio la soluzione che ci aspettavamo.

Abbiamo preso anche un'altra decisione in contrasto con la nostra analisi: eliminare la classe `AutenticazioneLauncher` che avrebbe dovuto contenere solo il metodo `main`. Questo perché sembrava una decisione forzata e priva di una qualche utilità.

Un altro dubbio ci è sorto nel momento in cui abbiamo dovuto verificare la correttezza della username e della password, è quello che non avevamo deciso in fase di progettazione se la verifica doveva tener conto delle lettere maiuscole o minuscole.

Non siamo riusciti neanche a creare un metodo `verificaPassword`, complementare al `verificaUsername` visto che se avessimo creato un metodo a parte, non avremmo potuto utilizzare la clausola `break` per uscire dal ciclo infinito. Insomma, rispetto all'analisi che abbiamo fatto, ci sono stati dei problemi che abbiamo risolto direttamente con il codice. Ma da dove sono scaturiti questi problemi? Perché il nostro processo non ha funzionato come speravamo?

La risposta è che essenzialmente ci mancano ancora concetti fondamentali che dobbiamo ancora studiare. E questo esercizio sarà ripreso dopo per essere reso più chiaro ed efficiente. In particolare ci è mancato un passaggio fondamentale, che consiste nell'assegnare le responsabilità alle classi che creiamo, e al passo di creare un diagramma delle classi che ci aiuti a meglio distribuire le responsabilità tra le classi. Ogni oggetto deve avere un'unica responsabilità, o più responsabilità relazionate strettamente tra loro. Le responsabilità verranno implementate o come metodi, o come variabili, e definiscono ruoli che si possono assegnare agli oggetti.

In qualsiasi caso, il programma che abbiamo scritto funziona correttamente. Anche se la nostra analisi non è stata perfetta, ci ha dato delle indicazioni importanti. Per esempio l'analisi degli scenari è stata fondamentale per capire cosa dovevamo fare. E i diagrammi di interazione ci hanno anche indirizzato verso la soluzione di codice (poi parzialmente disattesa)

Dopo l'esperienza fatta con questi esercizi, se nel caso non siate riuscite a risolvere gli esercizi 4.m, 4.q, 4.r, 4.s, 4.t, 4.u, 4.v e 4.z del capitolo precedente, potreste provare a riprogettarli da zero e cercare di trovare una soluzione da soli, magari diversa da quella proposta).

Soluzione 5.v)

Il listato di `Utente` incapsulato è il seguente:

```
package com.claudiodesio.autenticazione;

public class Utente {
    private String nome;
    private String username;
    private String password;

    public Utente(String n, String u, String p) {
        this.nome = n;
        this.username = u;
        this.password = p;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }
}
```

mentre `Autenticazione` cambia poco: bisogna solo sostituire l'accesso diretto alle variabili pubbliche di `Utente`, con le relative chiamate ai metodi accessor:

```
package com.claudiodesio.autenticazione;

import java.util.Scanner;
```

```
public class Autenticazione {

    private static final Utente[] utenti = {
        new Utente("Daniele", "dansap", "musica"),
        new Utente("Giovanni", "giobat", "scienze"),
        new Utente("Ligeia", "ligder", "arte")
    };

    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        while (true) {
            System.out.println("Inserisci username.");
            String username = scanner.nextLine();
            Utente utente = verificaUsername(username);
            if (utente == null) {
                System.out.println("Utente non trovato!");
                continue;
            }
            System.out.println("Inserisci password");
            String password = scanner.nextLine();
            if (password != null && password.equals(utente.getPassword())) {
                System.out.println("Benvenuto " + utente.getNome());
                break;
            } else {
                System.out.println("Autenticazione fallita");
            }
        }
    }

    private static Utente verificaUsername(String username) {
        if (username != null) {
            for (Utente utente : utenti) {
                if (username.equals(utente.getUsername())) {
                    return utente;
                }
            }
        }
        return null;
    }
}
```

Soluzione 5.w)

La figura 5.w.1 mostra il class diagram richiesto. Si noti come i membri della classe Autenticazione siano marcati statici con una notazione costituita da una sottolineatura. Inoltre l'aggregazione (che indica la *relazione di contenimento*) viene indirizzata dall'oggetto contenuto all'oggetto contenente, e si distingue sintatticamente da una semplice associazione (*relazione d'uso*) tramite il disegno di un rombo bianco sul lato dell'oggetto contenuto. Il simbolo di asterisco * accanto all'oggetto contenuto invece, descrive la molteplicità dell'oggetto contenuto.

Come abbiamo già notato in precedenza, la notazione di molteplicità che si inserisce nel diagramma delle classi, rappresenta un'informazione relativa alle possibili istanze della classe coinvolta nella relazione. Ricordiamo che la notazione della molteplicità, viene utilizzata nel diagramma delle classi sulle

notazioni di associazione, aggregazione e composizione. A volte, in altri diagrammi come il component diagram, la notazione di molteplicità, viene usata anche sulla notazione della relazione di dipendenza.

Si noti infine che sul lato dell'oggetto contenente non ci sono simboli di molteplicità, questo significa che è come se fosse presente la molteplicità di default, ovvero 1. Infatti un oggetto Autenticazione contiene più oggetti Utente.

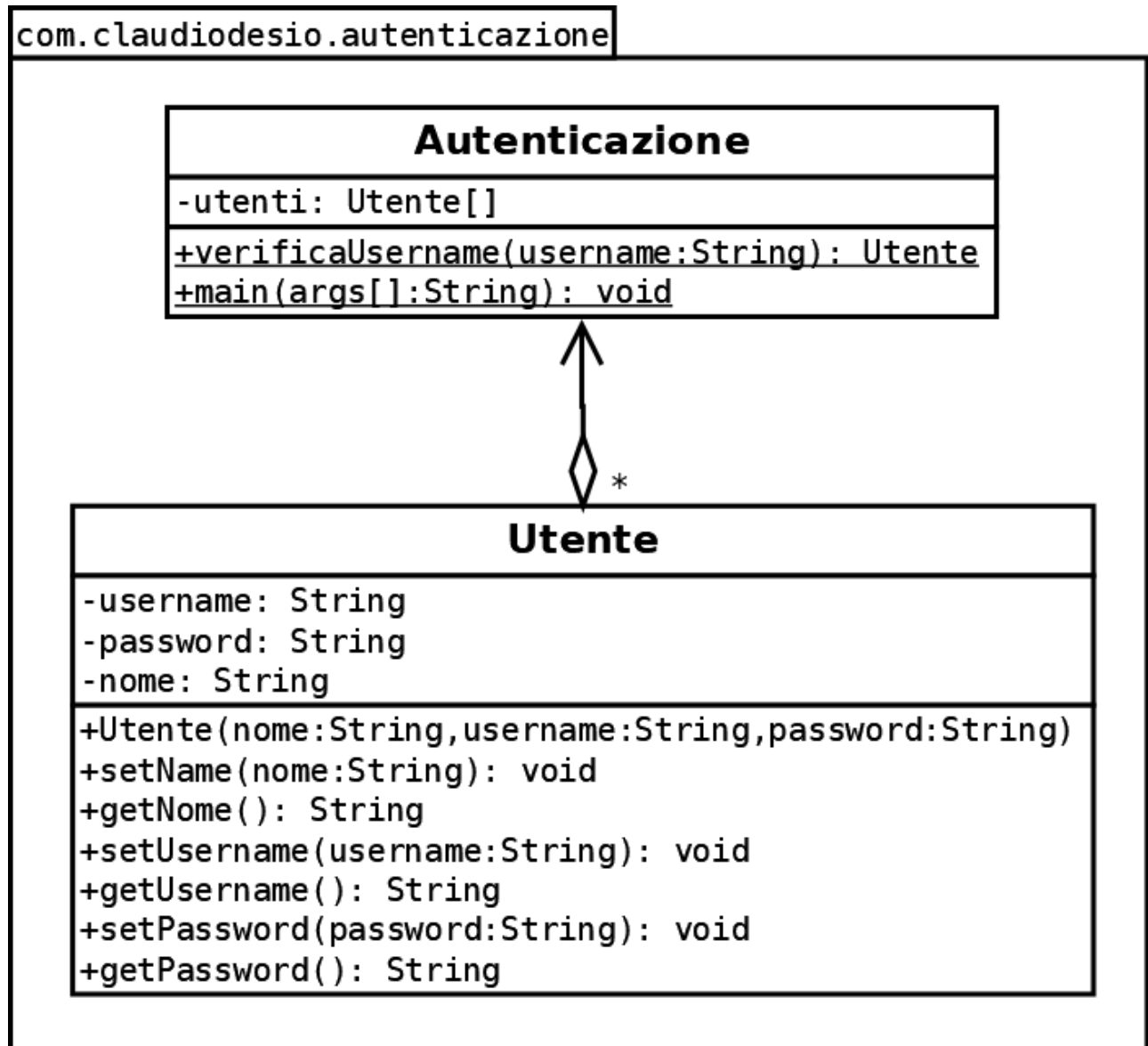


Figura 5.w.1 - Class diagram del package com.claudiodesio.autenticazione.

Soluzione 5.x)

La figura 5.x.1 mostra il class diagram richiesto. Notiamo che per trasformare la classe in singleton, abbiamo definito un costruttore privato, una variabile statica `instance` di tipo `Autenticazione`, e

un metodo `getInstance` che ha la responsabilità di restituire sempre la stessa istanza di `Autenticazione`, ovvero l'istanza `instance`, che viene istanziata opportunamente un'unica volta. Visto che la classe è ora un singleton e verrà istanziata, abbiamo fatto in modo che i suoi metodi e le sue variabili non siano più statici. Abbiamo scritto tutti i metodi che sono stati descritti nei sequence diagram e abbiamo aggiunto i tipi di ritorno e i parametri a nostro avviso più corretti. In realtà scopriremo se questi sono davvero corretti quando affronteremo l'implementazione, questa per ora è solo la nostra idea, ed anche un'idea superficiale (si pensi alla soluzione di codice che abbiamo implementato nell'esercizio 5.u, dove il risultato non è quello a cui volevamo arrivare). In particolare abbiamo inteso i metodi `richiediUsername` e `richiediPassword` come metodi di stampa. Infatti non prendono in input parametri e nemmeno dichiarano tipi di ritorno. I metodi `verificaUsername` e `verificaPassword` invece dovrebbero tornare un booleano (`true` se la verifica va a buon fine e `false` se non va a buon fine). Abbiamo anche aggiunto i metodi `auguraBenvenuto`, `autenticazioneFallita`, e `usernameInesistente` intesi come metodi di stampa, anche se non sono stati riportati nei sequence diagram (i nomi sono auto-esplicativi). Tutti questi metodi sono metodi privati, mentre l'unico metodo pubblico è il metodo `login`, che gestisce il flusso delle chiamate ai metodi privati.

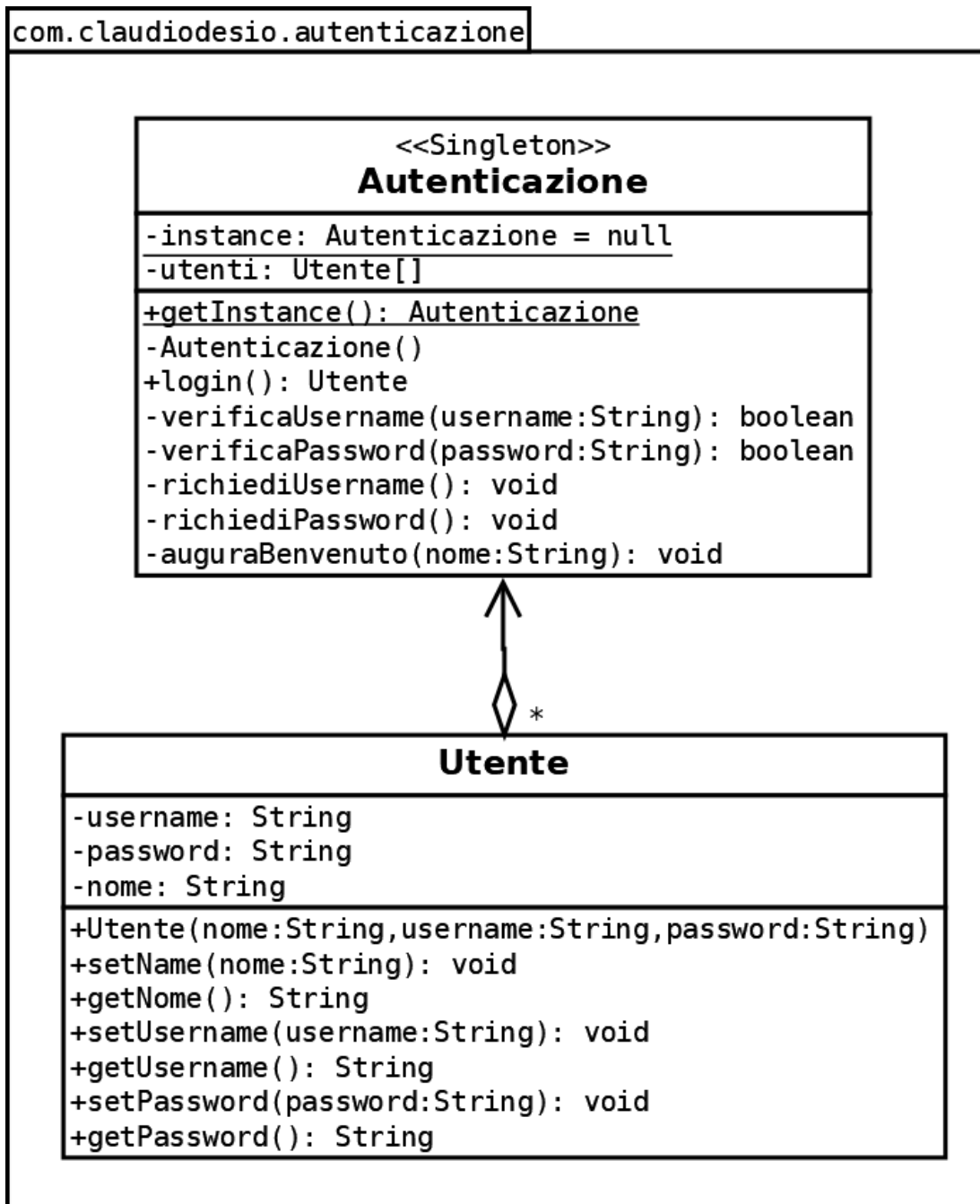


Figura 5.q.1 - Class diagram del package com.claudiodesio.autenticazione modificato come richiesto.

Nella nostra mente la classe `Autenticazione` deve funzionare così (per ora).

Soluzione 5.y)

Come è possibile verificare in figura 5.y.1, abbiamo creato una nuova classe chiamata `ProfiliUtenti`, che funge da base di dati e che contiene le informazioni sugli utenti (l'array `utenti`). Abbiamo deciso che un'istanza di questa classe sostituirà l'array `utenti` che prima risiedeva all'interno della classe `Autenticazione`, per non far perdere alla classe `Autenticazione` l'informazione sugli utenti. Adesso le classi sono astratte meglio, in quanto

ognuna ha un ruolo specifico. Ci siamo resi conto che il design pattern singleton ha più senso che sia implementato nella classe `ProfiliUtenti`, rispetto che in `Autenticazione`. Infatti sono i dati che devono essere unici per tutte le classi, non il processo di autenticazione. Quindi abbiamo agito di conseguenza.

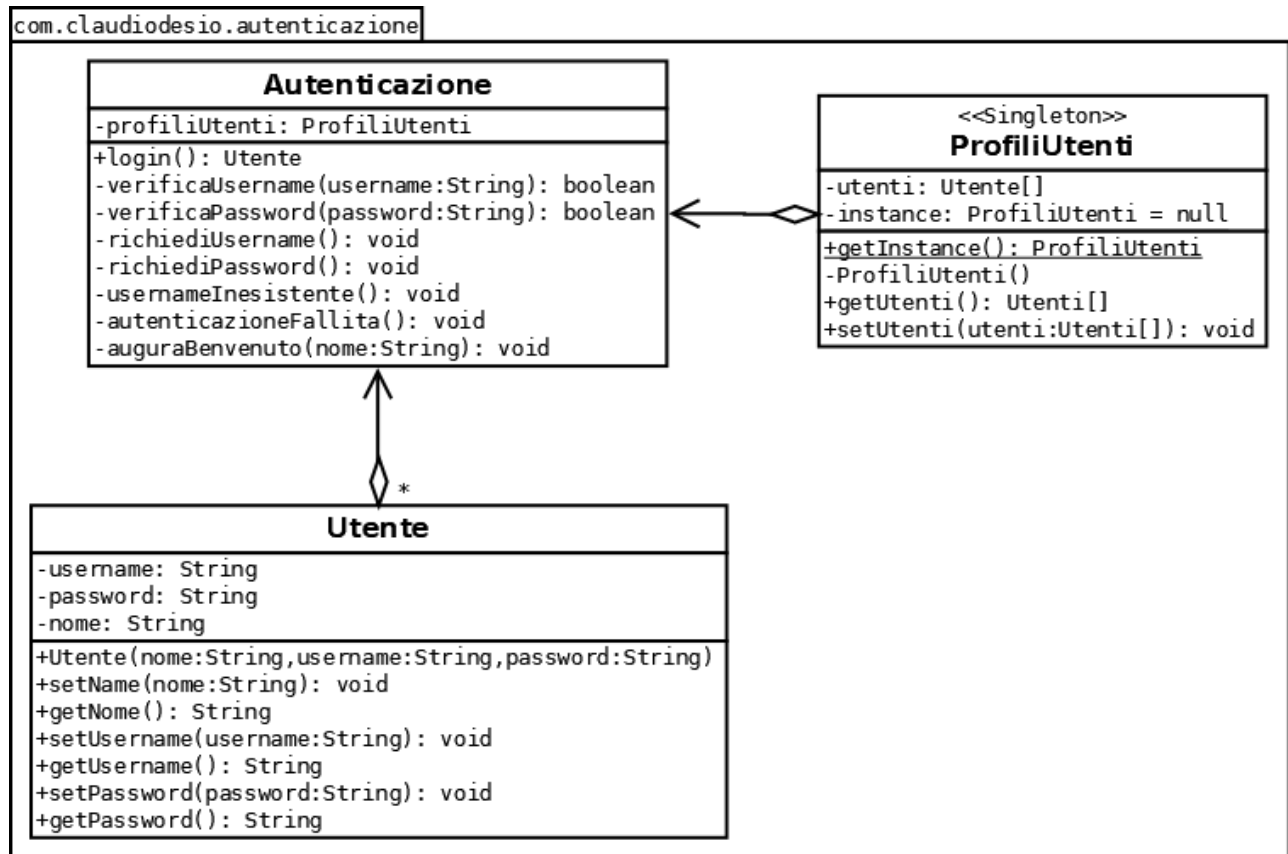


Figura 5.y.1 - Class diagram del package com.claudiodesio.autenticazione con ProfiliUtenti.

Soluzione 5.z)

Abbiamo creato una semplice classe di utilità chiamata `Stampa`, contenente tutti metodi che mandano messaggi all'utilizzatore dell'applicazione. Li abbiamo resi tutti statici, perché sembra superfluo istanziare una classe che contenga solo metodi di stampa senza che definisca variabili d'istanza.

Quest'ultima è una scelta come un'altra, non è detto che sia la migliore. Dichiarare statici i metodi implica ignorare i vantaggi dell'estensibilità che vedremo nei prossimi capitoli, ma la nostra scelta non è da condannare.

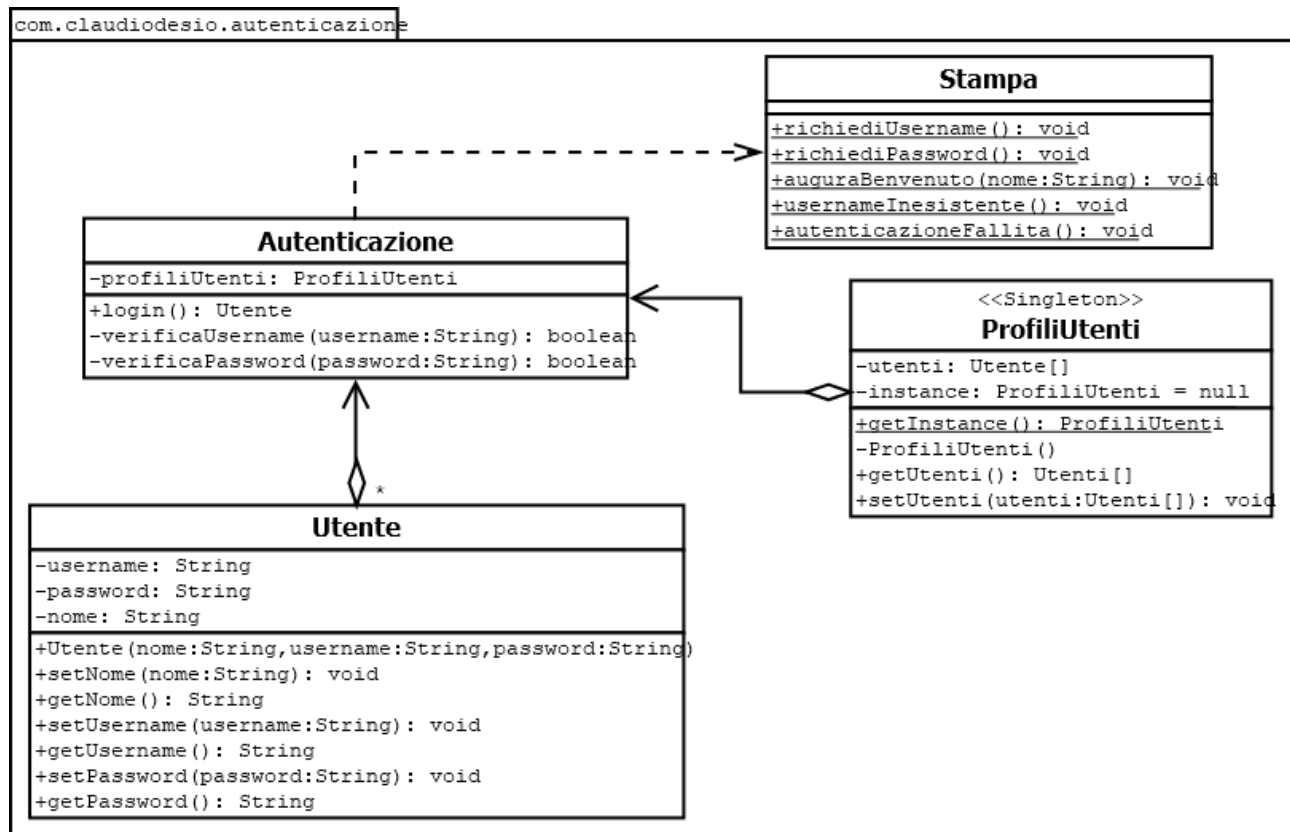


Figura 5.z.1 - Class diagram del package `com.claudiodesio.autenticazione` con **Stampa**.

Soluzione 5.aa)

La classe è astratta correttamente? Dipende dai punti di vista. La classe `Autenticazione`, ha come responsabilità la definizione del flusso della login, e la verifica della correttezza dei dati. Messa così sembra vada bene. Tuttavia si potrebbe anche pensare di delegare la verifica della correttezza dei dati ad un'altra classe d'utilità che potremmo chiamare `Verificatore`. Questa classe potrebbe contenere i due metodi di verifica dichiarati statici, oppure potrebbe contenere un costruttore a cui passiamo l'istanza di `Utente` che vogliamo verificare. Sono scelte tutte valide, ognuna delle quali ha delle conseguenze. Non creare la classe `Verificatore` implicherebbe avere una classe `Autenticazione` più corposa, ma crearla implicherebbe una classe in più (tra l'altro dipendente strettamente dalla classe `Utente`). Per ora optiamo per lasciare le cose come stanno. Decideremo più tardi quando avremo un quadro più chiaro.

Soluzione 5.bb)

Come abbiamo detto, la classe `Utente` rimane invariata:

```
package com.claudiodesio.autenticazione;

public class Utente {

    private String nome;
    private String username;
    private String password;
```

```
public Utente(String n, String u, String p) {
    this.nome = n;
    this.username = u;
    this.password = p;
}

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}

public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}
}
```

La classe `ProfiliUtenti` è stata implementata in maniera fedele rispetto a come era stata progettata:

```
package com.claudiodesio.autenticazione;

public class ProfiliUtenti {

    private static ProfiliUtenti instance;

    private Utente[] utenti;

    private ProfiliUtenti() {
        utenti = creaUtenti();
    }

    public static ProfiliUtenti getInstance() {
        if (instance == null) {
            instance = new ProfiliUtenti();
        }
        return instance;
    }

    private Utente[] creaUtenti() {
        Utente[] utenti = {
            new Utente("Daniele", "dansap", "musica"),
            new Utente("Giovanni", "giobat", "scienze"),
            new Utente("Ligeia", "ligder", "arte")
        };
        return utenti;
    }
}
```

```
}

public void setUtenti(Utente[] utenti) {
    this.utenti = utenti;
}
public void setUtenti(Utente[] utenti) {
    this.utenti = utenti;
}
public Utente[] getUtenti() {
    return utenti;
}
}
```

Anche la classe `Stampa` è fedele a come era stata progettata, a meno dell'introduzione di un metodo privato in più: `stampaMessaggio`, che centralizza l'istruzione di stampa. Questo può servirci in futuro se vorremo modificare il modo in cui stampiamo un messaggio, perché dovremo farlo solo in quel metodo e non in tutti gli altri:

```
package com.claudiodesio.autenticazione;

public class Stampa {

    public static void richiediUsername() {
        stampaMessaggio("Inserisci username.");
    }

    public static void richiediPassword() {
        stampaMessaggio("Inserisci password.");
    }

    public static void auguraBenvenuto(String nome) {
        stampaMessaggio("Benvenuto " + nome);
    }

    public static void usernameInesistente() {
        stampaMessaggio("Utente non trovato!");
    }

    public static void autenticazioneFallita() {
        stampaMessaggio("Autenticazione fallita");
    }

    private static void stampaMessaggio(String messaggio) {
        System.out.println(messaggio);
    }
}
```

Invece la classe `Autenticazione` ha subito dei cambiamenti:

```
package com.claudiodesio.autenticazione;

import java.util.Scanner;

public class Autenticazione {

    public void login() {
        boolean autorizzato = false;
        Scanner scanner = new Scanner(System.in);
        do {
            Stampa.richiediUsername();
            String username = scanner.nextLine();
            Utente utente = trovaUtente(username);
            if (utente != null) {
```

```
        Stampa.richiediPassword();
        String password = scanner.nextLine();
        if (verificaPassword(utente, password)) {
            Stampa.auguraBenvenuto(utente.getNome());
            autorizzato = true;
        } else {
            Stampa.autenticazioneFallita();
        }
    } else {
        Stampa.usernameInesistente();
    }
} while (!autorizzato);
}

private Utente trovaUtente(String username) {
    Utente[] utenti = ProfiliUtenti.getInstance().getUtenti();
    if (username != null) {
        for (Utente utente : utenti) {
            if (username.equals(utente.getUsername())) {
                return utente;
            }
        }
    }
    return null;
}

// private boolean verificaUsername(String username) {
//     Utente[] utenti = ProfiliUtenti.getInstance().getUtenti();
//     boolean trovato = false;
//     Utente utente = trovaUtente(username);
//     if (utente != null && username.equals(utente.getUsername())) {
//         trovato = true;
//     }
//     return trovato;
// }

private boolean verificaPassword(Utente utente, String password) {
    boolean trovato = false;
    if (password != null) {
        if (password.equals(utente.getPassword())) {
            trovato = true;
        }
    }
    return trovato;
}

public static void main(String args[]) {
    Autenticazione autenticazione = new Autenticazione();
    autenticazione.login();
}
}
```

In particolare, il metodo `verificaUsername`, che ritorna un booleano come avevamo progettato, è stato commentato e sostituito con il metodo `trovaUtente`, che ritorna direttamente l'oggetto `Utente` relativo allo `username` specificato come parametro. Se non viene trovato nessun utente, il metodo ritorna `null`. Questa sostituzione ci permette di non duplicare codice (sia per trovare un utente con `trovaUtente`). Infatti, per verificare lo `username` con `verificaUsername`, avremmo effettuato lo stesso ciclo, e il codice dei due metodi sarebbe stato quasi identico.

Il metodo `main` è stato introdotto solo come metodo per testare il funzionamento della funzionalità di login. Poteva esser posizionato in una classe qualsiasi, come per esempio `AutenticazioneLauncher` che avevamo inizialmente individuato nella nostra analisi tra le soluzioni degli esercizi del quinto capitolo.

Il metodo `login` ora contiene la cosiddetta *logica di business*, ovvero il codice che soddisfa i requisiti grazie ad un algoritmo opportuno. Rispetto alla soluzione dell'esercizio 5.u, si noti l'eliminazione dei costrutti `break` e `continue`, sostituiti dal più conveniente ciclo `do while`, supportato dalla variabile booleana `autorizzato`, che viene impostata a `true` solo nel momento in cui le procedure di verifica dello username e della password sono entrambe verificate. L'algoritmo risulta più chiaro e lineare, anche grazie al supporto della classe `Stampa` e dell'oggetto `Scanner`, che vengono utilizzati più volte rispettivamente per stampare messaggi in output e raccogliere input dell'utente dell'applicazione. Tuttavia a nostro parere, dei miglioramenti all'algoritmo e all'astrazione della classe si possono ancora fare. Infatti abbiamo dovuto improvvisare la nostra soluzione, perché quella progettata con il class diagram non si è rivelata meritevole di essere implementata. Cosa è mancato?

È mancato ridisegnare gli scenari con gli interaction diagram, alla luce delle nuove classi e dei nuovi metodi individuati. In questi interaction diagram, avremmo potuto specificare anche i dettagli come tipi di parametri, nomi di oggetti e tipi di ritorno. Infatti i primi sequence diagram che abbiamo creato nell'esercizio 5.t, erano basati solo sulle key abstraction, e servivano per verificare cosa fare (erano diagrammi di analisi). I diagrammi che avremmo potuto creare dopo le modifiche fatte al class diagram invece, avrebbero dovuto essere considerati diagrammi di progettazione che spiegassero come fare.

Per ora un passo in avanti grazie al class diagram lo abbiamo fatto, negli esercizi dei prossimi capitoli cercheremo di farne altri.

Soluzione 5.cc)

La soluzione migliore sembra essere la seguente:

```
public class BluRay {
    public final static int capacitaMassimaInGigaByte = 25;
    private byte[] contenuto;

    public BluRay() {
    }

    public void setContenuto(byte[] bytes) {
        this.contenuto = bytes;
    }

    public byte[] getContenuto() {
        return contenuto;
    }
}
```

Soluzione 5.dd)

Le dichiarazioni corrette sono la 3, la 4 e la 6.

Soluzione 5.ee)

L'output del programma è:

```
Static Initializer
Static Method
Initializer
Constructor
Method
```

Soluzione 5.ff)

La classe `Libro` richiesta potrebbe essere la seguente:

```
public class Libro {
    private String isbn;
    private String titolo;
    private String autore;
    private int prezzo;
    private String genere;
    public static final String [] generi = {"Romanzo", "Saggio", "Thriller",
        "Manuale"};

    public Libro(String isbn, String titolo, String autore, int prezzo,
        String genere) {
        setIsbn(isbn);
        setTitolo(titolo);
        setAutore(autore);
        setPrezzo(prezzo);
        setGenere(genere);
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }

    public String getTitolo() {
        return titolo;
    }

    public void setTitolo(String titolo) {
        this.titolo = titolo;
    }

    public String getAutore() {
        return autore;
    }

    public void setAutore(String autore) {
        this.autore = autore;
    }

    public int getPrezzo() {
        return prezzo;
    }
}
```

```
public void setPrezzo(int prezzo) {
    this.prezzo = prezzo;
}

public String getGenere() {
    return genere;
}

public void setGenere(String genere) {
    for (String generePredefinito : generi) {
        if (generePredefinito.equals(genere)) {
            this.genere = genere;
            return;
        }
    }
    System.out.println("Genere " + genere +
        " non valido! Usare uno dei seguenti generi:");
    for (String generePredefinito : generi) {
        System.out.println(generePredefinito);
    }
}
}
```

Notare che abbiamo creato un array di stringhe `generi` come costante statica per definire i generi validi predefiniti. Abbiamo utilizzato questi ultimi per testare la validità della chiamata dal metodo `setGenere`. Notare che l'utilizzo del comando `return` utilizzato all'interno del metodo, causa la terminazione immediata dello stesso. Il comando `return` non è seguito da nessun valore o variabile perché il metodo ha come tipo di ritorno `void`.

Notare che il controllo definito all'interno del metodo `setGenere`, permette di evitare il settaggio del campo `genere` nel caso non sia valido il parametro del metodo, ma l'oggetto di tipo `Libro` viene comunque creato con la variabile `genere` settata a `null`. Nel capitolo 8 vedremo come gestire questo tipo di situazione lanciando un'eccezione, evitando così di istanziare un oggetto `Libro` senza settare il campo `genere`.

Con la seguente classe abbiamo testato la classe `Libro`:

```
public class TestLibro {
    public static void main(String[] args) {
        Libro jfaVol1 = new Libro("979-12-200-4915-3", "Java for Aliens Vol. 1",
            "Claudio De Sio Cesari", 25, "Manuale");
        Libro jfaVol2 = new Libro("979-12-200-4916-0", "Java for Aliens Vol. 2",
            "Claudio De Sio Cesari", 25, "Biografia");
        System.out.println("Genere JFA Vol 1 = " + jfaVol1.getGenere());
        System.out.println("Genere JFA Vol 2 = " + jfaVol2.getGenere());
    }
}
```

Segue l'output della precedente classe:

```
Genere Biografia non valido! Usare uno dei seguenti generi:
Romanzo
Saggio
Thriller
Manuale
Genere JFA Vol 1 = Manuale
Genere JFA Vol 2 = null
```


Soluzione 5.gg)

Prima di definire la classe `Scaffale` abbiamo ritenuto necessario ridefinire la classe `Libro` nel seguente modo:

```
public class Libro {
    private String isbn;
    private String titolo;
    private String autore;
    private int prezzo;
    private String genere;

    public Libro(String isbn, String titolo, String autore, int prezzo,
        String genere) {
        setIsbn(isbn);
        setTitolo(titolo);
        setAutore-autore);
        setPrezzo(prezzo);
        setGenere(genere);
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }

    public String getTitolo() {
        return titolo;
    }

    public void setTitolo(String titolo) {
        this.titolo = titolo;
    }

    public String getAutore() {
        return autore;
    }

    public void setAutore(String autore) {
        this.autore = autore;
    }

    public int getPrezzo() {
        return prezzo;
    }

    public void setPrezzo(int prezzo) {
        this.prezzo = prezzo;
    }

    public String getGenere() {
        return genere;
    }

    public void setGenere(String genere) {
```

```
        if (GenereUtils.isGenereValido(genere)) {
            this.genere = genere;
        } else {
            GenereUtils.stampaErrore(genere);
        }
    }
}
```

Si può notare che abbiamo semplificato l'implementazione, eliminando l'array `generi`, visto che siccome dovremo utilizzare il concetto di genere anche per la classe `Scaffale`, abbiamo preferito creare anche una classe d'utilità che abbiamo chiamata `GenereUtils`, che abbiamo utilizzato anche per semplificare l'implementazione dei controlli del metodo `setGenere`. Riportiamo la classe `GenereUtils` di seguito:

```
public class GenereUtils {
    public static final String ROMANZO = "Romanzo";
    public static final String SAGGIO = "Saggio";
    public static final String THRILLER = "Thriller";
    public static final String MANUALE = "Manuale";
    public static final String FANTASCIENZA = "Fantascienza";
    public static final String[] generi =
        { ROMANZO, SAGGIO, THRILLER, MANUALE, FANTASCIENZA };

    public static boolean isGenereValido(String genere) {
        boolean genereValido = false;
        for (String generePredefinito : generi) {
            if (generePredefinito.equals(genere)) {
                genereValido = true;
                break;
            }
        }
        return genereValido;
    }

    public static void stampaErrore(String genere) {
        System.out.println("Genere + " + genere + " non valido! " +
            "Usare uno dei seguenti generi:");
        for (String generePredefinito : generi) {
            System.out.println(generePredefinito);
        }
    }
}
```

Nella classe `GenereUtils` abbiamo riportato i generi sotto forma di costanti statiche e pubbliche, che se usate al posto delle stringhe permettono di evitare che errori di battitura diventino bug. Ne abbiamo approfittato per aggiungere un nuovo genere `FANTASCIENZA`. Inoltre per comodità le abbiamo anche utilizzate per riempire l'array `generi`, in modo tale da usare i cicli per iterare sugli elementi dell'array. Infine abbiamo dichiarato due metodi statici e pubblici. Il metodo `isGenereValido`, ritorna `true` solo se il genere è valido. Il metodo `stampaErrore` stampa un messaggio di errore. Entrambi questi metodi sono stati utilizzati nel metodo `setGenere` della classe `Libro`.

La classe `Scaffale` quindi, l'abbiamo definita nel seguente modo:

```
public class Scaffale {
    private Libro[] libri;
    private String genere;

    public Scaffale(String genere) {
        libri = new Libro[100];
        setGenere(genere);
    }
}
```

```
}

public void aggiungiLibro(Libro libro) {
    if (genere == null) {
        System.out.println("Il genere di questo scaffale non è stato "
            + "ancora settato e quindi non possono essere aggiunti libri!");
        GenereUtils.stampaErrore(null);
        return;
    }
    for (int i = 0; i < libri.length; i++) {
        if (libri[i] == null) {
            libri[i] = libro;
            return;
        }
    }
    System.out.println("Scaffale pieno!");
}

public Libro[] getLibri() {
    return libri;
}

public void setGenere(String genere) {
    if (GenereUtils.isGenereValido(genere)) {
        this.genere = genere;
    } else {
        GenereUtils.stampaErrore(genere);
    }
}

public String getGenere() {
    return genere;
}
}
```

Notare che il metodo `setGenere`, è praticamente lo stesso metodo definito per la classe `Libro`, potendo sfruttare i metodi di `GenereUtils`. Il metodo `aggiungiLibro` invece, in primo luogo controlla se l'oggetto `Scaffale` corrente ha la variabile `genere` valorizzata. In caso negativo stampa un messaggio di errore e non esegue il resto del metodo. Poi invece cerca il primo posto libero nell'array `generi`, se non lo trova, allora stampa un messaggio di errore per avvertire che non c'è posto sullo scaffale per altri libri.

La classe singleton `Libreria` infine, potrebbe essere la seguente:

```
public class Libreria {
    private String nome;
    private Scaffale[] scaffali;
    private static Libreria instance;

    private Libreria() {
        scaffali = new Scaffale[GenereUtils.generi.length];
    }

    public static Libreria getInstance(){
        if (instance == null) {
            instance = new Libreria();
        }
        return instance;
    }

    public void setNome(String nome) {
```

```
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }

    public void aggiungiScaffale(Scaffale scaffale) {
        if (scaffali[scaffali.length-1] != null) {
            System.out.println("Questa libreria ha già tutti gli scaffali!");
            return;
        }
        for (int i = 0; i < scaffali.length; i++) {
            if (scaffali[i] == null) {
                scaffali[i] = scaffale;
                break;
            } else if (scaffali[i].getGenere().equals(scaffale.getGenere())) {
                System.out.println("Scaffale genere " + scaffale.getGenere()
                    + " già esistente!");
                break;
            }
        }
    }

    public Scaffale[] getScaffali() {
        return scaffali;
    }
}
```

Possiamo notare che all'interno di questa classe abbiamo utilizzato sempre la variabile `length` degli array coinvolti, per non essere soggetti a modifiche nel caso di variazioni del numero di generi utilizzati nel programma.

Il metodo `aggiungiScaffale` invece, implementa un algoritmo che prima controlla se tutti gli scaffali sono già stati settati. Poi all'interno di un ciclo che itera sull'array `scaffali`, cerca il primo posto libero nell'array. Per ogni scaffale già posizionato all'interno dell'array, il metodo controlla se esso ha un genere coincidente con quello dello scaffale che è stato passato come parametro (in caso affermativo, stampa un messaggio di errore e termina il metodo con un comando `return`).

Possiamo testare il tutto con la seguente classe:

```
public class TestLibreria {
    public static void main(String[] args) {
        Libro jfaVol1 = new Libro("979-12-200-4915-3", "Java for Aliens Vol. 1",
            "Claudio De Sio Cesari", 25, GenereUtils.MANUALE);
        Libro jfaVol2 = new Libro("979-12-200-4916-0", "Java for Aliens Vol. 2",
            "Claudio De Sio Cesari", 25, GenereUtils.MANUALE);
        Libro f451 = new Libro("978-88-046-6529-8", "Fahrenheit 451",
            "Ray Bradbury", 10, GenereUtils.FANTASCIENZA);
        Libro shining = new Libro("978-88-452-9530-0", "Shining",
            "Stephen King", 12, GenereUtils.THRILLER);
        Libro icda = new Libro("978-88-683-6730-5", "Il cacciatore di aquiloni",
            "Khaled Hosseini", 11, GenereUtils.ROMANZO);
        Libro ltdt = new Libro("978-88-170-7976-1", "La teoria del tutto",
            "Stephen Hawking", 10, GenereUtils.SAGGIO);
        Scaffale scaffaleManuali = new Scaffale(GenereUtils.MANUALE);
        Scaffale scaffaleFantascienza = new Scaffale(GenereUtils.FANTASCIENZA);
        Scaffale scaffaleFantascienza2 = new Scaffale(GenereUtils.FANTASCIENZA);
        Scaffale scaffaleThriller = new Scaffale(GenereUtils.THRILLER);
        Scaffale scaffaleRomanzi = new Scaffale(GenereUtils.ROMANZO);
        Scaffale scaffaleSaggi = new Scaffale(GenereUtils.SAGGIO);
        Scaffale scaffaleSaggi2 = new Scaffale(GenereUtils.SAGGIO);
    }
}
```

```
scaffaleManuali.aggiungiLibro(jfaVol1);
scaffaleManuali.aggiungiLibro(jfaVol2);
scaffaleFantascienza.aggiungiLibro(f451);
scaffaleThriller.aggiungiLibro(shining);
scaffaleRomanzi.aggiungiLibro(icda);
scaffaleSaggi.aggiungiLibro(ltdt);
Libreria libreria = Libreria.getInstance("Libreria per alieni");
libreria.aggiungiScaffale(scaffaleManuali);
libreria.aggiungiScaffale(scaffaleFantascienza);
libreria.aggiungiScaffale(scaffaleFantascienza2);
libreria.aggiungiScaffale(scaffaleThriller);
libreria.aggiungiScaffale(scaffaleRomanzi);
libreria.aggiungiScaffale(scaffaleSaggi);
libreria.aggiungiScaffale(scaffaleSaggi2);
Scaffale[] scaffali = libreria.getScaffali();
System.out.println("Elenco scaffali della libreria:");
for (Scaffale scaffale : scaffali) {
    System.out.println("Scaffale " + scaffale.getGenere() + ":");
    Libro[] libri = scaffale.getLibri();
    for (Libro libro : libri) {
        if (libro != null) {
            System.out.println("\t" + libro.getTitolo() + " di " +
                libro.getAutore() + " (Genere " + libro.getGenere() + ")");
        }
    }
}
}
```

il cui output sarà:

```
Scaffale genere Fantascienza già esistente!
Questa libreria ha già tutti gli scaffali!
Elenco scaffali della libreria:
Scaffale Manuale:
    Java for Aliens Vol. 1 di Claudio De Sio Cesari (Genere Manuale)
    Java for Aliens Vol. 2 di Claudio De Sio Cesari (Genere Manuale)
Scaffale Fantascienza:
    Fahrenheit 451 di Ray Bradbury (Genere Fantascienza)
Scaffale Thriller:
    Shining di Stephen King (Genere Thriller)
Scaffale Romanzo:
    Il cacciatore di aquiloni di Khaled Hosseini (Genere Romanzo)
Scaffale Saggio:
    La teoria del tutto di Stephen Hawking (Genere Saggio)
```

Soluzione 5.hh)

Le uniche risposte false sono la numero 2 e la numero 4. Infatti, per quanto riguarda l'affermazione numero 2, l'unico rettangolo che fa parte della sintassi del diagramma dei casi d'uso, è il system boundary, che circonda solitamente i casi d'uso e delimita il sistema. Per quanto riguarda l'affermazione numero 4, in realtà il concetto di attore, va interpretato come "ruolo". Un utente del sistema potrebbe essere anche un altro sistema.

Soluzione 5.ii)

Solo le ultime due affermazioni sono false. In particolare la definizione riportata nell'affermazione numero 5 è quella del diagramma dei casi d'uso, non del deployment diagram. Invece la definizione riportata nell'affermazione numero 6 è quella di astrazioni chiave (key abstractions), non di classi candidate.

Soluzione 5.jj)

Solo l'affermazione 2 è errata, in quanto riporta la definizione di linea di vita di un oggetto, e non di linea di attività.

Soluzioni degli esercizi del capitolo 6

Ereditarietà ed interfacce

Soluzione 6.a) *Object Orientation in Java (pratica), Vero o Falso:*

Falso, il processo di generalizzazione implica scrivere una classe in più e ciò non sempre implica scrivere qualche riga in meno.

Falso.

Falso, anche se dal punto di vista della programmazione la generalizzazione può non farci sempre risparmiare codice, essa ha comunque il pregio di farci gestire le classi in maniera più naturale, favorendo l'astrazione dei dati. Inoltre apre la strada all'implementazione del polimorfismo.

Vero.

Falso, l'ereditarietà multipla esiste nella realtà, e in Java esiste in una versione soft perché si eredita solo la parte funzionale.

Falso, è un'interfaccia che dichiara un unico metodo astratto.

Vero.

Falso, una squadra non "è un" giocatore, né un giocatore "è una" squadra. Semmai una squadra "ha un" giocatore ma questa non è la relazione di ereditarietà. Si tratta infatti della relazione di associazione.

Vero, infatti entrambe le classi potrebbero estendere una classe `Partecipante`.

Falso, un `Padre` è sempre un `Figlio`, o entrambe potrebbero estendere la classe `Persona`.

Soluzione 6.b)

```
public class Impiegato extends Persona { //Ereditarietà
    private int matricola;

    public void setDati(String nome, int matricola) {
        setNome(nome); //Riuso ed ereditarietà
        setMatricola(matricola); //Riuso
    }

    public void setMatricola(int matricola) {
        this.matricola = matricola; //incapsulamento
    }

    public int getMatricola() {
        return matricola; //incapsulamento
    }

    public String dammiDettagli() {
        //Riuso, incapsulamento ed ereditarietà
        return getNome() + ", matricola: " + getMatricola();
    }
}
```

Soluzione 6.c) Classi astratte ed interfacce, Vero o Falso:

Vero, i modificatori `abstract` e `final` sono in contraddizione.

Vero, manca il blocco di codice che definisce la classe.

Vero, un'interfaccia `final` non ha senso.

Falso.

Vero.

Vero.

Vero.

Falso, un'interfaccia non può dichiarare costruttori perché non si può istanziare.

Vero.

Vero.

Soluzione 6.d)

`Persona` potrebbe essere superclasse di `Professore` e `Studiante`, per tutto il resto non c'è ereditarietà.

L'ereditarietà si testa sempre e comunque con la relazione “is a”. Quindi risulta molto semplice verificare che per tutte le altre coppie di classi non è confermata questa relazione.

Soluzione 6.e)

Indubbiamente la classe `Persona` potrebbe essere una classe astratta, ma anche `Professore` e `Studiante` potrebbero essere dichiarate astratte nel caso si volessero a loro volta estendere con classi come `StudianteIngegneria` o `ProfessoreMatematica`.

Soluzione 6.f)

Premettendo che tutte le scelte object oriented sono soggettive, probabilmente l'implementazione astratta è la scelta più corretta per un concetto così astratto:

```
/**
 * Astrae il concetto di oggetto musicale.
 *
 * @author Claudio De Sio Cesari
 */
public interface Musicale {
    /**
     * Esegue la musica dell'oggetto musicale corrente.
     */
    void suona();
}
```


Soluzione 6.g)

Il listato dell'interfaccia `StrumentoMusicale` potrebbe essere il seguente:

```
/**
 * Astrae il concetto di strumento musicale.
 *
 * @author Claudio De Sio Cesari
 */
public interface StrumentoMusicale extends Musicale {
}
```

Il listato dell'interfaccia `Suoneria` potrebbe essere il seguente:

```
/**
 * Astrae il concetto di suoneria musicale.
 *
 * @author Claudio De Sio Cesari
 */
public interface Suoneria extends Musicale {
}
```

Il metodo `suona` per noi continua ad essere astratto.

Soluzione 6.h)

Mentre potrebbe essere plausibile considerare uno strumento musicale uno smartphone che usufruisce di una determinata app, è semplicemente scorretto che possa essere considerato una suoneria, infatti il test “is a” fallisce:

Domanda: “uno smartphone è uno strumento musicale?”

Risposta: Sì (a patto che sia installata una app che lo renda tale)

Domanda: “uno smartphone è una suoneria?”

Risposta: No (semmai contiene suonerie)

Soluzione 6.i) Interfacce dopo Java 8, Vero o Falso:

Vero.

Vero, il modificatore `static` non si può applicare a classi ed interfacce.

Vero, `class` è una parola chiave e non può essere utilizzata come identificatore.

Falso.

Falso.

Vero, in particolare è possibile riscrivere un metodo con la stessa firma in una sottointerfaccia (ma lo stesso concetto si applica alle classi), ma tecnicamente non si tratta di un override perché i metodi statici semplicemente non vengono ereditati. Infatti un eventuale uso nella sottointerfaccia dell'annotazione `@Override` per marcare il nuovo metodo statico provocherà un errore in compilazione (cfr. listati `SuperInterfaccia.java` e `SottoInterfaccia.java`).

Vero, la regola 5 caratterizzata dalla massima “class always win”, continua a valere anche se la classe interessata è astratta (cfr. listati `A.java`, `B.java`, `C.java`, `D.java`, `TestABCD.java`).

Falso, potrebbe trarre in inganno il fatto che la classe astratta `G` anche ereditando il metodo astratto dall'interfaccia `E` faccia valere i motivi della regola “class always win”. Invece in questo caso, vale la regola 4, ovvero vince l'implementazione più specifica. Infatti la classe `G`, eredita

un metodo dell'interfaccia *E*, che è meno specifico di quello ridefinito nell'interfaccia *F* (cfr. listati *E.java*, *F.java*, *G.java*, *H.java*, *TestEFGH.java*).

Vero, anche se potrebbe sorgere qualche dubbio, un'interfaccia può estendere più interfacce (cfr. listato *MiaInterfaccia.java*).

Falso, un'interfaccia non può estendere una classe in qualsiasi caso.

Soluzione 6.j)

L'astrazione della classe *Album* che abbiamo realizzato, ricorda molto l'astrazione della classe

Libro:

```
public class Album {
    private String ismn;
    private String titolo;
    private String artista;
    private String genere;
    private String prezzo;

    public Album(String ismn, String titolo, String autore, String genere,
        String prezzo) {
        setIsmn(ismn);
        setTitolo(titolo);
        setArtista(autore);
        setGenere(genere);
        setPrezzo(prezzo);
    }

    public String getIsmn() {
        return ismn;
    }

    public void setIsmn(String ismn) {
        this.ismn = ismn;
    }

    public String getTitolo() {
        return titolo;
    }

    public void setTitolo(String titolo) {
        this.titolo = titolo;
    }

    public String getArtista() {
        return artista;
    }

    public void setArtista(String autore) {
        this.artista = autore;
    }

    public String getGenere() {
        return genere;
    }

    public void setGenere(String genere) {
        this.genere = genere;
    }
}
```

```
    }

    public String getPrezzo() {
        return prezzo;
    }

    public void setPrezzo(String prezzo) {
        this.prezzo = prezzo;
    }
}
```

Notiamo che ci sono chiaramente tre campi in comune con la classe `Libro`. Per le variabili `prezzo` e `titolo` non ci sono dubbi, mentre per la variabile `genere` qualcuno potrebbe averne perché i generi musicali non corrispondono a quelli letterari. In realtà, avendo gestito i possibili valori dei generi letterari all'interno della classe di utilità `GenereUtils`, la variabile `genere` non dipende direttamente dai valori che può assumere. Inoltre notiamo che le variabili `isbn` e `ismn`, rappresentano sostanzialmente lo stesso concetto: un numero che identifica univocamente un oggetto. Ma anche le variabili `autore` in `Libro` e `artista` in `Album`, si assomigliano molto se ci caliamo nel contesto della vendita di libri e di album musicali. Benché ci siano delle differenze di significato delle due parole, un acquirente di un libro, solitamente lo identifica anche mediante il suo autore. Per esempio il libro “Shining” è di Stephen King. Allo stesso modo un acquirente di un album musicale, potrebbe identificare un album anche specificando il relativo l'artista, per esempio potremmo dire che l'album “The Wall” è dei Pink Floyd. Questo è vero perché ci troviamo nell'ambito della vendita di album e di libri, che per noi sono semplicemente “articoli”. Quindi abbiamo deciso di creare la superclasse astratta `Articolo` delle classi `Libro` e `Album` nel seguente modo:

```
public abstract class Articolo {
    private String id;
    private String titolo;
    private String nome;
    private int prezzo;
    private String genere;

    public Articolo(String id, String titolo, String nome, int prezzo,
        String genere) {
        super();
        setId(id);
        setTitolo(titolo);
        setNome(nome);
        setPrezzo(prezzo);
        setGenere(genere);
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getTitolo() {
        return titolo;
    }

    public void setTitolo(String titolo) {
        this.titolo = titolo;
    }
}
```

```
}

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public int getPrezzo() {
    return prezzo;
}

public void setPrezzo(int prezzo) {
    this.prezzo = prezzo;
}

public String getGenere() {
    return genere;
}

public void setGenere(String genere) {
    this.genere = genere;
}
}
```

Le classi `Libro` e `Album`, possono quindi essere semplificati nella seguente maniera:

```
public class Libro extends Articolo {
    public Libro(String isbn, String titolo, String autore, int prezzo,
        String genere) {
        super(isbn, titolo, autore, prezzo, genere);
    }
}

public class Album extends Articolo {
    public Album(String ismn, String titolo, String artista, int prezzo,
        String genere) {
        super(ismn, titolo, artista, prezzo, genere);
    }
}
```

in pratica abbiamo “normalizzato” i campi `isbn` della classe `Libro` e `ismn` della classe `Album`, in un campo `id` (abbreviativo di “numero identificativo”) nella superclasse `Articolo`. Stesso discorso per i campi `autore` della classe `Libro` e `artista` della classe `Album`, “normalizzati” in un campo `nome` nella superclasse `Articolo`.

Soluzione 6.k)

Con una soluzione non semplice, abbiamo implementato l’ereditarietà tra le classi di utilità richieste. In particolare abbiamo fatto estendere alle classi `GenereMusicaleUtils` e `GenereLetterarioUtils` una nuova classe generica per la quale abbiamo riutilizzato il nome `GenereUtils`. In quest’ultima abbiamo implementato i metodi statici `isGenereValido` e `stampaErrore` aggiungendo come secondo parametro l’array di generi su cui basare il controllo richiesto. Questo è necessario, perché non sappiamo a priori se in questa classe generica useremo

generi musicali o letterari, ma contemporaneamente non vogliamo duplicare codice nelle sottoclassi:

```
public class GenereUtils {

    public static boolean isGenereValido(String genere, String[] generiValidi) {
        boolean genereValido = false;
        for (String generePredefinito : generiValidi) {
            if (generePredefinito.equals(genere)) {
                genereValido = true;
            }
        }
        return genereValido;
    }

    public static void stampaErrore(String genere, String[] generiValidi) {
        System.out.println("Genere " + genere +
            " non valido! Usare uno dei seguenti generi:");
        for (String generePredefinito : generiValidi) {
            System.out.println(generePredefinito);
        }
    }
}
```

Quindi abbiamo definito la classe `GenereLetterarioUtils` nella seguente maniera:

```
public class GenereLetterarioUtils extends GenereUtils {
    public static final String ROMANZO = "Romanzo";
    public static final String SAGGIO = "Saggio";
    public static final String THRILLER = "Thriller";
    public static final String MANUALE = "Manuale";
    public static final String FANTASCIENZA = "Fantascienza";
    public static final String[] generi = { ROMANZO, SAGGIO, THRILLER, MANUALE,
        FANTASCIENZA };

    public static boolean isGenereValido(String genere) {
        return isGenereValido(genere, generi);
    }

    public static void stampaErrore(String genere) {
        stampaErrore(genere, generi);
    }
}
```

E la classe `GenereMusicaleUtils` in questo modo:

```
public class GenereMusicaleUtils extends GenereUtils {
    public static final String ROCK = "Rock";
    public static final String JAZZ = "Jazz";
    public static final String BLUES = "Blues";
    public static final String POP = "Pop";
    public static final String RAP = "Rap";
    public static final String[] generi = { ROCK, JAZZ, BLUES, POP, RAP };

    public static boolean isGenereValido(String genere) {
        return isGenereValido(genere, generi);
    }

    public static void stampaErrore(String genere) {
        stampaErrore(genere, generi);
    }
}
```

In queste due classi abbiamo fatto in modo che i metodi `isGenereValido` e `stampaErrore` invochino i metodi definiti nella superclasse. Inoltre abbiamo definito le informazioni sui generi.

Le difficoltà tecniche nell'implementare questa ereditarietà, dipendono soprattutto dal fatto che abbiamo a che fare con classi che definiscono metodi astratti, e che quindi non si adattano bene al discorso dell'estensione.

Nella classe `Libro`, abbiamo quindi ridefinito il metodo `setGenere` come si può vedere di seguito (in grassetto):

```
public class Libro extends Articolo {

    public Libro(String isbn, String titolo, String autore, int prezzo, String
        genere) {
        super(isbn, titolo, autore, prezzo, genere);
    }

    public void setGenere(String genere) {
        if (GenereLetterarioUtils.isGenereValido(genere)) {
            super.setGenere(genere);
        } else {
            GenereLetterarioUtils.stampaErrore(genere);
        }
    }
}
```

Notare che siamo stati costretti ad invocare il metodo `setGenere` della superclasse `Articolo` utilizzando il reference `super`, visto che avevamo la necessità di chiamare un metodo che aveva lo stesso nome del metodo che stavamo ridefinendo.

Lo stesso discorso per la classe `Album`:

```
public class Album extends Articolo {
    public Album(String ismn, String titolo, String artista, int prezzo, String
        genere) {
        super(ismn, titolo, artista, prezzo, genere);
    }

    public void setGenere(String genere) {
        if (GenereMusicaleUtils.isGenereValido(genere)) {
            super.setGenere(genere);
        } else {
            GenereMusicaleUtils.stampaErrore(genere);
        }
    }
}
```

infine abbiamo implementato la classe di test `TestArticoli` nella seguente maniera:

```
public class TestArticoli {
    public static void main(String[] args) {
        Libro jfaVol1 = new Libro("979-12-200-4915-3", "Java for Aliens Vol. 1",
            "Claudio De Sio Cesari", 25, GenereLetterarioUtils.MANUALE);
        Libro jfaVol2 = new Libro("979-12-200-4916-0", "Java for Aliens Vol. 2",
            "Claudio De Sio Cesari", 25, "NonEsistente");
        System.out.println("Genere JFA Vol 1 = " + jfaVol1.getGenere());
        System.out.println("Genere JFA Vol 2 = " + jfaVol2.getGenere());
        Album lad = new Album("979-0-236-44-3", "Live after Death",
            "Iron Maiden", 25, GenereMusicaleUtils.ROCK);
        Album mop = new Album("978-0-789-01-2", "Master of Puppets",
            "Metallica", 25, "NonEsistente");
        System.out.println("Genere Live after Death = " + lad.getGenere());
    }
}
```

```
        System.out.println("Genere Master of Puppets = " + mop.getGenere());  
    }  
}
```

L'output finale è il seguente:

```
Genere NonEsistente non valido! Usare uno dei seguenti generi:  
Romanzo  
Saggio  
Thriller  
Manuale  
Fantascienza  
Genere JFA Vol 1 = Manuale  
Genere JFA Vol 2 = null  
Genere NonEsistente non valido! Usare uno dei seguenti generi:  
Rock  
Jazz  
Blues  
Pop  
Rap  
Genere Live after Death = Rock  
Genere Master of Puppets = null
```

Soluzione 6.l)

Il compilatore in realtà trasformerà la classe in qualcosa di molto simile alla seguente (in grassetto gli inserimenti impliciti del compilatore):

```
import java.lang.*;  
  
public class CompilatorePensaciTu extends Object {  
  
    private int var;  
  
    public CompilatorePensaciTu() {  
    }  
  
    public void setVar(int v) {  
        this.var = v;  
    }  
  
    public int getVar() {  
        return this.var;  
    }  
}
```

Teniamo anche conto che, estendendo `Object`, questa classe eredita anche tutti i suoi metodi.

Soluzione 6.m)

La risposta esatta è la prima. Nessun errore da segnalare.

Soluzione 6.n)

La risposta esatta è la prima. Nessun errore da segnalare.

Soluzione 6.o)

Il concetto di aggregazione, utilizzato più volte negli esercizi precedenti, è una relazione che indica contenimento, che semmai può essere considerata un'alternativa all'estensione. Quindi l'unica risposta corretta è la terza.

Soluzione 6.p)

La quarta e la quinta opzione sono entrambe corrette per il compilatore. La quinta però, ha meno senso dal punto di vista della logica (un animale è un leone? Non per forza).

Soluzione 6.q)

Anche se un'interfaccia può potenzialmente definire metodi di diversi tipi, non può dichiarare variabili d'istanza, ma solo costanti statiche pubbliche.

Soluzione 6.r)

Gli statement corretti sono i numeri 1 e 3.

Il metodo <code>equals</code> è ereditato direttamente dalla classe <code>Object</code>.

Soluzione 6.s)

Solo la prima risposta è corretta. Essendo la prima corretta, ovviamente la seconda e la terza non possono essere corrette. È possibile dichiarare metodi statici e di default, ma i relativi modificatori non sono mai aggiunti automaticamente. Il dubbio può venire per la risposta 6, perché prima dell'avvento di Java 8 questa risposta sarebbe stata giusta, ora non più visto che possiamo dichiarare nelle interfacce anche metodi di default e statici. Infine, il modificatore `final` è implicitamente aggiunto agli attributi delle interfacce (che sono anche implicitamente dichiarati statici e pubblici).

Soluzione 6.t)

Le risposte sono tutte false tranne la 3.

Soluzione 6.u)

Le risposte corrette sono la 1, la 2 e la 5.

Soluzione 6.v)

Potremmo seguire il seguente ragionamento. Un tennista si allena e corre diversamente da un calciatore o da un corridore. Insomma, sia la classe `Tennista`, sia la classe `Corridore` che la classe `Calciatore`, ridefiniranno i metodi `allenati` e `corri`. In `Atleta`, questi due metodi invece li vorremmo definire astratti, perché, a priori, sarebbe difficile definire come si allena o come corre un atleta... dipende dal tipo di atleta. A questo punto si potrebbe pensare di definire `Atleta` come interfaccia visto che utilizza solo due metodi astratti, e lo si può fare. Ma con l'evoluzione di questo programma, è altamente probabile che si vogliano definire campi di anagrafica degli atleti come potrebbero essere `nome` e `cognome`. Un'interfaccia però non può dichiarare variabili, e quindi si potrebbe preferire dichiarare `Atleta` come classe astratta. Oppure potremmo creare una soluzione dove `Tennista`, `Calciatore` e `Corridore` implementino l'interfaccia `Atleta` (che definisce i due metodi astratti `allenati` e `corri`) ed estendono la classe astratta `Persona` (che definisce gli attributi `nome`, `cognome` etc.). Insomma sarà il contesto del programma in cui ci caleremo che ci porterà a definire la soluzione più corretta.

Soluzione 6.w)

Le affermazioni corrette sono la 2, 3, 4, 5. La numero 1 è errata perché il modificatore `protected` non può essere utilizzato con le classi (non esistono classi `protected`). La 6 è falsa perché i costruttori non si ereditano (neanche se sono pubblici). La numero 7 è errata per le ragioni esposte nel paragrafo 6.2.4.1.

Soluzione 6.x)

La classe `Contatto` incapsulata potrebbe essere la seguente (in grassetto le modifiche):

```
public class Contatto {  
    protected static final String SCONOSCIUTO = "sconosciuto";  
    private String nome;  
    private String numeroDiTelefono;  
    private String indirizzo;  
  
    public Contatto(String nome, String numeroDiTelefono) {  
        this.setName(nome) ;  
        this.setNumeroDiTelefono(numeroDiTelefono) ;  
        this.setIndirizzo(SCONOSCIUTO) ;  
    }  
  
    public Contatto(String nome, String numeroDiTelefono, String indirizzo) {  
        this.setName(nome) ;  
        this.setNumeroDiTelefono(numeroDiTelefono) ;  
        this.setIndirizzo(indirizzo) ;  
    }  
  
    public void setIndirizzo(String indirizzo) {
```

```
        this.indirizzo = indirizzo;
    }

    public String getIndirizzo() {
        return indirizzo;
    }

    public void setNumeroDiTelefono(String numeroDiTelefono) {
        this.numeroDiTelefono = numeroDiTelefono;
    }

    public String getNumeroDiTelefono() {
        return numeroDiTelefono;
    }

    public void setName(String nome) {
        this.nome = nome;
    }

    public String getName() {
        return nome;
    }

    public void stampaDettagli() {
        System.out.println(nome);
        System.out.println(indirizzo);
        System.out.println(numeroDiTelefono);
        System.out.println();
    }
}
```

Notare che abbiamo introdotto una costante statica privata `SCONOSCIUTO` per rappresentare un indirizzo sconosciuto quando si utilizza il primo costruttore dichiarato (nelle soluzioni dell'esercizio 4.z, il valore dell'indirizzo non veniva proprio settato, e di conseguenza valeva `null`).

La classe `Rubrica` incapsulata e resa Singleton potrebbe essere implementata nel modo seguente (in grassetto le modifiche):

```
public class Rubrica {
    private static Rubrica instance;
    public Contatto[] contatti;

    private Rubrica () {
        contatti = new Contatto[] {
            new Contatto("Claudio De Sio Cesari", " Via Java 13", "131313131313"),
            new Contatto("Stevie Wonder", "Via Musica 10", "1010101010"),
            new Contatto("Gennaro Capuozzo",
                "Piazza Quattro Giornate 1", "1111111111")
        };
    }

    public static Rubrica getInstance() {
        if (instance == null) {
            instance = new Rubrica();
        }
        return instance;
    }

    public Contatto[] cercaContattiPerNome(String nome) {
        Contatto []contattiTrovati = new Contatto[contatti.length];
        for (int i = 0, j = 0; i < contattiTrovati.length; i++) {
            if (contatti[i].getName().toUpperCase().

```

```

        contains(nome.toUpperCase())) {
            contattiTrovati[j] = contatti[i];
            j++;
        }
    }
    return contattiTrovati;
}

public Contatto[] getContatti() {
    return contatti;
}
}

```

Notare che abbiamo evitato di creare il metodo setter per la variabile `contatti`.

Abbiamo poi sostituito all'interno della classe `RicercaContatti`, solo la riga:

```
var rubrica = new Rubrica();
```

con:

```
var rubrica = Rubrica.getInstance();
```

E tutto ha continuato a funzionare come funzionava prima.

La classe `Speciale` potrebbe essere implementata così:

```

public class Speciale extends Contatto {

    private String suoneria;

    public Speciale(String nome, String numeroDiTelefono, String indirizzo,
        String suoneria) {
        super(nome, numeroDiTelefono, indirizzo);
        setSuoneria(suoneria);
    }

    public String getSuoneria() {
        return suoneria;
    }

    public void setSuoneria(String suoneria) {
        this.suoneria = suoneria;
    }

    public void stampaDettagli() {
        System.out.println(getNome());
        System.out.println(getIndirizzo());
        System.out.println(getNumeroDiTelefono());
        System.out.println(getSuoneria());
        System.out.println();
    }
}

```

Notare che per istanziare un oggetto di tipo `Speciale`, potremo utilizzare solo un costruttore che prende in input quattro parametri `nome`, `numeroDiTelefono`, `indirizzo` e `suoneria` (perché per la nostra astrazione, è la suoneria che dovrebbe distinguere un contatto speciale da uno contatto normale). Tale costruttore richiama il costruttore della superclasse `Contatto` tramite la parola chiave `super`. Notare anche che il metodo `stampaDettagli` è stato ridefinito aggiungendo la stampa anche della variabile `suoneria`.

Aggiorniamo quindi la classe `Rubrica` come richiesto (modifiche in grassetto):

```

public class Rubrica {
    private static Rubrica instance;
    public Contatto[] contatti;
    public Speciale[] contattiSpeciali;
}

```

```

private Rubrica() {
    contatti = new Contatto[] {
        new Contatto("Claudio De Sio Cesari", "Via Java 13","131313131313"),
        new Contatto("Stevie Wonder", "Via Musica 10","1010101010"),
        new Contatto("Gennaro Capuozzo",
            "Piazza Quattro Giornate 1","1111111111")
    } ;

    contattiSpeciali = new Speciale[] {
        new Speciale("Mario Ruoppolo","Via Neruda 3","333333","Il Postino"),
        new Speciale("Vincenzo Malinconico",
            "Via dei Tribunali 8","888888","Tuca Tuca"),
        new Speciale("Logan Howlett","Piazza Canada 6","66666","Hurt")
    } ;
}

public static Rubrica getInstance() {
    if (instance == null) {
        instance = new Rubrica();
    }
    return instance;
}

public Contatto[] cercaContattiPerNome(String nome) {
    Contatto []contattiTrovati = new Contatto[contatti.length];
    for (int i = 0, j = 0; i < contattiTrovati.length; i++) {
        if (contatti[i].getNome().toUpperCase().
            contains(nome.toUpperCase())) {
            contattiTrovati[j] = contatti[i];
            j++;
        }
    }
    return contattiTrovati;
}

public Speciale[] cercaContattiSpecialiPerNome(String nome) {
    Speciale []contattiSpecialiTrovati =
        new Speciale[contattiSpeciali.length];
    for (int i = 0, j = 0; i < contattiSpecialiTrovati.length; i++) {
        if (contattiSpeciali[i].getNome().toUpperCase().
            contains(nome.toUpperCase())) {
            contattiSpecialiTrovati[j] = contattiSpeciali[i];
            j++;
        }
    }
    return contattiSpecialiTrovati;
}

public Contatto[] getContatti() {
    return contatti;
}

public Speciale[] getContattiSpeciali() {
    return contattiSpeciali;
}
}

```

Ed infine possiamo creare la seguente classe RicercaContattiSpeciali:

```
import java.util.Scanner;
```

```
public class RicercaContattiSpeciali {
```

```

    public static void main(String args[]) {
        System.out.println("Ricerca Contatti Speciali");
        System.out.println();
        var rubrica = Rubrica.getInstance();
        System.out.println("Inserisci nome o parte del nome da ricercare");
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();
        Speciale[] contattiSpecialiTrovati =
            rubrica.cercaContattiSpecialiPerNome(input);
        System.out.println("Contatti Speciali trovati con nome contenente \""
            + input + "\"");
        for (Speciale speciale : contattiSpecialiTrovati) {
            if (speciale != null) {
                speciale.stampaDettagli();
            }
        }
    }
}

```

Soluzione 6.y)

Nel package `rubrica.dat`i, decidiamo di mettere le nuove astrazioni `Dato`:

```
package rubrica.dat;
```

```
public interface Dato {
}

```

Identificabile:

```
package rubrica.dat;
```

```
public interface Identificabile {
    int getId();
}

```

ed `Entita`:

```
package rubrica.dat;
```

```
import rubrica.util.Contatore;
```

```
public abstract class Entita implements Dato, Identificabile {
    private int id;

    public Entita () {
        setId(Contatore.dammiNumeroSeriale());
    }

    private void setId(int id) {
        this.id = id;
    }

    public int getId() {
        return id;
    }
}

```

Notare come quest'ultima classe utilizzi una classe di utilità `Contatore` (inserita in un package `rubrica.util`, creato ad hoc) che contiene un metodo statico che aggiorna un numero seriale ogni volta che viene istanziata un'entità (ovvero una sua sottoclasse visto che `Entita` è una classe astratta). Segue la classe `Contatore`:

```
package rubrica.util;
```

```
public class Contatore {
    private static int contatoreOggetti;

    public static int dammiNumeroSeriale() {
        return contatoreOggetti+=1;
    }
}
```

Come già detto, il numero seriale ci servirà negli esercizi futuri.

Abbiamo deciso di inserire nel package `rubrica.dati` anche le classi `Contatto` e `Rubrica`. Per quanto riguarda quest'ultima abbiamo ritenuto necessario modificarla nel seguente modo:

```
package rubrica.dati;

public class Rubrica implements Dato {
    private static Rubrica instance;

    public Contatto[] contatti;
    public Speciale[] contattiSpeciali;

    private Rubrica () {
        contatti = new Contatto[]{
            new Contatto("Claudio De Sio Cesari", "Via Java 13",
                        "13131313131313"),
            new Contatto("Stevie Wonder", "Via Musica 10", "1010101010"),
            new Contatto("Gennaro Capuozzo", "Piazza Quattro Giornate 1",
                        "111111111111")
        };

        contattiSpeciali = new Speciale[]{
            new Speciale("Mario Ruoppolo", "Via Neruda 3", "333333",
                        "Il Postino"),
            new Speciale("Vincenzo Malinconico", "Via dei Tribunali 8",
                        "888888", "Tuca Tuca"),
            new Speciale("Logan Howlett", "Piazza Canada 6", "66666", "Hurt")
        };
    }

    public static Rubrica getInstance() {
        if (instance == null) {
            instance = new Rubrica();
        }
        return instance;
    }

    public Contatto[] getContatti() {
        return contatti;
    }

    public Speciale[] getContattiSpeciali() {
        return contattiSpeciali;
    }
}
```

Notare che abbiamo rimosso i metodi `cercaContattiPerNome` e `cercaContattiSpecialiPerNome` perché da considerarsi *metodi di business*, e non metodi di una classe che si chiama `Rubrica`. Nella nostra astrazione di rubrica infatti, ci rifacciamo al concetto di

rubrica contenente dei contatti, non di una rubrica che può eseguire ricerche. Come per le antiche e ormai non più utilizzate rubriche cartacee, intendiamo un oggetto simile ad un quaderno in cui sono scritti i contatti. Una rubrica rappresenta quindi un contenitore di dati e non un oggetto che esegue azioni di business, infatti gli abbiamo fatto implementare l'interfaccia `Dato`. Per cercare un certo contatto in una rubrica cartacea, un utente deve sfogliare le pagine della rubrica. Quindi abbiamo deciso per ora, di spostare i metodi `cercaContattiPerNome` e

`cercaContattiSpecialiPerNome` in una nuova classe chiamata proprio `Utente`, che si occuperà di eseguire le ricerche nella rubrica:

```
package rubrica.business;
import rubrica.dat.*;

public class Utente {
    public Contatto[] cercaContattiPerNome(String nome) {
        Contatto []contatti = Rubrica.getInstance().getContatti();
        Contatto []contattiTrovati = new Contatto[contatti.length];
        for (int i = 0, j = 0; i < contattiTrovati.length; i++) {
            if (contatti[i].getNome().toUpperCase().
                                contains(nome.toUpperCase())) {
                contattiTrovati[j] = contatti[i];
                j++;
            }
        }
        return contattiTrovati;
    }

    public Speciale[] cercaContattiSpecialiPerNome(String nome) {
        Speciale []contattiSpeciali =
            Rubrica.getInstance().getContattiSpeciali();
        Speciale []contattiSpecialiTrovati =
            new Speciale[contattiSpeciali.length];
        for (int i = 0, j = 0; i < contattiSpecialiTrovati.length; i++) {
            if (contattiSpeciali[i].getNome().toUpperCase().
                                contains(nome.toUpperCase())) {
                contattiSpecialiTrovati[j] = contattiSpeciali[i];
                j++;
            }
        }
        return contattiSpecialiTrovati;
    }
}
```

Notare che tale classe, appartiene al package `rubrica.business`. Riassumendo, una rubrica la astraiamo come `dato`, le facciamo implementare l'interfaccia `Dato` e la posizioniamo nel package `rubrica.dat`, mentre la classe `Utente` rappresenta un oggetto di business e si trova nel package `rubrica.business`.

Infine abbiamo modificato le classi `RicercaContatti` e `RicercaContattiSpeciali` nel seguente modo (modifiche in grassetto):

```
package rubrica.interfaccia;

import rubrica.dat.*;
import rubrica.business.Utente;
import java.util.Scanner;

public class RicercaContatti {
    public static void main(String args[]) {
        System.out.println("Ricerca Contatti");
        System.out.println();
    }
}
```

```

        var utente = new Utente();
        System.out.println("Inserisci nome o parte del nome da ricercare");
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();
        Contatto[] contattiTrovati = utente.cercaContattiPerNome(input);
        System.out.println("Contatti trovati con nome contenente \"" +
            input + "\"");
        for (Contatto contatto : contattiTrovati) {
            if (contatto != null) {
                contatto.stampaDettagli();
            }
        }
    }
}

package rubrica.interfaccia;

import rubrica.dati.*;
import rubrica.business.Utente;
import java.util.Scanner;

public class RicercaContattiSpeciali {
    public static void main(String args[]) {
        System.out.println("Ricerca Contatti Speciali");
        System.out.println();
        var utente = new Utente();
        System.out.println("Inserisci nome o parte del nome da ricercare");
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();
        Speciale[] contattiSpecialiTrovati =
            utente.cercaContattiSpecialiPerNome(input);
        System.out.println("Contatti Speciali trovati con nome contenente \""
            + input + "\"");
        for (Speciale speciale : contattiSpecialiTrovati) {
            if (speciale != null) {
                speciale.stampaDettagli();
            }
        }
    }
}

```

Soluzione 6.z)

Una soluzione semplice e quasi automatica, consisterebbe nell'estendere direttamente la classe Utente, con le due sottoclassi Amministratore e Commesso:

```

package com.claudiodesio.autenticazione;

public class Amministratore extends Utente {
    public Amministratore(String nome, String username, String password) {
        super(nome, username, password);
    }
}

e
package com.claudiodesio.autenticazione;

public class Commesso extends Utente {
    public Commesso(String nome, String username, String password) {

```



```
        super(nome, username, password);  
    }  
}
```

Solo che non abbiamo ancora informazioni sufficienti per inserire campi e metodi specifici per queste due classi (infatti sono vuote). Quindi per ora decidiamo di non implementarle.

Soluzioni degli esercizi del capitolo 7

Polimorfismo

Soluzione 7.a) *Polimorfismo per metodi, Vero o Falso:*

1. **Falso**, l'overload di un metodo implica scrivere un altro metodo con lo stesso nome e diversa lista di parametri.
2. **Falso**, l'overload di un metodo implica scrivere un altro metodo con lo stesso nome e diversa lista di parametri.
3. **Vero**.
4. **Vero**.
5. **Falso**, l'overload di un metodo implica scrivere un altro metodo con lo stesso nome e diversa lista di parametri.
6. **Falso**, stiamo facendo override. L'unica differenza sta nel nome dell'identificatore di un parametro, che è influente al fine di distinguere metodi.
7. **Vero**, la lista dei parametri dei due metodi è diversa.
8. **Vero**, in caso di override il tipo di ritorno non può essere differente.
9. **Vero**, in caso di override il metodo riscritto non può essere meno accessibile del metodo originale.
10. **Falso**, otterremo un overload. Infatti, le due liste di parametri differiscono per posizioni.

Soluzione 7.b) *Polimorfismo per dati, Vero o Falso:*

1. **Vero**.
2. **Vero**.
3. **Falso**, la classe `Veicolo` essendo astratta non è istanziabile. Inoltre non è possibile inserire in una collezione eterogenea di aerei un `Veicolo` che è superclasse di `Aereo`.
4. **Falso**, la compilazione fallirebbe già dal momento in cui provassimo a compilare il metodo `viaggia`. Infatti non è possibile chiamare il metodo `accelera` con un reference di tipo `Object`.
5. **Falso**, c'è bisogno di un casting, perché il compilatore non sa a priori il tipo a cui punterà il reference al runtime.
6. **Vero**.
7. **Vero**.
8. **Vero**, infatti `Veicolo` è superclasse di `Piper`.
9. **Vero**, il metodo `toString` appartiene a tutte le classi perché ereditato dalla superclasse `Object`.
10. **Vero**, ma tutti i dipendenti saranno pagati allo stesso modo.

Soluzione 7.c)

Il listato potrebbe essere il seguente:

```
public interface TipoFile {
```

```
int JAVA = 1;
int C_SHARP = 2;
int C_PLUS_PLUS = 3;
int C = 4;
}
```

Si noti che non è necessario specificare modificatori per le costanti dato che è sottointeso che siano dichiarate implicitamente `public`, `static` e `final`. Inoltre abbiamo scelto come tipo delle costanti `int`, ma un qualsiasi altro tipo sarebbe andato bene, l'importante è che le costanti abbiano valori differenti.

Questo tipo di uso di interfacce è in disuso da anni, precisamente da quando in Java versione 5 sono state introdotte le cosiddette enumerazioni (cfr. capitolo 9). Tuttavia useremo ugualmente questa modalità di programmazione visto che non abbiamo ancora affrontato l'argomento enumerazioni.

Soluzione 7.d)

Il listato della classe `File` potrebbe essere il seguente:

```
public abstract class File {

    private String nome;

    private int tipo;

    public File(String nome, int tipo) {
        this.nome = nome;
        this.tipo = tipo;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public int getTipo() {
        return tipo;
    }

    public void setTipo(int tipo) {
        this.tipo = tipo;
    }
}
```

Si noti che abbiamo dichiarato la classe astratta, perché generica e creata ai fini dell'estensione.

Soluzione 7.e)

Il listato della classe `FileSorgente` dovrebbe essere il seguente:

```
public class FileSorgente extends File {
```

```
private String contenuto;

public FileSorgente(String nome, int tipo) {
    super(nome, tipo);
}

public FileSorgente(String nome, int tipo, String contenuto) {
    this(nome, tipo);
    this.contenuto = contenuto;
}

public String getContenuto() {
    return contenuto;
}

public void setContenuto(String contenuto) {
    this.contenuto = contenuto;
}
}
```

Si noti che abbiamo riutilizzato il costruttore della superclasse mediante la parola chiave `super`, ed abbiamo creato anche un costruttore equivalente a quello della superclasse.

Soluzione 7.f)

Il listato del metodo richiesto potrebbe essere:

```
public void aggiungiTesto(String testo) {
    if (contenuto == null) {
        contenuto = "";
    }
    if (testo != null) {
        contenuto += testo;
    }
}
```

Una stringa `null` “sommata” ad un'altra stringa viene rappresentata proprio con la stringa “null”. Da quest'affermazione si può intuire del perché abbiamo implementato il metodo in questo modo.

Soluzione 7.g)

Il listato del metodo richiesto potrebbe essere:

```
public void aggiungiTesto(String testo, int posizione) {
    final int length = contenuto.length();
    if (contenuto != null && testo != null && posizione > 0
        && posizione < length) {
        contenuto = contenuto.substring(0, posizione) + testo +
            contenuto.substring(posizione);
    }
}
```

Abbiamo scelto, per semplicità, di non aggiungere testo nel caso la posizione specificata non sia corretta. Tuttavia una clausola `else` che stampi un messaggio di errore potrebbe rappresentare una soluzione migliore. In realtà questo metodo si può migliorare molto...

Nel capitolo 8 vedremo come si gestiscono veramente le eccezioni in Java.
--

Soluzione 7.h)

Il listato per la classe `TestFileSorgente` potrebbe essere il seguente:

```
public class TestFileSorgente {
    public static void main(String args[]) {
        FileSorgente fileSorgente = new FileSorgente("Test.java",
            TipoFile.JAVA, "public class MyClass {\n\r"};
        System.out.println(fileSorgente.getContenuto());
        // Test aggiungiTesto (String) corretto
        fileSorgente.aggiungiTesto("{}");
        System.out.println(fileSorgente.getContenuto());
        // Test aggiungiTesto (String,int) corretto
        fileSorgente.aggiungiTesto("//Test aggiunta testo\n\r", 23);
        System.out.println(fileSorgente.getContenuto());
        // Test aggiungiTesto (String,int) scorretto
        fileSorgente.aggiungiTesto("//Test aggiunta testo\n\r", -1);
        System.out.println(fileSorgente.getContenuto());
        // Test aggiungiTesto (String,int) scorretto
        fileSorgente.aggiungiTesto("//Test aggiunta testo\n\r", 100);
        System.out.println(fileSorgente.getContenuto());
    }
}
```

L'output risultante è il seguente:

```
public class MyClass {

public class MyClass {
}
public class MyClass {
//Test aggiunta testo
}
public class MyClass {
//Test aggiunta testo
}
public class MyClass {
//Test aggiunta testo
}
}
```

Si potrebbero scrivere molti più casi di test, e si dovrebbe anche fare in modo che ogni caso di test non dipenda dal precedente, ma per ora va bene così.

Per effettuare casi di test con maggiore comodità, è consigliato l'utilizzo di un tool come JUnit. Potete trovare una breve descrizione di JUnit all'interno dell'approfondimento 8.2.

Soluzione 7.i)

Il listato dell'interfaccia richiesta potrebbe essere:

```
public interface Editor {
    default void salva(FileSorgente file) {
        System.out.println("File: " + file.getNome() + " salvato!");
    }
    default void apri(FileSorgente file) {
```

```
        System.out.println("File: " + file.getNome() + " aperto!");
    }
    default void chiudi(FileSorgente file) {
        System.out.println("File: " + file.getNome() + " chiuso!");
    }
    default void modifica(FileSorgente file, String testo) {
        System.out.println("File: " + file.getNome() + " modificato!");
    }
}
```

Abbiamo creato dei metodi di default che simulano con la stampa la loro effettiva esecuzione.

Soluzione 7.j)

Il listato dell'interfaccia richiesta potrebbe essere:

```
public interface IDE extends Editor {
    default void compila(FileSorgente file) {
        System.out.println("File: " + file.getNome() + " compilato!");
    }

    default void esegui(FileSorgente file) {
        System.out.println("File: " + file.getNome() + " eseguito!");
    }
}
```

Anche in questo caso abbiamo creato dei metodi di default che simulano con la stampa la loro effettiva esecuzione.

Soluzione 7.k)

Il listato della classe richiesta potrebbe essere:

```
public class JavaIDE implements IDE {
    @Override
    public void modifica(FileSorgente file, String testo) {
        IDE.super.modifica(file, testo);
        file.aggiungiTesto(testo);
        System.out.println("Contenuto modificato:\n" + file.getContenuto());
    }
}
```

Soluzione 7.l)

Il listato della classe richiesta potrebbe essere:

```
public class TestIDE {
    public static void main(String args[]) {
        IDE ide = new JavaIDE();
        FileSorgente fileSorgente = new FileSorgente("Test.java",
            TipoFile.JAVA, "public class MyClass {\n\r"});
        ide.modifica(fileSorgente, "}");
    }
}
```

L'output risultante sarà:

```
File: Test.java modificato!
```

```
Contenuto modificato:
public class MyClass {
}
```

Soluzione 7.m)

Inseriamo le classi `Articolo`, `Album` e `Libro` nel package `libreria.dati`, le classi `GenereUtils`, `GenereLetterarioUtils` e `GenereMusicaleUtils` all'interno del package `libreria.util`, la classe `TestArticoli` all'interno del package `libreria.test`, infine lasciamo vuoto il package `libreria.business`, visto che per ora non abbiamo ancora delle classi che incapsulano la logica di business.

Per creare nelle classi `Libro` e `Album` i metodi `toString` richiesti, andiamo a sfruttare anche la loro superclasse astratta `Articolo`, che riscriviamo di seguito (in grassetto le modifiche):

```
package libreria.dati;

public abstract class Articolo {
    private static final char VALUTA = '€';
    private String id;
    private String titolo;
    private String nome;
    private int prezzo;
    private String genere;

    public Articolo(String id, String titolo, String nome, int prezzo,
                                                             String genere) {

        super();
        setId(id);
        setTitolo(titolo);
        setNome(nome);
        setPrezzo(prezzo);
        setGenere(genere);
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getTitolo() {
        return titolo;
    }

    public void setTitolo(String titolo) {
        this.titolo = titolo;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
}
```

```
}

public int getPrezzo() {
    return prezzo;
}

public void setPrezzo(int prezzo) {
    this.prezzo = prezzo;
}

public String getGenere() {
    return genere;
}

public void setGenere(String genere) {
    this.genere = genere;
}

@Override
public String toString() {
    return ":" + getId() + " " + getTitolo() +
        " di " + getNome() + ", " + getGenere() + ", " + getPrezzo() +
        " " + VALUTA;
}
}
```

Notare che abbiamo anche introdotto una costante privata e statica `VALUTA` per rappresentare la valuta del prezzo.

Possiamo quindi riscrivere il metodo `toString` nella sottoclasse `Libro` nel seguente modo (modifiche in grassetto):

```
package libreria.dati;
import libreria.util.*;

public class Libro extends Articolo {

    public Libro(String isbn, String titolo, String autore, int prezzo, String
    genere) {
        super(isbn, titolo, autore, prezzo, genere);
    }

    public void setGenere(String genere) {
        if (GenereLetterarioUtils.isGenereValido(genere)) {
            super.setGenere(genere);
        } else {
            GenereLetterarioUtils.stampaErrore(genere);
        }
    }

    @Override
    public String toString() {
        return "Libro" + super.toString();
    }
}
```

E possiamo quindi riscrivere il metodo `toString` nella sottoclasse `Album` nel seguente modo (modifiche in grassetto):

```
package libreria.dati;
import libreria.util.*;

public class Album extends Articolo {
```



```

    public Album(String ismn, String titolo, String artista, int prezzo,
        String genere) {
        super(ismn, titolo, artista, prezzo, genere);
    }

    public void setGenere(String genere) {
        if (GenereMusicaleUtils.isGenereValido(genere)) {
            super.setGenere(genere);
        } else {
            GenereMusicaleUtils.stampaErrore(genere);
        }
    }

    @Override
    public String toString() {
        return "Album"+ super.toString();
    }
}

```

Infine modifichiamo la classe `TestArticoli` nel seguente modo (modifiche in grassetto):

```

package libreria.test;

import libreria.dati.*;
import libreria.util.*;

public class TestArticoli {
    public static void main(String[] args) {
        Libro jfaVol1 = new Libro("979-12-200-4915-3", "Java for Aliens Vol. 1",
            "Claudio De Sio Cesari", 25, GenereLetterarioUtils.MANUALE);
        Libro jfaVol2 = new Libro("979-12-200-4916-0", "Java for Aliens Vol. 2",
            "Claudio De Sio Cesari", 25, "NonEsistente");
//      System.out.println("Genere JFA Vol 1 = " + jfaVol1.getGenere());
//      System.out.println("Genere JFA Vol 2 = " + jfaVol2.getGenere());
        System.out.println(jfaVol1);
        System.out.println(jfaVol2);
        Album lad = new Album("979-0-236-44-3", "Live after Death",
            "Iron Maiden", 25, GenereMusicaleUtils.ROCK);
        Album mop = new Album("978-0-789-01-2", "Master of Puppets",
            "Metallica", 25, "NonEsistente");
//      System.out.println("Genere Live after Death = " + lad.getGenere());
//      System.out.println("Genere Master of Puppets = " + mop.getGenere());
        System.out.println(lad);
        System.out.println(mop);
    }
}

```

Ovviamente il metodo `toString` viene chiamato automaticamente dal metodo `println`. L'output della precedente classe sarà il seguente:

```

Genere NonEsistente non valido! Usare uno dei seguenti generi:
Romanzo
Saggio
Thriller
Manuale
Fantascienza
Libro: (979-12-200-4915-3) Java for Aliens Vol. 1 di Claudio De Sio
    Cesari, Manuale, 25 €
Libro: (979-12-200-4916-0) Java for Aliens Vol. 2 di Claudio De Sio
    Cesari, null, 25 €
Genere NonEsistente non valido! Usare uno dei seguenti generi:
Rock

```

```
Jazz
Blues
Pop
Rap
Album: (979-0-236-44-3) Live after Death di Iron Maiden, Rock, 25 €
Album: (978-0-789-01-2) Master of Puppets di Metallica, null, 25 €
```

Soluzione 7.n)

Potremmo implementare la classe Carrello nel seguente modo:

```
package libreria.dat;

import java.util.ArrayList;
import java.util.List;

public class Carrello {
    private static final int MASSIMO_NUMERO_DI_ARTICOLI = 4;
    private Articolo[] articoli;

    public Carrello() {
        articoli = new Articolo[MASSIMO_NUMERO_DI_ARTICOLI];
    }

    public void aggiungi(Articolo articolo) {
        for (int i = 0; i < articoli.length; i++) {
            if (articoli[i] == null) {
                articoli[i] = articolo;
                return;
            }
        }
        System.out.println("Articolo non inserito: carrello pieno!");
    }

    public boolean isVuoto() {
        return articoli[0] == null;
    }

    public int calcolaPrezzo() {
        int prezzoCarrello = 0;
        for (Articolo articolo : articoli) {
            prezzoCarrello += articolo.getPrezzo();
        }
        return prezzoCarrello;
    }

    public String toString() {
        String descrizioneCarrello =
            "Il carrello contiene i seguenti articoli:\n";
        for (Articolo articolo : articoli) {
            descrizioneCarrello += articolo + "\n";
        }
        return descrizioneCarrello;
    }
}
```

Mentre potremmo implementare la classe TestCarrello (all'interno del package libreria.test) nel seguente modo:

```
package libreria.test;

import libreria.dat.*;
import libreria.util.*;

public class TestCarrello {
    public static void main(String[] args) {
        Libro jfaVol1 = new Libro("979-12-200-4915-3",
            "Java for Aliens Vol. 1", "Claudio De Sio Cesari", 25,
            GenereLetterarioUtils.MANUALE);
        Libro jfaVol2 = new Libro("979-12-200-4916-0",
            "Java for Aliens Vol. 2", "Claudio De Sio Cesari",
            25, GenereLetterarioUtils.MANUALE);
        Album lad = new Album("979-0-236-44-3", "Live after Death",
            "Iron Maiden", 25, GenereMusicaleUtils.ROCK);
        Album mop = new Album("978-0-789-01-2", "Master of Puppets",
            "Metallica", 25, GenereMusicaleUtils.ROCK);
        Album tt = new Album("978-0-789-01-9", "Tokyo Tapes", "Scorpions",
            22, GenereMusicaleUtils.ROCK);
        Carrello carrello = new Carrello();
        System.out.println("Carrello vuoto = " + carrello.isVuoto());
        System.out.println("Aggiungiamo il libro " + jfaVol1);
        carrello.aggiungi(jfaVol1);
        System.out.println("Aggiungiamo il libro " + jfaVol2);
        carrello.aggiungi(jfaVol2);
        System.out.println("Aggiungiamo l'album " + lad);
        carrello.aggiungi(lad);
        System.out.println("Aggiungiamo l'album " + mop);
        carrello.aggiungi(mop);
        System.out.println("Aggiungiamo l'album " + tt);
        carrello.aggiungi(tt);
        System.out.println("Carrello vuoto = " + carrello.isVuoto());
        System.out.println(carrello);
    }
}
```

Segue l'output della classe TestCarrello:

```
Carrello vuoto = true
Aggiungiamo il libro Libro: (979-12-200-4915-3) Java for Aliens Vol. 1 di
  Claudio De Sio Cesari, Manuale, 25 €
Aggiungiamo il libro Libro: (979-12-200-4916-0) Java for Aliens Vol. 2 di
  Claudio De Sio Cesari, Manuale, 25 €
Aggiungiamo l'album Album: (979-0-236-44-3) Live after Death di Iron
  Maiden, Rock, 25 €
Aggiungiamo l'album Album: (978-0-789-01-2) Master of Puppets di
  Metallica, Rock, 25 €
Aggiungiamo il libro Album: (978-0-789-01-9) Tokyo Tapes di Scorpions,
  Rock, 22 €
Articolo non inserito: carrello pieno!
Carrello vuoto = false
Il carrello contiene i seguenti articoli:
Libro: (979-12-200-4915-3) Java for Aliens Vol. 1 di Claudio De Sio
  Cesari, Manuale, 25 €
Libro: (979-12-200-4916-0) Java for Aliens Vol. 2 di Claudio De Sio
  Cesari, Manuale, 25 €
Album: (979-0-236-44-3) Live after Death di Iron Maiden, Rock, 25 €
Album: (978-0-789-01-2) Master of Puppets di Metallica, Rock, 25 €
```

Soluzione 7.o) Varargs, Vero o Falso:

1. **Vero.**
2. **Falso.**
3. **Falso.**
4. **Vero.**
5. **Vero.**
6. **Vero.**
7. **Falso.**
8. **Vero.**
9. **Falso.**
10. **Vero.**

Soluzione 7.p)

Le risposte vere sono la 1, la 3 e la 5. La numero 5 è vera perché il metodo ereditato è implicitamente pubblico, e ridefinendolo senza il modificatore `public`, si sta cercando di renderlo meno accessibile di quello ereditato. Infatti compilando la classe `D` otterremo il seguente errore:

```
error: metodo() in D cannot implement metodo() in A
    void metodo() {}
        ^
    attempting to assign weaker access privileges; was public
1 error
```

Soluzione 7.q)

Le affermazioni corrette sono la 2, la 3 e la 4.

Soluzione 7.r)

La risposta giusta è la numero 3. Infatti, viene chiamato sempre il metodo `stampa` della superclasse `StampaNumero` che prende in input un `double`, e questo spiega il formato dell'output. Il motivo per cui non viene chiamato in maniera virtuale il metodo nella sottoclasse `StampaIntero`, è perché non si tratta di un override, visto che il tipo di parametro è diverso tra i due metodi. Non trattandosi di un override, usando un reference della superclasse, l'unico metodo che si può chiamare, è proprio quello della superclasse.

Soluzione 7.s)

L'unica risposta corretta è la numero 4.

Soluzione 7.t)

Overload: dal momento che un metodo è univocamente determinato dalla sua firma, in una classe (o un'interfaccia) è possibile creare più metodi con lo stesso identificatore ma con differente lista di parametri. In casi come questo si parla di overload di un (nome di un) metodo.

Override: permette di riscrivere in una sottoclasse un metodo ereditato da una superclasse (o interfaccia).

Un esempio di sottoclasse, che implementa entrambi i concetti, lo possiamo ricavare modificando le classi dell'esercizio 7.r:

```
public class StampaNumero {
    public void stampa(double numero) {
        System.out.print(numero);
    }
}

public class StampaIntero extends StampaNumero {
    //overload
    public void stampa(int numero) {
        System.out.print(numero);
    }

    //override
    public void stampa(double numero) {
        System.out.print(numero);
    }
}
```

Soluzione 7.u)

Le affermazioni corrette sono le numero 1 e 4.

L'affermazione 2 non è corretta perché il tipo di ritorno della classe `SommaIntero` è covariante (cfr. paragrafo 7.2.3.1). L'affermazione 3 invece non è corretta perché sussiste l'autoboxing-unboxing.

La numero 4 è corretta perché se per esempio eseguiamo questo codice:

```
SommaIntero sommaIntero = new SommaIntero();
sommaIntero.somma(1.0, 1.0);
```

otterremo quest'eccezione al runtime:

```
Exception in thread "main" java.lang.ClassCastException:
    java.base/java.lang.Double cannot be cast to
    java.base/java.lang.Integer
    at SommaIntero.somma(SommaIntero.java:4)
    at SommaIntero.main(SommaIntero.java:9)
```

Alla gestione delle eccezioni è dedicato gran parte del capitolo 8.

Soluzione 7.v)

Una possibile soluzione potrebbe essere la seguente:

```
public class SommaIntero extends SommaNumero {
    @Override
    public Integer somma(Number n1, Number n2) {
        if (n1 == null || n2 == null) {
            System.out.println("Impossibile sommare un operando null, " +
                "restituisco il valore di default");
        }
    }
}
```

```

        return Integer.MIN_VALUE;
    } else if (!(n1 instanceof Integer && n2 instanceof Integer)) {
        System.out.println("Passa solo variabili di tipo intero, " +
            "restituisco il valore di default");
        return Integer.MIN_VALUE;
    }
    return (Integer)n1 + (Integer)n2;
}
}

```

Eseguendo il seguente metodo main infatti:

```

public static void main(String args[]) {
    SommaIntero sommaIntero = new SommaIntero();
    sommaIntero.somma(1.0, 1.0);
    sommaIntero.somma(null, 1.0);
}

```

otterremo il seguente output e nessuna eccezione:

```

Passa solo variabili di tipo intero, restituisco il valore di default
Impossibile sommare un operando null, restituisco il valore di default

```

Si noti che il primo controllo sulla nullità dei parametri è necessario perché non è possibile utilizzare il cast su una variabile nulla, e nemmeno utilizzare l'operatore +.

Soluzione 7.w)

Un **parametro polimorfo** è un parametro di un metodo dichiarato di un certo tipo (magari astratto) ma che al runtime punterà ad un'istanza di una sua sottoclasse.

Le **collezioni eterogenee** sono collezioni di oggetti diversi, come per esempio un array di `Number`, che contiene oggetti delle sue sottoclassi come `Integer`.

Si parla di **chiamata virtuale ad un metodo** quando, utilizzando un reference di una superclasse che punta ad un oggetto di una, viene invocato un metodo che in realtà è ridefinito in una sottoclasse.

Soluzione 7.x)

Riscriviamo le classi `Contatto` e `Speciale` eliminando il metodo `stampaDettagli`, e lo sostituiamo con l'override del metodo `toString`. Prima però inseriamo anche il seguente metodo `toString` all'interno della classe `Entita`:

```

@Override
public String toString() {
    return "Id="+getId();
}

```

Richiamiamo il metodo precedente nel metodo `toString` della sottoclasse `Contatto` nel seguente modo:

```

@Override
public String toString() {
    return super.toString() + "\nNome=" + nome +
        "\nNumeroDiTelefono=" + numeroDiTelefono + "\nIndirizzo=" + indirizzo;
}

```

Quindi possiamo semplificare molto il metodo `toString` della classe `Speciale`:

```

@Override
public String toString() {
    return super.toString() + "\nSuoneria=" + suoneria;
}

```

Potremmo implementare la classe `Rubrica` nel seguente modo (modifiche in grassetto):

```
package rubrica.dat;

public class Rubrica implements Dato {
    private static Rubrica instance;

    public Contatto[] contatti;

    private Rubrica () {
        contatti = new Contatto[] {
            new Contatto("Claudio De Sio Cesari","Via Java 13","131313131313"),
            new Contatto("Stevie Wonder","Via Musica 10","1010101010"),
            new Contatto("Gennaro Capuozzo",
                "Piazza Quattro Giornate 1","1111111111"),
            new Speciale("Mario Ruoppolo","Via Neruda 3","333333","Il Postino"),
            new Speciale("Vincenzo Malinconico",
                "Via dei Tribunali 8","888888","Tuca Tuca"),
            new Speciale("Logan Howlett","Piazza Canada 6","66666","Hurt")
        } ;
    }

    public static Rubrica getInstance() {
        if (instance == null) {
            instance = new Rubrica();
        }
        return instance;
    }

    public Contatto[] getContatti() {
        return contatti;
    }

    public Speciale[] getContattiSpeciali() {
        Speciale[] contattiSpecialiTrovati = new Speciale[contatti.length];
        for (int i = 0, j = 0; i < contatti.length; ++i) {
            if (contatti[i] == null) {
                break;
            }
            if (contatti[i] instanceof Speciale) {
                contattiSpecialiTrovati[j] = (Speciale)contatti[i];
                j++;
            }
        }
        return contattiSpecialiTrovati;
    }
}
```

Mentre la classe `Utente` potrebbe subire piccole modifiche (in grassetto):

```
package rubrica.business;
import rubrica.dat.*;

public class Utente {
    public Contatto[] cercaContattiPerNome(String nome) {
        Contatto []contatti = Rubrica.getInstance().getContatti();
        Contatto []contattiTrovati = new Contatto[contatti.length];
        for (int i = 0, j = 0; i < contattiTrovati.length; i++) {
            if (contatti[i] == null) {
                break;
            }
            if (contatti[i].getNome().toUpperCase().
```

```

        contains(nome.toUpperCase())) {
            contattiTrovati[j] = contatti[i];
            j++;
        }
    }
    return contattiTrovati;
}

public Speciale[] cercaContattiSpecialiPerNome(String nome) {
    Speciale []contattiSpeciali =
        Rubrica.getInstance().getContattiSpeciali();
    Speciale []contattiSpecialiTrovati =
        new Speciale[contattiSpeciali.length];
    for (int i = 0, j = 0; i < contattiSpecialiTrovati.length; i++) {
        if (contattiSpeciali[i] == null) {
            break;
        }
        if (contattiSpeciali[i].getNome().toUpperCase().
            contains(nome.toUpperCase())) {
            contattiSpecialiTrovati[j] = contattiSpeciali[i];
            j++;
        }
    }
    return contattiSpecialiTrovati;
}
}

```

Anche per le classi `RicercaContatti` e `RicercaContattiSpeciali` le modifiche sono minime (in grassetto) e riguardano la stampa degli oggetti utilizzando il metodo `toString`, invece del metodo `stampaDettagli`. Segue la classe `RicercaContatti`:

```

package rubrica.interfaccia;

import rubrica.dati.*;
import rubrica.business.Utente;
import java.util.Scanner;

public class RicercaContatti {
    public static void main(String args[]) {
        System.out.println("Ricerca Contatti");
        System.out.println();
        var utente = new Utente();
        System.out.println("Inserisci nome o parte del nome da ricercare");
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();
        Contatto[] contattiTrovati = utente.cercaContattiPerNome(input);
        System.out.println("Contatti trovati con nome contenente \"" +
            input + "\"");
        for (Contatto contatto : contattiTrovati) {
            if (contatto != null) {
                System.out.println(contatto+"\n");
            }
        }
    }
}

```

Ed infine la classe `RicercaContattiSpeciali`:

```

package rubrica.interfaccia;
import rubrica.dati.*;
import rubrica.business.Utente;
import java.util.Scanner;

```



```

public class RicercaContattiSpeciali {
    public static void main(String args[]) {
        System.out.println("Ricerca Contatti Speciali");
        System.out.println();
        var utente = new Utente();
        System.out.println("Inserisci nome o parte del nome da ricercare");
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();
        Speciale[] contattiSpecialiTrovati =
            utente.cercaContattiSpecialiPerNome(input);
        System.out.println("Contatti Speciali trovati con nome contenente \""
            + input + "\"");
        for (Speciale speciale : contattiSpecialiTrovati) {
            if (speciale != null) {
                System.out.println(speciale+"\n");
            }
        }
    }
}

```

Soluzione 7.y)

Potremmo scrivere il metodo `getContattiOrdinari` nella classe `Rubrica` nel seguente modo:

```

public Contatto[] getContattiOrdinari() {
    Contatto[] contattiOrdinariTrovati = new Contatto[contatti.length];
    for (int i = 0, j = 0; i < contatti.length; ++i) {
        if (contatti[i] == null) {
            break;
        }
        if (!(contatti[i] instanceof Speciale)) {
            contattiOrdinariTrovati[j] = contatti[i];
            j++;
        }
    }
    return contattiOrdinariTrovati;
}

```

Mentre il metodo `cercaContattiOrdinariPerNome` nella classe `Utente` potrebbe essere il seguente:

```

public Contatto[] cercaContattiOrdinariPerNome(String nome) {
    Contatto []contattiOrdinari =
        Rubrica.getInstance().getContattiOrdinari();
    Contatto []contattiOrdinariTrovati =
        new Contatto[contattiOrdinari.length];
    for (int i = 0, j = 0; i < contattiOrdinariTrovati.length; i++) {
        if (contattiOrdinari[i] == null) {
            break;
        }
        if (contattiOrdinari[i].getNome().toUpperCase().
            contains(nome.toUpperCase())) {
            contattiOrdinariTrovati[j] = contattiOrdinari[i];
            j++;
        }
    }
    return contattiOrdinariTrovati;
}

```

Infine, potremmo implementare la classe `RicercaContattiOrdinari` così:

```
package rubrica.interfaccia;
```

```

import rubrica.dat.*;
import rubrica.business.Utente;
import java.util.Scanner;

public class RicercaContattiOrdinari {
    public static void main(String args[]) {
        System.out.println("Ricerca Contatti Ordinari");
        System.out.println();
        var utente = new Utente();
        System.out.println("Inserisci nome o parte del nome da ricercare");
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();
        Contatto[] contattiTrovati = utente.cercaContattiOrdinariPerNome(input);
        System.out.println("Contatti ordinari trovati con nome contenente \""
            + input + "\"");
        for (var contatto : contattiTrovati) {
            if (contatto != null) {
                System.out.println(contatto+"\n");
            }
        }
    }
}

```

Soluzione 7.z)

Potremmo implementare la classe `ContattoFactory` richiesta, creando usando un overload per il metodo chiamato `getContatto`:

```

package rubrica.dat;

import rubrica.dat.Speciale;
import rubrica.dat.Contatto;

public class ContattoFactory {

    public static Contatto getContatto(String nome, String numeroDiTelefono,
        String indirizzo) {
        return new Contatto(nome, numeroDiTelefono, indirizzo);
    }

    public static Speciale getContatto(String nome, String numeroDiTelefono,
        String indirizzo, String suoneria) {
        return new Speciale(nome, numeroDiTelefono, indirizzo, suoneria);
    }
}

```

Abbiamo modificato il costruttore della classe `Rubrica` nel seguente modo (modifiche in grassetto):

```

private Rubrica () {
    contatti = new Contatto[] {
        new Contatto("Claudio De Sio Cesari","Via Java 13","1313131313"),
        new Contatto("Stevie Wonder","Via Musica 10","1010101010"),
        new Contatto("Gennaro Capuozzo",
            "Piazza Quattro Giornate 1","1111111111"),
        new Speciale("Mario Ruoppolo","Via Neruda 3","333333","Il Postino"),
        new Speciale("Vincenzo Malinconico","Via dei Tribunali 8", "888888",
            "Tuca Tuca"),
        new Speciale("Logan Howlett","Piazza Canada 6","66666","Hurt"),
    }
}

```

```
        null,  
        null,  
        null,  
        null  
    } ;  
}
```

Decidiamo di modificare la classe `Utente`, non solo aggiungendo il metodo `aggiungi richiesto`, ma anche introducendo la variabile d'istanza `rubrica`, e un costruttore che la inizializza (modifiche in grassetto):

```
package rubrica.business;  
import rubrica.dat.*;  
  
public class Utente {  
  
    private Rubrica rubrica;  
  
    public Utente() {  
        rubrica = Rubrica.getInstance();  
    }  
  
    public Contatto[] cercaContattiPerNome(String nome) {  
        Contatto []contatti = rubrica.getContatti();  
        Contatto []contattiTrovati = new Contatto[contatti.length];  
        for (int i = 0, j = 0; i < contattiTrovati.length; i++) {  
            if (contatti[i] == null) {  
                break;  
            }  
            if (contatti[i].getNome().toUpperCase().  
                contains(nome.toUpperCase())) {  
                contattiTrovati[j] = contatti[i];  
                j++;  
            }  
        }  
        return contattiTrovati;  
    }  
  
    public Speciale[] cercaContattiSpecialiPerNome(String nome) {  
        Speciale []contattiSpeciali = rubrica.getContattiSpeciali();  
        Speciale []contattiSpecialiTrovati =  
            new Speciale[contattiSpeciali.length];  
        for (int i = 0, j = 0; i < contattiSpecialiTrovati.length; i++) {  
            if (contattiSpeciali[i] == null) {  
                break;  
            }  
            if (contattiSpeciali[i].getNome().toUpperCase().  
                contains(nome.toUpperCase())) {  
                contattiSpecialiTrovati[j] = contattiSpeciali[i];  
                j++;  
            }  
        }  
        return contattiSpecialiTrovati;  
    }  
  
    public Contatto[] cercaContattiOrdinariPerNome(String nome) {  
        Contatto []contattiOrdinari = rubrica.getContattiOrdinari();  
        Contatto []contattiOrdinariTrovati =  
            new Contatto[contattiOrdinari.length];  
        for (int i = 0, j = 0; i < contattiOrdinariTrovati.length; i++) {  
            if (contattiOrdinari[i] == null) {
```

```

        break;
    }
    if (contattiOrdinari[i].getNome().toUpperCase().
        contains(nome.toUpperCase())) {
        contattiOrdinariTrovati[j] = contattiOrdinari[i];
        j++;
    }
}
return contattiOrdinariTrovati;
}

public void inserisci(Contacto contattoDaAggiungere) {
    Contacto[] contatti = rubrica.getContatti();
    for (int i = 0; i < contatti.length; ++i) {
        if (contatti[i] == null) {
            contatti[i] = contattoDaAggiungere;
            break;
        }
    }
}
}

```

Infatti, se non avessimo creato anche la variabile d'istanza e il costruttore avremmo potuto avere una situazione imprevista per quanto riguarda la classe `RicercaContatti`, che abbiamo deciso di modificare nel seguente modo (modifiche in grassetto):

```

package rubrica.interfaccia;

import rubrica.dati.*;
import rubrica.business.Utente;
import java.util.Scanner;

public class RicercaContatti {
    public static void main(String args[]) {
        System.out.println("Ricerca Contatti");
        System.out.println();
        var utente = new Utente();
        Contacto nuovoContatto = ContactoFactory.getContatto(
            "Molly Malone", "123456789",
            "Suffolk St, Dublin 2, D02 KX03, Ireland");
        Speciale contattoSpeciale = ContactoFactory.getContatto(
            "Phil Lynott", "987654321", "Harry St, Dublin, Ireland", "Rosalie");
        utente.inserisci(nuovoContatto);
        utente.inserisci(contattoSpeciale);
        System.out.println("Inserisci nome o parte del nome da ricercare");
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();
        Contacto[] contattiTrovati = utente.cercaContattiPerNome(input);
        System.out.println("Contatti trovati con nome contenente \""
            + input + "\"");
        for (Contacto contatto : contattiTrovati) {
            if (contatto != null) {
                System.out.println(contatto+"\n");
            }
        }
    }
}

```

Se non avessimo introdotto nella classe `Rubrica` la variabile e il costruttore, avremmo avuto che gli identificatori degli oggetti istanziati nel metodo `main` `nuovoContatto` e `contattoSpeciale`, avrebbero avuto rispettivamente `id = 1` e `id = 2`. Infatti, il metodo `dammiNumeroSeriale` della

classe `Contatore` richiamata dal costruttore di `Contatto`, sarebbe stato chiamato prima per queste due istanze, mentre il metodo `getInstance` della classe `Rubrica`, che a sua volta avrebbe inizializzato l'array `contatti`, sarebbe stato chiamato solo nel successivamente, e quindi avremmo avuto che gli id dei contatti sarebbero partiti dal numero 3.

Soluzioni degli esercizi del capitolo 8

Eccezioni ed asserzioni

Soluzione 8.a) Gestione delle eccezioni e degli errori, Vero o Falso:

1. **Vero**, perché `ArithmeticException` è sottoclasse di `RuntimeException`.
2. **Falso**.
3. **Falso**, produrrà il seguente output:

```
Divisione per zero...  
Finally!
```
4. **Falso**, produrrà un errore in compilazione (l'ordine dei blocchi `catch` non è regolare).
5. **Falso**.
6. **Falso**, solo le sottoclassi di `Throwable`.
7. **Vero**.
8. **Falso**.
9. **Vero**.
10. **Vero**.

Soluzione 8.b) Gestione delle asserzioni, Vero o Falso:

1. **Vero**.
2. **Vero**.
3. **Falso**.
4. **Vero**.
5. **Vero**.
6. **Vero**.
7. **Vero**.
8. **Vero**.
9. **Vero**.
10. **Vero**.

Soluzione 8.c)

Come abbiamo visto nell'approfondimento 8.3, non dovremmo mai usare le asserzioni per testare i parametri di un metodo pubblico. Quindi indubbiamente è più corretto usare la gestione delle eccezioni.

Soluzione 8.d)

Una possibile implementazione potrebbe essere la seguente:

```
public void aggiungiTesto(String testo) throws RuntimeException {
    if (contenuto == null) {
        contenuto = "";
    }
    if (testo == null) {
        throw new RuntimeException("testo = null");
    }
    contenuto += testo;
}
```

Si noti che abbiamo rilanciato una `RuntimeException`, ma avremmo potuto rilanciare una qualsiasi altra eccezione (per esempio `Exception` stessa). Inoltre la clausola `throws` accanto alla dichiarazione del metodo non è tecnicamente obbligatoria ma consigliabile.

Soluzione 8.e)

Il listato potrebbe essere simile al seguente:

```
public void aggiungiTesto(String testo, int posizione) {
    try {
        if (testo != null) {
            contenuto = contenuto.substring(0, posizione) + testo +
                contenuto.substring(posizione);
        }
    } catch (NullPointerException exc) {
        System.out.println("Il contenuto è null : " + exc.getMessage());
        contenuto = "" + testo;
    } catch (StringIndexOutOfBoundsException exc) {
        System.out.println("L'indice " + posizione + " è invalido : " +
            exc.getMessage());
        contenuto = (posizione < 0 ? testo + contenuto : contenuto
            + testo);
    }
}
```

In questo esempio abbiamo solo controllato se il testo da aggiungere è `null`, in tal caso nessuna operazione viene eseguita. Poi abbiamo gestito la `NullPointerException` che si presenterebbe nel caso `contenuto` valga `null`. Nella clausola `catch` abbiamo stampato un messaggio significativo e mantenuto la coerenza con il metodo precedentemente presentato.

Abbiamo anche gestito una `StringIndexOutOfBoundsException` che scatterebbe se la posizione specificata contenesse un numero negativo o maggiore della dimensione del contenuto del file.

Anche in questo caso nella clausola `catch` abbiamo dapprima stampato un messaggio significativo, e poi “accomodato” la situazione. In particolare abbiamo fatto in modo che (sfruttando anche un operatore ternario), se la variabile `posizione` è specificata con un valore negativo, allora la variabile `testo` viene messa all’inizio prima della variabile `contenuto` (come se fosse stata specificata la posizione 0). Se invece viene impostata con un valore superiore all’ultimo indice disponibile per il contenuto, allora la variabile `testo` viene aggiunta alla fine del contenuto.

Infine, per quanto riguarda il `finally`, l’impostazione della nostra soluzione non ne richiede l’uso.

Soluzione 8.f)

Il listato della classe `TestFileSorgente` potrebbe essere il seguente:

```
public class TestFileSorgente {
    public static void main(String args[]) {
```

```
FileSorgente fileSorgente = new FileSorgente("Test.java",
    TipoFile.JAVA, "public class MyClass {\n\r"});
System.out.println(fileSorgente.getContenuto());
// Test aggiungiTesto(String) corretto
fileSorgente.aggiungiTesto("{}");
System.out.println(fileSorgente.getContenuto());
// Test aggiungiTesto(String,int) corretto
fileSorgente.aggiungiTesto("//Test aggiunta testo\n\r", 23);
System.out.println(fileSorgente.getContenuto());
// Test aggiungiTesto(String,int) scorretto
fileSorgente.aggiungiTesto("//Test aggiunta testo\n\r", -1);
System.out.println(fileSorgente.getContenuto());
// Test aggiungiTesto(String,int) scorretto
fileSorgente.aggiungiTesto("//Test aggiunta testo\n\r", 100);
System.out.println(fileSorgente.getContenuto());
FileSorgente fileSorgenteVuoto = new FileSorgente("FileVuoto.c",
    TipoFile.C);
fileSorgenteVuoto.aggiungiTesto("//Test aggiunta testo\n\r", 3);
System.out.println(fileSorgenteVuoto.getContenuto());
FileSorgente fileSorgenteVuoto2 = new FileSorgente("FileVuoto2.cpp",
    TipoFile.C_PLUS_PLUS);
fileSorgenteVuoto2.aggiungiTesto("//Test aggiunta testo\n\r");
}
}
```

Soluzione 8.g)

Il nuovo listato della classe `PortaMonetePienoException` potrebbe essere il seguente:

```
public class PortaMonetePienoException extends Exception {
    public PortaMonetePienoException(String message) {
        super(message);
    }
}
```

Il listato del costruttore della classe `PortaMonete`, con i nuovi requisiti potrebbe essere trasformato nel seguente modo:

```
public PortaMonete(int... valori) {
    try {
        int numeroMonete = valori.length;
        for (int i = 0; i < numeroMonete; i++) {
            if (i >= 10) {
                throw new PortaMonetePienoException (
                    "Sono state inserite solo le prime 10 monete!");
            }
            monete[i] = new Moneta(valori[i]);
        }
    } catch (PortaMonetePienoException | NullPointerException exc) {
        System.out.println(exc.getMessage());
    }
}
```

Si noti che passando `null` come parametro al costruttore, potremmo generare una `NullPointerException` quando viene richiesta la variabile `length` sul parametro `valori` (che varrebbe appunto `null`).

Con un `multi-catch` abbiamo garantito il funzionamento del costruttore senza che il programma si interrompa, stampando il messaggio del problema che si è presentato. Tuttavia non è corretto in

questo caso gestire allo stesso modo questi due tipi di eccezione, in quanto nel caso della `NullPointerException` il messaggio stampato sarà semplicemente:

`null`

che è poco esplicativo.

Sarebbe meglio gestire le due eccezioni in questo modo:

```
public PortaMonete(int... valori) {
    try {
        int numeroMonete = valori.length;
        for (int i = 0; i < numeroMonete; i++) {
            if (i >= 10) {
                throw new PortaMonetePienoException (
                    "Sono state inserite solo le prime 10 monete!");
            }
            monete[i] = new Moneta(valori[i]);
        }
    } catch (PortaMonetePienoException | NullPointerException exc) {
    } catch (PortaMonetePienoException exc) {
        System.out.println(exc.getMessage());
    } catch (NullPointerException exc) {
        System.out.println("Il portamonete è stato creato vuoto");
    }
}
```

Soluzione 8.h)

Il listato del metodo `aggiungi` della classe `PortaMonete` con i nuovi requisiti, potrebbe essere trasformato nel seguente modo:

```
public void aggiungi(Moneta moneta) throws PortaMonetePienoException {
    try {
        System.out.println("Proviamo ad aggiungere una " +
            moneta.getDescrizione());
    } catch (NullPointerException exc) {
        throw new NullPointerException("La moneta passata era null");
    }
    int indiceLibero = primoIndiceLibero();
    if (indiceLibero == -1) {
        throw new PortaMonetePienoException("Portamonete pieno! La moneta "
            + moneta.getDescrizione() + " non è stata aggiunta...");
    } else {
        monete[indiceLibero] = moneta;
        System.out.println("E' stata aggiunta una " +
            moneta.getDescrizione());
    }
}
```

Si noti che abbiamo gestito anche la `NullPointerException` catturandola e rilanciandola con un messaggio migliore del “`null`”. Anche in questo caso l’eccezione si verificherebbe nel caso il parametro passato fosse `null`, non appena fosse chiamato il metodo `getDescrizione` sull’oggetto `moneta` (che varrebbe `null`).

Soluzione 8.i)

L’eccezione richiesta potrebbe essere la seguente:

```
public class MonetaNonTrovataException extends Exception {
```

```
public MonetaNonTrovataException(String message) {
    super(message);
}
}
```

Il listato del metodo `preleva` della classe `PortaMonete`, con i nuovi requisiti, potrebbe essere trasformato nel seguente modo:

```
public Moneta preleva(Moneta moneta) throws MonetaNonTrovataException {
    try {
        System.out.println("Proviamo a prelevare una " +
            moneta.getDescrizione());
    } catch (NullPointerException exc) {
        throw new MonetaNullException();
    }
    Moneta monetaTrovata = null;
    int indiceMonetaTrovata = indiceMonetaTrovata(moneta);
    if (indiceMonetaTrovata == -1) {
        throw new MonetaNonTrovataException("Moneta non trovata!");
    } else {
        monetaTrovata = moneta;
        monete[indiceMonetaTrovata] = null;
        System.out.println("Una " + moneta.getDescrizione()
            + " prelevata");
    }
    return monetaTrovata;
}
```

Si noti che abbiamo gestito anche la `NullPointerException` catturandola e rilanciandola con un messaggio migliore del “null”. Anche in questo caso l’eccezione si verificherebbe nel caso il parametro passato fosse `null`, non appena viene chiamato il metodo `getDescrizione` sull’oggetto `moneta` (che varrebbe `null`). Possiamo migliorare questo meccanismo creando un’altra eccezione personalizzata:

```
public class MonetaNullException extends RuntimeException {
    public MonetaNullException() {
        super("La moneta passata era null");
    }
}
```

A `MonetaNullException` gli abbiamo fatto estendere `RuntimeException`, e quindi si tratta di una *unchecked exception* che non ha bisogno di essere dichiarata nella clausola `throws` del metodo che la lancia. E quindi il metodo `aggiungi` può essere modificato come segue:

```
public void aggiungi(Moneta moneta) throws PortaMonetePienoException {
    try {
        System.out.println("Proviamo ad aggiungere una " +
            moneta.getDescrizione());
    } catch (NullPointerException exc) {
        throw new MonetaNullException();
    }
    int indiceLibero = primoIndiceLibero();
    if (indiceLibero == -1) {
        throw new PortaMonetePienoException("Portamonete pieno! La moneta "
            + moneta.getDescrizione() + " non è stata aggiunta...");
    } else {
        monete[indiceLibero] = moneta;
        System.out.println("E' stata aggiunta una " +
            moneta.getDescrizione());
    }
}
```

Soluzione 8.1)

Equivalentemente all'esercizio precedente il listato richiesto potrebbe essere il seguente:

```
/**
 * Classe di test per la classe Moneta.
 *
 * @author Claudio De Sio Cesari
 */
public class TestMonete {

    public static void main(String args[]) {

        Moneta monetaDaVentiCentesimi = new Moneta(20);
        Moneta monetaDaUnCentesimo = new Moneta(1);
        Moneta monetaDaUnEuro = new Moneta(100);
        // Creiamo un portamonete con 11 monete
        PortaMonete portaMoneteInsufficiente = new PortaMonete(2, 5, 100, 10,
            50, 10, 100, 200, 10, 5, 2);
        // Creiamo un portamonete con 8 monete
        PortaMonete portaMonete = new PortaMonete(2, 5, 100, 10, 50, 10, 100,
            200);
        portaMonete.stato();

        try {
            // Aggiungiamo una moneta da 20 centesimi
            portaMonete.aggiungi(monetaDaVentiCentesimi);
        } catch (PortaMonetePienoException | MonetaNullException exc) {
            System.out.println(exc.getMessage());
        }

        try {
            // Aggiungiamo la decima moneta da 1 centesimo.
            portaMonete.aggiungi(monetaDaUnCentesimo);
        } catch (PortaMonetePienoException | MonetaNullException exc) {
            System.out.println(exc.getMessage());
        }

        try {
            // Aggiungiamo l'undicesima moneta (dovremmo ottenere un errore e la
            // moneta non sarà aggiunta)
            portaMonete.aggiungi(monetaDaUnEuro);
        } catch (PortaMonetePienoException | MonetaNullException exc) {
            System.out.println(exc.getMessage());
        }

        // Valutiamo lo stato del portamonete.
        portaMonete.stato();

        try {
            // preleviamo 20 centesimi
            portaMonete.preleva(monetaDaVentiCentesimi);
        } catch (MonetaNonTrovataException exc) {
            System.out.println(exc.getMessage());
        }

        try {
            // Aggiungiamo l'undicesima moneta (dovremmo ottenere un errore e la
            // moneta non sarà aggiunta)
            portaMonete.aggiungi(monetaDaUnEuro);
        } catch (PortaMonetePienoException | MonetaNullException exc) {
```

```
        System.out.println(exc.getMessage());
    }

    portaMonete.stato();

    try {
        //Cerchiamo una moneta non esistente (dovremmo ottenere una stampa
        //di errore)
        portaMonete.preleva(new Moneta(7));
    } catch (MonetaNonTrovataException exc) {
        System.out.println(exc.getMessage());
    }

    try {
        //Proviamo ad aggiungere null
        portaMonete.aggiungi(null);
    } catch (PortaMonetePienoException | MonetaNullException exc) {
        System.out.println(exc.getMessage());
    }

    try {
        //Proviamo a prelevare una moneta null
        portaMonete.preleva(null);
    } catch (MonetaNonTrovataException | MonetaNullException exc) {
        System.out.println(exc.getMessage());
    }

    //Testiamo il passaggio di null al costruttore del porta monete
    PortaMonete portaMoneteEccezione = new PortaMonete(null);
    portaMonete.stato();
}
}
```

Il cui output sarà:

```
Creata una moneta da 20 centesimi di EURO
Creata una moneta da 1 centesimo di EURO
Creata una moneta da 1 EURO
Creata una moneta da 2 centesimi di EURO
Creata una moneta da 5 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 50 centesimi di EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 2 EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 5 centesimi di EURO
Sono state inserite solo le prime 10 monete!
Creata una moneta da 2 centesimi di EURO
Creata una moneta da 5 centesimi di EURO
Creata una moneta da 1 EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 50 centesimi di EURO
Creata una moneta da 10 centesimi di EURO
Creata una moneta da 1 EURO
```

```
Creata una moneta da 2 EURO
Il portamonete contiene:
Una moneta da 2 centesimi di EURO
Una moneta da 5 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 10 centesimi di EURO
Una moneta da 50 centesimi di EURO
Una moneta da 10 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 2 EURO
Proviamo ad aggiungere una moneta da 20 centesimi di EURO
E' stata aggiunta una moneta da 20 centesimi di EURO
Proviamo ad aggiungere una moneta da 1 centesimo di EURO
E' stata aggiunta una moneta da 1 centesimo di EURO
Proviamo ad aggiungere una moneta da 1 EURO
Portamonete pieno! La moneta moneta da 1 EURO non è stata aggiunta...
Il portamonete contiene:
Una moneta da 2 centesimi di EURO
Una moneta da 5 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 10 centesimi di EURO
Una moneta da 50 centesimi di EURO
Una moneta da 10 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 2 EURO
Una moneta da 20 centesimi di EURO
Una moneta da 1 centesimo di EURO
Proviamo a prelevare una moneta da 20 centesimi di EURO
Una moneta da 20 centesimi di EURO prelevata
Proviamo ad aggiungere una moneta da 1 EURO
E' stata aggiunta una moneta da 1 EURO
Il portamonete contiene:
Una moneta da 2 centesimi di EURO
Una moneta da 5 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 10 centesimi di EURO
Una moneta da 50 centesimi di EURO
Una moneta da 10 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 2 EURO
Una moneta da 1 EURO
Una moneta da 1 centesimo di EURO
Creata una moneta da 7 centesimi di EURO
Proviamo a prelevare una moneta da 7 centesimi di EURO
Moneta non trovata!
La moneta passata era null
La moneta passata era null
Il portamonete è stato creato vuoto
Il portamonete contiene:
Una moneta da 2 centesimi di EURO
Una moneta da 5 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 10 centesimi di EURO
Una moneta da 50 centesimi di EURO
```

```
Una moneta da 10 centesimi di EURO
Una moneta da 1 EURO
Una moneta da 2 EURO
Una moneta da 1 EURO
Una moneta da 1 centesimo di EURO
```

Soluzione 8.m)

Potremmo modificare il listato di `PortaMonete` nel seguente modo:

```
public class PortaMonete {

    private static final int DIMENSIONE = 10;

    private final Moneta[] monete = new Moneta[DIMENSIONE];

    public PortaMonete(int... valori) {
        assert monete.length == DIMENSIONE;
        try {
            int numeroMonete = valori.length;
            for (int i = 0; i < numeroMonete; i++) {
                if (i >= DIMENSIONE) {
                    throw new PortaMonetePienoException(
                        "Sono state inserite solo le prime + "
                        + DIMENSIONE + " monete!");
                }
                monete[i] = new Moneta(valori[i]);
            }
        } catch (PortaMonetePienoException | NullPointerException exc) {
        } catch (PortaMonetePienoException exc) {
            System.out.println(exc.getMessage());
        } catch (NullPointerException exc) {
            System.out.println("Il portamonete è stato creato vuoto");
        }
        assert monete.length == DIMENSIONE;
    }
    //...
}
```

Si noti che abbiamo definito la costante `DIMENSIONE` impostata a 10. Poi come asserzioni abbiamo usato una pre-condizione e una post-condizione. In entrambi i casi abbiamo asserito lo stesso concetto: la lunghezza dell'array di monete è uguale al valore di `DIMENSIONE`. Si noti anche come con queste semplici asserzioni stiamo rinforzando la logica del costruttore, è come se stessi dicendo “qualsiasi cosa succeda, il valore della lunghezza dell'array non può cambiare”. In questo modo il nostro codice rimarrà coerente con questa asserzione attraverso tutte le modifiche che verranno fatte in seguito.

Soluzione 8.n)

Solo l'affermazione numero 3 è corretta.

Soluzione 8.o)

Sono corrette le affermazioni numero 3 e numero 6. Infatti la clausola `throws` e il comando `throw` possono essere utilizzati per ogni tipo di eccezione, e questo esclude che siano corrette le affermazioni 1, 2, 4 e 5. L'affermazione 7 è invece scorretta perché il metodo che invoca un metodo che dichiara la clausola `throws`, potrebbe a sua volta dichiarare la clausola `throws`.

Soluzione 8.p)

Nessuna delle affermazioni è corretta.

Soluzione 8.q)

Solo la prima affermazione è corretta.

Soluzione 8.r)

Tutti gli snippet sono validi perché, nella clausola `throws`, viene dichiarata la `ArrayIndexOutOfBoundsException` che è un'eccezione `unchecked`, e che quindi non è obbligatoria esplicitamente. Si noti che la risposta 2 fa scattare una `ArithmeticException`, la gestisce e rilancia una `ArrayIndexOutOfBoundsException`.

Soluzione 8.s)

L'unica affermazione corretta è la 3. La sua correttezza esclude la correttezza delle affermazioni 4, 5 e 6. La 1 e la 2 sono false perché `NullPointerException` è comunque una classe.

Soluzione 8.t)

Una semplice implementazione potrebbe essere la seguente:

```
public class PortaAutomatica implements AutoCloseable {
    public void close() {
        System.out.println("La porta si sta chiudendo");
    }

    public void open() {
        System.out.println("La porta si sta aprendo");
    }
}
```

Soluzione 8.u)

La soluzione potrebbe essere semplice come la seguente:

```
public class Esercizio8U {  
    public static void main(String args[]) {  
        try (PortaAutomatica portaAutomatica = new PortaAutomatica();) {  
            portaAutomatica.open();  
        }  
    }  
}
```

il cui output, una volta eseguita, sarà:

```
La porta si sta aprendo  
La porta si sta chiudendo
```

Soluzione 8.v)

La soluzione con la gestione delle eccezioni è indubbiamente più semplice ed elegante:

```
public class SommaIntero extends SommaNumero {  
    @Override  
    public Integer somma(Number n1, Number n2) {  
        Integer risultato = null;  
        try {  
            risultato = (Integer)n1 + (Integer)n2;  
        } catch (NullPointerException e) {  
            System.out.println("Impossibile sommare un operando null");  
        } catch (ClassCastException e) {  
            System.out.println("Passa solo variabili di tipo intero");  
        }  
        return risultato;  
    }  
}
```

Soluzione 8.z)

Con la seguente classe di test:

```
public class Esercizio8Z {  
    private static final String PREFISSO_FRASE = "Il risultato è ";  
    public static void main(String args[]) {  
        SommaIntero sommaIntero = new SommaIntero();  
        System.out.println(PREFISSO_FRASE + sommaIntero.somma(1.0, 1.0));  
        System.out.println(PREFISSO_FRASE + sommaIntero.somma(1, null));  
        System.out.println(PREFISSO_FRASE + sommaIntero.somma(1, 25));  
    }  
}
```

Possiamo verificare i casi possibili che abbiamo previsto.

Soluzioni degli esercizi riassuntivi Parte II

Un esempio guidato alla programmazione ad oggetti

Soluzione 8.aa)

Potremmo implementare la classe `Punto` nel seguente modo:

```
public class Punto {
    private int x;
    private int y;

    public Punto (int x, int y) {
        setX(x);
        setY(y);
    }

    private void setY(int y) {
        this.y = y;
    }

    public int getY() {
        return y;
    }

    private void setX(int x) {
        this.x = x;
    }

    public int getX() {
        return x;
    }

    public String toString() {
        return "("+x+", " + y + ")";
    }
}
```

Segue la classe `Righello` (notare che abbiamo scelto di dichiarare il metodo statico, visto che dipende solo dall'input e non da eventuali variabili d'istanza):

```
import static java.lang.Math.*;

public class Righello {
    public static double calcolaDistanza(Punto p1, Punto p2) {
        return sqrt(pow(p1.getX()-p2.getX(), 2) + pow(p1.getY()-p2.getY(), 2));
    }
}
```

Infine segue la classe `Esercizio8AA` che istanzia due punti, ne calcola la distanza e utilizza un'asserzione:

```
public class Esercizio8AA {
    private final static double RISULTATO_CORRETTO = 2;
    public static void main(String args[]) {
        Punto p1 = new Punto(0,0);
        Punto p2 = new Punto(2,0);
    }
}
```

```
double distanza = Righello.calcolaDistanza(p1,p2);
System.out.println("Distanza tra i punti: " + p1 + " e " + p2 + " = "
    + distanza);
assert distanza == RISULTATO_CORRETTO :
    "Errore! Il risultato dovrebbe essere " + RISULTATO_CORRETTO;
System.out.println("Distanza corretta!");
    }
}
```

Soluzione 8.bb)

Il listato della classe `Esercizio8BB` è il seguente:

```
public class Esercizio8BB {
    public static void main(String args[]) {
        if (args.length != 4) {
            System.out.println("Per calcolare la distanza tra due punti,"
                + " inserire le 4 coordinate");
        }
        try {
            Punto p1 = new Punto(Integer.parseInt(args[0]),
                Integer.parseInt(args[1]));
            Punto p2 = new Punto(Integer.parseInt(args[2]),
                Integer.parseInt(args[3]));
            System.out.println("Distanza tra i punti: "+ p1 + " e "+ p2 + " = "
                + Righello.calcolaDistanza(p1,p2));
        }
        catch (NumberFormatException exc) {
            System.out.println("Le coordinate devono essere di tipo intero!"
                + " Errore: "+ exc.getMessage());
        }
    }
}
```

Lanciando l'applicazione con il seguente comando:

```
java Esercizio8BB 2 1 1 2
```

otteremo l'output:

```
Distanza tra i punti: (2,1) e (1,2) = 1.4142135623730951
```

Soluzione 8.cc)

Segue il codice della classe `Esercizio8CC`:

```
import java.util.Scanner;
import java.util.InputMismatchException;

public class Esercizio8CC {
    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.println("Inserisci coordinata x per il punto 1");
            int x1 = scanner.nextInt();
            System.out.println("Inserisci coordinata y per il punto 1");
            int y1 = scanner.nextInt();
            Punto p1 = new Punto(x1, y1);
            System.out.println("Creato punto:"+ p1);
        }
    }
}
```

```
        System.out.println("Inserisci coordinata x per il punto 2");
        int x2 = scanner.nextInt();
        System.out.println("Inserisci coordinata y per il punto 2");
        int y2 = scanner.nextInt();
        Punto p2 = new Punto(x2, y2);
        System.out.println("Creato punto:" + p2);
        System.out.println("Distanza tra i punti: " + p1 + " e " + p2 + " = "
            + Righello.calcolaDistanza(p1,p2));
    } catch (InputMismatchException exc) {
        System.out.println("Le coordinate devono essere di tipo intero!");
    }
}
}
```

Soluzione 8.dd)

Segue il codice della classe `Esercizio8DD`, refactoring della classe `Esercizio8CC`:

```
import java.util.Scanner;
import java.util.InputMismatchException;

public class Esercizio8DD {
    private Scanner scanner;

    public Esercizio8DD () {
        scanner = new Scanner(System.in);
    }

    public void start() {
        try {
            Punto p1 = getPunto("1");
            Punto p2 = getPunto("2");
            stampaDistanza(p1, p2) ;
        }
        catch (InputMismatchException exc) {
            System.out.println("Le coordinate devono essere di tipo intero!");
        }
    }

    private int getCoordinata(String nomeCoordinata, String nomePunto) {
        System.out.println("Inserisci coordinata " + nomeCoordinata
            + " per il punto " + nomePunto);
        return scanner.nextInt();
    }

    private Punto getPunto(String nomePunto) {
        int x = getCoordinata("x", nomePunto);
        int y = getCoordinata("y", nomePunto);
        Punto p = new Punto(x,y);
        System.out.println("Creato punto: " + p);
        return p;
    }

    public void stampaDistanza(Punto p1, Punto p2) {
        System.out.println("Distanza tra i punti: " + p1 + " e " + p2 + " = "
            + Righello.calcolaDistanza(p1,p2));
    }
}
```

```
public static void main(String args[]) {  
    Esercizio8DD esercizio8DD = new Esercizio8DD();  
    esercizio8DD.start();  
}  
}
```

Soluzione 8.ee)

Segue una possibile implementazione della classe Esercizio8EE:

```
import java.util.Scanner;  
import java.util.InputMismatchException;  
  
public class Esercizio8EE {  
    private Scanner scanner;  
  
    public Esercizio8EE () {  
        scanner = new Scanner(System.in);  
    }  
  
    public void start() {  
        Punto p1 = getPunto("1");  
        Punto p2 = getPunto("2");  
        stampaDistanza(p1, p2) ;  
    }  
  
    private int getCoordinata(String nomeCoordinata, String nomePunto) {  
        System.out.println("Inserisci coordinata "+ nomeCoordinata  
            + " per il punto "+ nomePunto);  
        while (scanner.hasNext()) {  
            if (scanner.hasNextInt()) {  
                return scanner.nextInt();  
            } else {  
                System.out.println("Coordinata non valida " + scanner.next()  
                    + "! Le coordinate devono essere di tipo intero! "  
                    + "Prego reinserire");  
            }  
        }  
        return -1;  
    }  
  
    private Punto getPunto(String nomePunto) {  
        int x = getCoordinata("x", nomePunto);  
        int y = getCoordinata("y", nomePunto);  
        Punto p = new Punto(x,y);  
        System.out.println("Creato punto:"+ p);  
        return p;  
    }  
  
    public void stampaDistanza(Punto p1, Punto p2) {  
        System.out.println("Distanza tra i punti: "+ p1 +" e "+ p2 +" = "  
            + Righello.calcolaDistanza(p1,p2));  
    }  
  
    public static void main(String args[]) {  
        Esercizio8EE esercizio8EE = new Esercizio8EE();  
        esercizio8EE.start();  
    }  
}
```

```
}
```

Soluzione 8.ff)

Il listato della nostra classe `Segmento` potrebbe essere il seguente:

```
public class Segmento {
    private Punto estremo1;
    private Punto estremo2;

    private double lunghezza;

    public Segmento(Punto estremo1, Punto estremo2 ) {
        this.estremo1 = estremo1;
        this.estremo2 = estremo2;
        setLunghezza();
    }

    public Punto getEstremo1() {
        return estremo1;
    }

    public Punto getEstremo2() {
        return estremo2;
    }

    private void setLunghezza() {
        this.lunghezza = Righello.calcolaDistanza(estremo1, estremo2);
    }

    public String toString() {
        return "Segmento da P1"+estremo1+" a P2"+estremo2 + " con lunghezza = "
            + lunghezza;
    }
}
```

Infine abbiamo implementato la classe `Esercizio8FF` come segue:

```
import java.util.*;

public class Esercizio8FF {
    private Scanner scanner;

    public Esercizio8FF () {
        scanner = new Scanner(System.in);
    }

    public void start() {
        System.out.println("Definiamo un segmento specificando i due estremi:");
        Punto p1 = getPunto("1");
        Punto p2 = getPunto("2");
        stampa(new Segmento(p1, p2) );
    }

    private int getCoordinata(String nomeCoordinata, String nomePunto) {
        System.out.println("Inserisci coordinata "+ nomeCoordinata
            + " per l'estremo "+ nomePunto);
        while (scanner.hasNext()) {
            if (scanner.hasNextInt()) {
                return scanner.nextInt();
            }
        }
    }
}
```

```
        } else {
            System.out.println("Coordinata non valida " + scanner.next()
                + "! Le coordinate devono essere di tipo intero! "
                + "Prego reinserire");
        }
    }
    return -1;
}

private Punto getPunto(String nomePunto) {
    int x = getCoordinata("x", nomePunto);
    int y = getCoordinata("y", nomePunto);
    Punto p = new Punto(x,y);
    System.out.println("Creato estremo:" + p);
    return p;
}

public void stampa(Segmento segmento) {
    System.out.println(segmento);
}

public static void main(String args[]) {
    Esercizio8FF esercizio8FF = new Esercizio8FF();
    esercizio8FF.start();
}
}
```

Soluzione 8.gg)

Abbiamo modificato la classe Punto nel seguente modo (modifiche in grassetto):

```
public class Punto {
    private String nome;
    private int x;
    private int y;

    public Punto (String nome, int x, int y) {
        setNome(nome);
        setX(x);
        setY(y);
    }

    public Punto (int x, int y) {
        this("", x, y);
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }

    private void setY(int y) {
        this.y = y;
    }
}
```

```
public int getY() {
    return y;
}

private void setX(int x) {
    this.x = x;
}

public int getX() {
    return x;
}

public String toString() {
    return nome + "(" + x + ", " + y + ")";
}
}
```

Infine abbiamo implementato la classe `Esercizio8GG` come segue:

```
import java.util.*;
import javax.swing.*;

public class Esercizio8GG {
    private Scanner scanner;
    private CartesianPlane cartesianPlane;

    public Esercizio8GG () {
        scanner = new Scanner(System.in);
    }

    public void start() {
        SwingUtilities.invokeLater(() -> cartesianPlane = new CartesianPlane());
        Punto p1 = getPunto("P1");
        cartesianPlane.drawPoint(p1);
    }

    private int getCoordinata(String nomeCoordinata, String nomePunto) {
        System.out.println("Inserisci coordinata " + nomeCoordinata
            + " per il punto " + nomePunto);
        while (scanner.hasNext()) {
            if (scanner.hasNextInt()) {
                return scanner.nextInt();
            } else {
                System.out.println("Coordinata non valida " + scanner.next()
                    + "! Le coordinate devono essere di tipo intero! "
                    + "Prego reinserire");
            }
        }
        return -1;
    }

    private Punto getPunto(String nomePunto) {
        int x = getCoordinata("x", nomePunto);
        int y = getCoordinata("y", nomePunto);
        Punto p = new Punto(nomePunto, x, y);
        System.out.println("Creato punto: " + p);
        return p;
    }

    public static void main(String args[]) {
        Esercizio8GG esercizio8GG = new Esercizio8GG();
        esercizio8GG.start();
    }
}
```

```
}  
}
```

Soluzione 8.hh)

Il listato della classe Esercizio8HH è il seguente:

```
import java.util.*;  
import javax.swing.*;  
  
public class Esercizio8HH {  
    private Scanner scanner;  
    private CartesianPlane cartesianPlane;  
    private static int contatore = 1;  
  
    public Esercizio8HH () {  
        scanner = new Scanner(System.in);  
    }  
  
    public void start() {  
        SwingUtilities.invokeLater(() -> cartesianPlane = new CartesianPlane());  
        while(true) {  
            Punto p1 = getPunto("P"+(contatore++));  
            cartesianPlane.drawPoint(p1);  
        }  
    }  
  
    private int getCoordinata(String nomeCoordinata, String nomePunto) {  
        System.out.println("Inserisci coordinata "+ nomeCoordinata  
            + " per il punto "+ nomePunto);  
        while (scanner.hasNext()) {  
            if (scanner.hasNextInt()) {  
                return scanner.nextInt();  
            } else {  
                System.out.println("Coordinata non valida " + scanner.next()  
                    + "! Le coordinate devono essere di tipo intero! "  
                    + "Prego reinserire");  
            }  
        }  
        return -1;  
    }  
  
    private Punto getPunto(String nomePunto) {  
        int x = getCoordinata("x", nomePunto);  
        int y = getCoordinata("y", nomePunto);  
        Punto p = new Punto(nomePunto, x,y);  
        System.out.println("Creato punto:"+ p);  
        return p;  
    }  
  
    public static void main(String args[]) {  
        Esercizio8HH esercizio8HH = new Esercizio8HH();  
        esercizio8HH.start();  
    }  
}
```


Soluzione 8.ii)

Segue la classe `Esercizio8II`:

```
import java.util.*;
import javax.swing.*;

public class Esercizio8II {
    private Scanner scanner;
    private CartesianPlane cartesianPlane;
    private static int contatore = 1;

    public Esercizio8II () {
        scanner = new Scanner(System.in);
    }

    public void start() {
        SwingUtilities.invokeLater(() -> cartesianPlane = new CartesianPlane());
        while(true) {
            System.out.println("Definiamo un segmento");
            Punto p1 = getPunto("P" + (contatore++));
            Punto p2 = getPunto("P" + (contatore++));
            Segmento s1 = new Segmento(p1, p2);
            cartesianPlane.drawSegment(s1);
        }
    }

    private int getCoordinata(String nomeCoordinata, String nomePunto) {
        System.out.println("Inserisci coordinata " + nomeCoordinata
            + " per il punto " + nomePunto);
        while (scanner.hasNext()) {
            if (scanner.hasNextInt()) {
                return scanner.nextInt();
            } else {
                System.out.println("Coordinata non valida + scanner.next() "
                    + "!, Le coordinate devono essere di tipo intero! "
                    + "Prego reinserire");
            }
        }
        return -1;
    }

    private Punto getPunto(String nomePunto) {
        int x = getCoordinata("x", nomePunto);
        int y = getCoordinata("y", nomePunto);
        Punto p = new Punto(nomePunto, x, y);
        System.out.println("Creato punto: " + p);
        return p;
    }

    public static void main(String args[]) {
        Esercizio8II esercizio8II = new Esercizio8II();
        esercizio8II.start();
    }
}
```

Soluzione 8.jj)

Abbiamo implementato la classe `Polilinea` nella seguente maniera:

```
import java.util.*;

public class Polilinea {
    private List<Segmento> segmentiList;

    public Polilinea (Segmento... segmenti) throws PolilineaNonValidaException {
        this.segmentiList = new ArrayList(Arrays.asList(segmenti));
        if (segmenti.length < 2 || !controllaSegmentiConsecutivi()) {
            throw new PolilineaNonValidaException(segmentiList);
        }
    }

    public void aggiungiSegmento(Segmento nuovoSegmento)
        throws PolilineaNonValidaException {
        Segmento ultimoSegmento = segmentiList.get(segmentiList.size()-1);
        Punto estremo2UltimoSegmento = ultimoSegmento.getEstremo2();
        Punto estremo1NuovoSegmento = nuovoSegmento.getEstremo1();
        segmentiList.add(nuovoSegmento);
        if (!uguali(estremo1NuovoSegmento, estremo2UltimoSegmento)) {
            throw new PolilineaNonValidaException(segmentiList);
        }
    }

    private boolean controllaSegmentiConsecutivi() {
        int segmentiListSize = segmentiList.size();
        for (int i = 0; i < segmentiListSize-1; ) {
            Punto secondoEstremoSegmentoPrecedente =
                segmentiList.get(i++).getEstremo2();
            Punto primoEstremoSegmentoSuccessivo =
                segmentiList.get(i).getEstremo1();
            if (!uguali(secondoEstremoSegmentoPrecedente,
                primoEstremoSegmentoSuccessivo)) {
                return false;
            }
        }
        return true;
    }

    private boolean uguali(Punto p1, Punto p2) {
        return p1.getX()==p2.getX() && p1.getY()== p2.getY();
    }

    public String toString() {
        String descrizione ="Polilinea formata da:\n";
        for (Segmento segmento : segmentiList) {
            descrizione += segmento +"\n";
        }
        return descrizione;
    }
}
```

Dove la classe `PolilineaNonValidaException` è implementata come segue:

```
import java.util.List;

public class PolilineaNonValidaException extends Exception {
    public PolilineaNonValidaException (List<Segmento> segmenti) {
        super(segmenti.size() < 2 ? "Una polilinea deve essere costituita da"
            + " almeno 2 segmenti"
```

```
        : "Questi segmenti " + segmenti + " non costituiscono una polilinea");
    }

    public String toString() {
        return "Polilinea non valida:\n" + getMessage();
    }
}
```

Infine l'implementazione della classe TestPolilinea non è semplice:

```
public class TestPolilinea {
    private static final String TEST_OK = "TEST OK:\n";
    private static final String TEST_KO = "TEST FALLITO: ";

    public static void main(String args[]) {
        testPolilineaCorretta();
        testPolilineaCorrettaConQuattroPunti();
        testPolilineaSenzaSegmenti();
        testPolilineaConUnSoloSegmento();
        testPolilineaConDueSegmentiNonConsecutiviNelCostruttore();
        testPolilineaConTreSegmentiNonConsecutiviNelCostruttore();
        testPolilineaCorrettaAggiungendoUnSegmentoConsecutivo();
        testPolilineaScorrettaAggiungendoUnSegmentoNonConsecutivo();
    }

    public static void testPolilineaCorretta() {
        try {
            System.out.println("testPolilineaCorretta:");
            Punto p1 = new Punto(0,0);
            Punto p2 = new Punto(1,1);
            Segmento s1 = new Segmento(p1, p2);
            Punto p3 = new Punto(1,2);
            Segmento s2 = new Segmento(p2, p3);
            Polilinea poll = new Polilinea(s1, s2);
            System.out.println(TEST_OK + poll);
        } catch (Exception exc) {
            assert false : TEST_KO + exc.getMessage();
        }
    }

    public static void testPolilineaCorrettaConQuattroPunti() {
        try {
            System.out.println("testPolilineaCorrettaConQuattroPunti:");
            Punto p1 = new Punto(0,0);
            Punto p2 = new Punto(1,1);
            Segmento s1 = new Segmento(p1, p2);
            Punto p3 = new Punto(1,1);
            Punto p4 = new Punto(2,3);
            Segmento s2 = new Segmento(p3, p4);
            Polilinea poll = new Polilinea(s1, s2);
            System.out.println(TEST_OK + poll);
        } catch (Exception exc) {
            assert false : TEST_KO + exc.getMessage();
        }
    }

    public static void testPolilineaSenzaSegmenti() {
        try {
            System.out.println("testPolilineaSenzaSegmenti:");
            Polilinea poll = new Polilinea();
            System.out.println(TEST_KO + poll);
            assert false : "È stata creata un polilinea senza segmenti!";
        }
    }
}
```

```
        } catch (Exception exc) {
            System.out.println(TEST_OK + exc.getMessage());
            System.out.println();
        }
    }

    public static void testPolilineaConUnSoloSegmento() {
        try {
            System.out.println("testPolilineaConUnSoloSegmento:");
            Punto p1 = new Punto(0,0);
            Punto p2 = new Punto(1,1);
            Segmento s1 = new Segmento(p1, p2);
            Polilinea poll = new Polilinea(s1);
            System.out.println(TEST_KO + poll);
            assert false : "È stata creata un polilinea con un solo segmento!";
        } catch (Exception exc) {
            System.out.println(TEST_OK + exc.getMessage());
            System.out.println();
        }
    }

    public static void testPolilineaConDueSegmentiNonConsecutiviNelCostruttore()
    {
        try {
            System.out.println(
                "testPolilineaConDueSegmentiNonConsecutiviNelCostruttore:");
            Punto p1 = new Punto(0,0);
            Punto p2 = new Punto(1,1);
            Segmento s1 = new Segmento(p1, p2);
            Punto p3 = new Punto(1,2);
            Punto p4 = new Punto(2,2);
            Segmento s2 = new Segmento(p3, p4);
            Polilinea poll = new Polilinea(s1, s2);
            System.out.println(TEST_KO+ poll);
            assert false : "È stata creata un polilinea con due segmenti " +
                "non consecutivi (nel costruttore)!";
        } catch (Exception exc) {
            System.out.println(TEST_OK + exc.getMessage());
            System.out.println();
        }
    }

    public static void testPolilineaConTreSegmentiNonConsecutiviNelCostruttore()
    {
        try {
            System.out.println(
                "testPolilineaConTreSegmentiNonConsecutiviNelCostruttore:");
            Punto p1 = new Punto(0,0);
            Punto p2 = new Punto(1,1);
            Segmento s1 = new Segmento(p1, p2);
            Punto p3 = new Punto(1,1);
            Punto p4 = new Punto(2,2);
            Segmento s2 = new Segmento(p3, p4);
            Punto p5 = new Punto(2,3);
            Punto p6 = new Punto(3,2);
            Segmento s3 = new Segmento(p5, p6);
            Polilinea poll = new Polilinea(s1, s2, s3);
            System.out.println(TEST_KO+ poll);
            assert false :
                "È stata creata un polilinea con tre segmenti "
```

```

        + "non consecutivi (nel costruttore)!!";
    } catch (Exception exc) {
        System.out.println(TEST_OK + exc.getMessage());
        System.out.println();
    }
}

public static void testPolilineaCorrettaAggiungendoUnSegmentoConsecutivo() {
    try {
        System.out.println(
            "testPolilineaCorrettaAggiungendoUnSegmentoConsecutivo:");
        Punto p1 = new Punto(0,0);
        Punto p2 = new Punto(1,1);
        Segmento s1 = new Segmento(p1, p2);
        Punto p3 = new Punto(1,2);
        Segmento s2 = new Segmento(p2, p3);
        Polilinea pol1 = new Polilinea(s1, s2);
        Punto p4 = new Punto(3,2);
        Segmento s3 = new Segmento(p3, p4);
        pol1.aggiungiSegmento(s3);
        System.out.println(TEST_OK+ pol1);
    } catch (Exception exc) {
        assert false : TEST_KO + exc.getMessage();
    }
}

public static void
    testPolilineaScorrettaAggiungendoUnSegmentoNonConsecutivo() {
    try {
        System.out.println(
            "testPolilineaScorrettaAggiungendoUnSegmentoNonConsecutivo:");
        Punto p1 = new Punto(0,0);
        Punto p2 = new Punto(1,1);
        Segmento s1 = new Segmento(p1, p2);
        Punto p3 = new Punto(1,2);
        Segmento s2 = new Segmento(p2, p3);
        Polilinea pol1 = new Polilinea(s1, s2);
        Punto p4 = new Punto(3,2);
        Punto p5 = new Punto(3,3);
        Segmento s3 = new Segmento(p4, p5);
        pol1.aggiungiSegmento(s3);
        System.out.println(TEST_KO+ pol1);
        assert false : "È stata creata un polilinea con un segment"
            + " non consecutivo!";
    } catch (Exception exc) {
        System.out.println(TEST_OK + exc.getMessage());
    }
}
}

```

Soluzione 8.kk)

In realtà gli errori evidenziati dal nostro test, erano più che altro dovuti ad errori del test stesso. Questo sta a sottolineare che possiamo creare bachi anche nei test. Quindi bisogna fare molta attenzione. Infatti nel `testDammiDistanzaConNull` è corretto che venga lanciata una `NullPointerException` (anche se sarebbe il caso creare un'eccezione personalizzata per gestire

questo problema), mentre nel metodo `testDammiDistanzaDaUnPunto3D` avevamo previsto che il risultato della distanza sarebbe stato `-1`, ma ovviamente il valore di una distanza non può coincidere con un numero negativo. In questo caso abbiamo risolto cambiando il segno, visto che in fondo avevamo deciso di permettere questo tipo di operazione (avremmo anche potuto implementare una soluzione che lanciasse un'altra eccezione personalizzata visto che avremmo potuto supporre che non è possibile eseguire il calcolo della distanza tra un punto che si trova su un riferimento a due coordinate, da un punto che si trova su un riferimento a tre coordinate). In qualsiasi caso riportiamo di seguito la nuova classe `TestPunto`, con le modifiche in grassetto.

```
import org.junit.Assert;
import org.junit.Test;

public class TestPunto{
    @Test
    public void testDammiDistanzaSullAscissa() {
        Punto p1 = new Punto(1,1);
        Punto p2 = new Punto(1,2);
        double distanza = p1.dammiDistanza(p2);
        Assert.assertTrue(distanza == 1);
    }
    @Test (expected = NullPointerException.class)
    public void testDammiDistanzaConNull() {
        Punto p1 = new Punto(1,1);
        Punto p2 = null;
        double distanza = p1.dammiDistanza(p2);
        //Assert.assertTrue(distanza == -1);
    }
    @Test
    public void testDammiDistanzaDalloStessoPunto() {
        Punto p1 = new Punto(1,1);
        Punto p2 = new Punto(1,1);
        double distanza = p1.dammiDistanza(p2);
        Assert.assertTrue(distanza == 0);
    }
    @Test
    public void testDammiDistanzaDaUnPunto3D() {
        Punto p1 = new Punto(1,1);
        Punto p2 = new Punto3D(1,2,2);
        double distanza = p1.dammiDistanza(p2);
        Assert.assertTrue(distanza == 1);
    }
}
```

Soluzione 8.II)

Segue l'implementazione della nuova classe `TestPolilinea` (tra le soluzioni troverete l'intero progetto da importare in Eclipse):

```
import org.junit.Assert;
import org.junit.Test;
import static org.junit.jupiter.api.Assertions.assertThrows;

public class TestPolilinea {
    private static final String TEST_OK = "TEST OK:\n";
    private static final String TEST_KO = "TEST FALLITO: ";

    // public static void main(String args[]) {
```

```
//      testPolilineaCorretta();
//      testPolilineaCorrettaConQuattroPunti();
//      testPolilineaSenzaSegmenti();
//      testPolilineaConUnSoloSegmento();
//      testPolilineaConDueSegmentiNonConsecutiviNelCostruttore();
//      testPolilineaConTreSegmentiNonConsecutiviNelCostruttore();
//      testPolilineaCorrettaAggiungendoUnSegmentoConsecutivo();
//      testPolilineaScorrettaAggiungendoUnSegmentoNonConsecutivo();
//  }

@Test
public void testPolilineaCorretta() {
    try {
        System.out.println("testPolilineaCorretta:");
        Punto p1 = new Punto(0, 0);
        Punto p2 = new Punto(1, 1);
        Segmento s1 = new Segmento(p1, p2);
        Punto p3 = new Punto(1, 2);
        Segmento s2 = new Segmento(p2, p3);
        Polilinea poll = new Polilinea(s1, s2);
        Assert.assertNotNull(poll);
    } catch (Exception exc) {
        exc.printStackTrace();
    }
}

@Test
public void testPolilineaCorrettaConQuattroPunti() {
    try {
        System.out.println("testPolilineaCorrettaConQuattroPunti:");
        Punto p1 = new Punto(0, 0);
        Punto p2 = new Punto(1, 1);
        Segmento s1 = new Segmento(p1, p2);
        Punto p3 = new Punto(1, 1);
        Punto p4 = new Punto(2, 3);
        Segmento s2 = new Segmento(p3, p4);
        Polilinea poll = new Polilinea(s1, s2);
        Assert.assertNotNull(poll);
    } catch (Exception exc) {
        exc.printStackTrace();
    }
}

@Test
public void testPolilineaSenzaSegmenti() {
    try {
        System.out.println("testPolilineaSenzaSegmenti:");
        Polilinea poll = new Polilinea();
        assert false : "È stata creata un polilinea senza segmenti!";
    } catch (Exception exc) {
        Assert.assertNotNull(exc);
        System.out.println(TEST_OK + exc.getMessage());
        System.out.println();
    }
}

@Test
public void testPolilineaConUnSoloSegmento() {
    try {
        System.out.println("testPolilineaConUnSoloSegmento:");
    }
}
```

```
Punto p1 = new Punto(0, 0);
Punto p2 = new Punto(1, 1);
Segmento s1 = new Segmento(p1, p2);
Polilinea poll = new Polilinea(s1);
System.out.println(TEST_KO + poll);
assert false : "È stata creata un polilinea con un solo segmento!";
} catch (Exception exc) {
    Assert.assertNotNull(exc);
    System.out.println(TEST_OK + exc.getMessage());
    System.out.println();
}
}

public void testPolilineaConDueSegmentiNonConsecutiviNelCostruttore() {
    try {
        System.out.println(
            "testPolilineaConDueSegmentiNonConsecutiviNelCostruttore:");
        Punto p1 = new Punto(0, 0);
        Punto p2 = new Punto(1, 1);
        Segmento s1 = new Segmento(p1, p2);
        Punto p3 = new Punto(1, 2);
        Punto p4 = new Punto(2, 2);
        Segmento s2 = new Segmento(p3, p4);
        Polilinea poll = new Polilinea(s1, s2);
        System.out.println(TEST_KO + poll);
        assert false : "È stata creata un polilinea con due segmenti non "
            + "consecutivi (nel costruttore)!";
    } catch (Exception exc) {
        Assert.assertNotNull(exc);
        System.out.println(TEST_OK + exc.getMessage());
        System.out.println();
    }
}

public void testPolilineaConTreSegmentiNonConsecutiviNelCostruttore() {
    try {
        System.out.println(
            "testPolilineaConTreSegmentiNonConsecutiviNelCostruttore:");
        Punto p1 = new Punto(0, 0);
        Punto p2 = new Punto(1, 1);
        Segmento s1 = new Segmento(p1, p2);
        Punto p3 = new Punto(1, 1);
        Punto p4 = new Punto(2, 2);
        Segmento s2 = new Segmento(p3, p4);
        Punto p5 = new Punto(2, 3);
        Punto p6 = new Punto(3, 2);
        Segmento s3 = new Segmento(p5, p6);
        Polilinea poll = new Polilinea(s1, s2, s3);
        System.out.println(TEST_KO + poll);
        assert false : "È stata creata un polilinea con tre segmenti non "
            + "consecutivi (nel costruttore)!";
    } catch (Exception exc) {
        Assert.assertNotNull(exc);
        System.out.println(TEST_OK + exc.getMessage());
        System.out.println();
    }
}

@Test
public void testPolilineaCorrettaAggiungendoUnSegmentoConsecutivo() {
```



```

        try {
            System.out.println(
                "testPolilineaCorrettaAggiungendoUnSegmentoConsecutivo:");
            Punto p1 = new Punto(0, 0);
            Punto p2 = new Punto(1, 1);
            Segmento s1 = new Segmento(p1, p2);
            Punto p3 = new Punto(1, 2);
            Segmento s2 = new Segmento(p2, p3);
            Polilinea pol1 = new Polilinea(s1, s2);
            Punto p4 = new Punto(3, 2);
            Segmento s3 = new Segmento(p3, p4);
            pol1.aggiungiSegmento(s3);
            System.out.println(TEST_OK + pol1);
        } catch (Exception exc) {
            exc.printStackTrace();
        }
    }

    @Test
    public void testPolilineaScorrettaAggiungendoUnSegmentoNonConsecutivo() {
        try {
            System.out.println(
                "testPolilineaScorrettaAggiungendoUnSegmentoNonConsecutivo:");
            Punto p1 = new Punto(0, 0);
            Punto p2 = new Punto(1, 1);
            Segmento s1 = new Segmento(p1, p2);
            Punto p3 = new Punto(1, 2);
            Segmento s2 = new Segmento(p2, p3);
            Polilinea pol1 = new Polilinea(s1, s2);
            Punto p4 = new Punto(3, 2);
            Punto p5 = new Punto(3, 3);
            Segmento s3 = new Segmento(p4, p5);
            pol1.aggiungiSegmento(s3);
            System.out.println(TEST_KO + pol1);
            assert false : "È stata creata un polilinea con un segmento non "
                + "consecutivo!";
        } catch (Exception exc) {
            Assert.assertNotNull(exc);
            System.out.println(TEST_OK + exc.getMessage());
        }
    }
}

```

Soluzione 8.mm)

Segue l'implementazione della classe `Esercizio8MM` (modifiche rispetto alla classe `Esercizio8II` in grassetto):

```

import java.util.*;
import javax.swing.*;

public class Esercizio8MM {
    private Scanner scanner;
    private CartesianPlane cartesianPlane;
    private static int contatore = 1;

    public Esercizio8MM () {
        scanner = new Scanner(System.in);
    }
}

```

```
}

public void start() {
    SwingUtilities.invokeLater(() -> cartesianPlane = new CartesianPlane());
    System.out.println("Definiamo i primi due segmenti di una polilinea :");
    System.out.println("Definiamo il primo estremo del primo segmento della"
        + " polilinea:");
    Punto p1 = getPunto("P" + (contatore++));
    System.out.println("Definiamo il secondo estremo del primo segmento "
        + "della polilinea:");
    Punto p2 = getPunto("P" + (contatore++));
    Segmento s1 = new Segmento(p1, p2);
    cartesianPlane.drawSegment(s1);
    System.out.println("Definiamo il secondo estremo del secondo"
        + " segmento della"
        + " polilinea:");
    Punto p3 = getPunto("P" + (contatore++));
    Segmento s2 = new Segmento(p2, p3);
    cartesianPlane.drawSegment(s2);
    Punto ultimoEstremo = p3;
    while(true) {
        Punto prossimoEstremo = getPunto("P" + (contatore++));
        Segmento prossimoSegmento = new Segmento(ultimoEstremo,
            prossimoEstremo);
        cartesianPlane.drawSegment(prossimoSegmento);
        ultimoEstremo = prossimoEstremo;
    }
}

private int getCoordinata(String nomeCoordinata, String nomePunto) {
    System.out.println("Inserisci coordinata " + nomeCoordinata
        + " per il punto " + nomePunto);
    while (scanner.hasNext()) {
        if (scanner.hasNextInt()) {
            return scanner.nextInt();
        } else {
            System.out.println("Coordinata non valida " + scanner.next()
                + "! Le coordinate devono essere di tipo intero! "
                + "Prego reinserire");
        }
    }
    return -1;
}

private Punto getPunto(String nomePunto) {
    int x = getCoordinata("x", nomePunto);
    int y = getCoordinata("y", nomePunto);
    Punto p = new Punto(nomePunto, x, y);
    System.out.println("Creato punto:" + p);
    return p;
}

public static void main(String args[]) {
    Esercizio8MM esercizio8MM = new Esercizio8MM();
    esercizio8MM.start();
}
}
```

Soluzione 8.nn)

Creiamo dapprima una semplice eccezione per la creazione errata di un segmento in questo modo:

```
public class SegmentoNonValidoException extends Exception {  
  
    public String toString() {  
        return "Gli estremi di un segmento non possono coincidere!";  
    }  
}
```

Poi lanciamo questa eccezione nella classe `Segmento` nel caso ci sia coincidenza dei due estremi (modifiche in grassetto):

```
public class Segmento {  
    private Punto estremo1;  
    private Punto estremo2;  
  
    private double lunghezza;  
  
    public Segmento(Punto estremo1, Punto estremo2 ) throws  
                                                SegmentoNonValidoException {  
  
        this.estremo1 = estremo1;  
        this.estremo2 = estremo2;  
        setLunghezza();  
    }  
  
    public Punto getEstremo1() {  
        return estremo1;  
    }  
  
    public Punto getEstremo2() {  
        return estremo2;  
    }  
  
    private void setLunghezza() throws SegmentoNonValidoException {  
        this.lunghezza = Righello.calcolaDistanza(estremo1, estremo2);  
        if (lunghezza == 0) {  
            throw new SegmentoNonValidoException();  
        }  
    }  
  
    public String toString() {  
        return "Segmento da "+estremo1+" a "+estremo2 + " con lunghezza = "  
            + lunghezza;  
    }  
}
```

Infine facciamo evolvere la nostra classe `Esercizio8MM` nella classe `Esercizio8NN`, andando a creare un metodo ricorsivo `creaSegmento`, che gestisce l'eccezione richiamando sé stesso.

Notiamo anche che abbiamo fatto diventare la variabile temporanea `ultimoEstremo`, una variabile d'istanza (modifiche in grassetto):

```
import java.util.*;  
import javax.swing.*;  
  
public class Esercizio8NN {  
    private Scanner scanner;  
    private CartesianPlane cartesianPlane;  
    private Punto ultimoEstremo;  
    private static int contatore = 1;
```

```
public Esercizio8NN () {
    scanner = new Scanner(System.in);
}

public void start() {
    SwingUtilities.invokeLater(() -> cartesianPlane = new CartesianPlane());
    System.out.println("Definiamo i primi due segmenti di una polilinea:");
    System.out.println(
        "Definiamo il primo estremo del primo segmento della polilinea:");
    Punto p1 = getPunto("P" + (contatore++));
    System.out.println(
        "Definiamo il secondo estremo del primo segmento della polilinea:");
    creaSegmento(p1);
    /* Punto p2 = getPunto("P" + (contatore++));
    Segmento s1 = new Segmento(p1, p2);
    cartesianPlane.drawSegment(s1); */
    System.out.println(
        "Definiamo il secondo estremo del secondo segmento della"
        + " polilinea:");
    creaSegmento(ultimoEstremo);
    /* Punto p3 = getPunto("P" + (contatore++));
    Segmento s2 = new Segmento(p2, p3);
    cartesianPlane.drawSegment(s2);
    Punto ultimoEstremo = p3; */
    while(true) {
        /* Punto prossimoEstremo = getPunto("P" + (contatore++));
        Segmento prossimoSegmento = new Segmento(ultimoEstremo,
            prossimoEstremo);
        cartesianPlane.drawSegment(prossimoSegmento);
        ultimoEstremo = prossimoEstremo; */
        creaSegmento(ultimoEstremo);
    }
}

public void creaSegmento(Punto primoEstremo) {
    try {
        ultimoEstremo = getPunto("P" + (contatore++));
        Segmento segmento = new Segmento(primoEstremo, ultimoEstremo);
        cartesianPlane.drawSegment(segmento);
    } catch (SegmentoNonValidoException exc) {
        System.out.println(exc);
        contatore--;
        creaSegmento(primoEstremo);
    }
}

private int getCoordinata(String nomeCoordinata, String nomePunto) {
    System.out.println("Inserisci coordinata " + nomeCoordinata +
        " per il punto " + nomePunto);
    while (scanner.hasNext()) {
        if (scanner.hasNextInt()) {
            return scanner.nextInt();
        } else {
            System.out.println("Coordinata non valida " + scanner.next()
                + "! Le coordinate devono essere di tipo intero! "
                + "Prego reinserire");
        }
    }
    return -1;
}
```

```
}

private Punto getPunto(String nomePunto) {
    int x = getCoordinata("x", nomePunto);
    int y = getCoordinata("y", nomePunto);
    Punto p = new Punto(nomePunto, x, y);
    System.out.println("Creato punto:" + p);
    return p;
}

public static void main(String args[]) {
    Esercizio8NN esercizio8NN = new Esercizio8NN();
    esercizio8NN.start();
}
}
```

Soluzione 8.oo)

La soluzione potrebbe essere la seguente:

```
import java.util.*;

public class Esercizio800 {
    public static void main(String[] args) {
        Esercizio800 esercizio800 = new Esercizio800();
        esercizio800.start();
    }

    public void start() {
        try {
            Scanner scanner = new Scanner(System.in);
            System.out.println("Inserisci un valore in Kelvin");
            float kelvin = scanner.nextFloat();
            float celsius = kelvin - 273.15F;
            float fahrenheit = celsius * 9/5 + 32;
            System.out.println("Il valore di " + kelvin
                + " gradi Kelvin equivale a:\n"
                + celsius + " gradi Celsius\n"
                + fahrenheit + " gradi Fahrenheit");
        } catch (InputMismatchException exc) {
            System.out.println("Il valore deve essere numerico");
            start();
        }
    }
}
```

Soluzione 8.pp)

La soluzione potrebbe essere la seguente:

```
import java.util.*;
import java.math.*;

public class Esercizio8PP {
    public static void main(String[] args) {
        Esercizio8PP esercizio8PP = new Esercizio8PP();
    }
}
```

```
        esercizio8PP.start();
    }

    public void start() {
        try {
            Scanner scanner = new Scanner(System.in);
            System.out.println("Inserisci un valore in Kelvin");
            BigDecimal kelvin = scanner.nextBigDecimal();
            BigDecimal celsius = kelvin.subtract(BigDecimal.valueOf(273.15));
            BigDecimal fahrenheit = celsius.multiply(BigDecimal.valueOf(9)).
                divide(BigDecimal.valueOf(5)).add(BigDecimal.valueOf(32));
            System.out.println("Il valore di " + kelvin
                + " gradi Kelvin equivale a:\n"
                + celsius + " gradi Celsius\n"
                + fahrenheit + " gradi Fahrenheit");
        } catch (InputMismatchException exc) {
            System.out.println("Il valore deve essere numerico");
            start();
        }
    }
}
```

Soluzione 8.qq)

La soluzione potrebbe essere la seguente:

```
import java.util.*;
import java.math.*;

public class Esercizio8QQ {
    public static void main(String[] args) {
        Esercizio8QQ esercizio8QQ = new Esercizio8QQ();
        esercizio8QQ.start();
    }

    public void start() {
        try {
            System.out.println("Inserisci un valore in Kelvin (per esempio 12K) "
                + ", Celsius (25C) o Fahrenheit (451F)");
            Scanner scanner = new Scanner(System.in);
            String input = scanner.next();
            int lastCharIndex = input.length()-1;
            BigDecimal valore = new BigDecimal(
                input.substring(0, lastCharIndex));
            String scala = input.substring(lastCharIndex);
            switch(scala) {
                case "k":
                case "K":
                {
                    BigDecimal celsius = valore.subtract(
                        BigDecimal.valueOf(273.15));
                    BigDecimal fahrenheit =
                        celsius.multiply(BigDecimal.valueOf(9))
                            .divide(BigDecimal.valueOf(5), 2, RoundingMode.HALF_UP)
                            .add(BigDecimal.valueOf(32));
                    System.out.println("Il valore di " + valore
                        + " gradi Kelvin equivale a:\n"
                        + celsius + " gradi Celsius\n");
                }
            }
        }
    }
}
```

```
        + fahrenheit + " gradi Fahrenheit");
        break;
    }
    case "c":
    case "C":
    {
        BigDecimal kelvin = valore.add(BigDecimal.valueOf(273.15));
        BigDecimal fahrenheit =
            valore.multiply(BigDecimal.valueOf(9))
                .divide(BigDecimal.valueOf(5), 2, RoundingMode.HALF_UP)
                .add(BigDecimal.valueOf(32));
        System.out.println("Il valore di " + valore
            + " gradi Celsius equivale a:\n"
            + kelvin + " gradi Kelvin\n" + fahrenheit
            + " gradi Fahrenheit");
        break;
    }
    case "f":
    case "F":
    {
        BigDecimal celsius = valore.subtract(BigDecimal.valueOf(32))
            .multiply(BigDecimal.valueOf(5))
            .divide(BigDecimal.valueOf(9), 2, RoundingMode.HALF_UP);
        BigDecimal kelvin = celsius.add(BigDecimal.valueOf(273.15));
        System.out.println("Il valore di " + valore
            + " gradi Fahrenheit equivale a:\n"
            + celsius + " gradi Celsius\n" + kelvin
            + " gradi Kelvin");
        break;
    }
    default:
        System.out.println("Scala " + scala + " non valida. Usare"
            + " K per Kelvin, C per Celsius, F per Fahrenheit");
        start();
    }
}
}
catch (NumberFormatException exc) {
    System.out.println("Formato errato!");
    start();
}
}
```

Soluzione 8.rr)

La classe Convertitore potrebbe essere implementata nel seguente modo:

```
import java.math.*;

public class Convertitore {
    public static BigDecimal convertiKelvinInCelsius(BigDecimal kelvin) {
        return kelvin.subtract(BigDecimal.valueOf(273.15));
    }

    public static BigDecimal convertiKelvinInFahrenheit(BigDecimal kelvin) {
        BigDecimal celsius = convertiKelvinInCelsius(kelvin);
        return celsius.multiply(BigDecimal.valueOf(9))
            .divide(BigDecimal.valueOf(5), 2, RoundingMode.HALF_UP)
            .add(BigDecimal.valueOf(32));
    }
}
```

```
        .add(BigDecimal.valueOf(32));
    }

    public static BigDecimal convertiCelsiusInKelvin(BigDecimal celsius) {
        return celsius.add(BigDecimal.valueOf(273.15));
    }

    public static BigDecimal convertiCelsiusInFahrenheit(BigDecimal celsius) {
        return celsius.multiply(BigDecimal.valueOf(9))
            .divide(BigDecimal.valueOf(5), 2, RoundingMode.HALF_UP)
            .add(BigDecimal.valueOf(32));
    }

    public static BigDecimal convertiFahrenheitInKelvin(BigDecimal fahrenheit) {
        BigDecimal celsius = convertiFahrenheitInCelsius(fahrenheit);
        return celsius.add(BigDecimal.valueOf(273.15));
    }

    public static BigDecimal convertiFahrenheitInCelsius(BigDecimal fahrenheit){
        return fahrenheit.subtract(BigDecimal.valueOf(32))
            .multiply(BigDecimal.valueOf(5))
            .divide(BigDecimal.valueOf(9), 2, RoundingMode.HALF_UP);
    }
}
```

Quindi la classe `Esercizio8RR`, potrebbe essere implementata nel seguente modo:

```
import java.util.*;
import java.math.*;

public class Esercizio8RR {
    public static void main(String[] args) {
        Esercizio8RR esercizio8RR = new Esercizio8RR();
        esercizio8RR.start();
    }

    public void start() {
        try {
            System.out.println("Inserisci un valore in Kelvin (per esempio 12K)"
                + ", Celsius (25C) o Fahrenheit (451F)");
            Scanner scanner = new Scanner(System.in);
            String input = scanner.next();
            int lastCharIndex = input.length()-1;
            BigDecimal valore = new BigDecimal(
                input.substring(0, lastCharIndex));
            String scala = input.substring(lastCharIndex);
            switch(scala) {
                case "k":
                case "K": {
                    BigDecimal celsius =
                        Convertitore.convertiKelvinInCelsius(valore);
                    BigDecimal fahrenheit =
                        Convertitore.convertiKelvinInFahrenheit(valore);
                    System.out.println("Il valore di "+ valore
                        + " gradi Kelvin equivale a:\n"
                        + celsius + " gradi Celsius\n"
                        + fahrenheit + " gradi Fahrenheit");
                    break;
                }
                case "c":
                case "C": {
                    BigDecimal kelvin =
```



```
        Convertitore.convertiCelsiusInKelvin(valore);
        BigDecimal fahrenheit =
            Convertitore.convertiCelsiusInFahrenheit(valore);
        System.out.println("Il valore di "+ valore
            + " gradi Celsius equivale a:\n"
            + kelvin + " gradi Kelvin\n"+ fahrenheit
            + " gradi Fahrenheit");
        break;
    }
    case"f":
    case"F": {
        BigDecimal celsius =
            Convertitore.convertiFahrenheitInCelsius(valore);
        BigDecimal kelvin =
            Convertitore.convertiFahrenheitInKelvin(valore);
        System.out.println("Il valore di "+ valore
            + " gradi Fahrenheit equivale a:\n"
            + celsius + " gradi Celsius\n"+ kelvin
            + " gradi Kelvin");
        break;
    }
    default:
        System.out.println("Scala "+ scala + " non valida. Usare K per"
            + " Kelvin, C per Celsius, F per Fahrenheit");
        start();
    }
}
catch (NumberFormatException exc) {
    System.out.println("Formato errato!");
    start();
}
}
```

Soluzione 8.ss)

La classe Stampante potrebbe essere implementata nel seguente modo:

```
import java.math.*;

public class Stampante {
    public static void stampaIstruzioni() {
        System.out.println("Inserisci un valore in Kelvin (per esempio 12K), "
            + " Celsius (25C) o Fahrenheit (451F)");
    }

    public static void stampaMessaggio(BigDecimal valore1, String
        scala1, BigDecimal valore2, String scala2, BigDecimal valore3,
        String scala3) {
        System.out.println("Il valore di "+ valore1 + " gradi " + scala1
            + " equivale a:\n" + valore2 + " gradi " + scala2 + "\n"
            + valore3 + " gradi " + scala3);
    }

    public static void stampaScalaErrata(String scala) {
        System.out.println("Scala "+ scala + " non valida. Usare K per Kelvin, "
            + "C per Celsius, F per Fahrenheit");
    }
}
```

```
    public static void stampaFormatoErrato() {  
        System.out.println("Formato errato!");  
    }  
}
```

Quindi la classe `Esercizio8SS`, potrebbe essere implementata nel seguente modo:

```
import java.util.*;  
import java.math.*;  
  
public class Esercizio8SS {  
    private static final String K = "Kelvin";  
    private static final String C = "Celsius";  
    private static final String F = "Fahrenheit";  
  
    public static void main(String[] args) {  
        Esercizio8SS esercizio8SS = new Esercizio8SS();  
        esercizio8SS.start();  
    }  
  
    public void start() {  
        try {  
            Stampante.stampaIstruzioni();  
            Scanner scanner = new Scanner(System.in);  
            String input = scanner.next();  
            int lastCharIndex = input.length()-1;  
            BigDecimal valore = new BigDecimal(  
                input.substring(0, lastCharIndex));  
            String scala = input.substring(lastCharIndex);  
            switch(scala) {  
                case "k":  
                case "K": {  
                    BigDecimal celsius =  
                        Convertitore.convertiKelvinInCelsius(valore);  
                    BigDecimal fahrenheit =  
                        Convertitore.convertiKelvinInFahrenheit(valore);  
                    Stampante.stampaMessaggio(valore, K, celsius, C,  
                        fahrenheit, F);  
                    break;  
                }  
                case "c":  
                case "C": {  
                    BigDecimal kelvin =  
                        Convertitore.convertiCelsiusInKelvin(valore);  
                    BigDecimal fahrenheit =  
                        Convertitore.convertiCelsiusInFahrenheit(valore);  
                    Stampante.stampaMessaggio(valore, C, kelvin, K,  
                        fahrenheit, F);  
                    break;  
                }  
                case "f":  
                case "F": {  
                    BigDecimal celsius =  
                        Convertitore.convertiFahrenheitInCelsius(valore);  
                    BigDecimal kelvin =  
                        Convertitore.convertiFahrenheitInKelvin(valore);  
                    Stampante.stampaMessaggio(valore, F, celsius, C, kelvin, K);  
                    break;  
                }  
                default:  
                    Stampante.stampaScalaErrata(scala);  
            }  
        }  
    }  
}
```

```
        start();
    }
    } catch (NumberFormatException exc) {
        Stampante.stampaFormatoErrato();
        start();
    }
}
}
```

Notare che il metodo `stampaMessaggio` ha indubbiamente troppi parametri, ed il suo utilizzo sembra risultare abbastanza complesso. Tuttavia la soluzione sembra migliore rispetto a quella del precedente esercizio.

Soluzione 8.tt)

La seguente potrebbe essere una possibile implementazione della classe `Esercizio8TT`:

```
import java.util.*;
import java.io.*;
import java.math.*;
import java.util.logging.*;

public class Esercizio8TT {
    private static final LogManager logManager = LogManager.getLogManager();
    private static final Logger LOGGER = Logger.getLogger("Esercizio8TT");
    private static final String K = "Kelvin";
    private static final String C = "Celsius";
    private static final String F = "Fahrenheit";

    public static void main(String[] args) {
        initLogging();
        LOGGER.info("Metodo main() chiamato");
        Esercizio8TT esercizio8TT = new Esercizio8TT();
        esercizio8TT.start();
        LOGGER.info("Programma terminato");
    }

    private static void initLogging() {
        try {
            logManager.readConfiguration(
                new FileInputStream("logging.properties"));
        } catch (IOException exception) {
            LOGGER.log(Level.SEVERE,
                "Problema con la lettura del file di configurazione", exception);
        }
    }

    public void start() {
        try {
            LOGGER.info("Metodo start() chiamato");
            Stampante.stampaIstruzioni();
            Scanner scanner = new Scanner(System.in);
            String input = scanner.next();
            int lastCharIndex = input.length()-1;
            BigDecimal valore = new BigDecimal(input.substring(0,
                lastCharIndex));
            String scala = input.substring(lastCharIndex);
            switch(scala) {
                case "k":
```

```

        case "K": {
            LOGGER.fine("Inserito scala Kelvin");
            BigDecimal celsius =
                Convertitore.convertiKelvinInCelsius(valore);
            BigDecimal fahrenheit =
                Convertitore.convertiKelvinInFahrenheit(valore);
            Stampante.stampaMessaggio(valore, K, celsius, C,
                fahrenheit, F);
            break;
        }
        case "c":
        case "C": {
            LOGGER.fine("Inserito scala Celsius");
            BigDecimal kelvin =
                Convertitore.convertiCelsiusInKelvin(valore);
            BigDecimal fahrenheit =
                Convertitore.convertiCelsiusInFahrenheit(valore);
            Stampante.stampaMessaggio(valore, C, kelvin, K,
                fahrenheit, F);
            break;
        }
        case "f":
        case "F": {
            LOGGER.fine("Inserito scala Fahrenheit");
            BigDecimal celsius =
                Convertitore.convertiFahrenheitInCelsius(valore);
            BigDecimal kelvin =
                Convertitore.convertiFahrenheitInKelvin(valore);
            Stampante.stampaMessaggio(valore, F, celsius, C, kelvin, K);
            break;
        }
        default:
            LOGGER.fine("Inserito scala errata");
            Stampante.stampaScalaErrata(scala);
            start();
    }
} catch (NumberFormatException exc) {
    LOGGER.severe(exc.getMessage());
    Stampante.stampaFormatoErrato();
    start();
}
}
}

```

Lanciando questa classe con il livello di default ALL, nel file di log troveremo il seguente testo:

```

apr 27, 2020 12:53:01 PM Esercizio8TT main
INFO: Metodo main() chiamato
apr 27, 2020 12:53:01 PM Esercizio8TT start
INFO: Metodo start() chiamato
apr 27, 2020 12:53:03 PM Esercizio8TT start
FINE: Inserito scala Kelvin
apr 27, 2020 12:53:03 PM Esercizio8TT main
INFO: Programma terminato

```

Ma modificando il livello di log ad INFO, ed utilizzando l'applicazione correttamente, troveremo questo testo nel file di log:

```

apr 27, 2020 12:54:01 PM Esercizio8TT main
INFO: Metodo main() chiamato
apr 27, 2020 12:54:02 PM Esercizio8TT start
INFO: Metodo start() chiamato
apr 27, 2020 12:54:05 PM Esercizio8TT main

```

INFO: Programma terminato

Ovvero sono stati ignorati i log di tipo `FINE`.

Infine se modificassimo il livello di log a `SEVERE` vedremmo solo i log di livello `SEVERE`. Quindi il file risulterebbe non modificato se l'applicazione fosse usata correttamente, mentre conterrebbe i messaggi di log definiti per gestire l'eccezione nel caso l'utente specifichi una scala non valida. Per esempio utilizzando l'applicazione nel seguente modo:

```
Inserisci un valore in Kelvin (per esempio 12K), Celsius (25C) o
Fahrenheit (451F)
j
Formato errato!
Inserisci un valore in Kelvin (per esempio 12K), Celsius (25C) o
Fahrenheit (451F)
Ok
Il valore di 0 gradi Kelvin equivale a:
-273.15 gradi Celsius
-459.67 gradi Fahrenheit
```

Otterremmo il seguente file di log:

```
apr 27, 2020 12:56:21 PM Esercizio8TT start
SEVERE: Character j is neither a decimal digit number, decimal point, nor "e"
notation exponential mark.
```

Soluzione 8.uu)

Possiamo implementare la classe `GuessNumber` nel seguente modo:

```
import java.util.Random;

public class GuessNumber {
    private static final Random RANDOM = new Random();

    public static int generateRandomNumber(int max) {
        return 1+ RANDOM.nextInt(max);
    }
}
```

Mentre possiamo definire la classe `GuessNumberGame` come segue:

```
import java.util.Scanner;
import java.util.Random;

public class GuessNumberGame implements Game {
    private static final int MAX_NUMBER = 100;
    private Scanner scanner;
    private int attemptsNumber;
    private int numberToGuess;

    public GuessNumberGame () {
        init();
        start();
        play();
        end();
    }

    @Override
    public void init() {
        scanner = new Scanner(System.in);
        numberToGuess = GuessNumber.generateRandomNumber(MAX_NUMBER);
    }
}
```

```
@Override
public void start() {
    System.out.println("Ho pensato ad un numero tra 1 e "
        + MAX_NUMBER + ", indovinalo!");
}

@Override
public void play() {
    attemptsNumber++;
    int number = scanner.nextInt();
    if (number < numberToGuess) {
        System.out.println("Troppo basso, riprova");
    } else if (number > numberToGuess) {
        System.out.println("Troppo alto, riprova");
    } else {
        return;
    }
    play();
}

@Override
public void end() {
    System.out.println("Complimenti! Hai indovinato dopo " + attemptsNumber
        + " tentativi");
}

public static void main(String args[]) {
    new GuessNumberGame();
}
}
```

Notare che il metodo `main` ha semplicemente istanziato l'oggetto `GuessNumberGame` chiamandone il costruttore. Quest'ultimo ha invocato i metodi definiti dall'interfaccia che sono stati implementati in maniera molto semplice e coerente. Merita attenzione solo il metodo ricorsivo `play`, che richiama sé stesso sin quando l'utente non ha indovinato il numero.

Soluzione 8.vv)

Possiamo ridefinire la classe `GuessNumberGame` come segue (modifiche in grassetto), rimuovendo la natura ricorsiva del metodo `play`:

```
import java.util.Scanner;
import java.util.Random;

public class GuessNumberGame implements Game {
    private static final int MAX_NUMBER = 100;
    private Scanner scanner;
    private int attemptsNumber;
    private int numberToGuess;

    public GuessNumberGame () {
        init();
        start();
        play();
        end();
    }

    @Override
```

```
public void init() {
    scanner = new Scanner(System.in);
    numberToGuess = GuessNumber.generateRandomNumber(MAX_NUMBER);
}

@Override
public void start() {
    System.out.println("Ho pensato ad un numero tra 1 e "
        + MAX_NUMBER + ", indovinalo!");
}

@Override
public void play() {
    while (scanner.hasNext()) {
        if (scanner.hasNextInt()) {
            attemptsNumber++;
            int number = scanner.nextInt();
            if (number < numberToGuess) {
                System.out.println("Troppo basso, riprova");
            } else if (number > numberToGuess) {
                System.out.println("Troppo alto, riprova");
            } else {
                return;
            }
            //play()
        } else {
            System.out.println("Input '" + scanner.next()
                + "' non valido. Puoi inserire solo numeri interi, riprova");
        }
    }
}

@Override
public void end() {
    System.out.println("Complimenti! Hai indovinato dopo " + attemptsNumber
        + " tentativi");
}

public static void main(String args[]) {
    new GuessNumberGame();
}
}
```

Soluzione 8.ww)

Possiamo implementare la classe Player nel seguente modo:

```
public class Player {
    private int id;
    private String name;

    public Player (int id, String name) {
        this.setId(id);
        this.setName(name);
    }

    public void setId(int id) {
        this.id = id;
    }
}
```

```
    }

    public int getId() {
        return id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    @Override
    public String toString() {
        return "Player " + id + ": " + name;
    }
}
```

Poi implementiamo l'eccezione PlayerException:

```
public class PlayerException extends Exception {
    public PlayerException (String message){
        super("Eccezione riguardante il player: " + message);
    }
}
```

Segue l'implementazione della classe astratta MultiPlayerGame:

```
import java.util.List;
import java.util.ArrayList;

public abstract class MultiPlayerGame implements Game {
    private List<Player> players;

    public MultiPlayerGame () {
        init();
        start();
        play();
        end();
    }

    @Override
    public void init() {
        players = new ArrayList<>();
    }

    public void addPlayer(Player player) {
        players.add(player);
    }

    public void removePlayer(Player player) {
        players.add(player);
    }

    public Player getPlayer(int id) throws PlayerException {
        if (id < 0) {
            throw new PlayerException("L'identificativo del player non può"
                + " essere negativo");
        } else if (id >= players.size()) {
            throw new PlayerException("L'identificativo del player non esiste");
        }
    }
}
```



```
        return players.get(id);
    }

    public Player getPlayer(String name) throws PlayerException {
        Player playerFound = null;
        int counter = 0;
        for (Player player : players) {
            if (player.getName().equals(name)) {
                counter++;
                playerFound = player;
            }
        }
        if (counter == 0) {
            throw new PlayerException("Player con nome " + name + " non trovato");
        } else if (counter > 1) {
            throw new PlayerException("Trovati " + counter + " player con nome "
                + name);
        }
        return playerFound;
    }

    public List<Player> getPlayers() {
        return players;
    }
}
```

Poi abbiamo definito in maniera minimale la sua implementazione:

```
import java.util.Iterator;

public class MultiPlayerGameImpl extends MultiPlayerGame {

    public void start() {
        // per ora nessuna implementazione
    }

    public void play() {
        // per ora nessuna implementazione
    }

    public void end() {
        // per ora nessuna implementazione
    }

    public void printPlayers() {
        System.out.println("Lista dei giocatori:");
        for (Player player : getPlayers()) {
            System.out.println(player);
        }
    }
}
```

Infine segue la nostra classe di test:

```
public class MultiPlayerGameTest {
    private static int counter;
    public static void main(String args[]) {
        MultiPlayerGameImpl game = new MultiPlayerGameImpl();
        Player p1 = new Player(++counter, "John");
        Player p2 = new Player(++counter, "James");
        game.addPlayer(p1);
        game.addPlayer(p2);
        game.printPlayers();
    }
}
```

```
System.out.println();
//NAME OK
try {
    System.out.println("Recuperato player: "+ game.getPlayer("John"));
} catch (Exception exc) {
    assert false;
    System.out.println(exc.getMessage());
}
//NAME NOT OK
try {
    System.out.println("Recuperato player: "+ game.getPlayer("Jack"));
    assert false;
} catch (Exception exc) {
    System.out.println(exc.getMessage());
}
try {
    Player p3 = new Player(++counter,"John");
    game.addPlayer(p3);
    game.printPlayers();
    System.out.println();
    System.out.println("Recuperato player: "+game.getPlayer("John"));
    assert false;
} catch (Exception exc) {
    System.out.println(exc.getMessage());
}
//ID OK
try {
    System.out.println("Recuperato player: "+game.getPlayer(2));
} catch (Exception exc) {
    assert false;
    System.out.println(exc.getMessage());
}
//ID NOT OK
try {
    System.out.println("Recuperato player: "+game.getPlayer(-1));
    assert false;
} catch (Exception exc) {
    System.out.println(exc.getMessage());
}
try {
    System.out.println("Recuperato player: "+game.getPlayer(3));
    assert false;
} catch (Exception exc) {
    System.out.println(exc.getMessage());
}
}
}
```

che produce il seguente output:

Lista dei giocatori:

Player 1: John

Player 2: James

Recuperato player: Player 1: John

Eccezione riguardante il player: Player con nome Jack non trovato

Lista dei giocatori:

Player 1: John

Player 2: James

Player 3: John

```
Eccezione riguardante il player: Trovati 2 player con nome John
Recuperato player: Player 3: John
Eccezione riguardante il player: L'identificativo del player non può
essere negativo
Eccezione riguardante il player: L'identificativo del player non esiste
```

Soluzione 8.xx)

Una possibile soluzione è rappresentata dal seguente codice:

```
import java.util.Scanner;

public class MultiPlayerGameConfigurator implements Configurator {
    private static int counter;
    private Scanner scanner;
    private MultiPlayerGameImpl game;

    public MultiPlayerGameConfigurator() {
        game = new MultiPlayerGameImpl();
        scanner = new Scanner(System.in);
        config();
        game.printPlayers();
    }

    @Override
    public void config() {
        var players = game.getPlayers();
        System.out.println("Inserire nome player "+ (++counter) +
            (players.isEmpty() ? "": " oppure 'i' per iniziare a giocare"));
        String text = scanner.next();
        if (!players.isEmpty() && text.equals("i")) {
            return;
        }
        game.addPlayer(new Player (counter, text));
        config();
    }
}
```

Notare l'implementazione ricorsiva del metodo `config`, che ci permette di aggiungere quanti giocatori vogliamo.

Soluzione 8.yy)

Per completezza, riportiamo la classe `Player` che è stata resa astratta e modificata con l'aggiunta del metodo astratto `play` (modifiche in grassetto):

```
public abstract class Player {
    private int id;
    private String name;

    public Player(int id, String name) {
        this.setId(id);
        this.setName(name);
    }

    public abstract boolean play();
}
```

```
public void setId(int id) {
    this.id = id;
}

public int getId() {
    return id;
}

public void setName(String name) {
    this.name = name;
}

public String getName() {
    return name;
}

@Override
public String toString() {
    return "Player " + id + ": " + name;
}
}
```

segue invece l'implementazione della classe `GuessNumberPlayer` che definisce la logica per giocare nel suo metodo `play`, molto simile a quella che era la logica nella versione della classe `GuessNumberGame` dell'esercizio 8.vv:

```
import java.util.Scanner;

public class GuessNumberPlayer extends Player {
    private Scanner scanner;
    private int numberToGuess;
    private int attemptsNumber;

    public GuessNumberPlayer (int id, String name, Scanner scanner,
                               int numberToGuess) {
        super(id, name);
        this.scanner = scanner;
        this.numberToGuess = numberToGuess;
    }

    public void incrementAttemptsNumber() {
        attemptsNumber++;
    }

    public int getAttemptsNumber() {
        return attemptsNumber;
    }

    @Override
    public boolean play() {
        if (scanner.hasNextInt()) {
            incrementAttemptsNumber();
            int number = scanner.nextInt();
            if (number < numberToGuess) {
                System.out.println("Troppo basso, riprova");
            } else if (number > numberToGuess) {
                System.out.println("Troppo alto, riprova");
            } else {
                return true;
            }
        } else {

```

```

        System.out.println("Input '"+ scanner.next() +
            "' non valido. Puoi inserire solo numeri interi, riprova");
        play();
    }
    return false;
}
}

```

infine, di seguito è riportata la nostra implementazione della nuova classe `GuessNumberGame` (modifiche in grassetto):

```

import java.util.Scanner;
import java.util.Random;
import java.util.List;
import java.util.ArrayList;

public class GuessNumberGame extends MultiPlayerGame implements Configurator {
    private static final int MAX_NUMBER = 100;
    private static int counter;
    private Player currentPlayer;
    private Scanner scanner;
    // private int attemptsNumber;
    private int numberToGuess;

    /* public GuessNumberGame () {
        init();
        start();
        play();
        end();
    } */

    @Override
    public void init() {
        super.init();
        scanner = new Scanner(System.in);
        numberToGuess = GuessNumber.generateRandomNumber(MAX_NUMBER);
    }

    @Override
    public void start() {
        config();
        System.out.println("Ho pensato ad un numero tra 1 e "
            + MAX_NUMBER + ", indovinalo!");
    }

    @Override
    public void config() {
        System.out.println("Inserire nome player "+ (++counter)
            +(getPlayers().isEmpty() ? "" : " oppure 'i' per iniziare a giocare"));
        String text = scanner.next();
        if (!getPlayers().isEmpty() && text.equals("i")) {
            currentPlayer = getPlayers().get(0);
            System.out.println("Tocca a "+ currentPlayer);
            return;
        }
        addPlayer(new GuessNumberPlayer (counter, text, scanner, numberToGuess));
        config();
    }

    @Override
    public void play() {

```

```
        while (scanner.hasNext()) {
            boolean result = currentPlayer.play();
            if (result) {
                return;
            }
            currentPlayer = getCurrentPlayer();
            System.out.println("Tocca a " + currentPlayer);
        }
    }

    private Player getCurrentPlayer() {
        int nextPlayerIndex = currentPlayer.getId();
        var players = getPlayers();
        if (nextPlayerIndex == players.size()) {
            nextPlayerIndex = 0;
        }
        return players.get(nextPlayerIndex);
    }

    @Override
    public void end() {
        System.out.println("Complimenti " + currentPlayer.getName() +
            "! Hai indovinato dopo " + ((GuessNumberPlayer) currentPlayer)
                .getAttemptsNumber() + " tentativi");
    }

    public static void main(String args[]) {
        new GuessNumberGame();
    }
}
```

Soluzione 8.zz)

La soluzione dovrebbe essere:

- A `games.generic.exceptions` appartiene la classe `PlayerException`.
- A `games.generic.data` appartiene la classe `Player` e `MultiPlayerGame`.
- A `games.guessnumber.data` appartiene la classe `GuessNumberPlayer`.
- A `games.guessnumber.business` appartiene la classe `GuessNumberGame`.
- A `games.guessnumber.util` appartiene la classe `GuessNumber`.
- A `games.generic.business` appartiene la classe `Game`.
- A `games.generic.business` appartiene la classe `Configurator`.

Notare che tutte le classi appartenenti al package `games.generic`, sono classi che potrebbero essere riutilizzate in altri giochi. Quindi in pratica abbiamo creato, quello che in architettura è detto framework, diciamo un “mini-framework”, che possiamo riutilizzare per creare nuovi giochi configurabili e multi-utente.

Soluzioni degli esercizi del capitolo 9

Tipi Enumerazioni e Tipi Record

Soluzione 9.a) *Record, Vero o Falso:*

1. **Vero.**
2. **Falso**, un record estende la classe `java.lang.Record`, non la classe `java.util.Record`.
3. **Vero.**
4. **Falso.**
5. **Falso**, un record non implementa metodi setter.
6. **Vero.**
7. **Falso**, per esempio un record non può dichiarare metodi setter.
8. **Vero.**
9. **Falso.**
10. **Vero.**

Soluzione 9.b) *Enumerazioni, Vero o Falso:*

1. **Vero.**
2. **Vero.**
3. **Falso.**
4. **Falso.**
5. **Vero.**
6. **Falso**, non è possibile utilizzare il costruttore di default se ne viene dichiarato uno esplicitamente.
7. **Vero.**
8. **Vero.**
9. **Falso.**
10. **Vero.**

Soluzione 9.c)

Il listato della enumerazione `TipoFile` è molto semplice:

```
public enum TipoFile {  
    JAVA,  
    C_SHARP,  
    C_PLUS_PLUS,  
    C;  
}
```

Poi bisogna modificare la classe `File`, in modo tale da usare il tipo `TipoFile` in luogo del vecchio tipo intero:

```
public abstract class File {  
  
    private String nome;
```

```
private TipoFile tipo;

public File(String nome, TipoFile tipo) {
    this.nome = nome;
    this.tipo = tipo;
}

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public TipoFile getTipo() {
    return tipo;
}

public void setTipo(TipoFile tipo) {
    this.tipo = tipo;
}
}
```

Poi bisogna modificare la classe `FileSorgente`, in modo tale da usare il tipo `TipoFile` in luogo del vecchio tipo intero:

```
public class FileSorgente extends File {

    private String contenuto;

    public FileSorgente(String nome, TipoFile tipo) {
        super(nome, tipo);
    }

    public FileSorgente(String nome, TipoFile tipo, String contenuto) {
        this(nome, tipo);
        this.contenuto = contenuto;
    }

    public void aggiungiTesto(String testo) {
        if (contenuto != null) {
            contenuto = "";
        }
        if (testo != null) {
            contenuto += testo;
        }
    }

    public void aggiungiTesto(String testo, int posizione) {
        final int length = contenuto.length();
        if (contenuto != null && testo != null && posizione > 0 &&
            posizione < length) {
            contenuto = contenuto.substring(0, posizione) + testo +
                contenuto.substring(posizione);
        }
    }

    public String getContenuto() {
        return contenuto;
    }
}
```



```
    }

    public void setContenuto(String contenuto) {
        this.contenuto = contenuto;
    }
}
```

Tutto il resto può restare com'è.

Soluzione 9.d)

Il listato di File con l'enumerazione innestata TipoFile dovrebbe essere il seguente:

```
public abstract class File {

    public enum TipoFile {

        JAVA,
        C_SHARP,
        C_PLUS_PLUS,
        C;
    }

    private String nome;

    private TipoFile tipo;

    public File(String nome, TipoFile tipo) {
        this.nome = nome;
        this.tipo = tipo;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public TipoFile getTipo() {
        return tipo;
    }

    public void setTipo(TipoFile tipo) {
        this.tipo = tipo;
    }
}
```

Poi dobbiamo modificare le due classi di test per referenziare correttamente l'elemento innestato TipoFile:

```
public class TestFileSorgente {
    public static void main(String args[]) {
        FileSorgente fileSorgente = new FileSorgente("Test.java",
            File.TipoFile.JAVA, "public class MyClass {\n\r"});
        System.out.println(fileSorgente.getContenuto());
        // Test aggiungiTesto (String) corretto
        fileSorgente.aggiungiTesto("{}");
        System.out.println(fileSorgente.getContenuto());
        // Test aggiungiTesto (String,int) corretto
    }
}
```

```

        fileSorgente.aggiungiTesto("//Test aggiunta testo\n\r", 23);
        System.out.println(fileSorgente.getContenuto());
        // Test aggiungiTesto (String,int) scorretto
        fileSorgente.aggiungiTesto("//Test aggiunta testo\n\r", -1);
        System.out.println(fileSorgente.getContenuto());
        // Test aggiungiTesto (String,int) scorretto
        fileSorgente.aggiungiTesto("//Test aggiunta testo\n\r", 100);
        System.out.println(fileSorgente.getContenuto());
    }
}

```

e:

```

public class TestIDE {

    public static void main(String args[]) {
        IDE ide = new JavaIDE();
        FileSorgente fileSorgente = new FileSorgente("Test.java",
            File.TipoFile.JAVA, "public class MyClass {\n\r"};
        ide.modifica(fileSorgente, "{}");
    }
}

```

Non è stato necessario invece modificare la classe `FileSorgente`, perché essendo sottoclasse di `File` ha ereditato l'enumerazione.

Soluzione 9.e)

Il listato della nostra enumerazione è abbastanza complesso:

```

public enum Valore {

    UN_CENTESIMO(1) {
        @Override
        public String getStringaDescrittiva() {
            return getValore() + " centesimi di ";
        }
    },
    DUE_CENTESIMI(2),
    CINQUE_CENTESIMI(5),
    DIECI_CENTESIMI(10),
    VENTI_CENTESIMI(20),
    CINQUANTA_CENTESIMI(50),
    UN_EURO(1) {
        @Override
        public String getStringaDescrittiva() {
            return getValore() + " ";
        }
    },
    DUE_EURO(2) {
        @Override
        public String getStringaDescrittiva() {
            return getValore() + " ";
        }
    };

    private int valore;

    private Valore(int valore) {
        this.valore = valore;
    }
}

```

```
public String getStringaDescrittiva() {
    return getValore() + " centesimi di ";
}

public int getValore() {
    return valore;
}
}
```

Abbiamo definito tutti i possibili valori per una moneta (supposta di tipo Euro). Ogni elemento dell'enumerazione usa un costruttore che imposta il valore numerico della variabile `valore`. Inoltre è definito il metodo `getStringaDescrittiva` che semplificherà la gestione fatta nella classe `Moneta` originale. Tale metodo è sottoposto a override tramite una classe anonima per tre elementi dell'enumerazione. La classe `Moneta` va modificata (semplificata) come segue:

```
public class Moneta {

    public final static String VALUTA = "EURO";

    private final Valore valore;

    public Moneta(Valore valore) {
        this.valore = valore;
        System.out.println("Creata una " + getDescrizione());
    }

    public Valore getValore() {
        return valore;
    }

    public String getDescrizione() {
        String descrizione = "moneta da " + valore.getStringaDescrittiva()
            + VALUTA;

        return descrizione;
    }
}
```

Soluzione 9.f)

Il listato della classe `PortaMonete` va modificato nel seguente modo (commenti omissi):

```
public class PortaMonete {

    private static final int DIMENSIONE = 10;
    private final Moneta[] monete = new Moneta[DIMENSIONE];

    public PortaMonete(Valore... valori) {
        assert monete.length == DIMENSIONE;
        try {
            int numeroMonete = valori.length;
            for (int i = 0; i < numeroMonete; i++) {
                if (i >= DIMENSIONE) {
                    throw new PortaMonetePienoException(
                        "Sono state inserite solo le prime + "
                        + DIMENSIONE + " monete!");
                }
                monete[i] = new Moneta(valori[i]);
            }
        }
    }
}
```

```
    }
//    } catch (PortaMonetePienoException | NullPointerException exc) {
//    } catch (PortaMonetePienoException exc) {
//        System.out.println(exc.getMessage());
//    } catch (NullPointerException exc) {
//        System.out.println("Il portamonete è stato creato vuoto");
//    }
    assert monete.length == DIMENSIONE;
}

public void aggiungi(Moneta moneta) throws PortaMonetePienoException {
    try {
        System.out.println("Proviamo ad aggiungere una " +
            moneta.getDescrizione());
    } catch (NullPointerException exc) {
        throw new MonetaNullException();
    }
    int indiceLibero = primoIndiceLibero();
    if (indiceLibero == -1) {
        throw new PortaMonetePienoException("Portamonete pieno! La moneta "
            + moneta.getDescrizione() + " non è stata aggiunta...");
    } else {
        monete[indiceLibero] = moneta;
        System.out.println("E' stata aggiunta una " +
            moneta.getDescrizione());
    }
}

public Moneta preleva(Moneta moneta) throws MonetaNonTrovataException {
    try {
        System.out.println("Proviamo a prelevare una " +
            moneta.getDescrizione());
    } catch (NullPointerException exc) {
        throw new MonetaNullException();
    }
    Moneta monetaTrovata = null;
    int indiceMonetaTrovata = indiceMonetaTrovata(moneta);
    if (indiceMonetaTrovata == -1) {
        throw new MonetaNonTrovataException("Moneta non trovata!");
    } else {
        monetaTrovata = moneta;
        monete[indiceMonetaTrovata] = null;
        System.out.println("Una " + moneta.getDescrizione()
            + " prelevata");
    }
    return monetaTrovata;
}

public void stato() {
    System.out.println("Il portamonete contiene:");
    for (Moneta moneta : monete) {
        if (moneta == null) {
            break;
        }
        System.out.println("Una " + moneta.getDescrizione());
    }
}

private int primoIndiceLibero() {
```

```
int indice = -1;
for (int i = 0; i < 10; i++) {
    if (monete[i] == null) {
        indice = i;
        break;
    }
}
return indice;
}

private int indiceMonetaTrovata(Moneta moneta) {
    int indiceMonetaTrovata = -1;
    for (int i = 0; i < 10; i++) {
        if (monete[i] == null) {
            break;
        }
        Valore valoreMonetaNelPortaMoneta = monete[i].getValore();
        Valore valore = moneta.getValore();
        if (valore == valoreMonetaNelPortaMoneta) {
            indiceMonetaTrovata = i;
            break;
        }
    }
    return indiceMonetaTrovata;
}
}
```

Soluzione 9.g)

Il listato della classe TestMonete è il seguente:

```
public class TestMonete {

    public static void main(String args[]) {

        Moneta monetaDaVentiCentesimi = new Moneta(Valore.VENTI_CENTESIMI);
        Moneta monetaDaUnCentesimo = new Moneta(Valore.UN_CENTESIMO);
        Moneta monetaDaUnEuro = new Moneta(Valore.UN_EURO);
        // Creiamo un portamonete con 11 monete
        PortaMonete portaMoneteInsufficiente =
            new PortaMonete(Valore.DUE_CENTESIMI,
                Valore.CINQUE_CENTESIMI, Valore.UN_EURO, Valore.DIECI_CENTESIMI,
                Valore.CINQUANTA_CENTESIMI, Valore.DIECI_CENTESIMI,
                Valore.UN_EURO, Valore.DUE_EURO, Valore.DIECI_CENTESIMI,
                Valore.CINQUE_CENTESIMI, Valore.DUE_CENTESIMI);
        // Creiamo un portamonete con 8 monete
        PortaMonete portaMonete = new PortaMonete(Valore.DUE_CENTESIMI,
            Valore.CINQUE_CENTESIMI, Valore.UN_EURO, Valore.DIECI_CENTESIMI,
            Valore.CINQUANTA_CENTESIMI, Valore.DIECI_CENTESIMI,
            Valore.UN_EURO, Valore.DUE_EURO);
        portaMonete.stato();
        try {
            // Aggiungiamo una moneta da 20 centesimi
            portaMonete.aggiungi(monetaDaVentiCentesimi);
        } catch (PortaMonetePienoException | MonetaNullException exc) {
            System.out.println(exc.getMessage());
        }
        try {
```

```
// Aggiungiamo la decima moneta da 1 centesimo.
portaMonete.aggiungi(monetaDaUnCentesimo);
} catch (PortaMonetePienoException | MonetaNullException exc) {
    System.out.println(exc.getMessage());
}
try {
    // Aggiungiamo l'undicesima moneta (dovremmo ottenere un errore e
    //la moneta non sarà aggiunta)
    portaMonete.aggiungi(monetaDaUnEuro);
} catch (PortaMonetePienoException | MonetaNullException exc) {
    System.out.println(exc.getMessage());
} // Valutiamo lo stato del portamonete.
portaMonete.stato();
try {
    // preleviamo 20 centesimi
    portaMonete.preleva(monetaDaVentiCentesimi);
} catch (MonetaNonTrovataException exc) {
    System.out.println(exc.getMessage());
}
try {
    // Aggiungiamo l'undicesima moneta (dovremmo ottenere un
    //errore e la moneta non sarà aggiunta)
    portaMonete.aggiungi(monetaDaUnEuro);
} catch (PortaMonetePienoException | MonetaNullException exc) {
    System.out.println(exc.getMessage());
}
portaMonete.stato();
//il prossimo esempio non è più riproducibile
//    try {
//
//
//Cerchiamo una moneta non esistente (dovremmo ottenere una stampa di errore)
//
//        portaMonete.preleva(new Moneta(7));
//    } catch (MonetaNonTrovataException exc) {
//        System.out.println(exc.getMessage());
//    }
//testiamo il passaggio di null al costruttore del porta monete
PortaMonete portaMoneteEccezione = new PortaMonete(null);
portaMonete.stato();
try {
    // proviamo ad aggiungere null
    portaMonete.aggiungi(null);
} catch (PortaMonetePienoException | MonetaNullException exc) {
    System.out.println(exc.getMessage());
}
try {
    //Proviamo a prelevare una moneta null
    portaMonete.preleva(null);
} catch (MonetaNonTrovataException | MonetaNullException exc) {
    System.out.println(exc.getMessage());
}
}
}
```

Soluzione 9.h)

Il listato del metodo `equals` l'abbiamo generato (insieme al metodo `hashCode` che riportiamo per completezza anche se non necessario ai fini dell'esercizio) tramite Netbeans (abbiamo anche utilizzato la classe `java.util.Objects` che spiegheremo più avanti nel libro):

```
@Override
public boolean equals(Object obj) {
    if (obj == null) {
        return false;
    }
    if (getClass() != obj.getClass()) {
        return false;
    }
    final Moneta other = (Moneta) obj;
    if (this.valore != other.valore) {
        return false;
    }
    return true;
}

@Override
public int hashCode() {
    int hash = 7;
    hash = 97 * hash + Objects.hashCode(this.valore);
    return hash;
}
```

Quindi il metodo `indiceMonetaTrovata` si può semplificare nel seguente modo:

```
private int indiceMonetaTrovata(Moneta moneta) {
    int indiceMonetaTrovata = -1;
    final int size = monete.size();
    for (int i = 0; i < size; i++) {
        if (monete.get(i) == null) {
            break;
        }
        if (monete.get(i).equals(moneta)) {
            indiceMonetaTrovata = i;
            break;
        }
    }
    return indiceMonetaTrovata;
}
```

Soluzione 9.i)

La soluzione potrebbe essere la seguente:

```
public record Persona(String nome, String cognome, String dataDiNascita, String
professione, String indirizzo) {
```

```
    @Override
    public String toString() {
        String string = "Nome: \t\t\t" + nome();
        string += "\nCognome: \t\t" + cognome();
        string += "\nProfessione: \t\t" + professione();
        string += "\nData di Nascita: \t" + dataDiNascita();
        string += "\nIndirizzo: \t\t" + indirizzo();
        return string;
    }
}
```

```
}
```

Soluzione 9.j)

L'output della classe è:

```
false  
true  
true  
true  
false
```

Infatti `stringa1` e `stringa2` sono due stringhe diverse, quindi è ovvio che la prima istruzione di stampa, che confronta le stringhe con l'operatore `==` che si basa sugli indirizzi dei reference, stampi `false`. La seconda e la terza istruzione di stampa invece stamperanno `true`, perché il metodo `equals` confronta i contenuti delle stringhe e non gli indirizzi dei reference. Il risultato della quarta e la quinta istruzione di stampa, invece dipende dal fatto che la stringa `Claudio`, è una stringa che è stata messa nella pool di stringhe (cfr. paragrafo F.4.1 dell'appendice F) e il cui indirizzo coincide con l'indirizzo di `stringa1` (vedi la prima istruzione del metodo) ma non con quella di `stringa2`.

Soluzione 9.k) Approfondimento `import static`, Vero o Falso:

1. **Vero.**
2. **Falso.**
3. **Falso**, `out` non è importato staticamente.
4. **Vero.**
5. **Vero.**
6. **Vero.**
7. **Vero.**
8. **Vero.**
9. **Vero.**
10. **Falso.**

Soluzione 9.q)

Le risposte corrette sono le 2, 5 e persino la 6.

Soluzione 9.r)

La risposta giusta è la 2, ovvero viene stampato 29. Infatti il metodo `metodo`, ereditato dall'interfaccia `InnerInterface`, è riscritto dall'enumerazione `Esercizio9R` con un'implementazione che restituisce il numero 22. Ma per lo specifico elemento dell'enumerazione `TRE`, il metodo viene nuovamente riscritto in modo tale da restituire appunto il valore 29. Nel metodo `main` l'elemento `Esercizio9R.TRE` viene assegnato al reference `i` dell'interfaccia `Interface` (è legale per il polimorfismo per dati). Poi viene invocato virtualmente il metodo

metodo su `i`, che in realtà punta all'elemento `TRE`, che come abbiamo detto, ha ridefinito un'implementazione del metodo `metodo`, che restituisce 29.

Soluzione 9.v)

Abbiamo deciso di utilizzare le enumerazioni sia per astrarre il concetto di seme che quello del numero delle carte. Un numero potrebbe essere rappresentato con un tipo primitivo come un `int`, o anche un `byte`. Ma tutto sommato per il nostro scopo l'enumerazione è parsa la scelta migliore per poter rappresentare anche testualmente le varie carte. Di seguito il codice dell'enumerazione

Numero:

```
public enum Numero {  
  
    UNO("Asso"),  
    DUE("Due"),  
    TRE("Tre"),  
    QUATTRO("Quattro"),  
    CINQUE("Cinque"),  
    SEI("Sei"),  
    SETTE("Sette"),  
    OTTO("Otto"),  
    NOVE("Nove"),  
    DIECI("Dieci");  
  
    String rappresentazione;  
  
    Numero(String rappresentazione) {  
        this.rappresentazione = rappresentazione;  
    }  
  
    @Override  
    public String toString() {  
        return rappresentazione;  
    }  
}
```

Si noti che abbiamo sfruttato la variabile `rappresentazione` per gestire la rappresentazione testuale degli elementi dell'enumerazione. Questa viene impostata nel momento della definizione sfruttando l'unico costruttore fornito. Con questa variabile abbiamo potuto far stampare la stringa `Asso` in luogo di `Uno`, in modo piuttosto semplice (ovvero senza utilizzare particolari algoritmi che usano condizioni come `switch` o `if`). Se avessimo utilizzato un tipo primitivo come `int` invece avremmo dovuto gestire la situazione con un algoritmo, che implica maggiori probabilità di errore. Si noti che abbiamo anche ridefinito il metodo `toString` ereditato dalla classe `Object` per facilitare la stampa degli elementi dell'enumerazione.

Per essere sicuri di aver scritto del codice corretto, abbiamo anche creato una classe di test, praticamente uno unit test (cfr. approfondimento 8.2) artigianale e semplificato per testare solo la stampa degli elementi dell'enumerazione:

```
/**  
 * Classe che testa l'enumerazione Numero.  
 */  
public class TestNumero {  
    public static void main(String args[]) {  
        for (Object object : Numero.values()) {  
            System.out.println(object);  
        }  
    }  
}
```

```
}  
}
```

Il cui output ha verificato la correttezza del codice (secondo le nostre intenzioni):

```
Asso  
Due  
Tre  
Quattro  
Cinque  
Sei  
Sette  
Otto  
Nove  
Dieci
```

Passiamo all'enumerazione che rappresenta il Seme di una carta:

```
public enum Seme {  
  
    COPPE,  
    BASTONI,  
    DENARI,  
    SPADE;  
  
    public String toString() {  
        return rendiMaiuscolo(this.name());  
    }  
  
    private String rendiMaiuscolo(String stringa) {  
        //rendiamo minuscola la stringa  
        String minuscolo = stringa.toLowerCase();  
        //recuperiamo la prima lettera della stringa  
        String iniziale = minuscolo.substring(0,1);  
        //rendiamo maiuscola la prima lettera  
        String inizialeMaiuscola = iniziale.toUpperCase();  
        //ritorniamo la concatenazione tra la lettera maiuscola  
        //e il resto della stringa minuscola  
        return inizialeMaiuscola + minuscolo.substring(1);  
    }  
}
```

In questo caso, per scopi didattici, abbiamo modificato il modo in cui vengono rappresentati gli elementi dell'enumerazione. Abbiamo realizzato un metodo d'utilità chiamato `rendiMaiuscolo`, che rende maiuscola la stringa passata in input. Le istruzioni di tale metodo sono commentate e facilmente comprensibili. Il metodo `toString`, non fa altro che restituire la stringa maiuscola del nome dell'elemento dell'enumerazione. Anche in questo caso abbiamo creato una classe di test:

```
/**  
 * Classe che testa l'enumerazione Seme.  
 */  
public class TestSeme {  
    public static void main(String args[]) {  
        for (Object object : Seme.values()) {  
            System.out.println(object);  
        }  
    }  
}
```

che una volta eseguita produce l'output voluto:

```
Coppe  
Bastoni  
Denari  
Spade
```

La classe Carta è molto semplice:

```
public class Carta {  
  
    private Seme seme;  
    private Numero numero;  
  
    public Carta (Numero numero, Seme seme) {  
        this.numero = numero;  
        this.seme = seme;  
    }  
  
    public void setNumero(Numero numero) {  
        this.numero = numero;  
    }  
  
    public Numero getNumero() {  
        return numero;  
    }  
  
    public void setSeme(Seme seme) {  
        this.seme = seme;  
    }  
  
    public Seme getSeme() {  
        return seme;  
    }  
  
    public String toString() {  
        return numero + " di " + seme;  
    }  
}
```

La classe più complicata è indubbiamente la classe `MazzoDiCarte`, che abbiamo voluto astrarre con un array bidimensionale di 4 righe (una per ogni seme) e 10 colonne (una per ogni numero):

```
public class MazzoDiCarte {  
  
    private Carta[][] carte;  
  
    public MazzoDiCarte() {  
        carte = new Carta[4][10];  
        caricaCarte();  
    }  
  
    public void caricaCarte() {  
        Seme[] semi = Seme.values();  
        Numero[] numeri = Numero.values();  
        int semiLength = semi.length;  
        int numeriLength = numeri.length;  
        for (int i = 0; i < semiLength; i++) {  
            for (int j = 0; j < numeriLength; j++) {  
                carte[i][j] = new Carta(numeri[j], semi[i]);  
            }  
        }  
    }  
  
    public String toString() {  
        String string = "";  
        for (int i = 0; i < carte.length; i++) {  
            for (int j = 0; j < carte[i].length; j++) {  
                string += carte[i][j] + ", ";  
            }  
        }  
    }  
}
```

```
        }
        string += "\n";
    }
    return string;
}

public void setCarte(Carta[][] carte) {
    this.carte = carte;
}

public Carta[][] getCarte() {
    return carte;
}
}
```

Si noti che il costruttore istanzia l'unica variabile d'istanza (ovvero l'array bidimensionali `carte`) e poi invoca il metodo `caricaCarte`, che si occupa di caricare nell'array tutte le carte del mazzo con un doppio ciclo `for`. Il metodo `toString` invece crea e restituisce la stringa rappresentativa del mazzo di carte.

Infine, la classe principale (quella con il metodo `main`) è la seguente:

```
public class Esercizio9V {
    public static void main(String args[]) {
        MazzoDiCarte mazzoDiCarte = new MazzoDiCarte();
        System.out.println(mazzoDiCarte);
    }
}
```

Dove con un doppio ciclo abbiamo stampato le quaranta carte del mazzo.

L'output è il seguente (purtroppo la pagina è troppo stretta rispetto ad ogni riga da stampare, quindi l'output risulta un po' sfalsato):

```
Asso di Coppe, Due di Coppe, Tre di Coppe, Quattro di Coppe, Cinque di
Coppe, Sei di Coppe, Sette di Coppe, Otto di Coppe, Nove di Coppe,
Dieci di Coppe,
```

```
Asso di Bastoni, Due di Bastoni, Tre di Bastoni, Quattro di Bastoni,
Cinque di Bastoni, Sei di Bastoni, Sette di Bastoni,
Otto di Bastoni, Nove di Bastoni, Dieci di Bastoni,
```

```
Asso di Denari, Due di Denari, Tre di Denari, Quattro di Denari,
Cinque di Denari, Sei di Denari, Sette di Denari, Otto di Denari,
Nove di Denari, Dieci di Denari,
```

```
Asso di Spade, Due di Spade, Tre di Spade, Quattro di Spade, Cinque di
Spade, Sei di Spade, Sette di Spade, Otto di Spade, Nove di Spade,
Dieci di Spade,
```

Si noti anche che la formattazione è tutt'altro che perfetta. Cercheremo di risolvere il problema nel prossimo esercizio.

Soluzione 9.z)

I problemi di formattazione sono:

- Alla fine di ogni riga viene stampato una virgola e uno spazio non necessari.
- Alla fine della stringa vengono stampati due a capo non necessari.

Risolviamo i due problemi con due controlli (in grassetto nel codice sottostante) all'interno dei due cicli del metodo, uno con un operatore ternario che aggiunge una virgola e uno spazio solo se non siamo all'ultima iterazione sui numeri, e uno con un `if` che aggiunge un "a capo" solo se non siamo all'ultima iterazione dei semi:

```
//...
public String toString() {
    int carteLength = carte.length;
    String string = "";
    for (int i = 0; i < carteLength; i++) {
        int rigaCarteLength = carte[i].length;
        for (int j = 0; j < rigaCarteLength; j++) {
            string += carte[i][j] + (j != (rigaCarteLength-1) ? ", " : "");
        }
        if (i != (carteLength-1)) {
            string += "\n";
        }
    }
    return string;
}
//...
```

Soluzioni esercizi del capitolo 10

Tipi Generici e Collezioni

Soluzione 10.a) Generics, Vero o Falso:

1. **Falso.**
2. **Vero.**
3. **Vero.**
4. **Vero.**
5. **Vero.**
6. **Vero.**
7. **Falso.**
8. **Falso**, otterremo il seguente errore:

```
error: incompatible types: Object cannot be converted to Pippo
    add(o);
      ^
   where Pippo is a type-variable:
     Pippo extends Number declared in class MyGeneric
```

9. **Vero.**
10. **Falso**, il metodo helper bisogna crearselo da soli.
11. **Falso.**
12. **Falso.**
13. **Falso.**
14. **Vero.**
15. **Vero.**

Soluzione 10.b)

Il listato della classe `PortaMonete` cambierà molto:

```
import java.util.ArrayList;
import java.util.List;

/**
 * Astrae il concetto di portamonete che può contenere un numero
 * limitato di monete.
 *
 * @author Claudio De Sio Cesari
 */
public class PortaMonete {

    /**
     * Costante che rappresenta il numero massimo di monete che questo
     * portamonete può contenere.
     */
    private static final int DIMENSIONE = 10;

    /**
     * Un array list che contiene un numero limitato di monete.
     */
}
```

```
*/
private final List<Moneta> monete = new ArrayList<>(DIMENSIONE);

/**
 * Crea un oggetto portamonete contenente monete i cui valori sono
 * specificati dal varargs valori. Se
 * il numero di elementi del varargs valori è maggiore del numero massimo di
 * monete che il portamonete corrente può contenere, allora viene gestita
 * un'eccezione
 *
 * @param valori un varargs di valori di monete.
 */
public PortaMonete(Valore... valori) {
    assert monete.size() <= DIMENSIONE;
    try {
        int numeroMonete = valori.length;
        for (int i = 0; i < numeroMonete; i++) {
            if (i >= DIMENSIONE) {
                throw new PortaMonetePienoException(
                    "Sono state inserite solo le prime + " + DIMENSIONE
                    + " monete!");
            }
            monete.add(new Moneta(valori[i]));
        }
    } catch (PortaMonetePienoException exc) {
        System.out.println(exc.getMessage());
    } catch (NullPointerException exc) {
        System.out.println("Il portamonete è stato creato vuoto");
    }
    assert monete.size() <= DIMENSIONE;
}

/**
 * Aggiunge una moneta al portamonete. Se questo è pieno la moneta non è
 * aggiunta e viene stampato un errore significativo.
 *
 * @param moneta la moneta da aggiungere.
 * @throws PortaMonetePienoException
 */
public void aggiungi(Moneta moneta) throws PortaMonetePienoException {
    try {
        System.out.println("Proviamo ad aggiungere una "
            + moneta.getDescrizione());
    } catch (NullPointerException exc) {
        throw new MonetaNullException();
    }
    int indiceLibero = primoIndiceLibero();
    if (indiceLibero == -1) {
        throw new PortaMonetePienoException("Portamonete pieno! La moneta "
            + moneta.getDescrizione() + " non è stata aggiunta...");
    } else {
        monete.set(indiceLibero, moneta);
        System.out.println("E' stata aggiunta una "
            + moneta.getDescrizione());
    }
}

/**
 * Esegue un prelievo della moneta specificata dal portamonete corrente. Nel
 * caso non sia presente la moneta specificata, un errore significativo

```

```
* verrà stampato e null verrà ritornato.
*
* @param moneta la moneta da prelevare.
* @return la moneta trovata o null se non trovata.
* @throws MonetaNonTrovataException
*/
public Moneta preleva(Moneta moneta) throws MonetaNonTrovataException {
    try {
        System.out.println("Proviamo a prelevare una "
            + moneta.getDescrizione());
    } catch (NullPointerException exc) {
        throw new MonetaNullException();
    }
    Moneta monetaTrovata = null;
    int indiceMonetaTrovata = indiceMonetaTrovata(moneta);
    if (indiceMonetaTrovata == -1) {
        throw new MonetaNonTrovataException("Moneta non trovata!");
    } else {
        monetaTrovata = moneta;
        monete.set(indiceMonetaTrovata, moneta);
        System.out.println("Una " + moneta.getDescrizione() + " prelevata");
    }
    return monetaTrovata;
}

/**
 * Stampa il contenuto del portamonete.
 */
public void stato() {
    System.out.println("Il portamonete contiene:");
    for (Moneta moneta : monete) {
        if (moneta == null) {
            break;
        }
        System.out.println("Una " + moneta.getDescrizione());
    }
}

/**
 * Restituisce il primo indice libero nell'array delle monete o -1 se il
 * portamonete è pieno.
 *
 * @return il primo indice libero nell'array delle monete o -1 se il
 * portamonete è pieno.
 */
private int primoIndiceLibero() {
    int indice = monete.indexOf(null);
    return indice;
}

private int indiceMonetaTrovata(Moneta moneta) {
    int indiceMonetaTrovata = -1;
    final int size = monete.size();
    for (int i = 0; i < size; i++) {
        if (monete.get(i) == null) {
            break;
        }
        Valore valoreMonetaNelPortaMoneta = monete.get(i).getValore();
        Valore valore = moneta.getValore();
        if (valore == valoreMonetaNelPortaMoneta) {
```



```
        indiceMonetaTrovata = i;
        break;
    }
}
return indiceMonetaTrovata;
}
```

Il costruttore ora ha delle asserzioni diverse, perché la grandezza di un `ArrayList` si misura diversamente dalla grandezza di un array, ma ha mantenuto la sua logica algoritmica ed ha cambiato solo il modo in cui viene aggiunto un elemento all'array list (metodo `add`).

Stessa logica anche per il metodo `aggiungi`, che ha modificato solo il modo in cui viene aggiunto un elemento con indice specificato (metodo `set`).

Il metodo privato `primoIndiceLibero` è stato notevolmente semplificato, grazie all'utilizzo del metodo `indexOf` dell'array list.

Il lettore dovrebbe essere in grado ora di trovare anche le differenze nel metodo `preleva` e nel metodo `indiceMonetaTrovata`, visto che sono molto simili a quanto abbiamo visto per i metodi `aggiungi` e `primoIndiceTrovato`.

Non è necessario modificare nessun'altra classe visto che la variabile `monete` che abbiamo modificato è incapsulata.

Soluzione 10.c)

Riportiamo di seguito i listati della classe `Frutta` e delle sue sottoclassi:

```
public abstract class Frutta {

    private final int peso;

    public Frutta(int peso) {
        this.peso = peso;
    }

    public int getPeso() {
        return peso;
    }
}

public class Mela extends Frutta {
    public Mela(int peso) {
        super(peso);
    }
}

public class Pesca extends Frutta {
    public Pesca(int peso) {
        super(peso);
    }
}

public class Arancia extends Frutta {
    public Arancia(int peso) {
        super(peso);
    }
}
```

La classe che rappresenta l'eccezione potrebbe essere la seguente:

```
public class PesoException extends Exception {
    public PesoException(String messaggio) {
        super(messaggio);
    }
}
```

La classe Cesta, invece potremmo codificarla così:

```
import java.util.ArrayList;

public class Cesta<F extends Frutta> {

    private ArrayList<F> frutta;

    public Cesta() {
        frutta = new ArrayList<>();
    }

    public ArrayList<F> getFrutta() {
        return frutta;
    }

    public void aggiungiFrutta(F frutto) throws PesoException {
        final int nuovoPeso = getPeso() + frutto.getPeso();
        if (nuovoPeso > 2000) {
            throw new PesoException("Tropo peso: " + nuovoPeso + " grammi!");
        }
        frutta.add(frutto);
        System.out.println(frutto.getClass().getName() + " da "
            + frutto.getPeso() + " grammi aggiunta alla cesta");
    }

    public int getPeso() {
        int peso = 0;
        for (F frutto : frutta) {
            peso += frutto.getPeso();
        }
        return peso;
    }
}
```

Si noti che con il tipo parametro possiamo usare una qualsiasi sottoclasse di `Frutta`. Il tipo `F`, viene usato poi nei metodi della classe e sarà sostituito durante l'esecuzione con una sottoclasse di `Frutta`.

Infine possiamo codificare una semplice classe di test nella seguente maniera:

```
public class TestFrutta {
    public static void main(String args[]) {
        Cesta<Mela> cestaDiMele = new Cesta<>();
        for (int i = 0; i <= 20; i++) {
            try {
                Mela mela = new Mela(100);
                cestaDiMele.aggiungiFrutta(mela);
            } catch (PesoException exc) {
                System.out.println(exc.getMessage());
            }
        }
    }
}
```

Segue l'output dell'esecuzione della classe `TestFrutta`:

```
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
```

```
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Mela da 100 grammi aggiunta alla cesta
Tropo peso: 2100 grammi!
```

Soluzione 10.d)

La risposta corretta è la numero 1. La numero 2 è errata perché l'operatore diamond si utilizza sull'istanza non sul reference. La numero 3 è errata per l'ordine errato delle parentesi tonde e i parametri generici dell'istanza (invertendo l'ordine sarebbe stato uno statement compilabile). Le ultime due sono errate perché un `HashMap` ha sempre due parametri.

Soluzione 10.e)

Le risposte corrette sono la numero 1, la numero 2 e la numero 4. La numero 4 in particolare è un caso particolare perché si tratta di un raw type che usa l'operatore diamond, che in questo particolare caso è praticamente opzionale. La numero 3 è errata perché un `ArrayList` ha sempre un unico parametro generico.

Soluzione 10.f)

Le risposte corrette sono la numero 1 e la numero 4. La numero 2 è scorretta perché una mappa ha sempre due parametri. La numero 3 non compila perché non c'è coincidenza tra i parametri del reference (due stringhe, e questo è già un errore perché una lista ha sempre un solo parametro) e l'istanza (un `HashMap`). La numero 5 è errata perché i parametri generici tra reference e istanza non coincidono. Infatti `Map` non coincide con `HashMap`.

Ecco infatti l'output dell'esecuzione:

```
jshell> List<Map<String, String>> al =
    new ArrayList<HashMap<String, String>>() ;
| Error:
| incompatible types: java.util.ArrayList<
| java.util.HashMap<java.lang.String,java.lang.String>> cannot be
```

```
converted to java.util.List<java.util.Map<
java.lang.String,java.lang.String>>
| List<Map<String, String>> al =
new ArrayList<HashMap<String, String>>();
| ^-----^
```

Soluzione 10.g)

La risposta corretta è la 1. Infatti non avendo specificato il parametro generico (si parla in questi casi di raw type), sarà possibile aggiungere elementi di ogni tipo alla collezione. Si noti che aggiungendo parametri primitivi, questi vengono automaticamente promossi ai relativi oggetti wrapper (in questo caso '2' viene promossa a Character, mentre 3 ad Integer). Segue l'output dell'esecuzione:

```
123
```

Soluzione 10.h)

La risposta corretta è la 1. Segue l'output dell'esecuzione:

```
123
```

Soluzione 10.i)

La risposta corretta è la 2. Non è possibile aggiungere un tipo primitivo (né altri tipi complessi diversi da String) ad una collezione che è parametrizzata con un tipo String. Infatti l'output della compilazione è il seguente:

```
Esercizio10I.java:8: error: no suitable method found for add(char)
    list.add('3');
        ^
    method Collection.add(String) is not applicable
      (argument mismatch; char cannot be converted to String)
    method List.add(String) is not applicable
      (argument mismatch; char cannot be converted to String)
Note: Some messages have been simplified; recompile with
-Xdiags:verbose to get full output
1 error
```

Soluzione 10.j)

Se parliamo di un array di interi, esistendo l'autoboxing-unboxing ci possiamo anche rifare ad un array di Integer. Visto che questa classe è dichiarata final, e che quindi non si può estendere, è impensabile crearne una sottoclasse che implementa Comparable. L'unica soluzione che ci rimane è creare una classe Comparator che ordina gli interi in maniera inversa:

```
import java.util.Comparator;

public class IntegerComparator implements Comparator<Integer> {
    @Override
```

```
    public int compare(Integer o1, Integer o2) {  
        return -(o1.compareTo(o2));  
    }  
}
```

La classe di test potrebbe limitarsi a questa:

```
import java.util.Arrays;  
  
public class TestIntegerComparator {  
    public static void main(String args[]) {  
        Integer []array = {1942, 1947, 1971, 1984, 1976, 1974};  
        Arrays.sort(array, new IntegerComparator());  
        for (int intero : array) {  
            System.out.println(intero);  
        }  
    }  
}
```

il cui output sarà:

```
1984  
1976  
1974  
1971  
1947  
1942
```

Soluzione 10.k)

Il listato per la classe WrapperComparable dovrebbe essere definito così:

```
public class WrapperComparable implements Comparable<WrapperComparable> {  
  
    private Integer integer;  
  
    public WrapperComparable(Integer integer) {  
        if (integer == null) {  
            integer = 0;  
        }  
        this.integer = integer;  
    }  
  
    public Integer getInteger() {  
        return integer;  
    }  
  
    public void setInteger(Integer integer) {  
        this.integer = integer;  
    }  
  
    @Override  
    public int compareTo(WrapperComparable otherWrapperComparable) {  
        return -(integer.compareTo(otherWrapperComparable.getInteger()));  
    }  
  
    @Override  
    public String toString() {  
        return "WrapperComparable("+integer+ ")";  
    }  
}
```

Il listato per la classe `TestWrapperComparable` potrebbe essere il seguente:

```
import java.util.Arrays;

public class TestWrapperComparable {
    public static void main(String args[]) {
        WrapperComparable[] array = {new WrapperComparable(1942),
            new WrapperComparable(1974), new WrapperComparable(1907)};
        Arrays.sort(array);
        for (WrapperComparable wrapperComparable : array) {
            System.out.println(wrapperComparable);
        }
    }
}
```

Soluzione 10.l)

L'unica affermazione corretta è la numero 3.

L'affermazione numero 1 non è valida perché le collezioni sono ridimensionabili di default, e il valore 1 passato al costruttore dell'`HashMap`, rappresenta solo la dimensione iniziale della collection.

Anche l'affermazione numero 2 non è corretta. Infatti i valori 07 e 0x0ABC sono valori interi e quindi con l'autoboxing sono correttamente "wrappati" all'interno di oggetti `Integer`.

Invece i valori 'a' e 'b' di cui si parla nell'affermazione 3 sono caratteri e sono "wrappati" all'interno di oggetti `Character`.

Segue l'output della compilazione della classe `Esercizio10L`:

```
Esercizio10L.java:7: error: incompatible types: char cannot be
    converted to Integer
        map.put('a','b');
                ^
Note: Some messages have been simplified; recompile with
    -Xdiags:verbose to get full output
1 error
```

Soluzione 10.m)

La risposta è l'ultima, visto che per aggiungere coppie chiave-valore ad una mappa, si usa il metodo `put`, non il metodo `add` (che è un metodo dichiarato nelle liste).

Soluzione 10.n)

Le opzioni corrette sono la 4 e la 6.

L'opzione numero 1 è errata perché non si può assegnare un reference con tipo generico `String`, ad un'istanza con tipo generico `Object`. Infatti, compilando il file si ottiene il seguente output:

```
Esercizio10N.java:2: error: incompatible types: Esercizio10N<Object>
    cannot be converted to Esercizio10N<String>
        Esercizio10N<String> g = new Esercizio10N<Object>();
                                   ^
where Object is a type-variable:
```

```
Object extends java.lang.Object declared in class Esercizio10N
1 error
```

La numero 2 fallirà alla prima riga, perché non è possibile utilizzare un operatore diamond come tipo generico di una classe. La numero 3 fallirà anch'essa in compilazione perché non abbiamo dichiarato un tipo generico per la classe. Segue l'output che riporta gli errori derivanti dal processo di compilazione:

```
Esercizio10N.java:2: error: type Esercizio10N does not take parameters
    Esercizio10N<String> g = new Esercizio10N<>();
    ^
Esercizio10N.java:3: error: type Esercizio10N does not take parameters
    Esercizio10N<Object> g2 = new Esercizio10N<>();
    ^
Esercizio10N.java:2: error: cannot infer type arguments for
    Esercizio10N
    Esercizio10N<String> g = new Esercizio10N<>();
                                ^
    reason: cannot use '<>' with non-generic class Esercizio10N
Esercizio10N.java:3: error: cannot infer type arguments for
    Esercizio10N
    Esercizio10N<Object> g2 = new Esercizio10N<>();
                                ^
    reason: cannot use '<>' with non-generic class Esercizio10N
4 errors
```

La numero 5 è anch'essa errata. Infatti le wildcard si utilizzano con i parametri di metodi. Segue l'output della compilazione:

```
Esercizio10N.java:1: error: <identifier> expected
public class Esercizio10N<? extends Object> {
    ^
1 error
```

La numero 7 infine, produce il seguente output in compilazione:

```
Esercizio10N.java:2: error: incompatible types: Esercizio10N<T>
    cannot be converted to Esercizio10N<String>
    Esercizio10N<String> g = new Esercizio10N<T>();
                                ^
    where T is a type-variable:
      T extends Object declared in class Esercizio10N
1 error
```

dove veniamo avvisati che `Esercizio10M<T>`, non può essere convertito a `Esercizio10N<String>`.

Soluzione 10.o)

Le opzioni corrette sono la 3 e la 5, che sono sostanzialmente equivalenti, ma anche la 7 dove utilizziamo un raw type. Infatti compilando utilizzando l'opzione 7 otterremo un warning:

```
Esercizio10O.java:4: warning: [rawtypes] found raw type: List
    public static int getSize(List/*INSERISCI CODICE QUI*/ list) {
                             ^
    missing type arguments for generic class List<E>
    where E is a type-variable:
      E extends Object declared in interface List
1 warning
```

L'opzione 1 è errata, perché il parametro `Object` non ammette altri tipi parametri al di fuori di `Object`. Infatti ecco l'output:

```
Esercizio100.java:9: error: incompatible types: ArrayList<Integer>
    cannot be converted to List<Object>
        System.out.println(getSize(new ArrayList<Integer>()));
                                   ^
Esercizio100.java:10: error: incompatible types:
    ArrayList<HashMap<String,List<String>>> cannot be converted to
    List<Object>
        System.out.println(getSize(new ArrayList<
    HashMap<String, List<String>>>()));
                                   ^
Note: Some messages have been simplified; recompile with
    -Xdiags:verbose to get full output
2 errors
```

L'opzione 2 non ha senso (sintassi errata). Infatti ecco l'output:

```
Esercizio100.java:4: error: illegal start of type
    public static int getSize(List<> list) {
                               ^
1 error
```

L'opzione 4 e l'opzione 6 sono entrambe errate per lo stesso motivo. Il tipo `T` non è definito da nessuna parte. Segue l'output con l'opzione 4:

```
Esercizio100.java:4: error: cannot find symbol
    public static int getSize(List<T> list) {
                               ^
    symbol:   class T
    location: class Esercizio100
1 error
```

Che è praticamente identico a quello dove si utilizza l'opzione 6:

```
Esercizio100.java:4: error: > expected
    public static int getSize(List<T extends Object> list) {
                               ^
1 error
```

Soluzione 10.p)

L'affermazione corretta è la numero 3. Infatti l'output è il seguente:

```
Esercizio10P.java:4: warning: [static] static method should be
    qualified by type name, T, instead of by an expression
        return t.values();
               ^
1 warning
```

Ovvero il compilatore ci sta avvertendo che il metodo `values` è statico e quindi si dovrebbe chiamare utilizzando il nome dell'enumerazione, e non su un suo elemento.

Per il resto delle risposte è facile capire perché non siano corrette. In particolare per la numero 1 la lettera `T` è proprio l'identificatore dell'enumerazione, quindi la domanda è posta in maniera completamente errata.

Soluzione 10.q)

L'opzione corretta è la numero 6. Laddove non c'è il bounded parameter di tipo `Number`, otterremo un errore in compilazione come il seguente:

```
Esercizio10Q.java:4: error: cannot find symbol
    return Integer.max(n1.intValue(), n2.intValue());
                        ^
symbol:   method intValue()
location: variable n1 of type N
where N is a type-variable:
  N extends Object declared in class Esercizio10Q
Esercizio10Q.java:4: error: cannot find symbol
    return Integer.max(n1.intValue(), n2.intValue());
                        ^
symbol:   method intValue()
location: variable n2 of type N
where N is a type-variable:
  N extends Object declared in class Esercizio10Q
2 errors
```

Soluzione 10.r)

La soluzione potrebbe essere la seguente (il metodo richiesto è evidenziato in grassetto):

```
import java.util.List;
import java.util.ArrayList;

public class Esercizio10R<T> {

    private <L extends List<Integer>> Integer sumElements(L list) {
        int size = list.size();
        int result = 0;
        for (int i = 0; i < size; i++) {
            Integer item = list.get(i);
            result += item;
        }
        return result;
    }

    public static void main(String args[]) {
        Esercizio10R<Void> e = new Esercizio10R<>();
        ArrayList<Integer> a = new ArrayList<>();
        a.add(2);
        a.add(6);
        a.add(10);
        a.add(24);
        a.add(17);
        System.out.println("La somma degli elementi della lista è "
            + e.sumElements(a));
    }
}
```

Soluzione 10.s)

Una possibile soluzione è rappresentata dal seguente codice, che dichiara un classico bounded parameter, e, come richiesto, dichiara il metodo `stampa` che delega il suo reale funzionamento all'implementazione del metodo `stampa` della tecnologia:

```
public class Stampante<T extends Tecnologia> {  
  
    private T tecnologia;  
  
    public Stampante(T tecnologia) {  
        this.tecnologia = tecnologia;  
    }  
  
    public void stampa() {  
        tecnologia.stampa();  
    }  
}
```

L'effettivo funzionamento può essere testato dalla seguente classe di test:

```
public class TestStampante {  
  
    public static void main(String args[]) {  
        Stampante<Laser> stampante = new Stampante<>(new Laser());  
        stampante.stampa();  
        Stampante<Inkjet> stampante2 = new Stampante<>(new Inkjet());  
        stampante2.stampa();  
    }  
}
```

che produce il seguente output:

```
Stampa laser  
Stampa getto d'inchiostro
```

Soluzione 10.t)

La classe `Operazione` potrebbe essere la seguente:

```
public class Operazione<Integer, O extends Operatore> {  
  
    private Integer operand1;  
    private O operatore;  
    private Integer operand2;  
  
    public Operazione(Integer operand1, O operatore, Integer operand2) {  
        this.operand1 = operand1;  
        this.operatore = operatore;  
        this.operand2 = operand2;  
    }  
  
    public void stampa() {  
        System.out.println(operand1 + " " + operatore + " " + operand2);  
    }  
}
```

Soluzione 10.u)

Le affermazioni corrette sono la 2 e la 4.

Soluzione 10.v)

Le affermazioni corrette sono la 1 e la 4. In particolare bisognerebbe creare un metodo helper come il seguente in grassetto:

```
import java.util.List;

public class Soluzione11V {
    protected final void modifica(List<?> list) {
        helperMethod(list);
    }

    private <T> void helperMethod(List<T> list) {
        list.add(list.get(0));
    }
}
```

Soluzione 10.w)

La nostra soluzione consiste nel creare un'enumerazione `Segno`, che definisce i tre segni del gioco della morra cinese:

```
public enum Segno {
    SASSO, CARTA, FORBICI;
}
```

Poi abbiamo creato una classe che abbiamo chiamato `Regole`, che rappresenta il “nucleo di business” del gioco. Qui è presente l'algoritmo che definisce il vincitore. Abbiamo definito tale algoritmo implementando il metodo `compare` dell'interfaccia `Comparator`. La scelta è ricaduta su tale metodo più per scopi didattici che per reale utilità, probabilmente esistevano soluzioni migliori:

```
import java.util.Comparator;

public class Regole implements Comparator<Segno> {
    @Override
    public int compare(Segno segno1, Segno segno2) {
        int risultato = 0;
        switch (segno1) {
            case CARTA: {
                if (segno2 == Segno.SASSO) {
                    risultato = 1;
                } else if (segno2 == Segno.FORBICI) {
                    risultato = -1;
                }
            }
            break;
            case SASSO: {
                if (segno2 == Segno.FORBICI) {
                    risultato = 1;
                } else if (segno2 == Segno.CARTA) {
                    risultato = -1;
                }
            }
            break;
            case FORBICI: {
                if (segno2 == Segno.CARTA) {
                    risultato = 1;
                } else if (segno2 == Segno.SASSO) {

```

```
        risultato = -1;
    }
}
break;
default: {
    risultato = 0;
}
break;

}
return risultato;
}
}
```

La classe più importante è la classe `MorraCinese` che definisce un unico metodo pubblico `gioca`, e tre metodi privati:

`getSegno` che recupera un elemento dell'enumerazione `Segno` in base alla sua posizione (`id`).

`getSegnoComputer` che riutilizza il metodo `getSegno` passandogli un numero casuale tra 0 e 2.

`stabilisciVincitore` che restituisce la stringa da stampare come output del programma, basandosi sulla comparazione definita nell'oggetto `Regole`.

```
import java.util.Random;

public class MorraCinese {

    public void gioca(int id) {
        Segno segnoGiocatore = getSegno(id);
        Segno segnoComputer = getSegnoComputer();
        System.out.println(segnoGiocatore + " VS " + segnoComputer);
        String risultato = stabilisciVincitore(segnoGiocatore, segnoComputer);
        System.out.println(risultato);
    }

    private String stabilisciVincitore(Segno segnoGiocatore,
                                       Segno segnoComputer) {
        Regole regole = new Regole();
        int risultato = regole.compare(segnoGiocatore, segnoComputer);
        if (risultato > 0) {
            return "Vince " + segnoGiocatore + "!\nHai vinto!";
        } else if (risultato < 0) {
            return "Vince " + segnoComputer + "!\nHai perso!";
        } else {
            return "Pari!";
        }
    }

    private Segno getSegno(int id) {
        Segno[] segni = Segno.values();
        Segno segnoComputer = segni[id];
        return segnoComputer;
    }

    private Segno getSegnoComputer() {
        Random random = new Random();
        return getSegno(random.nextInt(3));
    }
}
```

Infine la classe del main, gestisce un'eventuale input da riga di comando dell'utente e chiama il metodo gioca di MorraCinese:

```
import java.util.Random;
```

```
public class Esercizio10W {  
  
    public static void main(String args[]) {  
        int id = getId(args);  
        MorraCinese morraCinese = new MorraCinese();  
        morraCinese.gioca(id);  
    }  
  
    public static int getId(String args[]) {  
        int id = 0;  
        if (args.length != 0) {  
            try {  
                id = Integer.parseInt(args[0]);  
                if (id < 0 || id > 2) {  
                    System.out.println("Inserire un numero compreso tra 0 e 2");  
                    System.exit(1);  
                }  
            } catch (Exception exc) {  
                System.out.println("Input non valido: "+ args[0]);  
                System.exit(1);  
            }  
        } else {  
            id = new Random().nextInt(3);  
        }  
        return id;  
    }  
}
```

Ecco qualche esempio di output:

```
SASSO VS FORBICI
```

```
Vince SASSO!
```

```
Hai vinto!
```

```
FORBICI VS CARTA
```

```
Vince FORBICI!
```

```
Hai vinto!
```

```
SASSO VS SASSO
```

```
Pari!
```

```
FORBICI VS SASSO
```

```
Vince SASSO!
```

```
Hai perso!
```

Soluzione 10.z)

Gli statement corretti sono i primi 3, solo l'uso dello statement 4 causerebbe un errore in compilazione, segue l'output:

```
Esercizio10Z.java:6: error: method printException in class  
    Esercizio10Z cannot be applied to given types;  
        /*INSERISCI CODICE QUI*/Esercizio10Z.printException(  
                                           ^  
                                           required: ()
```

```
new Throwable("Exception"));  
^  
required: E  
found: Throwable  
reason: inference variable E has incompatible bounds  
  upper bounds: Exception  
  lower bounds: Throwable  
where E is a type-variable:  
  E extends Exception declared in method <E>printException(E)  
1 error
```

Nel caso della risposta numero 1, abbiamo usato una sintassi che si utilizza raramente.

Soluzioni degli esercizi del capitolo 11 e dell'appendice E

Tipi Annotazioni, Reflection e la variabile d'ambiente CLASSPATH

Soluzione 11.a) Annotazioni, dichiarazioni ed uso, Vero o Falso:

1. **Falso**, è un tipo annotazione.
2. **Falso**, è un tipo annotazione.
3. **Vero**.
4. **Falso**, un elemento di un'annotazione non può avere come tipo `void`.
5. **Falso**, un metodo di un'annotazione non può avere parametri in input.
6. **Vero**.
7. **Falso**, infatti è legale sia il codice del metodo `m()`, sia dichiarare `public` prima dell'annotazione (ma ovviamente è un modificatore del metodo). Non è legale però passare in input all'annotazione il valore `MiaAnnotazione.MiaEnum.VERO` senza specificare una sintassi del tipo `chiave = valore`.
8. **Falso**, infatti la sintassi:

```
return @MiaAnnotazione.MyEnum;
```

non è valida. Non si può utilizzare un'annotazione come se fosse una classe con variabili statiche pubbliche.
9. **Falso**, infatti l'annotazione in questione non è a valore singolo, perché il suo unico elemento non si chiama `value`.
10. **Vero**.

Soluzione 11.b) Annotazioni e libreria, Vero o Falso:

1. **Vero**, infatti se non si specifica con la meta-annotazione `Target` quali sono gli elementi a cui è applicabile l'annotazione in questione, l'annotazione sarà di default applicabile a qualsiasi elemento tranne tipi parametro e uso dei tipi.
2. **Falso**, il valore `ElementType.SOURCE` non esiste.
3. **Falso**, il valore `ElementType.@INTERFACE` non esiste.
4. **Falso**, non è neanche applicabile a metodi per via del valore di `Target`, che è `ElementType.ANNOTATION_TYPE`.
5. **Falso**, infatti non può essere applicata ad una classe se è annotata con `@Target(ElementType.METHOD)`.
6. **Vero**.
7. **Falso**, al compilatore, non al runtime.
8. **Vero**.
9. **Vero**.
10. **Vero**, `Override` non è applicabile a classi.

Soluzione 11.c)

Il listato dell'annotazione Breve dovrebbe essere simile al seguente:

```
package metadati;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Inherited;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
public @interface Breve {
}
```

Si noti che per i nostri scopi abbiamo dovuto specificare il livello di Retention a RetentionPolicy.RUNTIME.

La classe ClasseBreve potrebbe invece essere modificata nel seguente modo:

```
package dati;

import metadati.Breve;

@Breve
public class ClasseBreve {
    public void metodo1() {
        System.out.println("metodo1");
    }
}
```

La classe ClasseLunga invece, potrebbe essere simile alla seguente:

```
package dati;

import metadati.Breve;

@Breve
public class ClasseLunga {
    public void metodo1() {
        System.out.println("metodo1");
    }
    public void metodo2() {
        System.out.println("metodo2");
    }
    public void metodo3() {
        System.out.println("metodo3");
    }
    public void metodo4() {
        System.out.println("metodo4");
    }
}
```

Infine ecco l'eccezione richiesta:

```
package eccezioni;

public class AnnotationException extends Exception {
    public AnnotationException(String msg) {
```



```

        super(msg);
    }

    @Override
    public String toString() {
        return "AnnotationException{" + getMessage() + "}";
    }
}

```

Soluzione 11.d)

Il listato del verificatore richiesto potrebbe essere il seguente:

```

package test;

import eccezioni.AnnotationException;
import java.lang.annotation.Annotation;
import java.lang.reflect.Method;
import java.util.Scanner;
import metadati.Breve;

public class VerificatoreInterattivo {

    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        String stringa = "";
        System.out.println("Digita il nome di un file java presente nella "
            + " cartella corrente e batti enter, oppure scrivi \"fine\" "
            + "per terminare il programma");
        while (!(stringa = scanner.next()).equals("fine")) {
            System.out.println("Hai digitato " + stringa.toUpperCase() + "!");
            try {
                verificaClasse(stringa);
            } catch (Exception ex) {
                ex.printStackTrace();
            }
        }
        System.out.println("Fine programma!");
    }

    private static void verificaClasse(String stringa) throws Exception {
        Class objectClass = Class.forName(stringa);
        try {
            System.out.println("Inizio verifica annotazione @Breve per " +
                stringa);
            Annotation breve = objectClass.getAnnotation(Breve.class);
            if (breve != null) {
                Method[] methods = objectClass.getDeclaredMethods();
                final int numeroMetodi = methods.length;
                if (numeroMetodi > 3) {
                    throw new AnnotationException("Ci sono " + numeroMetodi
                        + " metodi nella classe " + stringa);
                }
                System.out.println("Classe " + stringa +
                    " corretta!\nlista metodi:");
                for (Method method : methods) {
                    System.out.println(method);
                }
            }
        }
    }
}

```

```

        } else {
            System.out.println("Questa classe non è annotata con @Breve");
        }
    } finally {
        System.out.println("Fine verifica annotazione @Breve per " +
            stringa);
    }
}
}

```

In pratica si tratta solo di concentrarsi sul metodo `verificaClasse`, il cui codice è comunque molto intuitivo.

Il seguente output viene prodotto specificando prima le due classi davvero annotate con `@Breve`, poi una che non è annotata (la classe `AnnotationException`) e infine una classe inesistente:

```

Digita il nome di un file java presente nella cartella corrente e
batti enter, oppure scrivi "fine" per terminare il programma
dati.ClasseBreve
Hai digitato DATI.CLASSEBREVE!
Inizio verifica annotazione @Breve per dati.ClasseBreve
Classe dati.ClasseBreve corretta!
lista metodi:
public void dati.ClasseBreve.metodo1()
public void dati.ClasseBreve.setVariabile(java.lang.String)
public java.lang.String dati.ClasseBreve.getVariabile()
Fine verifica annotazione @Breve per dati.ClasseBreve
dati.ClasseLunga
Hai digitato DATI.CLASSELUNGA!
Inizio verifica annotazione @Breve per dati.ClasseLunga
Fine verifica annotazione @Breve per dati.ClasseLunga
AnnotationException{Ci sono 4 metodi nella classe dati.ClasseLunga}
    at
    test.VerificatoreInterattivo.verificaClasse(
    VerificatoreInterattivo.java:37)
    at
    test.VerificatoreInterattivo.main(VerificatoreInterattivo.java:20)

eccezioni.AnnotationException
Hai digitato ECCEZIONI.ANNOTATIONEXCEPTION!
Inizio verifica annotazione @Breve per eccezioni.AnnotationException
Questa classe non è annotata con @Breve
Fine verifica annotazione @Breve per eccezioni.AnnotationException

ClasseInesistente
Hai digitato CLASSEINESISTENTE!
java.lang.ClassNotFoundException: ClasseInesistente
    at java.net.URLClassLoader$1.run(URLClassLoader.java:372)
    at java.net.URLClassLoader$1.run(URLClassLoader.java:361)
    at java.security.AccessController.doPrivileged(Native Method)
    at java.net.URLClassLoader.findClass(URLClassLoader.java:360)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:424)
    at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:308)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:357)
    at java.lang.Class.forName0(Native Method)
    at java.lang.Class.forName(Class.java:259)
    at test.VerificatoreInterattivo.verificaClasse(
    VerificatoreInterattivo.java:29)

```

```
at
test.VerificatoreInterattivo.main(VerificatoreInterattivo.java:20)
```

Soluzione 11.e)

Il listato dell'annotazione Specifica dovrebbe essere il seguente:

```
package metadati;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Inherited;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
public @interface Specifica {
    int value();
}
```

Il listato delle classi ClasseBreve e ClasseLunga potrebbe essere modificato come segue:

```
package dati;

import metadati.Breve;
import metadati.Specifica;

@Breve
@Specifica(1)
public class ClasseBreve {

    public void metodo1() {
        System.out.println("metodo1");
    }
    private String variabile;

    public String getVariabile() {
        return variabile;
    }

    public void setVariabile(String variabile) {
        this.variabile = variabile;
    }
}
```

e:

```
package dati;

import metadati.Specifica;

@Breve
@Specifica(3)
public class ClasseLunga {

    public String variabile1;
    public String variabile2;
```

```
public void metodo1() {
    System.out.println("metodo1");
}

public void metodo2() {
    System.out.println("metodo2");
}

public void metodo3() {
    System.out.println("metodo3");
}

public void metodo4() {
    System.out.println("metodo4");
}
}
```

Soluzione 11.f)

Modifichiamo il codice del nostro verificatore, estraendo come metodo il codice di verifica dell'annotazione `Breve`, e commentandone la chiamata. Creiamo un equivalente metodo per verificare l'annotazione `Specifica`. Di seguito riportiamo solo il codice modificato:

```
private static void verificaClasse(String stringa) throws Exception {
    Class objectClass = Class.forName(stringa);
    //verificaBreve(stringa, objectClass);
    verificaSpecifica(stringa, objectClass);
}

private static void verificaBreve(String stringa, Class objectClass) throws
    AnnotationException {
    try {
        System.out.println("Inizio verifica annotazione @Breve per " +
            stringa);
        Annotation breve = objectClass.getAnnotation(Breve.class);
        if (breve != null) {
            Method[] methods = objectClass.getDeclaredMethods();
            final int numeroMetodi = methods.length;
            if (numeroMetodi > 3) {
                throw new AnnotationException("Ci sono " + numeroMetodi
                    + " metodi nella classe " + stringa);
            }
            System.out.println("Classe " + stringa +
                " corretta!\nlista metodi:");
            for (Method method : methods) {
                System.out.println(method);
            }
        } else {
            System.out.println("Questa classe non è annotata con @Breve");
        }
    } finally {
        System.out.println("Fine verifica annotazione @Breve per " +
            stringa);
    }
}

private static void verificaSpecifica(String stringa, Class objectClass)
    throws AnnotationException {
```

```

    try {
        System.out.println("Inizio verifica annotazione @Specifica per "
            + stringa);
        Specifica specifica = (Specifica)objectClass.getAnnotation(
            Specifica.class);
        if (specifica != null) {
            int numeroVariabiliDaSpecifica = specifica.value();
            Field[] fields = objectClass.getDeclaredFields();
            final int numeroVariabili = fields.length;
            if (numeroVariabili != numeroVariabiliDaSpecifica) {
                throw new AnnotationException("Ci sono " + numeroVariabili
                    + " variabili nella classe " + stringa +
                    " ma dovrebbero essere " +
                    numeroVariabiliDaSpecifica);
            }
            System.out.println("Classe " + stringa +
                " corretta!\nlista variabili:");
            for (Field field : fields) {
                System.out.println(field);
            }
        } else {
            System.out.println(
                "Questa classe non è annotata con @Specifica");
        }
    } finally {
        System.out.println("Fine verifica annotazione @Breve per " +
            stringa);
    }
}

```

Ecco un esempio di output:

```

Digita il nome di un file java presente nella cartella corrente e
batti enter, oppure scrivi "fine" per terminare il programma
dati.ClasseBreve
Hai digitato DATI.CLASSEBREVE!
Inizio verifica annotazione @Specifica per dati.ClasseBreve
Classe dati.ClasseBreve corretta!
lista variabili:
private java.lang.String dati.ClasseBreve.variabale
Fine verifica annotazione @Breve per dati.ClasseBreve
dati.ClasseLunga
Hai digitato DATI.CLASSELUNGA!
Inizio verifica annotazione @Specifica per dati.ClasseLunga
Fine verifica annotazione @Breve per dati.ClasseLunga
AnnotationException{Ci sono 2 variabili nella classe dati.ClasseLunga
ma dovrebbero essere 3}
    at
    test.VerificatoreInterattivo.verificaSpecifica(
VerificatoreInterattivo.java:69)
    at
    test.VerificatoreInterattivo.verificaClasse(
VerificatoreInterattivo.java:33)
    at
    test.VerificatoreInterattivo.main(VerificatoreInterattivo.java:22)

```

Soluzione 11.g)

Il listato dell'annotazione Bean potrebbe essere il seguente:

```
package metadati;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Inherited;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
public @interface Bean {
    int numeroMassimoMetodi();
    int numeroMinimoVariabili();
}
```

Il listato delle classi ClasseBreve e ClasseLunga potrebbe essere modificato come segue:

```
package dati;

import metadati.Bean;
import metadati.Breve;
import metadati.Specifica;

@Breve
@Specifica(1)
@Bean(numeroMassimoMetodi = 10, numeroMinimoVariabili = 1)
public class ClasseBreve {

    public void metodo1() {
        System.out.println("metodo1");
    }

    private String variabile;

    public String getVariabile() {
        return variabile;
    }

    public void setVariabile(String variabile) {
        this.variabile = variabile;
    }
}
```

e:

```
package dati;

import metadati.Bean;
import metadati.Breve;
import metadati.Specifica;

@Breve
@Specifica(3)
@Bean(numeroMassimoMetodi = 10, numeroMinimoVariabili = 1)
public class ClasseLunga {

    public String variabile1;
```

```
public String variabile2;

public void metodo1() {
    System.out.println("metodo1");
}

public void metodo2() {
    System.out.println("metodo2");
}

public void metodo3() {
    System.out.println("metodo3");
}

public void metodo4() {
    System.out.println("metodo4");
}
}
```

Soluzione 11.h)

Modifichiamo il codice del nostro verificatore e, sul modello della soluzione dell'esercizio 11.f, creiamo un metodo per verificare l'annotazione Bean. Di seguito riportiamo solo il codice modificato:

```
private static void verificaClasse(String stringa) throws Exception {
    Class<?> objectClass = Class.forName(stringa);
    //verificaBreve(stringa, objectClass);
    //verificaSpecifica(stringa, objectClass);
    verificaBean(stringa, objectClass);
}

private static void verificaBean(String stringa, Class<?> objectClass)
    throws AnnotationException {
    try {
        System.out.println("Inizio verifica annotazione @Bean per "
            + stringa);
        Bean bean = objectClass.getAnnotation(Bean.class);
        if (bean != null) {
            controlloNumeroVariabili(bean, objectClass, stringa);
            controlloNumeroMetodi(bean, objectClass, stringa);
            controlloCostruttoreSenzaParametro(objectClass, stringa);
            controlloIncapsulamento(objectClass, stringa);
        } else {
            System.out.println("Questa classe non è annotata con @Bean");
        }
    } finally {
        System.out.println("Fine verifica annotazione Bean per " + stringa);
    }
}

private static void controlloNumeroVariabili(Bean bean, Class<?>
    objectClass, String stringa) throws AnnotationException {
    int numeroMinimoVariabili = bean.numeroMinimoVariabili();
    Field[] fields = objectClass.getDeclaredFields();
    final int numeroVariabili = fields.length;
    if (numeroVariabili < numeroMinimoVariabili) {
        throw new AnnotationException("Ci sono " + numeroVariabili
```

```
        + " variabili nella classe " + stringa
        + " ma dovrebbero essere almeno " + numeroMinimoVariabili);
    }
    System.out.println("Classe " + stringa +
        " numero variabili ok!\nlista variabili:");
    for (Field field : fields) {
        System.out.println(field);
    }
}

private static void controlloNumeroMetodi(Beans bean, Class<?> objectClass,
    String stringa) throws AnnotationException {
    int numeroMassimoMetodi = bean.numeroMassimoMetodi();
    Method[] methods = objectClass.getDeclaredMethods();
    final int numeroMetodi = methods.length;
    if (numeroMetodi > numeroMassimoMetodi) {
        throw new AnnotationException("Ci sono " + numeroMetodi
            + " metodi nella classe " + stringa
            + " ma dovrebbero essere al massimo "
            + numeroMassimoMetodi);
    }
    System.out.println("Classe " + stringa +
        " numero metodi ok!\nlista metodi:");
    for (Method method : methods) {
        System.out.println(method);
    }
}

private static void controlloCostruttoreSenzaParametro(Class<?> objectClass,
    String stringa) throws AnnotationException {
    try {
        Constructor<?> constructor = objectClass.getConstructor();
        System.out.println("Classe " + stringa
            + " costruttore senza parametri presente!");
        System.out.println(constructor);
    } catch (NoSuchMethodException exc) {
        throw new AnnotationException(
            "Niente costruttore senza parametri!");
    }
}

private static void controlloIncapsulamento(Class<?> objectClass, String
    stringa) throws AnnotationException {
    String nomeVariabile = null;
    try {
        Field[] fields = objectClass.getDeclaredFields();
        for (Field field : fields) {
            nomeVariabile = field.getName();
            final Class<?> type = field.getType();
            final Method setMethod = objectClass.getDeclaredMethod("set" +
                capitalize(nomeVariabile), type);
            final Method getMethod = objectClass.getDeclaredMethod("get" +
                capitalize(nomeVariabile));
        }
        System.out.println("Classe " + stringa + " incapsulamento ok!");
    } catch (NoSuchMethodException exc) {
        throw new AnnotationException("Variabile " + nomeVariabile +
            " non incapsulata correttamente nella classe " + stringa);
    }
}
```



```

    private static String capitalize(String string) {
        return string.substring(0, 1).toUpperCase() + string.substring(1);
    }
}

```

Ecco un esempio di output:

```

Digita il nome di un file java presente nella cartella corrente e
batti enter, oppure scrivi "fine" per terminare il programma
dati.ClasseBreve
Hai digitato DATI.CLASSEBREVE!
Inizio verifica annotazione @Bean per dati.ClasseBreve
Classe dati.ClasseBreve numero variabili ok!
lista variabili:
private java.lang.String dati.ClasseBreve.variabile
Classe dati.ClasseBreve numero metodi ok!
lista metodi:
public void dati.ClasseBreve.metodo1()
public java.lang.String dati.ClasseBreve.getVariabile()
public void dati.ClasseBreve.setVariabile(java.lang.String)
Classe dati.ClasseBreve costruttore senza parametri presente!:
public dati.ClasseBreve()
Classe dati.ClasseBreve incapsulamento ok!
Fine verifica annotazione @Breve per dati.ClasseBreve
dati.ClasseLunga
Hai digitato DATI.CLASSELUNGA!
Inizio verifica annotazione @Bean per dati.ClasseLunga
Classe dati.ClasseLunga numero variabili ok!
lista variabili:
public java.lang.String dati.ClasseLunga.variabile1
public java.lang.String dati.ClasseLunga.variabile2
Classe dati.ClasseLunga numero metodi ok!
lista metodi:
public void dati.ClasseLunga.metodo3()
public void dati.ClasseLunga.metodo2()
public void dati.ClasseLunga.metodo4()
java.lang.NoSuchMethodException:
dati.ClasseLunga.setVariabile1(java.lang.String)
public void dati.ClasseLunga.metodo1()
    at java.lang.Class.getDeclaredMethod(Class.java:2117)
Classe dati.ClasseLunga costruttore senza parametri presente!:
public dati.ClasseLunga()
    at test.VerificatoreInterattivo.controlloIncapsulamento(
    VerificatoreInterattivo.java:152)
Fine verifica annotazione @Breve per dati.ClasseLunga
    at test.VerificatoreInterattivo.verificaBean(
    VerificatoreInterattivo.java:97)
    at test.VerificatoreInterattivo.verificaClasse(
    VerificatoreInterattivo.java:36)
    at test.VerificatoreInterattivo.main(
    VerificatoreInterattivo.java:24)

```

Soluzione 11.i)

Tutte le affermazioni sull'annotazione Override sono corrette.

Soluzione 11.j)

Il listato della classe ReflectionInterattiva potrebbe essere il seguente:

```
import java.lang.reflect.Method;
import java.util.Scanner;

public class ReflectionInterattiva {

    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        String stringa = "";
        System.out.println("Digita il nome di una classe presente nella"
            + " cartella corrente e batti enter, oppure scrivi \"fine\" "
            + " per terminare il programma");
        while (!(stringa = scanner.next()).equals("fine")) {
            System.out.println("Hai digitato " + stringa.toUpperCase() + "!");
            try {
                stampaMetodi(stringa);
            } catch (ClassNotFoundException ex) {
                ex.printStackTrace();
            }
            System.out.println("Fine programma!");
        }

        private static void stampaMetodi(String stringa) throws
            ClassNotFoundException {
            Class objectClass = Class.forName(stringa);
            Method[] methods = objectClass.getDeclaredMethods();
            for (Method method : methods) {
                System.out.println(method);
            }
        }
    }
}
```

Si noti che abbiamo utilizzato il metodo `getDeclaredMethods` invece di `getMethods`, perché quest'ultimo avrebbe anche stampato i metodi ereditati dalle superclassi (nel caso si specifichi la classe `Moneta` i metodi di `Object`).

L'output del precedente listato è il seguente:

```
Digita il nome di una classe presente nella cartella corrente e batti
enter, oppure scrivi "fine" per terminare il programma
Moneta
Hai digitato MONETA!
Caricata la classe Moneta con valuta = EURO
public boolean Moneta.equals(java.lang.Object)
public int Moneta.hashCode()
public java.lang.String Moneta.getDescrizione()
public Valore Moneta.getValore()
fine
Fine programma!
```

Soluzione 11.k)

Il listato della classe `CompilatoreInterattivo` potrebbe essere il seguente:

```
import java.util.Scanner;

public class CompilatoreInterattivo {

    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        String stringa = "";
        System.out.println("Digita il nome di un file java presente nella "
            + " cartella corrente e batti enter, oppure scrivi \"fine\" "
            + "per terminare il programma");
        while (!(stringa = scanner.next()).equals("fine")) {
            System.out.println("Hai digitato " + stringa.toUpperCase() + "!");
            try {
                compilaClasse(stringa);
            } catch (Exception ex) {
                ex.printStackTrace();
            }
        }
        System.out.println("Fine programma!");
    }

    private static void compilaClasse(String stringa) throws Exception {
        Runtime runtime = Runtime.getRuntime();
        Process process = runtime.exec("javac " + stringa);
        final int exitValue = process.waitFor();
        System.out.println(exitValue == 0 ? stringa + " compilato!" :
            "Impossibile compilare " + stringa);
    }
}
```

Si noti che avremmo potuto anche utilizzare la `Compiler API`, con un codice semplice come il seguente:

```
JavaCompilerTool compiler = ToolProvider.defaultJavaCompiler();
compiler.run(new FileInputStream("MyClass.java"), null, null);
```

Si noti inoltre che abbiamo catturato con il metodo `waitFor` il codice di uscita del processo di compilazione. Se questo vale 0 allora il file è stato compilato correttamente.

L'output del nostro programma sarà:

```
D:\esercizi>dir *.class

D:\esercizi>javac CompilatoreInterattivo.java

D:\esercizi>dir *.class
Il volume nell'unità C è OS
Numero di serie del volume: 48D1-FF51

Directory di D:\esercizi>

08/06/2014  15.11          1.693 CompilatoreInterattivo.class
                1 File              1.693 byte
                0 Directory  98.803.150.848 byte disponibili

D:\esercizi>java CompilatoreInterattivo
Digita il nome di un file java presente nella cartella corrente e batti
enter, oppure scrivi "fine"
Moneta
```

```

Hai digitato MONETA!
Impossibile compilare Moneta
Moneta.java
Hai digitato MONETA.JAVA!
Moneta.java compilato!
fine
Fine programma!

D:\esercizi>dir *.class
Il volume nell'unità C è OS
Numero di serie del volume: 48D1-FF51

Directory di D:\esercizi>

08/06/2014  13.47          1.693 CompilatoreInterattivo.class
08/06/2014  13.47          1.345 Moneta.class
08/06/2014  13.33           627 Valore$1.class
08/06/2014  13.33           636 Valore$2.class
08/06/2014  13.33           636 Valore$3.class
08/06/2014  13.33          1.786 Valore.class
                6 File              6.723 byte
                0 Directory 98.804.051.968 byte disponibili

```

Da riga di comando abbiamo prima cercato di visualizzare tutti i file .class con il comando `dir *.class` (cfr. appendice C), ma non ce n'erano. Poi abbiamo compilato la classe `CompilatoreInterattivo.java` e abbiamo controllato che il file `CompilatoreInterattivo.class` fosse stato generato. Poi abbiamo eseguito il nostro programma e nella nostra sessione interattiva abbiamo prima cercato di compilare il file `Moneta`, ma questo non esiste. Poi abbiamo provato a compilare il file `Moneta.java` e abbiamo verificato che effettivamente la compilazione sia andata a buon fine. Naturalmente è stata compilata anche la classe `Valore` e le sue classi anonime.

Soluzione 11.I)

Solo lo statement 1 non è corretto in quanto, non esplicitando il modificatore `public`, si sta rendendo il metodo che applica l'override, meno accessibile rispetto all'originale ereditato. Quest'ultimo infatti, essendo un metodo dichiarato in un'interfaccia, è implicitamente pubblico. Insomma è stata violata la regola di cui abbiamo parlato nel paragrafo 7.2.3.1 (terza regola "il metodo ridefinito nella sottoclasse non deve essere meno accessibile del metodo originale della superclasse").

Infatti l'eventuale output sarebbe il seguente:

```

Bambino.java:2: error: gioca() in Bambino cannot implement gioca() in
    Giocatore
@Override void gioca(){}      /*INSERISCI CODICE QUI*/
                ^
    attempting to assign weaker access privileges; was public
1 error

```

Soluzione 11.m)

Solo la seconda affermazione è corretta. La prima non è corretta perché semmai l'annotazione e il tag javadoc vanno usati in contemporanea. La terza è anch'essa scorretta perché l'annotazione appartiene a `java.lang`. Infine la numero 4 è sbagliata perché tale annotazione può annotare diversi elementi della programmazione Java, ma non altre annotazioni.

Soluzione 11.n)

Le affermazioni corrette sono la 1, la 2 e la 4. La terza è errata visto che è possibile dichiarare un'interfaccia funzionale senza obbligatoriamente annotarla.

La numero 5 è errata invece, perché come `FunctionalInterface` anche l'annotazione `@Override` è sempre facoltativa.

Soluzione 11.o)

Per ottenere solo due warning vi sono più soluzioni: aggiungere un'annotazione per sopprimere le annotazioni o sul costruttore, o sulla variabile d'istanza `objects`, o sul metodo `rimuovi`. Per esempio scegliamo la variabile d'istanza `objects`:

```
import java.util.*;

public class Esercizio11O {

    @SuppressWarnings({"rawtypes"})
    List objects;

    public Esercizio11O() {
        objects = new ArrayList();
    }

    public void rimuovi(Object object) {
        Iterator iterator = objects.iterator();
        if (iterator.hasNext()) {
            Object item = iterator.next();
            if (object.toString().equals(item.toString())) {
                iterator.remove();
            }
        }
    }
}
```

che produce il seguente output:

```
Esercizio16O.java:8: warning: [rawtypes] found raw type: ArrayList
    objects = new ArrayList();
                   ^
missing type arguments for generic class ArrayList<E>
where E is a type-variable:
  E extends Object declared in class ArrayList
Esercizio11O.java:16: warning: [rawtypes] found raw type: Iterator
    Iterator iterator = objects.iterator();
           ^
missing type arguments for generic class Iterator<E>
where E is a type-variable:
```

```
E extends Object declared in interface Iterator
2 warnings
```

È facile immaginare come ottenere un solo warning. Per esempio, annotiamo il costruttore e il metodo rimuovi:

```
import java.util.*;
```

```
public class Esercizio110 {

    List objects;

    @SuppressWarnings({"rawtypes"})
    public Esercizio110() {
        objects = new ArrayList();
    }

    @SuppressWarnings({"rawtypes"})
    public void rimuovi(Object object) {
        Iterator iterator = objects.iterator();
        if (iterator.hasNext()) {
            Object item = iterator.next();
            if (object.toString().equals(item.toString())) {
                iterator.remove();
            }
        }
    }
}
```

In questo caso otterremo il seguente output:

```
Esercizio110.java:5: warning: [rawtypes] found raw type: List
    List objects;
    ^
missing type arguments for generic class List<E>
where E is a type-variable:
  E extends Object declared in interface List
1 warning
```

Infine è ovvio che annotando anche la variabile d'istanza non otterremo nessun warning, ma è più comodo annotare l'intera classe:

```
import java.util.*;
```

```
@SuppressWarnings({"rawtypes"})
public class Esercizio110 {

    List objects;

    public Esercizio110() {
        objects = new ArrayList();
    }

    public void rimuovi(Object object) {
        Iterator iterator = objects.iterator();
        if (iterator.hasNext()) {
            Object item = iterator.next();
            if (object.toString().equals(item.toString())) {
                iterator.remove();
            }
        }
    }
}
```

Che non produce warning in compilazione.

Soluzione 11.p)

Le affermazioni corrette sono 1, 4 e 5. Con il termine “linguaggio nativo” ci si riferisce solitamente al linguaggio utilizzato dalla piattaforma su cui gira il programma, ovvero il sistema operativo, che solitamente coincide con C/C++, quindi l’affermazione numero 2 è errata.

L’annotazione `Native` appartiene al package `java.lang.annotation`, quindi anche l’affermazione numero 3 è scorretta.

Soluzione 11.q)

Le affermazioni corrette sono 1 e 4. La numero 2 non è corretta perché `Target` è un’annotazione a valore singolo di tipo array di `ElementType`.

La numero 3 non è corretta perché `ElementType` non è un’interfaccia ma un’enumerazione.

Soluzione 11.r)

Tutte le affermazioni sono vere.

Soluzione 11.s)

Le affermazioni corrette sono 1, 3 e 4.

La numero 2 non è corretta perché `RetentionPolicy` dichiara anche un terzo elemento (`RUNTIME`).

La numero 5 non è corretta perché `Target` è un’annotazione a valore singolo di tipo `RetentionPolicy`.

Soluzione 11.t)

Le affermazioni corrette sono la 2 e la 5.

La numero 1 non è corretta perché l’annotazione `Documented` è un’annotazione che annota annotazioni riportate all’interno della documentazione javadoc generata.

Le numero 3 e 4 sono semplicemente inventate.

Soluzione 11.u)

Il programma che legge le annotazioni di `Esercizio11U` stamperà solo l’annotazione che è stata dichiarata per la classe stessa, ma non erediterà quella dall’interfaccia `Interface16U`. Infatti l’annotazione `Inherited` funziona solo sulle classi e non sulle interfacce.

Ecco l’output:

```
@DifferentAnnotation()
```

Soluzione 11.v)

La reflection in teoria non è necessaria. Infatti, supponiamo di aver definito il blocco statico seguente nella classe `Moneta`:

```
static {
    System.out.println("Caricata la classe Moneta con valuta = " + VALUTA);
}
```

Basterebbe semplicemente definire una variabile della classe `Moneta` come segue:

```
public class TestReflection {
    public static void main(String args[]) {
        Moneta moneta = new Moneta(Valore.CINQUANTA_CENTESIMI);
    }
}
```

Il codice precedente produrrebbe il seguente output:

```
Caricata la classe Moneta con valuta = EURO
Creata una moneta da 50 centesimi di EURO
```

Tuttavia, se volessimo usare la reflection non sarà sufficiente il seguente codice:

```
Class<Moneta> classMoneta = Moneta.class;
try {
    classMoneta.newInstance();
} catch (InstantiationException | IllegalAccessException ex) {
    ex.printStackTrace();
}
```

Che infatti produrrà il seguente output:

```
java.langInstantiationException: Moneta
    at java.lang.Class.newInstance(Class.java:418)
    at TestReflection.main(TestReflection.java:10)
Caused by: java.lang.NoSuchMethodException: Moneta.<init>()
    at java.lang.Class.getConstructor0(Class.java:2971)
    at java.lang.Class.newInstance(Class.java:403)
    ... 1 more
```

Questo succede in quanto con il metodo `newInstance` viene chiamato il costruttore senza parametri, che però non esiste.

La soluzione è chiamare il costruttore corretto con il seguente codice:

```
try {
    Constructor<Moneta> costruttore = classMoneta.getConstructor(Valore.class);
    costruttore.newInstance(Valore.CINQUANTA_CENTESIMI);
} catch (NoSuchMethodException | SecurityException | InstantiationException |
        IllegalAccessException | IllegalArgumentException |
        InvocationTargetException ex) {
    ex.printStackTrace();
}
```

Esercizio 11.x)

Solo la quinta risposta è falsa. Un IDE invece esonera il programmatore dalla complessa gestione dei package.

Esercizio 11.y)

Solo la prima risposta è falsa. Un architetto infatti si dedica essenzialmente ai requisiti non funzionali dell'applicazione.

Esercizio 11.z)

Sono errate le affermazioni:

- numero 2: perché non sono evidenziati flussi di esecuzione.
- la numero 5: è vero che il nodo è il principale elemento del diagramma, ma viene rappresentato come un cubo tridimensionale. La descrizione riportata invece si riferisce all'elemento *componente*.

Esercizio 11.aa)

Se si lavora nel campo web, avere delle conoscenze basiche sul protocollo HTTP, i linguaggi HTML, Javascript e CSS, librerie come Bootstrap, e framework come Spring, o Angular JS e così via.

Esercizio 11.bb)

1. Per definizione un *client* è un programma che richiede servizi ad un altro programma che si chiama server.
2. Per definizione un *server*, è un programma sempre in esecuzione, che mette a disposizione dei servizi. Un client e un server comunicano solitamente tramite rete, con un protocollo ben definito.
3. Le *applicazioni standalone*, (dette anche *applicazioni desktop*). Questo tipo di applicazioni vengono eseguite su computer desktop e portatili, e solitamente hanno un'interfaccia grafica.
4. Un' *applicazione mobile* (spesso chiamata semplicemente *app*), sono applicazioni che vengono eseguite su client mobili, come smartphone e tablet, e possono anche avere una controparte server.
5. Le *applicazioni web* per esempio sono applicazioni che hanno un'architettura suddivisa in una parte client e una parte server.
6. Un *client web* richiede pagine web, e coincide con i programmi che chiamiamo comunemente *browser* (Mozilla Firefox, Google Chrome e così via).
7. Un *server web*, è invece un'applicazione che mette a disposizione servizi disponibili in rete.
8. Le *applicazioni enterprise* (potremmo tradurre come *applicazioni aziendali*), sono un'evoluzione delle applicazioni web, e solitamente mettono a disposizione servizi più complessi come scaricamento di risorse, *web services* (ovvero applicazioni di comunicazione tra sistemi eterogenei che usano il protocollo HTTP), servizi di reportistica, eccetera, e che quindi come *client enterprise* possono avere anche programmi appositamente creati per interagire con lo strato *server enterprise*. Quest'ultimo è costituito a sua volta da vari strati che utilizzano tecnologie diverse per adempiere a vari scopi.

Esercizio 11.cc)

1. Il *capo progetto* (in inglese *project manager*) dovrebbe pianificare il lavoro a livello temporale, assegnare il lavoro alle risorse, interagire con tutti i principali membri del gruppo di sviluppo, con il management e con il cliente.
2. Il *business analyst* ha il compito di raccogliere requisiti dal cliente, formarli e risolvere dubbi funzionali.
3. L'*IT manager* (in italiano meglio noto come *sistemista*) si occupa di preparare gli ambienti, di installare i software, di configurare la rete, le macchine, i permessi etc.
4. Il *deployer* o *release manager*, ha la responsabilità di creare e installare le versioni del software per i vari ambienti in cui deve essere utilizzato.
5. Il *database administrator* (o *DBA*) è l'esperto del database e si occupa di tutto ciò che concerne l'interazione e la configurazione del database.
6. Il *grafico* è il responsabile dell'aspetto della grafica dell'applicazione.

Esercizio 11.dd)

Una *metodologia object oriented*, nella sua definizione più generale, potrebbe intendersi come una coppia costituita da un processo e da un linguaggio di modellazione.

A sua volta un *processo* potrebbe essere definito come la serie di indicazioni riguardanti i passi da intraprendere per portare a termine con successo un progetto.

Un *linguaggio di modellazione* è invece lo strumento che le metodologie utilizzano per descrivere (possibilmente in maniera grafica) tutte le caratteristiche statiche e dinamiche di un progetto. Il linguaggio di modellazione considerato standard de facto è l'UML.

Soluzione E.a)

Come soluzione creiamo un file JAR eseguibile, aggiungendo un file manifest.txt che specifica il nome della classe con il main. Il file manifest conterrà il seguente testo:

```
Main-Class: com.claudiodesio.autenticazione.Autenticazione
```

Notare che si deve andare da capo dopo la riga, altrimenti il comando non verrà considerato.

Il comando per impacchettare i file è:

```
jar -cvfm autenticazione.jar manifest.txt  
com/claudiodesio/autenticazione/*.class
```

Il file manifest.txt deve essere presente nella cartella da dove si sta lanciando il comando.

Infine, per lanciare l'applicazione utilizzare il seguente comando:

```
java -jar autenticazione.jar
```

Soluzioni degli esercizi del capitolo 12

Gestione dei thread

Soluzione 12.a) Creazione di thread, Vero o Falso:

1. **Falso**, `Runnable` è un'interfaccia.
2. **Vero**.
3. **Vero**, il cosiddetto thread "main".
4. **Vero**.
5. **Falso**, il programmatore può invocare il metodo `start` e lo scheduler invocherà il metodo `run`.
6. **Vero**.
7. **Falso**.
8. **Vero**.
9. **Vero**.
10. **Falso**.

Soluzione 12.b) Gestione del multithreading, Vero o Falso:

1. **Vero**.
2. **Falso**.
3. **Vero**.
4. **Falso**.
5. **Vero**.
6. **Vero**.
7. **Vero**.
8. **Falso**, il primo thread che partirà sarà quello a priorità più alta.
9. **Vero**.
10. **Vero**.

Soluzione 12.c)

Per prima cosa astraiamo il concetto di stato di cui si è parlato nel quesito 12.c, tramite un'enumerazione. Creiamo anche il messaggio da stampare per ogni stato:

```
package com.claudiodesio.osservatorio.dati;

public enum Stato {

    IN_ATTESA("\Sono in attesa...\"),
    OSSERVAZIONE("\Tocca a me... che meraviglia!\"),
    FINITO("\Ho finito.\");

    private String messaggio;

    private Stato(String messaggio) {
```

```
        this.messaggio = messaggio;
    }

    public String getMessaggio() {
        return messaggio;
    }
}
```

Poi creiamo una semplice classe `Partecipante`, che ha tra le sue caratteristiche proprio un oggetto `Stato`. Tutti i partecipanti hanno un nome e un telescopio a cui fanno riferimento per l'osservazione. La nostra classe estende `Thread`, e il suo metodo `run` definisce l'azione dell'osservazione.

```
package com.claudiodesio.osservatorio.dat;

public class Partecipante extends Thread {

    private final String nome;

    private Stato stato;

    private final Telescopio telescopio;

    public Telescopio getTelescopio() {
        return telescopio;
    }

    public Partecipante(String nome, Telescopio telescopio) {
        this.nome = nome;
        this.telescopio = telescopio;
        this.setStato(Stato.IN_ATTESA);
        stato();
    }

    public Stato getStato() {
        return stato;
    }

    public void setStato(Stato stato) {
        this.stato = stato;
    }

    public String getNome() {
        return nome;
    }

    @Override
    public void run() {
        telescopio.permettiOsservazione(this);
    }

    public void stato() {
        System.out.println(nome + " dice: " + stato.getMessaggio());
    }
}
```

Si noti che, quando viene creato un `Partecipante`, lo stato viene impostato a `IN_ATTESA`. Poi creiamo la classe più importante, ovvero quella che astrae il concetto di `Telescopio`:

```
package com.claudiodesio.osservatorio.dat;

public class Telescopio {
```

```
public synchronized void permettiOsservazione(Partecipante partecipante) {
    partecipante.setStato(Stato.OSSERVAZIONE);
    partecipante.stato();
    try {
        Thread.sleep(3000);
    } catch (InterruptedException ex) {
        ex.printStackTrace();
    }
    partecipante.setStato(Stato.FINITO);
    partecipante.stato();
}
```

Nel suo metodo di business sincronizzato, si limita a passare da uno stato ad un altro dopo una pausa di 3 secondi.

Infine, segue una classe di test:

```
package com.claudiodesio.osservatorio.test;

import com.claudiodesio.osservatorio.dat.Partecipante;
import com.claudiodesio.osservatorio.dat.Telescopio;

public class Osservazione {

    public static void main(String args[]) {
        Telescopio telescopio = new Telescopio();
        Partecipante[] partecipanti = getPartecipanti(telescopio);
        for (Partecipante partecipante : partecipanti) {
            partecipante.start();
        }
    }

    private static Partecipante[] getPartecipanti(Telescopio telescopio) {
        Partecipante[] partecipanti = {
            new Partecipante("Ciro", telescopio),
            new Partecipante("Gianluca", telescopio),
            new Partecipante("Pierluigi", telescopio),
            new Partecipante("Gigi", telescopio),
            new Partecipante("Nicola", telescopio),
            new Partecipante("Pino", telescopio),
            new Partecipante("Maurizio", telescopio),
            new Partecipante("Raffaele", telescopio),
            new Partecipante("Fabio", telescopio),
            new Partecipante("Vincenzo", telescopio)};
        return partecipanti;
    }
}
```

L'output è il seguente (che cambia ad ogni lancio):

```
Ciro dice: "Sono in attesa..."
Gianluca dice: "Sono in attesa..."
Pierluigi dice: "Sono in attesa..."
Gigi dice: "Sono in attesa..."
Nicola dice: "Sono in attesa..."
Pino dice: "Sono in attesa..."
Maurizio dice: "Sono in attesa..."
Raffaele dice: "Sono in attesa..."
Fabio dice: "Sono in attesa..."
Vincenzo dice: "Sono in attesa..."
Ciro dice: "Tocca a me... che meraviglia!"
```

```
Ciro dice: "Ho finito."
Vincenzo dice: "Tocca a me... che meraviglia!"
Vincenzo dice: "Ho finito."
Maurizio dice: "Tocca a me... che meraviglia!"
Maurizio dice: "Ho finito."
Pino dice: "Tocca a me... che meraviglia!"
Pino dice: "Ho finito."
Fabio dice: "Tocca a me... che meraviglia!"
Fabio dice: "Ho finito."
Gigi dice: "Tocca a me... che meraviglia!"
Gigi dice: "Ho finito."
Nicola dice: "Tocca a me... che meraviglia!"
Nicola dice: "Ho finito."
Raffaele dice: "Tocca a me... che meraviglia!"
Raffaele dice: "Ho finito."
Pierluigi dice: "Tocca a me... che meraviglia!"
Pierluigi dice: "Ho finito."
Gianluca dice: "Tocca a me... che meraviglia!"
Gianluca dice: "Ho finito."
```

Soluzione 12.d)

Creiamo una classe `TimeUtils` che definisce un metodo d'utilità per generare un numero casuale tra 5 e 10:

```
package com.claudiodesio.sportello.dati;

import java.util.*;

public class TimeUtils {

    private static final Random RANDOM = new Random();

    public static int getNumeroRandom() {
        return (RANDOM.nextInt(6) + 5);
    }
}
```

La classe `Richiedente` estende `Thread`:

```
package com.claudiodesio.sportello.dati;

public class Richiedente extends Thread {

    private final String nome;

    public Richiedente(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }

    @Override
    public void run() {
        Sportello.getInstance().gestisciRichiesta(this);
    }
}
```

```
@Override
public String toString() {
    return nome;
}
}
```

La classe Stampante sarà usata per stampare i documenti:

```
package com.claudiodesio.sportello.dati;

public class Stampante {

    private static Stampante instance;

    private Stampante() {
    }

    public static Stampante getInstance() {
        if (instance == null) {
            instance = new Stampante();
        }
        return instance;
    }

    public synchronized void stampa(Richiedente richiedente) {
        System.out.println("Stampa carta d'identità di " + richiedente
            + " in corso...");
        try {
            wait(3000);
        } catch (InterruptedException ex) {
            ex.printStackTrace();
        }
        System.out.println("Stampa completata! " + richiedente
            + " grazie e arrivederci!");
    }
}
```

La classe Sportello è la classe chiave:

```
package com.claudiodesio.sportello.dati;

public class Sportello {

    private final Stampante stampante;
    private static Sportello instance;

    public synchronized static Sportello getInstance() {
        if (instance == null) {
            instance = new Sportello();
        }
        return instance;
    }

    private Sportello() {
        stampante = Stampante.getInstance();
    }

    public synchronized void gestisciRichiesta(Richiedente richiedente) {
        System.out.println("Buongiorno " + richiedente);
        System.out.println("Impiegato dice: \"Prego compili il modulo \"
            + richiedente + "\"");
        compilaModulo(richiedente);
    }
}
```

```
stampante.stampa(richiedente);
System.out.println(richiedente + " dice: \"Grazie a lei!\");
}

private synchronized void compilaModulo(Richiedente richiedente) {
    System.out.println("Richiedente " + richiedente
        + " dice: \"OK lo compilo subito ma...\");
    final int attesa = TimeUtils.getNumeroRandom();
    try {
        System.out.println("...mi servono " + attesa + " minuti...");
        wait(attesa * 1000);
    } catch (InterruptedException ex) {
        ex.printStackTrace();
    }
    System.out.println("Richiedente " + richiedente
        + " dice: \"...ho compilato il modulo!\");
}
}
```

Segue la classe di test:

```
package com.claudiodesio.sportello.test;

import com.claudiodesio.sportello.dat.Richiedente;

public class TestSportello {

    public static void main(String args[]) {
        final Richiedente[] richiedenti = getRichiedenti();
        for (Richiedente richiedente : richiedenti) {
            richiedente.start();
        }
    }

    private static Richiedente[] getRichiedenti() {
        Richiedente[] partecipanti = {
            new Richiedente("Ciro"),
            new Richiedente("Mario"),
            new Richiedente("Massimo"),
            new Richiedente("Chicco"),
            new Richiedente("Enrico"),
            new Richiedente("Lorenzo"),
            new Richiedente("Emanuele"),
            new Richiedente("Cosimo"),
            new Richiedente("Alessandro"),
            new Richiedente("Salvatore")};
        return partecipanti;
    }
}
```

Infine, segue l'output (che cambia sempre):

```
Buongiorno Ciro
Impiegato dice: "Prego compili il modulo Ciro"
Richiedente Ciro dice: "OK lo compilo subito ma..."
...mi servono 9 minuti...
Buongiorno Salvatore
Impiegato dice: "Prego compili il modulo Salvatore"
Richiedente Salvatore dice: "OK lo compilo subito ma..."
...mi servono 9 minuti...
Buongiorno Cosimo
Impiegato dice: "Prego compili il modulo Cosimo"
```



```
Richiedente Cosimo dice: "OK lo compilo subito ma..."
...mi servono 7 minuti...
Buongiorno Mario
Impiegato dice: "Prego compili il modulo Mario"
Richiedente Mario dice: "OK lo compilo subito ma..."
...mi servono 7 minuti...
Buongiorno Chicco
Impiegato dice: "Prego compili il modulo Chicco"
Richiedente Chicco dice: "OK lo compilo subito ma..."
...mi servono 5 minuti...
Buongiorno Alessandro
Impiegato dice: "Prego compili il modulo Alessandro"
Richiedente Alessandro dice: "OK lo compilo subito ma..."
...mi servono 9 minuti...
Buongiorno Enrico
Impiegato dice: "Prego compili il modulo Enrico"
Richiedente Enrico dice: "OK lo compilo subito ma..."
...mi servono 8 minuti...
Buongiorno Massimo
Impiegato dice: "Prego compili il modulo Massimo"
Richiedente Massimo dice: "OK lo compilo subito ma..."
...mi servono 8 minuti...
Buongiorno Lorenzo
Impiegato dice: "Prego compili il modulo Lorenzo"
Richiedente Lorenzo dice: "OK lo compilo subito ma..."
...mi servono 5 minuti...
Buongiorno Emanuele
Impiegato dice: "Prego compili il modulo Emanuele"
Richiedente Emanuele dice: "OK lo compilo subito ma..."
...mi servono 9 minuti...
Richiedente Lorenzo dice: "...ho compilato il modulo!"
Stampa carta d'identità di Lorenzo in corso...
Stampa completata! Lorenzo grazie e arrivederci!
Lorenzo dice: "Grazie a lei!"
Richiedente Massimo dice: "...ho compilato il modulo!"
Stampa carta d'identità di Massimo in corso...
Stampa completata! Massimo grazie e arrivederci!
Massimo dice: "Grazie a lei!"
Richiedente Enrico dice: "...ho compilato il modulo!"
Stampa carta d'identità di Enrico in corso...
Stampa completata! Enrico grazie e arrivederci!
Enrico dice: "Grazie a lei!"
Richiedente Mario dice: "...ho compilato il modulo!"
Stampa carta d'identità di Mario in corso...
Stampa completata! Mario grazie e arrivederci!
Mario dice: "Grazie a lei!"
Richiedente Cosimo dice: "...ho compilato il modulo!"
Stampa carta d'identità di Cosimo in corso...
Stampa completata! Cosimo grazie e arrivederci!
Cosimo dice: "Grazie a lei!"
Richiedente Chicco dice: "...ho compilato il modulo!"
Stampa carta d'identità di Chicco in corso...
Stampa completata! Chicco grazie e arrivederci!
Chicco dice: "Grazie a lei!"
```

```
Richiedente Emanuele dice: "...ho compilato il modulo!"
Stampa carta d'identità di Emanuele in corso...
Stampa completata! Emanuele grazie e arrivederci!
Emanuele dice: "Grazie a lei!"
Richiedente Salvatore dice: "...ho compilato il modulo!"
Stampa carta d'identità di Salvatore in corso...
Stampa completata! Salvatore grazie e arrivederci!
Salvatore dice: "Grazie a lei!"
Richiedente Alessandro dice: "...ho compilato il modulo!"
Stampa carta d'identità di Alessandro in corso...
Stampa completata! Alessandro grazie e arrivederci!
Alessandro dice: "Grazie a lei!"
Richiedente Ciro dice: "...ho compilato il modulo!"
Stampa carta d'identità di Ciro in corso...
Stampa completata! Ciro grazie e arrivederci!
Ciro dice: "Grazie a lei!"
```

Soluzione 12.e)

La risposta corretta è la numero 6, in quanto la classe `RunnableArrayList` non implementa correttamente l'interfaccia `Runnable`, il cui metodo `run` non prende nessun tipo di parametri in input. Infatti l'output della compilazione è il seguente (sono presenti anche due warning relativi all'utilizzo e alla dichiarazione di raw type):

```
Esercizio12E.java:3: warning: [rawtypes] found raw type: ArrayList
class RunnableArrayList extends ArrayList implements Runnable {
    ^
    missing type arguments for generic class ArrayList<E>
    where E is a type-variable:
      E extends Object declared in class ArrayList
Esercizio12E.java:3: error: RunnableArrayList is not abstract and does
    not override abstract method run() in Runnable
class RunnableArrayList extends ArrayList implements Runnable {
    ^
Esercizio12E.java:3: warning: [serial] serializable class
    RunnableArrayList has no definition of serialVersionUID
class RunnableArrayList extends ArrayList implements Runnable {
    ^
1 error
2 warnings
```

Soluzione 12.f)

Solo la risposta 3 è corretta, tutte le altre no.

In particolare, riguardo la risposta 4, la classe `Monitor` non esiste.

Soluzione 12.g)

Le risposte corrette sono la numero 2 e la numero 3.

La numero 1 è falsa perché la classe `Thread` implementa `Runnable`. Infatti quest'ultima è un'interfaccia e in quanto tale va implementata e non estesa da un'altra classe.

La numero 4 è sbagliata visto che i metodi `wait` e `notify` sono dichiarati nella classe `Object`.

Soluzione 12.h)

La classe `ContoAllaRovescia` potrebbe essere la seguente:

```
public class ContoAllaRovescia {  
  
    public void attiva(int secondi) throws InterruptedException {  
        for (int i = secondi; i > 0; i--) {  
            System.out.println(i);  
            Thread.sleep(1000);  
        }  
        System.out.println("Tempo scaduto!");  
    }  
}
```

Poi possiamo creare una classe che la testi:

```
public class Esercizio12H {  
    public static void main(String args[]) throws Exception {  
        ContoAllaRovescia contoAllaRovescia = new ContoAllaRovescia();  
        int secondi = 10;  
        if (args.length > 0) {  
            try {  
                secondi = Integer.parseInt(args[0]);  
            }  
            catch (Exception exc) {  
                System.out.println("L'input deve essere un numero "  
                    + "intero positivo, usiamo il valore di default 10...");  
            }  
        }  
        contoAllaRovescia.attiva(secondi);  
    }  
}
```

Eseguendo quest'ultima senza specificare parametri otterremo il seguente output:

```
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
Tempo scaduto!
```

Se passiamo il parametro 3 allora l'output sarà limitato a:

```
3  
2  
1  
Tempo scaduto!
```

Se invece specifichiamo una lettera (supponiamo `F`) avremo il seguente output:

```
L'input deve essere un numero intero positivo, usiamo il valore di  
default 10...
```

```
10
9
8
7
6
5
4
3
2
1
Tempo scaduto!
```

Infine, specificando un numero negativo oppure 0, otterremo direttamente il seguente output:

```
Tempo scaduto!
```

Soluzione 12.i)

Le risposte corrette sono le numero 1, 2 e 3. In particolare la 3 non asserisce che il metodo `run` viene eseguito in un thread a parte, altrimenti sarebbe stata errata. Il metodo `run`, rimane comunque un metodo, e quindi è invocabile come qualsiasi altro metodo. L'affermazione numero 4 è errata perché il metodo `start` è dichiarato dalla classe `Thread`, e non dalla classe `Object`.

L'affermazione numero 5 è errata perché la frase giusta sarebbe: "in ogni programma c'è almeno un thread in esecuzione".

Soluzione 12.l)

Le specifiche dell'esercizio 12.1 sono volutamente non troppo dettagliate, per lasciare più spazio alla fantasia del lettore. Quindi in questo caso un'eventuale soluzione può differire davvero molto dalla soluzione che presentiamo di seguito.

Per esempio potremmo implementare la classe `Corridore` nel seguente modo:

```
public class Corridore extends Thread {

    private boolean viaLibera;

    private boolean inAzione;

    private int gap;

    public Corridore() {
        inAzione = true;
        gap = 1000;
    }

    public void run() {
        while (inAzione) {
            try {
                Thread.sleep(gap);
                if (viaLibera) {
                    System.out.println("|");
                    Thread.sleep(gap);
                    System.out.println("  |");
                }
            } catch (InterruptedException exc) {
```

```
        assert false;
    }
}

public void inizia() {
    start();
}

public void corri() {
    System.out.println("Ok, vado...");
    gap = 400;
    viaLibera = true;
}

public void fermati() {
    System.out.println("Ok, mi fermo.");
    System.out.println("| |");
    viaLibera = false;
}

public void cammina() {
    System.out.println("Ok, mi riposo un po'...");
    gap = 1000;
    viaLibera = true;
}

public void basta() {
    System.out.println("Meno male, non ce la facevo più...");
    inAzione = false;
}
}
```

Il nucleo della logica di business si trova proprio nel metodo `run`, che con un ciclo che si basa sulla variabile `inAzione`, stampa con dei simboli `|` (si legge “pipe” in inglese) una trama che assomiglia a dei passi. Si noti che quando si deve correre, la variabile `gap`, che rappresenta l’attesa da un punto all’altro, è di 400 millisecondi, mentre quando si deve camminare il `gap` viene allungato ad un secondo. A seconda che sia chiamato il metodo `corri` o il metodo `cammina` quindi, questi passi saranno più veloci o meno veloci.

Ma vediamo ora come potremmo implementare la classe principale `Esercizio12L`:

```
import java.util.Scanner;

public class Esercizio12L {
    public static void main(String args[]) {
        Corridore corridore = new Corridore();
        corridore.inizia();
        Scanner scanner = new Scanner(System.in);
        boolean cicla = true;
        System.out.println(
            "Ciao allenatore, il corridore è a tua disposizione!");
        System.out.println("Scrivi i comandi e batti invio");
        System.out.println("(corri, cammina, fermati, basta)");
        while (cicla) {

            String comando = scanner.nextLine();
            switch (comando) {
                case "corri":
                    corridore.corri();
                    break;
            }
        }
    }
}
```

```

        case "cammina":
            corridore.cammina();
            break;
        case "fermati":
            corridore.fermati();
            break;
        case "basta":
            corridore.basta();
            cicla = false;
            break;

        default:
            break;
    }
}
try {
    Thread.sleep(2000);
}
catch (Exception exc) {
    assert false;
}
System.out.println("Fine allenamento");
}
}

```

Il codice non differisce molto da altro che abbiamo già visto tra questi esercizi. Tramite un'oggetto `Scanner` vengono letti i comandi, che si traducono in chiamate ai metodi di `Corridore`.

Se eseguiamo quest'ultima classe potremmo ottenere il seguente output, che non rende bene l'idea se non si vede in azione “dal vivo” (meglio eseguire le classi già pronte dalla cartella

Codice\capitolo_12\esercizi\12.1 del file degli esercizi che avete probabilmente già scaricato insieme a questi esercizi all'indirizzo <https://www.nuovojava.it>):

```
Ciao allenatore, il corridore è a tua disposizione!
```

```
Scrivi i comandi e batti invio
```

```
(corri, cammina, fermati, basta)
```

```
corri
```

```
Ok, vado...
```

```
|
```

```
|
```

```
|
```

```
|
```

```
|
```

```
|
```

```
|
```

```
|
```

```
c|
```

```
a |
```

```
mmi|
```

```
na |
```

```
|
```

```
Ok, mi riposo un po'...
```

```
|
```

```
|
```

```
|
```

```
|
```

```
|
```

```
ferm|
```

```
ati |
Ok, mi fermo.
| |
corri
Ok, vado...
|
| |
| |
| |
| |
ca|
|
mm|
ina |
|
| |
|
Ok, mi riposo un po'...
|
| |
| |
|
fe |
rmat|
i
Ok, mi fermo.
| |
basta
Meno male, non ce la facevo più...
Fine allenamento
```

Si noti che, i comandi inseriti, si estendono spesso su più righe, perché intanto il programma sta stampando “i passi”.

Soluzione 12.m)

Tutte le affermazioni sono corrette tranne la numero 2. Infatti anche implementando l'interfaccia `Runnable` si ha la possibilità di implementare altre interfacce.

Soluzione 12.n)

Tutte le affermazioni sono corrette tranne la numero 3. Infatti il preemptive scheduling è il comportamento di default della maggior parte dei sistemi Unix.

Soluzione 12.o)

Tutte le affermazioni sono corrette tranne la numero 1. Infatti il modificatore `volatile`, si può usare solo su variabili d'istanza.

Soluzione 12.p)

Tutte le affermazioni sono corrette.

Soluzione 12.q)

Tutte le affermazioni sono corrette tranne la numero 1. Infatti metodi `wait`, `notify` e `notifyAll` sono definiti della classe `Object`.

Soluzione 12.r)

Una possibile soluzione è la seguente:

```
import java.util.Date;

public final class Esercizio12R {

    private final Integer intero;

    private final Date date;

    public Esercizio12R (Integer intero, Date date){
        this.intero = intero;
        this.date = (Date)date.clone();
    }

    public final Date getStringBuilder() {
        return (Date)date.clone();
    }

    public final Integer getIntero() {
        return intero;
    }
}
```

Soluzione 12.s)

Tutte le affermazioni sono corrette tranne la numero 4. Infatti `AtomicInteger` è una classe non un'interfaccia.

Soluzione 12.t)

La compilazione dello snippet causerà tre errori, uno per ogni riga:


```
Esercizio12T.java:6: error: <anonymous Esercizio12T$1> is not abstract
and does not override abstract method call() in Callable
    Callable<String> callable = new Callable<>() {public void
    call(){} };
                                         ^
Esercizio12T.java:6: error: call() in <anonymous Esercizio12T$1>
cannot implement call() in Callable
    Callable<String> callable = new Callable<>() {public void
    call(){} };
                                         ^
    return type void is not compatible with String
    where V is a type-variable:
      V extends Object declared in interface Callable
Esercizio12T.java:7: error: cannot find symbol
    Future<String> future =
    Executors.newFixedThreadPool(3).start(callable);
                                         ^
    symbol:   method start(Callable<String>)
    location: interface ExecutorService
3 errors
```

Infatti nella prima riga abbiamo provato a creare una classe anonima che estende `Callable` parametrizzata con una stringa. Poi viene invocato il metodo `call` che ritorna `void` in luogo di `String`, che però non esiste. Nella seconda riga l'errore consiste nel chiamare il metodo `start` (come per `Runnable`), ma il metodo da chiamare è `submit`. Infine nella terza riga, viene invocato il metodo `get` di `Future` che richiede la gestione dell'`InterruptedException`.

Quindi, il codice corretto sarebbe simile al seguente:

```
Callable<String> callable = new Callable<>() {
    public String call(){return "";}
};
ExecutorService service = Executors.newFixedThreadPool(3);
Future<String> future = service.submit(callable);
String result = null;
try {
    result = future.get();
}
catch (Exception exc) {
    exc.printStackTrace();
}
```

Soluzione 12.u)

Una possibile (e semplice) soluzione potrebbe essere la seguente:

```
import java.time.format.*;

public class Esercizio12U extends TimerTask {

    private Timer timer;

    public Esercizio12U() {
        timer = new Timer();
    }

    @Override
    public void run() {
```

```
        DateTimeFormatter timeFormatter =
            DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT);
        System.out.println("Sveglia! Sono le " +
            timeFormatter.format(LocalTime.now()));
        timer.cancel();
    }

    public static void main(String args[]) throws Exception {
        int seconds = Integer.parseInt(args[0])*1000;
        Esercizio12U timerTask = new Esercizio12U();
        timerTask.timer.schedule(timerTask, seconds);
    }
}
```

Soluzione 12.v)

Le risposte corrette sono la numero 2 e la numero 4.

La numero 1 definisce uno statement non compilabile, perché bisogna specificare quantomeno il numero di permits per quanto riguarda il costruttore di Semaphore.

La numero 3 è falsa perché Semaphore è una classe (facile intuirlo visto che viene istanziata con un costruttore).

Soluzione 12.z)

Le risposte corrette sono le numero 1 e 2.

La numero 3 è falsa perché signalAll non è un metodo di CyclicBarrier ma di Condition, a cui abbiamo accennato tra le ultime righe del paragrafo 12.6.3.1.

La numero 4 è falsa, perché CyclicBarrier fornisce solo la possibilità ad un gruppo di thread di sincronizzarsi e attendersi l'un l'altro in un determinato punto del codice

Soluzioni degli esercizi del capitolo 13 e dell'appendice F

Espressioni Lambda e package java.lang

Soluzione 13.a) *Espressioni lambda e reference a metodi:*

1. **Falso**, per esempio in una classe anonima possiamo definire anche variabili d'istanza.
2. **Falso**.
3. **Falso**.
4. **Vero**.
5. **Falso**, mancano le parentesi graffe che circondano le istruzioni del metodo.
6. **Falso**, le parentesi vicino al metodo `println` non fanno parte della sintassi.
7. **Falso**.
8. **Falso**.
9. **Vero**.
10. **Vero**.

Soluzione 13.b)

Il listato richiesto potrebbe essere come il seguente:

```
package com.claudiodesio.lambda.test;

import java.util.Arrays;
import java.util.Comparator;

public class TestComparators {

    static String nomi[] = {"Clarissa", "Jem", "Top", "Ermeringildo",
        "Iamaca", "Tom", "Arlequin", "Francesca", "Cumbus", "Blue"};

    public static void main(String args[]) {

        Comparator<String> comparatorLunghezza = (first, second)
            -> -(Integer.compare(first.length(), second.length()));

        Comparator<String> comparatorLunghezzaAlContrario = (first, second)
            -> (Integer.compare(first.length(), second.length()));

        Comparator<String> comparatorAlfabeticoAlContrario = (first, second)
            -> -(first.compareTo(second));

        Comparator<String> comparatorLunghezzaEAlfabeticoAlContrario =
            (first, second)
            -> {
                int result = -Integer.compare(first.length(),
                    second.length());
                if (result == 0) {
                    result = first.compareTo(second);
                }
            }
    }
}
```

```

        return result;
    };
    Arrays.sort(nomi, comparatorLunghezza);
    System.out.println("Nomi ordinati per lunghezza: " +
        Arrays.asList(nomi));

    Arrays.sort(nomi, comparatorLunghezzaAlContrario);
    System.out.println("Nomi ordinati per lunghezza al contrario: " +
        Arrays.asList(nomi));

    Arrays.sort(nomi, String::compareTo);
    System.out.println("Nomi ordinati: " + Arrays.asList(nomi));

    Arrays.sort(nomi, comparatorAlfabeticoAlContrario);
    System.out.println("Nomi ordinati al contrario: " +
        Arrays.asList(nomi));

    Arrays.sort(nomi, comparatorLunghezzaEAlfabeticoAlContrario);
    System.out.println(
        "Nomi ordinati per lunghezza al contrario e in ordine alfabetico: "
        + Arrays.asList(nomi));
    }
}

```

Si noti che il metodo `asList` della classe `Arrays` è stato usato solo per sfruttare la rappresentazione testuale della collezione.

L'output sarà:

```

Nomi ordinati per lunghezza: [Ermeringildo, Francesca, Clarissa,
Arlequin, Iamaca, Cumbus, Blue, Jem, Top, Tom]
Nomi ordinati per lunghezza al contrario: [Jem, Top, Tom, Blue, Iamaca,
Cumbus, Clarissa, Arlequin, Francesca, Ermeringildo]
Nomi ordinati: [Arlequin, Blue, Clarissa, Cumbus, Ermeringildo,
Francesca, Iamaca, Jem, Tom, Top]
Nomi ordinati al contrario: [Top, Tom, Jem, Iamaca, Francesca,
Ermeringildo, Cumbus, Clarissa, Blue, Arlequin]
Nomi ordinati per lunghezza al contrario e in ordine alfabetico:
[Ermeringildo, Francesca, Arlequin, Clarissa, Cumbus, Iamaca, Blue,
Jem, Tom, Top]

```

Soluzione 13.c)

Il listato della classe `Citta`, dovrebbe essere il seguente:

```

package com.claudiodesio.lambda.dat;

public class Citta {

    private String nome;

    private boolean capoluogo;

    private boolean diMare;

    public Citta(String nome, boolean capoluogo, boolean diMare) {
        this.nome = nome;
        this.capoluogo = capoluogo;
        this.diMare = diMare;
    }
}

```

```

    public boolean isDiMare() {
        return diMare;
    }

    public void setDiMare(boolean diMare) {
        this.diMare = diMare;
    }

    public boolean isCapoluogo() {
        return capoluogo;
    }

    public void setCapoluogo(boolean capoluogo) {
        this.capoluogo = capoluogo;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    @Override
    public String toString() {
        return getNome();
    }
}

```

Il listato della classe Esercizio13C invece, potrebbe essere codificato nel seguente modo:

```

package com.claudiodesio.lambda.test;

import com.claudiodesio.lambda.dati.Citta;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.function.Consumer;
import java.util.function.Predicate;

public class Esercizio13C {

    public static void main(String args[]) {
        List<Citta> listaCitta = getCitta();
        System.out.println("Città di mare: " +
            filtraCitta(listaCitta, Esercizio13C::isDiMare));
        listaCitta = getCitta();
        System.out.println("Città capoluogo: " +
            filtraCitta(listaCitta, Esercizio13C::isCapoluogo));
        listaCitta = getCitta();
    }

    public static List<Citta> filtraCitta(List<Citta> listaCitta,
                                           Predicate<Citta> p) {
        final Iterator<Citta> iterator = listaCitta.iterator();
        while (iterator.hasNext()) {
            Citta citta = iterator.next();
            if (!p.test(citta)) {
                iterator.remove();
            }
        }
    }
}

```

```

    }
    return listaCitta;
}

private static List<Citta> getCitta() {
    List<Citta> citta = new ArrayList<>();
    citta.add(new Citta("Milano", true, false));
    citta.add(new Citta("Rovigo", false, false));
    citta.add(new Citta("Potenza", true, false));
    citta.add(new Citta("Siracusa", false, true));
    citta.add(new Citta("Perugia", true, false));
    citta.add(new Citta("Napoli", true, true));
    citta.add(new Citta("Pescara", false, true));
    citta.add(new Citta("Taranto", false, true));
    citta.add(new Citta("Siena", false, false));
    return citta;
}

public static boolean isDiMare(Citta citta) {
    return citta.isDiMare();
}

public static boolean isCapoluogo(Citta citta) {
    return citta.isCapoluogo();
}
}

```

L'output sarà:

```

Città di mare: [Siracusa, Napoli, Pescara, Taranto]
Città capoluogo: [Milano, Potenza, Perugia, Napoli]

```

Soluzione 13.d)

Il listato della classe `Esercizio13D` potrebbe essere il seguente:

```

package com.claudiodesio.lambda.test;

import com.claudiodesio.lambda.dati.Citta;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.function.Predicate;

public class Esercizio13D {

    public static void main(String args[]) {
        List<Citta> listCitta = getCitta();
        System.out.println("Città di mare: " +
            filtraCitta(listCitta, (citta) -> citta.isDiMare()));
        listCitta = getCitta();
        System.out.println("Città capoluogo: " +
            filtraCitta(listCitta, (citta) -> citta.isCapoluogo()));
    }

    public static List<Citta> filtraCitta(List<Citta> listaCitta,
        Predicate<Citta> p) {
        final Iterator<Citta> iterator = listaCitta.iterator();
        while (iterator.hasNext()) {
            Citta citta = iterator.next();

```

```

        if (!p.test(citta)) {
            iterator.remove();
        }
    }
    return listaCitta;
}

private static List<Citta> getCitta() {
    List<Citta> citta = new ArrayList<>();
    citta.add(new Citta("Milano", true, false));
    citta.add(new Citta("Rovigo", false, false));
    citta.add(new Citta("Potenza", true, false));
    citta.add(new Citta("Siracusa", false, true));
    citta.add(new Citta("Perugia", true, false));
    citta.add(new Citta("Napoli", true, true));
    citta.add(new Citta("Pescara", false, true));
    citta.add(new Citta("Taranto", false, true));
    citta.add(new Citta("Siena", false, false));
    return citta;
}
}

```

L'output rimarrà lo stesso dell'esercizio precedente:

```

Città di mare: [Siracusa, Napoli, Pescara, Taranto]
Città capoluogo: [Milano, Potenza, Perugia, Napoli]

```

Soluzione 13.e)

La classe `Esercizio13E` potrebbe essere la seguente:

```

package com.claudiodesio.lambda.test;

import com.claudiodesio.lambda.dati.Citta;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.function.Consumer;
import java.util.function.Predicate;

public class Esercizio13E {

    public static void main(String args[]) {
        List<Citta> listaCitta = getCitta();
        System.out.println("Città di mare: " +
            filtraCitta(listaCitta, Esercizio13E::isDiMare));
        listaCitta = getCitta();
        System.out.println("Città capoluogo: " +
            filtraCitta(listaCitta, Esercizio13E::isCapoluogo));
        listaCitta = getCitta();
        stampaDettagli(listaCitta, (citta) -> System.out.println(citta.getNome()
            + (citta.isCapoluogo() ? " è capoluogo," : "")
            + (citta.isDiMare() ? " è città di mare," : "")));
    }

    public static List<Citta> filtraCitta(List<Citta> listaCitta,
        Predicate<Citta> p) {
        final Iterator<Citta> iterator = listaCitta.iterator();
        while (iterator.hasNext()) {
            Citta citta = iterator.next();

```

```

        if (!p.test(citta)) {
            iterator.remove();
        }
    }
    return listaCitta;
}

public static void stampaDettagli(List<Citta> listaCitta,
                                   Consumer<Citta> p) {
    for (Citta citta : listaCitta) {
        p.accept(citta);
    }
}

private static List<Citta> getCitta() {
    List<Citta> citta = new ArrayList<>();
    citta.add(new Citta("Milano", true, false));
    citta.add(new Citta("Rovigo", false, false));
    citta.add(new Citta("Potenza", true, false));
    citta.add(new Citta("Siracusa", false, true));
    citta.add(new Citta("Perugia", true, false));
    citta.add(new Citta("Napoli", true, true));
    citta.add(new Citta("Pescara", false, true));
    citta.add(new Citta("Taranto", false, true));
    citta.add(new Citta("Siena", false, false));
    return citta;
}

public static boolean isDiMare(Citta citta) {
    return citta.isDiMare();
}

public static boolean isCapoluogo(Citta citta) {
    return citta.isCapoluogo();
}
}

```

E quindi l'interfaccia funzionale da usare era l'interfaccia `Consumer`.

È bastato poi invocare questo metodo usando un'espressione lambda (ed un paio di operatori ternari) nel metodo `main`:

```

stampaDettagli(listaCitta, (citta) -> System.out.println(citta.getNome()
    + (citta.isCapoluogo() ? " è capoluogo, " : "")
    + (citta.isDiMare() ? " è città di mare" : "")));

```

L'output sarà:

```

Città di mare: [Siracusa, Napoli, Pescara, Taranto]
Città capoluogo: [Milano, Potenza, Perugia, Napoli]
Milano è capoluogo,
Rovigo
Potenza è capoluogo,
Siracusa è città di mare,
Perugia è capoluogo,
Napoli è capoluogo, è città di mare,
Pescara è città di mare,
Taranto è città di mare,
Siena

```


Soluzione 13.f)

La soluzione è molto semplice utilizzando una classe anonima al volo come nel seguente listato, la difficoltà semmai è nella formattazione del codice sfruttando tabulazioni (“\t”):

```
public class TestPersona {

    public static void main(String args[]) {
        System.out.println(new Persona("Arjen Anthony", "Lucassen",
            "03/04/1960", "Compositore", "Olanda") {

            @Override
            public String toString() {
                String string = "Nome: \t\t\t" + getNome();
                string += "\nCognome: \t\t" + getCognome();
                string += "\nProfessione: \t\t" + getProfessione();
                string += "\nData di Nascita: \t" + getDataDiNascita();
                string += "\nIndirizzo: \t\t" + getIndirizzo();
                return string;
            }
        });
    }
}
```

Soluzione 13.g)

Anche in questo caso, è impossibile usare un'espressione lambda. Invece è possibile utilizzare una classe anonima:

```
package com.claudiodesio.osservatorio.test;

import com.claudiodesio.osservatorio.dati.Partecipante;
import com.claudiodesio.osservatorio.dati.Telescopio;

public class Osservazione {

    public static void main(String args[]) {
        Telescopio telescopio = new Telescopio();
        Partecipante[] partecipanti = getPartecipanti(telescopio);
        for (Partecipante partecipante : partecipanti) {
            partecipante.start();
        }
    }

    private static Partecipante[] getPartecipanti(Telescopio telescopio) {
        Partecipante[] partecipanti = {
            new Partecipante("Ciro", telescopio),
            new Partecipante("Gianluca", telescopio),
            new Partecipante("Pierluigi", telescopio),
            new Partecipante("Gigi", telescopio),
            new Partecipante("Nicola", telescopio) {
                @Override
                public void run() {
                    System.out.println(getNome() + " sono pronto!");
                    super.run();
                }
            },
            new Partecipante("Pino", telescopio),
        }
    }
}
```

```

        new Partecipante("Maurizio", telescopio),
        new Partecipante("Raffaele", telescopio),
        new Partecipante("Fabio", telescopio),
        new Partecipante("Vincenzo", telescopio));
    return partecipanti;
}
}

```

Soluzione 13.h)

Solo la terza affermazione è corretta. L'output di JShell è il seguente:

```

| Error:
| ')' expected
| new
| Thread(()->System.out.print("Java");System.out.print("Java");).start();
|

```

Infatti mancano le parentesi graffe intorno al blocco di codice. Se ci fossero:

```

new Thread(()->{System.out.print("Java");
System.out.print("Java");}).start();

```

allora la prima affermazione sarebbe stata quella corretta:

```

jshell> JavaJava

```

Soluzione 13.i)

Nessuna delle affermazioni è corretta. Infatti il codice non compilerà perché al costruttore di un Thread bisogna passare un'implementazione del metodo run, che non prende in input parametri come specificato nel codice. Eseguendo questo snippet su JShell otterremo il seguente output:

```

jshell> new Thread((int a)->{
...>   int b = 0;
...>   b = a/b;
...> }).start();
| Error:
| no suitable constructor found for Thread((int a)->{[...] b; })
|   constructor java.lang.Thread.Thread(java.lang.Runnable) is not
|   applicable
|   (argument mismatch; incompatible parameter types in lambda
|   expression)
|   constructor java.lang.Thread.Thread(java.lang.String) is not
|   applicable
|   (argument mismatch; java.lang.String is not a functional
|   interface)
|   new Thread((int a)->{
|   ^-----...

```

Notiamo che `ArithmeticException` estende `RuntimeException` non una checked exception. Il compilatore quindi non avrebbe segnalato errori, e per questa ragione la prima affermazione era sicuramente scorretta. La seconda e la quarta affermazione sono ovviamente false. La terza affermazione sarebbe stata corretta se l'implementazione del metodo `run` fosse stata corretta, ma non lo è.

Soluzione 13.j) Tipi innestati, Vero o Falso:

1. **Vero.**
2. **Vero.**
3. **Vero.**
4. **Falso**, le classi anonime devono per forza essere istanziate.
5. **Vero**, ma non l'abbiamo visto all'interno del libro.
6. **Falso.**
7. **Vero.**
8. **Falso**, una classe anonima non ha nome.
9. **Vero.**
10. **Falso**, una classe anonima non può essere dichiarata astratta.

Soluzione 13.k)

Le affermazioni corrette sono le 1, 2, 3, 4 e 5.

Soluzione 13.l)

Le affermazioni sono tutte corrette tranne la numero 3.

Soluzione 13.m)

La soluzione è:

```
public interface Operazione {  
    double operazione(double x, double y);  
}
```

Soluzione 13.n)

Sì, esiste la classe `DoubleBinaryOperator` che definisce il metodo:

```
double applyAsDouble(double left, double right)
```

che coincide come parametri e tipo di ritorno con il metodo `operazione` dell'interfaccia `Operazione` dell'esercizio precedente (ovviamente il nome non conta).

Soluzione 13.o)

La soluzione potrebbe essere la seguente:

```
package com.claudiodesio.lambda.test;  
  
import java.util.Arrays;  
import java.util.Comparator;  
  
public class Esercizio13O {
```

```

static String nomi[] = {"Clarissa", "Jem", "Top", "Ermeringildo", "Iamaca",
                        "Tom", "Arlequin", "Francesca", "Cumbus", "Blue"};
};

static int compareLunghezza(String first, String second) {
    return -(Integer.compare(first.length(), second.length()));
}

static int compareLunghezzaAlContrario(String first, String second) {
    return (Integer.compare(first.length(), second.length()));
}

static int compareAlfabeticoAlContrario(String first, String second) {
    return -(first.compareTo(second));
}

static int compareLunghezzaEAlfabeticoAlContrario(String first,
                                                    String second) {
    int result = -Integer.compare(first.length(),
                                   second.length());
    if (result == 0) {
        result = first.compareTo(second);
    }
    return result;
}

public static void main(String args[]) {

    Arrays.sort(nomi, Esercizio130::compareLunghezza);
    System.out.println("Nomi ordinati per lunghezza: "
        + Arrays.asList(nomi));

    Arrays.sort(nomi, Esercizio130::compareLunghezzaAlContrario);
    System.out.println("Nomi ordinati per lunghezza al contrario: "
        + Arrays.asList(nomi));

    Arrays.sort(nomi, String::compareTo);
    System.out.println("Nomi ordinati : " + Arrays.asList(nomi));

    Arrays.sort(nomi, Esercizio130::compareAlfabeticoAlContrario);
    System.out.println("Nomi ordinati al contrario: "
        + Arrays.asList(nomi));

    Arrays.sort(nomi, Esercizio130::compareLunghezzaEAlfabeticoAlContrario);
    System.out.println(
        "Nomi ordinati per lunghezza al contrario e in ordine alfabetico: "
        + Arrays.asList(nomi));
}
}

```

Soluzione 13.p)

L'esercizio era davvero semplice, la risposta giusta è la numero 4, Supplier, in quanto la definizione del suo metodo è la seguente:

```
T get()
```

dove T è un parametro generico dell'interfaccia.

Si noti che l'interfaccia funzionale Factory della risposta 2 non esiste.

Soluzione 13.q)

Nessuna delle affermazioni è corretta.

Solamente la numero 2 potrebbe non risultare scorretta in maniera evidente. Infatti un'interfaccia funzionale deve dichiarare un unico metodo astratto, ma è possibile dichiarare anche metodi di default e statici, sia pubblici che privati.

Soluzione 13.r)

Le affermazioni corrette sono la numero 3 (il che esclude la correttezza della 1 e della 2) e la numero 5 (che è coerente con la definizione delle espressioni lambda). L'affermazione numero 4 è scorretta perché il metodo SAM non può restituire `void`, ma deve restituire lo stesso tipo del costruttore.

Soluzione 13.s)

La classe `Persona` potrebbe essere la seguente:

```
import java.util.*;

public class Persona implements Comparable<Persona> {

    private String nome;

    private int anni;

    public Persona(String nome, int anni) {
        this.nome = nome;
        this.anni = anni;
    }

    public void setName(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }

    public void setAnni(int anni) {
        this.anni = anni;
    }

    public int getAnni() {
        return anni;
    }

    @Override
    public int compareTo(Persona altraPersona) {
        int result = Integer.valueOf(this.anni).compareTo(
            Integer.valueOf(altraPersona.anni));
    }
}
```

```

        if (result == 0) {
            result = this.nome.compareTo(altraPersona.nome);
        }
        return result;
    }

    public String toString() {
        return nome;
    }
}

```

Invece la classe `Esercizio13S` può essere completata con il “reference a metodo d’istanza di un certo tipo” nel seguente modo (in grassetto il codice aggiunto):

```

import java.util.Arrays;

public class Esercizio13S {
    public static void main(String args[]) {
        Persona [] persone = {
            new Persona("Antonio",21),
            new Persona("Bruno",20),
            new Persona("Giorgio",19),
            new Persona("Martino",22),
            new Persona("Daniele",21)
        };
        Arrays.sort(persone, Persona::compareTo);
        System.out.println(Arrays.toString(persone));
    }
}

```

Soluzione 13.t)

La risposta giusta è: `BiPredicate`. Infatti essa definisce il metodo:

```
boolean test(T t, U u)
```

Soluzione 13.u)

La soluzione potrebbe essere la seguente (il codice aggiunto è in grassetto):

```

import java.util.Arrays;
import java.util.function.BiPredicate;

public class Esercizio13U {
    public static void main(String args[]) {
        Persona [] persone = {
            new Persona("Antonio",21),
            new Persona("Bruno",20),
            new Persona("Giorgio",19),
            new Persona("Martino",22),
            new Persona("Daniele",21)
        };
        Persona personaCheIniziaPerD = getPersonaCheIniziaPer("D", persone,
            (String iniziale, Persona persona) ->
            persona.getNome().startsWith(iniziale));
        System.out.println(personaCheIniziaPerD);
    }

    static Persona getPersonaCheIniziaPer(String iniziale, Persona[] persone,

```

```
BiPredicate<String, Persona> biPredicate){
    for(Persona persona : persone) {
        if (biPredicate.test(iniziale, persona)) {
            return persona;
        }
    }
    return null;
}
}
```

Soluzione 13.v)

Le risposte corrette sono la numero 1, 3 e 4. La numero 2 è falsa, anzi il fatto che le espressioni lambda non ereditino le complicate regole delle classi anonime è uno dei vantaggi dell'utilizzare espressioni lambda al posto delle classi anonime. Le numero 5 e 6 sono false perché la sintassi di un reference a metodo non prevede codice innestato, e quindi è impossibile utilizzare altre espressioni lambda innestate o altri reference a metodo.

Soluzione 13.w)

È impossibile in questo caso sostituire la classe anonima con un'espressione lambda. Ricordiamo che un'espressione lambda funziona implementando interfacce funzionali (con un solo metodo astratto).

Soluzione 13.x)

La risposta corretta è l'ultima. Infatti l'espressione `super.intero`, implicherebbe che la superclasse della classe `Interna` (ovvero la classe `Object`) abbia una variabile che si chiama `intero`, cosa che in realtà non sussiste.

Segue il messaggio di errore della compilazione:

```
Esterna.java:11: error: cannot find symbol
    System.out.println(super.intero);
                        ^
  symbol:   variable intero
  1 error
```

Soluzione 13.y)

La soluzione che appare più semplice è la seguente:

```
public class Esterna {
    private int intero = 5;

    public static void main(String[] args) {
        Esterna.Interna interna = new Esterna().new Interna();
        interna.metodoInterno();
    }
    private class Interna extends Esterna {
        private int intero = 10;
```

```
protected void metodoInterno() {  
    System.out.println(super.intero);  
    System.out.println(this.intero);  
}  
}  
}
```

Abbiamo evidenziato in grassetto le modifiche apportate.

Soluzione 13.z)

Le risposte corrette sono la numero 1, 3, 4, 6 e 8. La numero 2 è falsa visto che è corretta la numero 3.

La numero 5 è falsa perché `TriFunction` non esiste, ma si potrebbe creare facilmente. La numero 7 è falsa perché non esiste il metodo `and` all'interno di `UnaryOperator` (semmai esiste `andThen`).

Soluzione 13.aa)

L'affermazione corretta è la 3. Infatti il codice del metodo `main` istanzia una classe anonima al volo, ridefinisce il metodo `metodo` e, senza assegnare la nuova istanza ad un reference, chiama (sempre al volo) il metodo appena ridefinito.

Sarebbe stato opportuno anche annotare il metodo ridefinito con l'annotazione `Override`.

Soluzione 13.bb)

Tutte le affermazioni sono vere. In particolare la numero 1 è vera, perché “Una classe innestata può dichiarare membri statici solo se dichiarata statica.”.

Soluzione 13.cc)

L'affermazione 1 è corretta. Infatti non è possibile compilare questo codice, perché non è possibile dichiarare una classe innestata all'interno di un costruttore (o di un metodo).

La numero 2 è falsa perché, mentre la prima affermazione potrebbe considerarsi corretta (se è dichiarata statica una variabile non può essere locale, e solo le variabili locali possono essere effettivamente `final`), la seconda non è vera perché la classe innestata `Innestata`, è a sua volta dichiarata statica, e quindi potrebbe utilizzare le variabili statiche della classe `Esterna`.

Per lo stesso motivo la numero 3 è anch'essa falsa.

La numero 4 è ovviamente falsa perché, come abbiamo detto per l'affermazione 1, non è possibile dichiarare una classe innestata all'interno di un costruttore (o di un metodo).

Soluzione 13.dd)

La seguente risposta vale se stiamo utilizzando una versione di Java non superiore alla 15. Infatti, con l'introduzione ufficiale dei tipi record nella versione 16 le regole del compilatore sono cambiate. In pratica è stato reso possibile la definizione di membri statici all'interno delle classe interne (cfr. capitolo 21.1.2 del capitolo extra online denominato "Aggiornamento a Java 17"). Quindi se usiamo una versione di Java dalla 16 in poi, allora la risposta giusta sarà la numero 4.

La risposta corretta è l'ultima (la numero 9). Infatti non è possibile dichiarare una enumerazione all'interno di una classe interna.

L'output della compilazione è il seguente:

```
Esercizio11S.java:4: error: enum declarations allowed only in static
  contexts
    public enum EnumInterna {
           ^
1 error
```

Soluzione 13.ee)

La seguente risposta vale se stiamo utilizzando una versione di Java non superiore alla 15. Infatti, con l'introduzione ufficiale dei tipi record nella versione 16 le regole del compilatore sono cambiate. In pratica è stato reso possibile la definizione di membri statici all'interno delle classi interne (cfr. capitolo 21.1.2 del capitolo extra online denominato "Aggiornamento a Java 17"). Quindi se usiamo una versione di Java dalla 16 in poi, questo esercizio è già stato risolto nel precedente.

Interpretando l'output della soluzione precedente:

```
Esercizio11S.java:4: error: enum declarations allowed only in static
  contexts
    public enum EnumInterna {
           ^
1 error
```

Si arriva facilmente a capire che il problema è che le classi interne (che ricordiamo essere classi innestate non statiche) non possono dichiarare enumerazioni. Il compilatore però ci dice che le dichiarazioni di enumerazioni sono permesse solo in contesti statici, così viene naturale dichiarare la classe innestata come statica in questo modo:

```
public enum Esercizio13EE {

    A, B, C;

    public static class Interna {
        public enum EnumInterna {
            D, E, F;
        }
    }
}
```

```

    public static void main(String args[]) {
        for (Esercizio11S.Interna.EnumInterna item :
            Esercizio11S.Interna.EnumInterna.values()) {
            System.out.println(item);
        }
    }
}

```

Compilando questo codice, otterremo il seguente output:

```

Esercizio13EE.java:9: error: package Esercizio13EE.Interna does not exist
    for (Esercizio13EE.Interna.EnumInterna item :
      ^
Esercizio13EE.java:9: error: package Esercizio11S.Interna does not
exist for (Esercizio13EE.Interna.EnumInterna item : Esercizio13EE.
Interna.EnumInterna.values()) {
      ^
2 errors

```

Il che ci fa capire che il tipo `Esercizio13EE.Interna.EnumInterna` non viene riconosciuto come tipo valido. La soluzione consiste nell'utilizzare il tipo corretto per il nostro contesto di visibilità: `Interna.EnumInterna` (ovvero non abbiamo specificato l'enumerazione che contiene sia il metodo `main`, sia la classe `Interna`). Quindi il codice dovrebbe essere il seguente:

```

public enum Esercizio13EE {

    A, B, C;

    public static class Interna {
        public enum EnumInterna {
            D, E, F;
        }
    }

    public static void main(String args[]) {
        for (Interna.EnumInterna item : Interna.EnumInterna.values()) {
            System.out.println(item);
        }
    }
}

```

che verrà compilato senza errori, ed il cui output risulterà essere il seguente:

```

D
E
F

```

come richiesto.

Soluzione 13.ff)

La risposta è la numero 6, come è possibile verificare nel paragrafo 13.1.7 quando nell'esempio della videoteca avevamo creato il codice da far eseguire al volo ad un metodo con la seguente sintassi:

```

public class TestVideoteca {
    public static void main(String args[]) {
//...
        Videoteca videoteca = new Videoteca();
        System.out.println("Bei Film:");
        Film[] beiFilms = videoteca.getFilmFiltrati(new FiltroFilm() {

```

```

        @Override
        public boolean filtra(Film film) {
            return film.getMediaRecensioni() > 3;
        }
    });
//...
    System.out.println("\nFilm di Fantascienza:");
    Film[] filmDiFantascienza = videoteca.getFilmFiltrati(
        new FiltroFilm() {
            @Override
            public boolean filtra(Film film) {
                return "Fantascienza".equals(film.getGenere());
            }
        });
//...

```

dove la classe Videoteca era così codificata:

```

public class Videoteca {
    private Film[] films;

    public Videoteca () {
        films = new Film[10];
        caricaFilms();
    }
    public void setFilms(Film[] films) {
        this.films = films;
    }

    public Film[] getFilms() {
        return films;
    }

    /* I SEGUENTI METODI SONO STATI SOSTITUITI DAL METODO DEFINITO SUBITO DOPO
    public Film[] getFilmDiFantascienza() {
        Film [] filmDiFantascienza = new Film[10];
        for (int i = 0, j= 0; i< 10;i++) {
            if ("Fantascienza".equals(films[i].getGenere())) {
                filmDiFantascienza[j] = films[i];
                j++;
            }
        }
        return filmDiFantascienza;
    }

    public Film[] getBeiFilm() {
        Film [] beiFilms = new Film[10];
        for (int i = 0, j= 0; i< 10;i++) {
            if (films[i].getMediaRecensioni() > 3) {
                beiFilms[j] = films[i];
                j++;
            }
        }
        return beiFilms;
    } */

    /* QUESTO METODO SOSTITUISCE I DUE PRECEDENTI (COMMENTATI) */
    public Film[] getFilmFiltrati(FiltroFilm filtroFilm) {
        Film [] filmFiltrati = new Film[10];
        for (int i = 0, j= 0; i< 10;i++) {
            if (filtroFilm.filtra(films[i])) {
                filmFiltrati[j] = films[i];
            }
        }
    }
}

```

```
        j++;  
    }  
    }  
    return filmFiltrati;  
}  
  
private void caricaFilms() {  
    //Caricamento film...  
}  
}
```

e dove l'interfaccia `FiltroFilm`, che viene ridefinita con le classi anonime nella classe `TestVideoteca`, era semplicemente la seguente:

```
public interface FiltroFilm {  
    boolean filtra(Film film);  
}
```

Soluzione F.a) Autoboxing, autounboxing e `java.lang`, Vero o Falso:

1. **Falso.**
2. **Vero.**
3. **Vero.**
4. **Vero.**
5. **Vero.**
6. **Vero.**
7. **Vero.**
8. **Falso**, dalla versione 1.5.
9. **Falso.**
10. **Falso**, non si può istanziare perché ha un costruttore privato ed è dichiarata `final` per non poter essere estesa.

Soluzione F.b)

Tutte le affermazioni sono corrette. In particolare nell'ultima si parla di modificare lo stato esterno di un oggetto... ma in realtà non esiste uno "stato esterno" da modificare.

Soluzione F.c)

Tutte le affermazioni sono false. In particolare la numero 2 e la numero 3 sono scorrette perché un carattere di una stringa è immagazzinato in 16 bit (non una stringa), mentre se la stringa è compatta un suo carattere è immagazzinato utilizzando 8 bit. La numero 4 è scorretta perché l'opzione `-XX:-CompactStrings`, va utilizzata quando si esegue il programma non quando lo si compila.

Soluzione F.d)

In teoria il risultato giusto sarebbe 13, ma la risposta giusta in realtà è l'ultima. Infatti nell'ultima istruzione mancano le parentesi tonde per invocare il metodo `length`, il che provoca il seguente errore in compilazione:

```
EsercizioFD.java:10: error: cannot find symbol
    System.out.println(stringa.length);
                               ^
  symbol:   variable length
  location: variable stringa of type String
1 error
```

Il compilatore ci avverte che non trova la variabile `length` (dato che mancano le parentesi tonde che caratterizzano la sintassi dei metodi). Notare che tutte le istruzioni (tranne la penultima), non riassegnano il risultato del metodo invocato a `stringa`, che quindi punta sempre allo stesso oggetto immutabile dichiarato inizialmente. Solo nella penultima istruzione viene riassegnata alla variabile `stringa` un nuovo oggetto (che concatena un punto esclamativo alla fine della stringa) con l'operatore `+=`. Ecco perché se non ci fosse l'errore all'ultimo statement, sarebbe stato stampato il valore 13.

Soluzione F.e)

L'output della classe `EsercizioFE` è il seguente:

```
intern() == equals()
```

quindi la risposta giusta è la numero 6. Infatti la chiamata al metodo `intern`, prova a recuperare l'oggetto `String` su cui è chiamato dal pool di stringhe, utilizzando il confronto che fornisce il metodo `equals`. Nel caso nel pool non esista la stringa desiderata, questa viene aggiunta, e viene restituito un reference ad essa. Quindi la stringa viene aggiunta alla pool, e le successive condizioni degli `if`, vengono verificate.

Soluzione F.f)

L'output della classe `EsercizioFF` è il seguente:

```
123456
```

Si noti che l'output comprende l'andare da capo finale. Infatti dopo aver dato come valore 123789 alla variabile `stringa1`, otteniamo `stringa2` concatenando a `stringa1` un separatore di linea (che fa andare a capo) grazie al metodo statico `lineSeparator` della classe `System`. Chiamando il metodo `toCharArray` su `stringa2`, immagazziniamo in `array1` l'array di caratteri che costituivano la stringa `stringa2`. Quindi tale array ha dimensioni 8 se lanciato su Windows (dove `System.lineSeparator` contiene due caratteri `\r` e `\n`), mentre ha dimensione 7 su altre piattaforme come Linux (dove `System.lineSeparator` contiene solo il carattere di escape `\n`). Poi all'array `array2` vengono assegnati tre elementi sempre di tipo carattere (4, 5 e 6), che vengono copiati tutti tramite il metodo `System.arraycopy` nell'array1 a partire dall'indice 2. Infine si stampa l'array `array2`, e poi si esce dal programma tramite il metodo `System.exit` (superfluo in questo caso perché il programma sarebbe terminato lo stesso subito dopo).

Soluzione F.g)

La soluzione potrebbe essere implementata nella seguente maniera. Creiamo la seguente enumerazione che definisce qualche tipologia di font semplicemente.

```
public enum Font {
    ARIAL, TIMES_NEW_ROMAN, COURIER, MONOSPACED;
}
```

Poi creiamo la classe `TestoRTF` richiesta in questa maniera:

```
public class TestoRTF implements Cloneable {
    String testo;
    Font carattere;
    boolean sottolineato;

    public TestoRTF (String testo, Font carattere, boolean sottolineato){
        this.testo = testo;
        this.carattere = carattere;
        this.sottolineato = sottolineato;
    }

    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }

    public String toString(){
        return "Testo = " + testo + ", carattere = " + carattere + ",
            sottolineato = " + sottolineato;
    }
}
```

Infine possiamo scrivere la classe di test in questa maniera:

```
public class EsercizioFG {
    public static void main(String args[]) throws CloneNotSupportedException {
        TestoRTF testoRTF1 = new TestoRTF("Java", Font.ARIAL, false);
        System.out.println(testoRTF1);
        System.out.println(testoRTF1.clone());
    }
}
```

Che produrrà il seguente output.

```
Testo = Java, carattere = ARIAL, sottolineato = false
Testo = Java, carattere = ARIAL, sottolineato = false
```

Soluzione F.h)

Una delle possibili soluzioni è la ridefinizione della classe `Autenticazione`:

```
package com.claudiodesio.autenticazione;

import java.util.Scanner;

public class Autenticazione {
    public void login() {
        String username = System.getProperty("username");
        System.out.println(username);
        String password = System.getProperty("password");
        System.out.println(password);
        Utente utente = trovaUtente(username);
        if (utente != null) {
            if (verificaPassword(utente, password)) {
                Stampa.auguraBenvenuto(utente.getNome());
            } else {

```

```

        Stampa.autenticazioneFallita();
    }
    } else {
        Stampa.usernameInesistente();
    }
}

private Utente trovaUtente(String username) {
    Utente[] utenti = ProfiliUtenti.getInstance().getUtenti();
    if (username != null) {
        for (Utente utente : utenti) {
            if (username.equals(utente.getUsername())) {
                return utente;
            }
        }
    }
    return null;
}

private boolean verificaPassword(Utente utente, String password) {
    boolean trovato = false;
    if (password != null) {
        if (password.equals(utente.getPassword())) {
            trovato = true;
        }
    }
    return trovato;
}

public static void main(String args[]) {
    Autenticazione autenticazione = new Autenticazione();
    autenticazione.login();
}
}

```

che lanciata con questi parametri:

```
java -Dusername=dansap -Dpassword=musica
com.claudiodesio.autenticazione.Autenticazione
```

darà luogo al seguente output:

```
dansap
musica
Benvenuto Daniele
```

Soluzione F.i)

Sono corrette le affermazioni numero 3 e 6. La numero 1 è falsa perché in realtà è possibile utilizzare anche altri algoritmi come SerialGC. Nell'affermazione numero 2 la prima parte della frase non è corretta: gli interventi della ParallelGC sono meno frequenti (ma più intensi). La “finalizzazione” consiste nel testare se esistono oggetti non più “raggiungibili” da qualche reference e quindi non utilizzabili, quindi anche l'affermazione 4 è errata. Il metodo `runFinalization` richiede la finalizzazione degli oggetti, ma risiede nella classe `Runtime` non `Object`, ecco perché anche l'affermazione 5 è errata.

Soluzione F.j)

Basterà utilizzare la variabile `PI` (che rappresenta il π della classe `Math`), e il metodo `pow` (che rappresenta la funzione potenza nel seguente modo:

```
public class EsercizioFJ {  
    public static void main(String args[]) {  
        int raggio = 7;  
        double area = Math.PI * Math.pow(raggio, 2);  
        System.out.println("L'area delle circonferenza di raggio 7 è " + area);  
    }  
}
```

E questo è il suo output:

```
L'area delle circonferenza di raggio 7 è 153.93804002589985
```

Soluzione F.k)

La risposta corretta è la 6, infatti l'output del è il seguente:

```
Exception in thread "main" java.lang.NullPointerException  
    at EsercizioFK.main(EsercizioFU.java:5)
```

questo perché il risultato della chiamata del metodo `floor`, non viene riassegnato a nessuna variabile, e quindi la variabile `e` rimane con il valore iniziale (2.718281828459045). Quindi il metodo `check` ritorna `null`, perché 2.0 è diverso da 2.718281828459045. Ma `null` non si può assegnare ad un tipo primitivo `boolean`.

Soluzione F.l)

L'output della classe `EsercizioFL`, è il seguente:

```
long 128
```

Infatti, come abbiamo asserito nel capitolo 3, il valore 128 viene considerato di default un `int`. Quindi, nonostante l'autoboxing sia sempre valido, viene invocato il metodo che ha come parametro un tipo primitivo, per ragioni di retro-compatibilità del codice, come spiegato nel paragrafo F.5.1.7.

Soluzioni degli Esercizi del capitolo 14 e dell'appendice H

Stream API e Framework Collections

Soluzione 14.a) *Framework Collections, Vero o Falso:*

1. **Vero.**
2. **Falso.**
3. **Vero.**
4. **Vero.**
5. **Vero.**
6. **Vero.**
7. **Vero.**
8. **Falso.**
9. **Vero.**
10. **Falso.**

Soluzione 14.b) *Stream API, Vero o Falso:*

1. **Falso**, `Stream` è un'interfaccia.
2. **Falso.**
3. **Vero.**
4. **Vero.**
5. **Falso**, `Optional` è una classe (peraltro dichiarata `final` e quindi non estendibile).
6. **Falso**, sono operazioni terminali.
7. **Falso**, il metodo `joining` appartiene alla classe `Collectors`.
8. **Falso.**
9. **Vero.**
10. **Falso.**

Soluzione 14.c)

L'eccezione personalizzata potrebbe essere la seguente:

```
package com.claudiodesio.eccezioni;

public class DuplicatoException extends RuntimeException {

    public DuplicatoException(Object elementoDuplicato) {
        super("Impossibile aggiungere l'elemento \""
            + elementoDuplicato + "\" perché già presente");
    }
}
```

Mentre potremmo codificare la collection richiesta in questo modo:

```
package com.claudiodesio.collections;
```

```
import com.claudiodesio.eccezioni.DuplicatoException;
import java.util.HashSet;

public class SetRobusto<E> extends HashSet<E> {

    @Override
    public boolean add(E e) {
        boolean result = super.add(e);
        if (!result) {
            throw new DuplicatoException(e);
        }
        return result;
    }
}
```

Ed ecco una classe di test:

```
package com.claudiodesio;

import com.claudiodesio.collections.SetRobusto;
import com.claudiodesio.eccezioni.DuplicatoException;

public class TestSetRobusto {

    public static void main(String args[]) {
        SetRobusto<String> set = getSetRobusto();
        try {
            set.add("Italia");
        } catch (DuplicatoException duplicatoException) {
            System.out.println(duplicatoException.getMessage());
        }
        System.out.println(set);
    }

    public static SetRobusto<String> getSetRobusto() {
        SetRobusto<String> set = new SetRobusto<>();
        set.add("Italia");
        set.add("Francia");
        set.add("Polonia");
        set.add("Germania");
        set.add("Inghilterra");
        set.add("Spagna");
        set.add("Grecia");
        set.add("Olanda");
        set.add("Portogallo");
        set.add("Belgio");
        return set;
    }
}
```

L'output sarà:

```
Impossibile aggiungere l'elemento "Italia" perché già presente
[Germania, Inghilterra, Francia, Belgio, Polonia, Olanda, Italia,
Spagna, Grecia, Portogallo]
```

Soluzione 14.d)

Il listato della mappa richiesta potrebbe essere il seguente:

```
package com.claudiodesio.collections;
```

```

import java.util.ArrayList;
import java.util.Collection;
import java.util.HashMap;

public class MappaIncrementale<K, V> extends HashMap<K, Collection<V>> {

    public void add(K key, V value) {
        if (this.get(key) == null) {
            Collection<V> collection = new ArrayList<>();
            collection.add(value);
            this.put(key, collection);
        } else {
            Collection<V> collection = this.get(key);
            collection.add(value);
        }
    }
}

```

Si noti che il metodo `add` non è un override (il metodo per aggiungere coppie chiave-valore è il metodo `put`) bensì un metodo ad hoc.

Segue una classe di test:

```

package com.claudiodesio.test;

import com.claudiodesio.collections.MappaIncrementale;
import com.claudiodesio.collections.SetRobusto;
import java.util.Iterator;

public class TestMappaIncrementale {

    public static void main(String args[]) {
        MappaIncrementale<Integer, String> mappa = new MappaIncrementale<>();
        riempiMappaIncrementale(mappa);
        System.out.println(mappa);
    }

    public static void riempiMappaIncrementale(
        MappaIncrementale<Integer, String> mappa) {
        SetRobusto<String> set = TestSetRobusto.getSetRobusto();
        int i = 1;
        int j = 1;
        Iterator<String> iterator = set.iterator();
        while (iterator.hasNext()) {
            if (i % 3 == 0) {
                j++;
            }
            String string = iterator.next();
            mappa.add(j, string);
            i++;
        }
    }
}

```

che genera il seguente output:

```
{1=[Germania, Inghilterra], 2=[Francia, Belgio, Polonia], 3=[Olanda, Italia, Spagna], 4=[Grecia, Portogallo]}
```

Soluzione 14.e)

Il listato della mappa richiesta potrebbe essere il seguente:

```
package com.claudiodesio.collections;

import java.util.Collection;

public class MappaIncrementaleRobusta<K, V> extends MappaIncrementale<K, V> {

    @Override
    public void add(K key, V value) {
        if (this.get(key) == null) {
            Collection<V> setRobusto = new SetRobusto<>();
            setRobusto.add(value);
            this.put(key, setRobusto);
        } else {
            Collection<V> setRobusto = this.get(key);
            setRobusto.add(value);
        }
    }
}
```

Si noti che questa volta il metodo `add` è un `override`, e semplicemente cambiando l'`ArrayList` definito nella classe `MappaIncrementale` con un `SetRobusto` abbiamo risolto la situazione. Volendo potremmo fare un po' di refactoring su queste due classi in modo tale da migliorare il nostro codice. Per prima cosa ritorniamo alla classe `MappaIncrementale` e riscriviamo il metodo `add` con dei piccoli accorgimenti:

```
package com.claudiodesio.collections;

import java.util.ArrayList;
import java.util.Collection;
import java.util.HashMap;

public class MappaIncrementale<K, V> extends HashMap<K, Collection<V>> {

    public void add(K key, V value) {
        if (this.get(key) == null) {
            Collection<V> collection = getCollection();
            collection.add(value);
            this.put(key, collection);
        } else {
            Collection<V> arrayList = this.get(key);
            arrayList.add(value);
        }
    }

    protected Collection<V> getCollection() {
        return new ArrayList<>();
    }
}
```

In questo modo possiamo semplificare la sottoclasse:

```
package com.claudiodesio.collections;

import java.util.Collection;

public class MappaIncrementaleRobusta<K, V> extends MappaIncrementale<K, V> {

    @Override
    protected Collection<V> getCollection() {
```

```
        return new SetRobusto<>();
    }
}
```

Ed ottenere lo stesso risultato, senza duplicazioni di codice.

Segue la classe di test:

```
package com.claudiodesio.test;

import com.claudiodesio.collections.MappaIncrementale;
import com.claudiodesio.collections.MappaIncrementaleRobusta;
import com.claudiodesio.eccezioni.DuplicatoException;

public class TestMappaIncrementaleRobusta {

    public static void main(String args[]) {

        MappaIncrementale<Integer, String> mappa =
            new MappaIncrementaleRobusta<>();
        TestMappaIncrementale.riempiMappaIncrementale(mappa);
        try {
            mappa.add(4, "Grecia");
        } catch (DuplicatoException duplicatoException) {
            System.out.println(duplicatoException.getMessage());
        }
        System.out.println(mappa);
    }
}
```

che genera il seguente output:

```
Impossibile aggiungere l'elemento "Grecia" perché già presente
{1=[Germania, Inghilterra], 2=[Francia, Belgio, Polonia], 3=[Olanda,
Italia, Spagna], 4=[Grecia, Portogallo]}
```

Soluzione 14.f)

Nella classe `Citta` aggiungiamo i metodi richiesti:

```
@Override
public int hashCode() {
    int hash = 7;
    hash = 19 * hash + Objects.hashCode(this.nome);
    return hash;
}

@Override
public boolean equals(Object obj) {
    if (obj == null) {
        return false;
    }
    if (getClass() != obj.getClass()) {
        return false;
    }
    final Citta other = (Citta) obj;
    if (!Objects.equals(this.nome, other.nome)) {
        return false;
    }
    return true;
}
```

La classe `TestStreams` invece potrebbe essere codificata come segue:

```
package com.claudiodesio.test;

import com.claudiodesio.collections.SetRobusto;
import com.claudiodesio.dati.Citta;

public class TestStreams {

    public static void main(String args[]) {
        SetRobusto<Citta> set = getSetRobusto();
        System.out.println("Città di mare:");
        set.stream().filter(e->e.isDiMare()).forEach(System.out::println);
        System.out.println("\nCittà capoluogo:");
        set.stream().filter(e->e.isCapoluogo()).forEach(System.out::println);
        System.out.println("\nCittà che finiscono con 'a':");
        set.stream().filter(e->e.getNome().endsWith("a")).
            forEach(System.out::println);
    }

    public static SetRobusto<Citta> getSetRobusto() {
        SetRobusto<Citta> set = new SetRobusto<>();
        set.add(new Citta("Milano", true, false));
        set.add(new Citta("Rovigo", false, false));
        set.add(new Citta("Potenza", true, false));
        set.add(new Citta("Siracusa", false, true));
        set.add(new Citta("Perugia", true, false));
        set.add(new Citta("Napoli", true, true));
        set.add(new Citta("Pescara", false, true));
        set.add(new Citta("Taranto", false, true));
        set.add(new Citta("Siena", false, false));
        return set;
    }
}
```

Segue l'output:

Città di mare:

Napoli
Siracusa
Taranto
Pescara

Città capoluogo:

Napoli
Potenza
Perugia
Milano

Città che finiscono con 'a':

Potenza
Perugia
Siena
Siracusa
Pescara

Soluzione 14.g)

Le affermazioni corrette sono le numero 2, 4 e 5.

Soluzione 14.h)

La risposta corretta è la numero 3 in quanto `HashMap` non estende `Collection`, e quindi non si può assegnare un reference di tipo `Collection` ad un `HashMap`. L'output della compilazione è infatti il seguente:

```
Esercizio14H.java:5: warning: [rawtypes] found raw type: Collection
    Collection map = new HashMap(10);
    ^
    missing type arguments for generic class Collection<E>
    where E is a type-variable:
      E extends Object declared in interface Collection
Esercizio14H.java:5: warning: [rawtypes] found raw type: HashMap
    Collection map = new HashMap(10);
    ^
    missing type arguments for generic class HashMap<K,V>
    where K,V are type-variables:
      K extends Object declared in class HashMap
      V extends Object declared in class HashMap
Esercizio14H.java:5: error: incompatible types: HashMap cannot be
    converted to Collection
    Collection map = new HashMap(10);
    ^
Esercizio14H.java:6: error: cannot find symbol
    map.put(1,1);
    ^
    symbol:   method put(int,int)
    location: variable map of type Collection
2 errors
2 warnings
```

Si noti che vengono mostrati anche due warning perché abbiamo usato raw type, e che anche gli errori sono due, perché il reference `map`, essendo di tipo `Collection`, non dichiara il metodo `put` (che viene dichiarato nell'interfaccia `Map`).

Soluzione 14.i)

Le risposte corrette sono le numero 1 e 3. `Iterator` non estende `Iterable`, quindi l'affermazione 2 è non corretta. Nel caso dell'affermazione 4 l'affermazione sarebbe stata corretta se fosse stata: `Collection` **estende** l'interfaccia `Iterable`. Infatti sia `Collection` che `Iterable` sono interfacce e non classi. Questo implica che una può estendere l'altra, non implementarla.

Soluzione 14.l)

L'unica risposta corretta è la numero 4. L'affermazione numero 1 è falsa semplicemente perché `Collection` è un'interfaccia e non una classe. Per quanto riguarda l'affermazione 2, il metodo

`toArray` è dichiarato dall'interfaccia `Collection`. Inoltre, non esiste un metodo `toCollection` nella classe `Arrays`, quindi anche l'affermazione 3 non è corretta.

Soluzione 14.m)

L'output della classe `Esercizio14M` è il seguente:

```
*
$
$
$
*
```

Infatti vengono inseriti due elementi (le stringhe `*` e `@`) con il metodo `add` nella lista `list`, e poi con il metodo `set` viene sovrascritto la stringa `@` con la stringa `$`. I successivi cicli stampano gli elementi della lista ciclando con un `ListIterator` prima in avanti e poi all'indietro.

Soluzione 14.n)

La soluzione potrebbe essere la seguente classe:

```
import java.util.*;

public class Esercizio14N {
    public static void main(String args[]) {
        ArrayList<Integer> list = new ArrayList<>();
        list.ensureCapacity(1000000);
        long startTime = System.currentTimeMillis();
        for (int i = 1; i <= 2000000; ++i) {
            if (i % 2 == 0) {
                list.add(i);
            }
        }
        long endTime = System.currentTimeMillis();
        System.out.println("Tempo = " + (endTime - startTime));
    }
}
```

Soluzione 14.o)

Tutte le affermazioni sono corrette tranne la numero 1. Infatti essendo `Set` estesa da `SortedSet` (vedi affermazione numero 3), le implementazioni di `SortedSet` come `TreeSet` saranno ordinate (“sorted” in inglese significa “ordinato”). La numero 2 è vera proprio in virtù del fatto che qualsiasi `Set` non ammette duplicati, quindi non è possibile neanche aggiungere due volte l'elemento `null`.

Soluzione 14.p)

Le affermazioni numero 1, 3 e 4 sono corrette.

Soluzione 14.q)

La soluzione potrebbe essere la seguente classe:

```
import java.util.*;

public class Esercizio14Q {

    public static void main(String args[]) {
        ClaudioLinkedList claudioLinkedList = new ClaudioLinkedList();
        claudioLinkedList.removeFirst();
        claudioLinkedList.addFirst("C");
        claudioLinkedList.set(2, "A");
        claudioLinkedList.removeLast();
        claudioLinkedList.addLast("O");
        System.out.println(claudioLinkedList);
    }
}
```

Soluzione 14.r)

Potremmo utilizzare un `Hashtable`, una `ConcurrentHashMap`, oppure utilizzare un reference che punta al risultato dell'invocazione di un metodo sincronizzatore `synchronizedMap`.

Soluzione 14.s)

Potremmo utilizzare un reference che punta al risultato dell'invocazione di un metodo sincronizzatore `unmodifiableList`, come riportato di seguito:

```
List<String> immutableList = Arrays.asList("a", "b", "c");
immutableList = Collections.unmodifiableList(immutableList);
```

Oppure è possibile utilizzare il metodo di convenienza statico `of` dell'interfaccia `List` introdotto in Java 9:

```
List immutableList = List.of("a", "b", "c");
```

Stesso discorso per le implementazioni immutabili di `Set`. Ecco i due esempi richiesti:

```
Set<String> immutableSet = new HashSet<>(Arrays.asList("a", "b", "c"));
immutableSet = Collections.unmodifiableSet(immutableSet);
```

e sfruttando il metodo statico `of` dell'interfaccia `Set`:

```
Set<String> immutableSet = Set.of("a", "b", "c");
```

Soluzione 14.t)

Una possibile soluzione potrebbe essere la seguente:

```
import java.util.*;

public class Esercizio14T {

    public static void main(String args[]) {
        List<String> stringList = getStringList();
        Double average = stringList.stream()
            .filter(s -> !s.startsWith("a")).mapToInt(
                String::length).average().getAsDouble();
        System.out.println(average);
    }
}
```

```

    }

    public static List<String> getStringList() {
        String stringa = "I ragazzi che si amano si baciano in piedi "
            + "Contro le porte della notte "
            + "E i passanti che passano li segnano a dito "
            + "Ma i ragazzi che si amano "
            + "Non ci sono per nessuno "
            + "Ed è la loro ombra soltanto "
            + "Che trema nella notte "
            + "Stimolando la rabbia dei passanti "
            + "La loro rabbia il loro disprezzo le risa la loro invidia "
            + "I ragazzi che si amano non ci sono per nessuno "
            + "Essi sono altrove molto più lontano della notte "
            + "Molto più in alto del giorno "
            + "Nell'abbagliante splendore del loro primo amore ";
        String[] stringhe = stringa.split(" ");
        return Arrays.asList(stringhe);
    }
}

```

Che produrrà il seguente output:

```
4.607142857142857
```

Soluzione 14.u)

Le affermazioni corrette sono le numero 2 e 3. L'affermazione numero 1 è scorretta perché `Optional` è una classe generica. La 4 è scorretta perché `findFirst` è dichiarata nell'interfaccia `Stream`.

Soluzione 14.v)

La soluzione potrebbe essere la seguente (la pipeline introdotta è in grassetto):

```

import java.util.*;
import java.util.stream.*;

public class Esercizio14V {

    public static void main(String args[]) {
        List<String> stringList = getStringList();
        Map<String, List<String>> map =
            stringList.stream().collect(Collectors.groupingBy(
                s -> (""+s.charAt(0)).toLowerCase());
        System.out.println(map);
    }

    public static List<String> getStringList() {
        String stringa = "I ragazzi che si amano si baciano in piedi "
            + "Contro le porte della notte "
            + "E i passanti che passano li segnano a dito "
            + "Ma i ragazzi che si amano "
            + "Non ci sono per nessuno "
            + "Ed è la loro ombra soltanto "
            + "Che trema nella notte "
            + "Stimolando la rabbia dei passanti "
    }
}

```

```

        + "La loro rabbia il loro disprezzo le risa la loro invidia "
        + "I ragazzi che si amano non ci sono per nessuno "
        + "Essi sono altrove molto più lontano della notte "
        + "Molto più in alto del giorno "
        + "Nell'abbagliante splendore del loro primo amore ";
String[] stringhe = stringa.split(" ");
return Arrays.asList(stringhe);
    }
}

```

Si noti che la `Function` passata al metodo `groupingBy`, ritorna il primo carattere, che viene “sommato” ad una stringa vuota per essere tramutato in stringa, per poi essere reso minuscolo per evitare di avere distinzioni tra maiuscole e minuscole. Purtroppo l’output non è molto leggibile (vedi esercizio successivo):

```

{a=[amano, a, amano, amano, altrove, alto, amore], b=[baciano], c=[che,
Contro, che, che, ci, Che, che, ci], d=[della, dito, dei, disprezzo,
della, del, del], e=[E, Ed, Essi], g=[giorno], è=[è], i=[I, in, i, i,
il, invidia, I, in], l=[le, li, la, loro, la, La, loro, loro, le, la,
loro, lontano, loro], m=[Ma, molto, Molto], n=[notte, Non, nessuno,
nella, notte, non, nessuno, notte, Nell'abbagliante], o=[ombra],
p=[piedi, porte, passanti, passano, per, passanti, per, più, più,
primo], r=[ragazzi, ragazzi, rabbia, rabbia, risa, ragazzi], s=[si, si,
segnano, si, sono, soltanto, Stimolando, si, sono, sono, splendore],
t=[trema]}

```

Soluzione 14.z)

La soluzione potrebbe essere la seguente (in grassetto la pipeline richiesta):

```

import java.util.*;
import java.util.stream.*;

public class Esercizio14Z {

    public static void main(String args[]) {
        List<String> stringList = getStringList();
        Map<String, List<String>> map = stringList.stream().collect(
            Collectors.groupingBy(s -> (""+s.charAt(0)).toLowerCase()));
        map.entrySet().stream().forEach(System.out::println);
    }

    public static List<String> getStringList() {
        String stringa ="I ragazzi che si amano si baciano in piedi "
        + "Contro le porte della notte "
        + "E i passanti che passano li segnano a dito "
        + "Ma i ragazzi che si amano "
        + "Non ci sono per nessuno "
        + "Ed è la loro ombra soltanto "
        + "Che trema nella notte "
        + "Stimolando la rabbia dei passanti "
        + "La loro rabbia il loro disprezzo le risa la loro invidia "
        + "I ragazzi che si amano non ci sono per nessuno "
        + "Essi sono altrove molto più lontano della notte "
        + "Molto più in alto del giorno "
        + "Nell'abbagliante splendore del loro primo amore ";
        String[] stringhe = stringa.split(" ");
        return Arrays.asList(stringhe);
    }
}

```

```
}  
}
```

L'output è decisamente più chiaro:

```
a=[amano, a, amano, amano, altrove, alto, amore]  
b=[baciano]  
c=[che, Contro, che, che, ci, Che, che, ci]  
d=[della, dito, dei, disprezzo, della, del, del]  
e=[E, Ed, Essi]  
g=[giorno]  
è=[è]  
i=[I, in, i, i, il, invidia, I, in]  
l=[le, li, la, loro, la, La, loro, loro, le, la, loro, lontano, loro]  
m=[Ma, molto, Molto]  
n=[notte, Non, nessuno, nella, notte, non, nessuno, notte,  
    Nell'abbagliante]  
o=[ombra]  
p=[piedi, porte, passanti, passano, per, passanti, per, più, più,  
    primo]  
r=[ragazzi, ragazzi, rabbia, rabbia, risa, ragazzi]  
s=[si, si, segnano, si, sono, soltanto, Stimolando, si, sono, sono,  
    splendore]  
t=[trema]
```

Soluzione 14.aa)

Solo l'affermazione 6 è corretta.

Soluzioni degli esercizi del capitolo 15 e dell'appendice I

Input-Output e Date-Time API

Soluzione 15.a) *Input - Output, Vero o Falso:*

1. **Vero.**
2. **Vero.**
3. **Falso.**
4. **Falso.**
5. **Vero.**
6. **Vero.**
7. **Vero.**
8. **Vero.**
9. **Falso.**
10. **Falso.**

Soluzione 15.b) *Serializzazione, Vero o Falso:*

1. **Vero.**
2. **Vero.**
3. **Falso.**
4. **Falso.**
5. **Falso.**
6. **Vero.**
7. **Vero.**
8. **Falso.**
9. **Vero.**
10. **Vero.**

Soluzione 15.c) *New Input Output, Vero o Falso:*

1. **Vero.**
2. **Vero.**
3. **Vero.**
4. **Vero.**
5. **Falso.**
6. **Vero.**
7. **Vero.**
8. **Falso.**
9. **Falso.**
10. **Vero.**

Soluzione 15.d)

Il listato dovrebbe essere simile al seguente:

```
package com.claudiodesio.io;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class EditorInterattivo {

    public static void main(String args[]) {
        File file = new File("file.txt");
        try (Scanner scanner = new Scanner(System.in);
            FileWriter fileWriter = new FileWriter(file);) {
            String stringa = "";
            System.out.println("Digita qualcosa e batti enter, oppure scrivi "
                + "\"fine\" per terminare il programma");
            while (!(stringa = scanner.nextLine()).equals("fine")) {
                System.out.println("Hai digitato " + stringa);
                fileWriter.append(stringa);
                fileWriter.flush();
            }

        } catch (IOException ex) {
            ex.printStackTrace();
        }
        System.out.println("Fine programma!");
    }
}
```

Soluzione 15.e)

Il listato dovrebbe essere simile al seguente:

```
package com.claudiodesio.io;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class More {

    private final static int numeroRighe = 10;
    private static final String QUIT_COMMAND = "q";
    private static final String NEXT_COMMAND = "n";

    public static void main(String args[]) {
        String carattere;
        System.out.println("Digita il nome di un file e batti enter, oppure "
            + "scrivi \"\" + QUIT_COMMAND + "\" per terminare il programma");
        try (Scanner in = new Scanner(System.in);) {
            String nomeFile;
            if (!(nomeFile = in.nextLine()).equals(QUIT_COMMAND)) {
```

```

        File file = new File(nomeFile);
        try (Scanner fileScanner = new Scanner(file)) {
            System.out.println("File trovato! \"\" + nomeFile +
                "\", scrivi \"n\" per vedere le prossime 10 righe");
            while (!(carattere = in.nextLine()).equals(QUIT_COMMAND)) {
                if (carattere.equals(NEXT_COMMAND)) {
                    for (int i = 0; fileScanner.hasNext() &&
                        i < numeroRighe; i++) {
                        System.out.println(fileScanner.nextLine());
                    }
                }
            }
        } catch (FileNotFoundException exc) {
            System.out.println("File non trovato!");
        }
    }
} catch (Exception ex) {
    ex.printStackTrace();
}
System.out.println("Fine programma!");
}
}

```

Segue un esempio di output:

```

java com.claudiodesio.io.More

Digita il nome di un file e batti enter, oppure scrivi "q" per
terminare il programma
More.java
File trovato! "More.java"
n
package com.claudiodesio.io;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class More {

    private final static int numeroRighe = 10;
    private static final String QUIT_COMMAND = "q";
n
    private static final String NEXT_COMMAND = "n";

    public static void main(String args[]) {
        String carattere;
        System.out.println("Digita il nome di un file e batti enter,
oppure " + "scrivi \"\" + QUIT_COMMAND + "\" per terminare il
programma");
        try (Scanner in = new Scanner(System.in);) {
            String nomeFile;
            if (!(nomeFile = in.nextLine()).equals(QUIT_COMMAND)) {
                File file = new File(nomeFile);
n
                try (Scanner fileScanner = new Scanner(file)) {
                    System.out.println("File trovato! \"\" +
nomeFile + "\"");

```

```

        while (!(carattere =
in.nextLine()).equals(QUIT_COMMAND)) {
            if (carattere.equals(NEXT_COMMAND)) {
                for (int i = 0; fileScanner.hasNext()
&& i < numeroRighe; i++) {
                    System.out.println(
fileScanner.nextLine());
                }
            }
        }
    } catch (FileNotFoundException exc) {
n
        System.out.println("File non trovato!");
    }
}
} catch (Exception ex) {
    ex.printStackTrace();
}
System.out.println("Fine programma!");
}
}
q
Fine programma!

```

Soluzione 15.f)

Abbiamo deciso di definire definito una classe di utilità FileUtils nel seguente modo:

```

package rubrica.util;

public class FileUtils {

    public static final String SUFFIX = ".con";

    public static String getNomeFile(String nome) {
        return nome + SUFFIX;
    }
}

```

Poi abbiamo implementato la classe GestoreFile, che si occupa di implementare i due metodi richiesti, inserisci e recupera, nel seguente modo:

```

package rubrica.integrazione;
import java.util.*;
import java.io.*;
import rubrica.dati.*;
import rubrica.util.*;

public class GestoreFile {

    public void inserisci(Contatto contatto) throws IOException {
        try (FileOutputStream fos =
            new FileOutputStream (FileUtils.getNomeFile(contatto.getNome()));
            ObjectOutputStream s = new ObjectOutputStream (fos);) {
            s.writeObject (contatto);
        }
    }
}

```



```
public Contatto recupera(String nome) throws IOException,
                                                                    ClassNotFoundException {
    Contatto contatto = null;
    try (FileInputStream fis = new FileInputStream(
                                                FileUtils.getNomeFile(nome));
         ObjectInputStream ois = new ObjectInputStream (fis);) {
        contatto = (Contatto)ois.readObject();
    }
    return contatto;
}
}
```

Si noti che abbiamo utilizzato il costrutto try-with-resources in entrambi i metodi.

Soluzione 15.g)

Segue la classe di test richiesta:

```
package rubrica.test;
import rubrica.dati.*;
import rubrica.integrazione.*;

public class Esercizio15G {
    private GestoreFile gestoreFile;
    private Contatto[] contatti;

    Esercizio15G() {
        contatti = getContatti();
        gestoreFile = new GestoreFile();
    }

    private void eseguiTest() {
        System.out.println("TESTIAMO LA CREAZIONE DEI TRE CONTATTI");
        creaContatti();
        System.out.println("RECUPERIAMO I TRE CONTATTI");
        recuperaContatti();
    }

    public void creaContatti() {
        for (Contatto contatto : contatti) {
            creaContatto(contatto);
        }
    }

    public void recuperaContatti() {
        for (Contatto contatto : contatti) {
            recuperaContatto(contatto.getNome());
        }
    }

    private void creaContatto(Contatto contatto) {
        try {
            gestoreFile.inserisci(contatto);
            System.out.println("Contatto creato:\n"+ contatto);
        }
        catch (Exception exc) {
            exc.printStackTrace();
        }
    }
}
```

```
public void recuperaContatto(String nomeContatto) {
    try {
        Contatto contatto = gestoreFile.recupera(nomeContatto);
        System.out.println("Contatto recuperato:\n"+contatto);
    }
    catch (Exception exc) {
        exc.printStackTrace();
    }
}

private Contatto[] getContatti() {
    Contatto contatto1 = new Contatto("Daniele", "01234560",
        "Via delle chitarre 1" );
    Contatto contatto2 = new Contatto("Giovanni", "0565432190",
        "Via delle scienze 2");
    Contatto contatto3 = new Contatto("Ligeia", "07899921",
        "Via dei segreti 3");
    Contatto[] contatti = {
        contatto1, contatto2, contatto3
    } ;
    return contatti;
}

public static void main(String args[]) {
    Esercizio15G esercizio15G = new Esercizio15G();
    esercizio15G.eseguiTest();
}
}
```

che produrrà il seguente output:

```
TESTIAMO LA CREAZIONE DEI TRE CONTATTI
Contatto creato:
Nome: Daniele
Telefono: 01234560
Indirizzo: Via delle chitarre 1
Contatto creato:
Nome: Giovanni
Telefono: 0565432190
Indirizzo: Via delle scienze 2
Contatto creato:
Nome: Ligeia
Telefono: 07899921
Indirizzo: Via dei segreti 3
RECUPERIAMO I TRE CONTATTI
Contatto recuperato:
Nome: Daniele
Telefono: 01234560
Indirizzo: Via delle chitarre 1
Contatto recuperato:
Nome: Giovanni
Telefono: 0565432190
Indirizzo :      Via delle scienze 2
Contatto recuperato:
Nome: Ligeia
Telefono: 07899921
Indirizzo: Via dei segreti 3
```

Soluzione 15.h)

Potremmo codificare i nuovi metodi come nel seguente file (in grassetto il codice aggiunto):

```
package rubrica.test;
import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.integrazione.*;

public class Esercizio15H {
    private GestoreFile gestoreFile;
    private Contatto[] contatti;

    Esercizio15H() {
        contatti = getContatti();
        gestoreFile = new GestoreFile();
    }

    private void eseguiTest() {
        System.out.println("TESTIAMO LA CREAZIONE DEI TRE CONTATTI");
        creaContatti();
        System.out.println("RECUPERIAMO I TRE CONTATTI");
        recuperaContatti();
        System.out.println(
            "TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE");
        creaContattoEsistente();
        System.out.println("PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE");
        recuperaContattoNonEsistente();
    }

    public void creaContattoEsistente() {
        creaContatto(contatti[0]);
    }

    public void recuperaContattoNonEsistente() {
        recuperaContatto("Pippo");
    }

    public void creaContatti() {
        for (Contatto contatto : contatti) {
            creaContatto(contatto);
        }
    }

    public void recuperaContatti() {
        for (Contatto contatto : contatti) {
            recuperaContatto(contatto.getNome());
        }
    }

    private void creaContatto(Contatto contatto) {
        try {
            gestoreFile.inserisci(contatto);
            System.out.println("Contatto creato:\n"+ contatto);
        }
        catch (Exception exc) {
            exc.printStackTrace();
        }
    }
}
```

```
public void recuperaContatto(String nomeContatto) {
    try {
        Contatto contatto = gestoreFile.recupera(nomeContatto);
        System.out.println("Contatto recuperato:\n"+contatto);
    }
    catch (Exception exc) {
        exc.printStackTrace();
    }
}

private Contatto[] getContatti() {
    Contatto contatto1 = new Contatto("Daniele", "01234560",
        "Via delle chitarre 1" );
    Contatto contatto2 = new Contatto("Giovanni", "0565432190",
        "Via delle scienze 2");
    Contatto contatto3 = new Contatto("Ligeia", "07899921",
        "Via dei segreti 3");
    Contatto[] contatti = {
        contatto1, contatto2, contatto3
    } ;
    return contatti;
}

public static void main(String args[]) {
    Esercizio15H esercizio15H = new Esercizio15H();
    esercizio15H.eseguiTest();
}
}
```

L'output risultante sarà:

```
TESTIAMO LA CREAZIONE DEI TRE CONTATTI
Contatto creato:
Nome:   Daniele
Telefono:   01234560
Indirizzo:   Via delle chitarre 1
Contatto creato:
Nome:   Giovanni
Telefono:   0565432190
Indirizzo:   Via delle scienze 2
Contatto creato:
Nome:   Ligeia
Telefono:   07899921
Indirizzo:   Via dei segreti 3
RECUPERIAMO I TRE CONTATTI
Contatto recuperato:
Nome:   Daniele
Telefono:   01234560
Indirizzo:   Via delle chitarre 1
Contatto recuperato:
Nome:   Giovanni
Telefono:   0565432190
Indirizzo:   Via delle scienze 2
Contatto recuperato:
Nome:   Ligeia
Telefono:   07899921
Indirizzo:   Via dei segreti 3
TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE
```

```

Contatto creato:
Nome:    Daniele
Telefono:    01234560
Indirizzo:    Via delle chitarre 1
PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE
java.io.FileNotFoundException: Pippo.con (Impossibile trovare il file
specificato)
    at java.base/java.io.FileInputStream.open0(Native Method)
    at
java.base/java.io.FileInputStream.open(FileInputStream.java:213)
    at
java.base/java.io.FileInputStream.<init>(FileInputStream.java:155)
    at
java.base/java.io.FileInputStream.<init>(FileInputStream.java:110)
    at rubrica.integrazione.GestoreFile.recupera(GestoreFile.java:51)
    at
rubrica.test.Esercizio15H.recuperaContattoNonEsistente(Esercizio15H.java:
37)
    at rubrica.test.Esercizio15H.eseguiTest(Esercizio15H.java:23)
    at rubrica.test.Esercizio15H.main(Esercizio15H.java:88)

```

Si noti che il test ha dimostrato che:

- aggiungere un contatto esistente provoca la sovrascrittura del vecchio contatto;
- recuperare un contatto inesistente provoca un'eccezione di tipo `FileNotFoundException`.

Entrambi i risultati dei test non ci soddisfano. Nel primo caso, non ci sembra corretto che il vecchio contatto sia sostituito senza neanche che l'utente ne venga a conoscenza. Decidiamo per l'implementazione di una semplice soluzione che prevede di controllare se il contatto esiste già, prima di serializzarlo, e nel caso lanciare un'eccezione personalizzata che potrebbe chiamarsi `ContattoEsistenteException`.

Nel secondo caso sarebbe preferibile creare un'eccezione personalizzata che potremmo chiamare `ContattoInesistenteException`, contenente un messaggio leggibile.

Implementiamo ora la soluzione.

Per prima cosa definiamo le eccezioni in maniera semplicistica, segue l'eccezione

`ContattoInesistenteException`:

```

package rubrica.eccezioni;
import java.io.IOException;

public class ContattoInesistenteException extends IOException {

    private static final long serialVersionUID = 8942402240056525663L;

    public ContattoInesistenteException (String message) {
        super(message);
    }
}

```

E l'altra eccezione `ContattoEsistenteException`:

```

package rubrica.eccezioni;
import java.io.IOException;

public class ContattoEsistenteException extends IOException {

    private static final long serialVersionUID = 8942402240056525662L;

    public ContattoEsistenteException (String message) {
        super(message);
    }
}

```

```

    }
}

```

Notare che abbiamo fatto estendere ad entrambe le classi l'eccezione `IOException`. Modifichiamo la classe `GestoreFile` nel seguente modo:

```

import java.util.*;
import java.io.*;

package rubrica.integrazione;
import java.util.*;
import java.io.*;
import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.util.*;

public class GestoreFile {
    public void inserisci(Contatto contatto) throws ContattoEsistenteException,
        FileNotFoundException, IOException {
        Contatto contattoEsistente = getContatto(
            FileUtils.getFileName(contatto.getNome()));
        if (contattoEsistente != null) {
            throw new ContattoEsistenteException(contatto.getNome()
                + ": contatto già esistente!");
        }
        try (FileOutputStream fos = new FileOutputStream (
            new File(FileUtils.getNomeFile(contatto.getNome())));
            ObjectOutputStream s = new ObjectOutputStream (fos);) {
            s.writeObject (contatto);
        }
    }

    public Contatto recupera(String nome) throws ContattoInesistenteException {
        Contatto contatto = getContatto(FileUtils.getFileName(nome));
        if (contatto == null) {
            throw new ContattoInesistenteException(nome +
                ": contatto non trovato!");
        }
        return contatto;
    }

    private Contatto getContatto(String nome) {
        try (FileInputStream fis = new FileInputStream (new File(nome));
            ObjectInputStream ois = new ObjectInputStream (fis);) {
            Contatto contatto = (Contatto)ois.readObject();
            return contatto;
        } catch (Exception exc) {
            return null;
        }
    }
}

```

Si noti che abbiamo aggiunto il metodo privato `getContatto`, che ritorna il contatto se esiste, oppure `null` se non viene trovato. Tale metodo viene utilizzato sia dal metodo `inserisci` che dal metodo `recupera`, al fine di gestire le eventuali eccezioni. Infine modifichiamo anche la classe di test sostituendo, con la stampa del metodo `getMessage` delle eccezioni, lo statement che stampava lo stack trace dell'eccezione (in grassetto le modifiche):

```

package rubrica.test;
import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.integrazione.*;

```

```
public class Esercizio15H {
    private GestoreFile gestoreFile;
    private Contatto[] contatti;

    Esercizio15H() {
        contatti = getContatti();
        gestoreFile = new GestoreFile();
    }

    private void eseguiTest() {
        System.out.println("TESTIAMO LA CREAZIONE DEI TRE CONTATTI");
        creaContatti();
        System.out.println("RECUPERIAMO I TRE CONTATTI");
        recuperaContatti();
        System.out.println(
            "TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE");
        creaContattoEsistente();
        System.out.println("PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE");
        recuperaContattoNonEsistente();
    }

    public void creaContattoEsistente() {
        creaContatto(contatti[0]);
    }

    public void recuperaContattoNonEsistente() {
        recuperaContatto("Pippo");
    }

    public void creaContatti() {
        for (Contatto contatto : contatti) {
            creaContatto(contatto);
        }
    }

    public void recuperaContatti() {
        for (Contatto contatto : contatti) {
            recuperaContatto(contatto.getNome());
        }
    }

    private void creaContatto(Contatto contatto) {
        try {
            gestoreFile.inserisci(contatto);
            System.out.println("Contatto creato:\n" + contatto);
        }
        catch (Exception exc) {
            System.out.println(exc.getMessage());
        }
    }

    public void recuperaContatto(String nomeContatto) {
        try {
            Contatto contatto = gestoreFile.recupera(nomeContatto);
            System.out.println("Contatto recuperato:\n" + contatto);
        }
        catch (Exception exc) {
            System.out.println(exc.getMessage());
        }
    }
}
```

```
}

private Contatto[] getContatti() {
    Contatto contatto1 = new Contatto("Daniele", "01234560",
        "Via delle chitarre 1" );
    Contatto contatto2 = new Contatto("Giovanni", "0565432190",
        "Via delle scienze 2");
    Contatto contatto3 = new Contatto("Ligeia", "07899921",
        "Via dei segreti 3");
    Contatto[] contatti = {
        contatto1, contatto2, contatto3
    };
    return contatti;
}

public static void main(String args[]) {
    Esercizio15H esercizio15H = new Esercizio15H();
    esercizio15H.eseguiTest();
}
}
```

Cancellare i file .con generati dalla precedente esecuzione per non ottenere i risultati falsati nel test.

L'output di questa classe adesso risulterà essere:

```
TESTIAMO LA CREAZIONE DEI TRE CONTATTI
Contatto creato:
Nome: Daniele
Indirizzo: Via delle chitarre 1
Telefono: 01234560
Contatto creato:
Nome: Giovanni
Indirizzo: Via delle scienze 2
Telefono: 0565432190
Contatto creato:
Nome: Ligeia
Indirizzo: Via dei segreti 3
Telefono: 07899921
RECUPERIAMO I TRE CONTATTI
Contatto recuperato:
Nome: Daniele
Indirizzo: Via delle chitarre 1
Telefono: 01234560
Contatto recuperato:
Nome: Giovanni
Indirizzo: Via delle scienze 2
Telefono: 0565432190
Contatto recuperato:
Nome: Ligeia
Indirizzo: Via dei segreti 3
Telefono: 07899921
TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE
Daniele: contatto già esistente!
PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE
Pippo: contatto non trovato!
```


Soluzione 15.i)

Coerentemente con la linea di sviluppo tenuta sinora, il file `GestoreFile`, potrebbe evolversi nel seguente modo (in grassetto le modifiche apportate):

```
package rubrica.integrazione;

import java.util.*;
import java.io.*;
import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.util.*;

public class GestoreFile {

    public void inserisci(Contatto contatto)
        throws ContattoEsistenteException, FileNotFoundException, IOException {
        Contatto contattoEsistente = getContatto(
            FileUtils.getFileName(contatto.getNome()));
        if (contattoEsistente != null) {
            throw new ContattoEsistenteException(
                contatto.getNome() + ": contatto già esistente!");
        }
        registra(contatto);
    }

    public Contatto recupera(String nome)
        throws ContattoInesistenteException, ContattoEsistenteException {
        Contatto contatto = getContatto(FileUtils.getFileName(nome));
        if (contatto == null) {
            throw new ContattoInesistenteException(
                nome + ": contatto non trovato!");
        }
        return contatto;
    }

    public void modifica(Contatto contatto) throws ContattoInesistenteException,
        ContattoEsistenteException, FileNotFoundException, IOException {
        if (isContattoEsistente(contatto.getNome())) {
            registra(contatto);
        } else {
            throw new ContattoInesistenteException(contatto.getNome() +
                ": contatto non trovato!");
        }
    }

    public void rimuovi(String nome) throws ContattoInesistenteException,
        ContattoEsistenteException, FileNotFoundException, IOException {
        File file = new File(FileUtils.getNomeFile(nome));
        if (!file.delete()) {
            throw new ContattoInesistenteException(
                nome + ": contatto non trovato!");
        }
    }

    private void registra(Contatto contatto)
        throws FileNotFoundException, IOException {
```

```

        try (FileOutputStream fos = new FileOutputStream (
            new File(FileUtils.getNomeFile(contatto.getNome())));
            ObjectOutputStream s = new ObjectOutputStream (fos);) {
            s.writeObject (contatto);
        }
    }

    private boolean isContattoEsistente(String nome) {
        File file = new File(FileUtils.getNomeFile(nome));
        return file.exists();
    }

    private Contatto getContatto(String nome) {
        try (FileInputStream fis = new FileInputStream (new File());
            ObjectInputStream ois = new ObjectInputStream (fis);) {
            Contatto contatto = (Contatto)ois.readObject();
            return contatto;
        } catch (Exception exc) {
            return null;
        }
    }
}

```

Si noti che il metodo privato `isContattoEsistente` controlla solo se un file con il nome del contatto esiste, e questo nel nostro caso dovrebbe essere sufficiente. Invochiamo questo metodo dal metodo `modifica` prima di registrare (e sovrascrivere) il vecchio contatto. Nel caso il file non esistesse, otterremmo il lancio di un'eccezione `ContattoInesistenteException`. Il metodo `rimuovi` invece, sfrutta il metodo `delete` della classe `File`. Questo ritorna `true` se e solo se è riuscito a cancellare il file. La classe di test potremmo modificarla nel seguente modo, per andare a testare gli scenari della modifica e della cancellazione (in grassetto le modifiche apportate):

```

package rubrica.test;

import rubrica.dat.*;
import rubrica.eccezioni.*;
import rubrica.integrazione.*;

public class Esercizio15I {

    private GestoreFile gestoreFile;

    private Contatto[] contatti;

    Esercizio15I() {
        contatti = getContatti();
        gestoreFile = new GestoreFile();
    }

    private void eseguiTest() {
        System.out.println("TESTIAMO LA CREAZIONE DEI TRE CONTATTI");
        creaContatti();
        System.out.println("RECUPERIAMO I TRE CONTATTI");
        recuperaContatti();
        System.out.println(
            "TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE");
        creaContattoEsistente();
        System.out.println("PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE");
        recuperaContattoNonEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO ESISTENTE");
        modificaContattoEsistente();
    }
}

```

```
        System.out.println("RIMUOVIAMO UN CONTATTO ESISTENTE");
        rimuoviContattoEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO NON ESISTENTE");
        modificaContattoNonEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO NON ESISTENTE");
        rimuoviContattoNonEsistente();
    }

    public void creaContatti() {
        for (Contatto contatto : contatti) {
            creaContatto(contatto);
        }
    }

    public void recuperaContatti() {
        for (Contatto contatto : contatti) {
            recuperaContatto(contatto.getNome());
        }
    }

    public void creaContattoEsistente() {
        creaContatto(contatti[0]);
    }

    public void modificaContattoEsistente() {
        try {
            Contatto contatto = new Contatto("Daniele",
                                                "Via dei microfoni 1","07890");
            gestoreFile.modifica(contatto);
            System.out.println("Contatto "+ contatto.getNome() +
                                " modificato!\n" + contatto);
        } catch (Exception exc) {
            System.out.println(exc.getMessage());
        }
    }

    public void rimuoviContattoEsistente() {
        try {
            gestoreFile.rimuovi(contatti[2].getNome());
            System.out.println("Contatto "+ contatti[2].getNome()
                                + " cancellato!");
        } catch (Exception exc) {
            System.out.println(exc.getMessage());
        }
    }

    public void modificaContattoNonEsistente() {
        try {
            Contatto contatto = new Contatto("Pluto",
                                                "Via dei microfoni 1","07890");
            gestoreFile.modifica(contatto);
            System.out.println("Contatto "+ contatto.getNome() +" modificato!");
        } catch (Exception exc) {
            System.out.println(exc.getMessage());
        }
    }

    public void rimuoviContattoNonEsistente() {
        try {
            String nome ="Ligeia";
```

```
        gestoreFile.rimuovi(nome);
        System.out.println("Contatto "+ nome +" cancellato!");
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

public void recuperaContattoNonEsistente() {
    recuperaContatto("Pippo");
}

public void recuperaContatto(String nomeContatto) {
    try {
        gestoreFile.recupera(nomeContatto);
        System.out.println("Contatto "+ nomeContatto +" recuperato!");
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

private void creaContatto(Contatto contatto) {
    try {
        gestoreFile.inserisci(contatto);
        System.out.println("Contatto creato:\n"+ contatto);

    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

private Contatto[] getContatti() {
    Contatto contatto1 = new Contatto("Daniele", "01234560",
        "Via delle chitarre 1" );
    Contatto contatto2 = new Contatto("Giovanni", "0565432190",
        "Via delle scienze 2");
    Contatto contatto3 = new Contatto("Ligeia", "07899921",
        "Via dei segreti 3");
    Contatto[] contatti = {contatto1, contatto2, contatto3};
    return contatti;
}

public static void main(String args[]) {
    Esercizio15I Esercizio15I = new Esercizio15I();
    Esercizio15I.eseguiTest();
}
}
```

L'output risultante sarà il seguente:

```
TESTIAMO LA CREAZIONE DEI TRE CONTATTI
Contatto Daniele creato!
Nome: Daniele
Indirizzo: Via delle chitarre 1
Telefono: 01234560
Contatto Giovanni creato!
Nome: Giovanni
Indirizzo: Via delle scienze 2
Telefono: 0565432190
Contatto Ligeia creato!
Nome: Ligeia
```

```
Indirizzo: Via dei segreti 3
Telefono: 07899921
RECUPERIAMO I TRE CONTATTI
Contatto recuperato!
Nome: Daniele
Indirizzo: Via delle chitarre 1
Telefono: 01234560
Contatto recuperato!
Nome: Giovanni
Indirizzo: Via delle scienze 2
Telefono: 0565432190
Contatto recuperato!
Nome: Ligeia
Indirizzo: Via dei segreti 3
Telefono: 07899921
TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE
Daniele: contatto già esistente!
PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE
Pippo: contatto non trovato!
MODIFICHIAMO UN CONTATTO ESISTENTE
Contatto Daniele modificato!
RIMUOVIAMO UN CONTATTO ESISTENTE
Contatto Ligeia cancellato!
MODIFICHIAMO UN CONTATTO NON ESISTENTE
Pluto: contatto non trovato!
RIMUOVIAMO UN CONTATTO NON ESISTENTE
Ligeia: contatto non trovato!
```

Soluzione 15.j)

Potremmo creare la seguente classe astratta `GestoreSerializzazione`:

```
package rubrica.integrazione;

import java.util.*;
import java.io.*;
import rubrica.dati.*;
import rubrica.eccezioni.*;

public interface GestoreSerializzazione<T extends Dato> {

    void inserisci(T dato) throws IOException;

    T recupera(String id) throws IOException, ClassNotFoundException;

    void modifica(T dato) throws IOException;

    void rimuovi(String id) throws IOException;
}
```

per farla implementare a `GestoreFile`:

```
package rubrica.integrazione;

import java.util.*;
import java.io.*;
import rubrica.dati.*;
```

```
import rubrica.eccezioni.*;
import rubrica.util.*;

public class GestoreFile implements GestoreSerializzazione<Contatto> {

    @Override
    public void inserisci(Contatto contatto)
        throws ContattoEsistenteException, FileNotFoundException, IOException {
        Contatto contattoEsistente = getContatto(
            FileUtils.getFileName(contatto.getNome()));
        if (contattoEsistente != null) {
            throw new ContattoEsistenteException(
                contatto.getNome() + ": contatto già esistente!");
        }
        registra( contatto);
    }

    @Override
    public Contatto recupera(String nome)
        throws ContattoInesistenteException, ContattoEsistenteException {
        Contatto contatto = getContatto(FileUtils.getFileName(nome));
        if (contatto == null) {
            throw new ContattoInesistenteException(
                nome + ": contatto non trovato!");
        }
        return contatto;
    }

    @Override
    public void modifica(Contatto contatto)
        throws ContattoInesistenteException, ContattoEsistenteException,
        FileNotFoundException, IOException {
        if (isContattoEsistente(contatto.getNome())) {
            registra(contatto);
        } else {
            throw new ContattoInesistenteException(
                contatto.getNome() + ": contatto non trovato!");
        }
    }

    @Override
    public void rimuovi(String nome)
        throws ContattoInesistenteException, ContattoEsistenteException,
        FileNotFoundException, IOException {
        File file = new File(FileUtils.getNomeFile(nome));
        if (!file.delete()){
            throw new ContattoInesistenteException(
                nome + ": contatto non trovato!");
        }
    }

    private void registra(Contatto contatto)
        throws FileNotFoundException, IOException {
        try (FileOutputStream fos = new FileOutputStream (
            new File(FileUtils.getNomeFile(contatto.getNome())));
            ObjectOutputStream s = new ObjectOutputStream (fos);) {
            s.writeObject (contatto);
        }
    }
}
```

```

    private boolean isContattoEsistente(String nome) {
        File file = new File(FileUtils.getNomeFile(nome));
        return file.exists();
    }

    private Contatto getContatto(String nome) {
        try (FileInputStream fis = new FileInputStream(new File(nome));
            ObjectInputStream ois = new ObjectInputStream(fis);) {
            Contatto contatto = (Contatto)ois.readObject();
            return contatto;
        } catch (Exception exc) {
            return null;
        }
    }
}

```

Infine ecco la classe di test:

```

package rubrica.test;

import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.integrazione.*;

public class Esercizio15J {

    private GestoreSerializzazione<Contatto> gestoreFile;

    private Contatto[] contatti;

    Esercizio15J() {
        contatti = getContatti();
        gestoreFile = new GestoreFile();
    }

    private void eseguiTest() {
        System.out.println("TESTIAMO LA CREAZIONE DEI TRE CONTATTI");
        creaContatti();
        System.out.println("RECUPERIAMO I TRE CONTATTI");
        recuperaContatti();
        System.out.println(
            "TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE");
        creaContattoEsistente();
        System.out.println("PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE");
        recuperaContattoNonEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO ESISTENTE");
        modificaContattoEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO ESISTENTE");
        rimuoviContattoEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO NON ESISTENTE");
        modificaContattoNonEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO NON ESISTENTE");
        rimuoviContattoNonEsistente();
    }

    public void creaContatti() {
        for (Contatto contatto : contatti) {
            creaContatto(contatto);
        }
    }

    public void recuperaContatti() {
        for (Contatto contatto : contatti) {

```

```
        recuperaContatto(contatto.getNome());
    }
}

public void creaContattoEsistente() {
    creaContatto(contatti[0]);
}

public void modificaContattoEsistente() {
    try {
        Contatto contatto = new Contatto("Daniele",
                                           "Via dei microfoni 1","07890");
        gestoreFile.modifica(contatto);
        System.out.println("Contatto "+ contatto.getNome() +
                           " modificato!\n" + contatto);
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

public void rimuoviContattoEsistente() {
    try {
        gestoreFile.rimuovi(contatti[2].getNome());
        System.out.println("Contatto "+ contatti[2].getNome()
                           + " cancellato!");
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

public void modificaContattoNonEsistente() {
    try {
        Contatto contatto = new Contatto("Pluto",
                                           "Via dei microfoni 1","07890");
        gestoreFile.modifica(contatto);
        System.out.println("Contatto "+ contatto.getNome() +" modificato!");
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

public void rimuoviContattoNonEsistente() {
    try {
        String nome ="Ligeia";
        gestoreFile.rimuovi(nome);
        System.out.println("Contatto "+ nome +" cancellato!");
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

public void recuperaContattoNonEsistente() {
    recuperaContatto("Pippo");
}

public void recuperaContatto(String nomeContatto) {
    try {
        gestoreFile.recupera(nomeContatto);
        System.out.println("Contatto "+ nomeContatto +" recuperato!");
    } catch (Exception exc) {
```



```

        System.out.println(exc.getMessage());
    }
}

private void creaContatto(Contacto contatto) {
    try {
        gestoreFile.inserisci(contatto);
        System.out.println("Contatto creato:\n"+ contatto);

    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

private Contacto[] getContatti() {
    Contacto contatto1 = new Contacto("Daniele", "01234560",
        "Via delle chitarre 1" );
    Contacto contatto2 = new Contacto("Giovanni", "0565432190",
        "Via delle scienze 2");
    Contacto contatto3 = new Contacto("Ligeia", "07899921",
        "Via dei segreti 3");
    Contacto[] contatti = {contatto1, contatto2, contatto3};
    return contatti;
}

public static void main(String args[]) {
    Esercizio15J esercizio15J = new Esercizio15J();
    esercizio15J.eseguiTest();
}
}

```

Soluzione 15.k)

Una soluzione potrebbe essere quella di usare delle espressioni lambda per evitare di riscrivere i vari metodi gestendo sempre allo stesso modo le eccezioni. In particolare possiamo creare due tipologie di interfacce funzionali. Una che definisce un metodo che restituisce void:

```

package rubrica.util;

@FunctionalInterface
public interface Executor {

    void esegui() throws Exception;
}

```

Ed un'altra che restituisce un tipo generico:

```

package rubrica.util;

@FunctionalInterface
public interface Retriever<O> {

    O esegui() throws Exception;
}

```

Non abbiamo utilizzato un'interfaccia funzionale `java.util.function.Consumer` al posto di `Executor`, e l'interfaccia

funzionale `java.util.function.Supplier` al posto di `Retriever`, perché ci occorreva che i metodi SAM dichiarassero la clausola `throws` ad `Exception`.

Possiamo sfruttare queste interfacce funzionali nella classe `Esercizio15K` nel seguente modo (le parti modificate come al solito sono in grassetto):

```
package rubrica.test;

import rubrica.dat.*;
import rubrica.eccezioni.*;
import rubrica.integrazione.*;
import rubrica.util.*;

public class Esercizio15K {

    private GestoreSerializzazione<Contatto> gestoreFile;

    private Contatto[] contatti;

    Esercizio15K() {
        contatti = getContatti();
        gestoreFile = new GestoreFile();
    }

    private void eseguiTest() {
        System.out.println("TESTIAMO LA CREAZIONE DEI TRE CONTATTI");
        creaContatti();
        System.out.println("RECUPERIAMO I TRE CONTATTI");
        recuperaContatti();
        System.out.println(
            "TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE");
        creaContattoEsistente();
        System.out.println("PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE");
        recuperaContattoNonEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO ESISTENTE");
        modificaContattoEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO ESISTENTE");
        rimuoviContattoEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO NON ESISTENTE");
        modificaContattoNonEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO NON ESISTENTE");
        rimuoviContattoNonEsistente();
    }

    public void creaContatti() {
        for (Contatto contatto : contatti) {
            creaContatto(contatto);
        }
    }

    public void recuperaContatti() {
        for (Contatto contatto : contatti) {
            recuperaContatto(contatto.getNome());
        }
    }

    public void creaContattoEsistente() {
        creaContatto(contatti[0]);
    }
}
```

```
public void modificaContattoEsistente() {
    Contatto contatto = new Contatto("Daniele", "Via dei microfoni 1",
                                     "07890");
    String messaggio = "Contatto " + contatto.getNome() + " modificato!\n" +
                       contatto;
    esegui()->gestoreFile.modifica(contatto), messaggio);
}

public void rimuoviContattoEsistente() {
    String nome = contatti[2].getNome();
    String messaggio = "Contatto " + nome + " cancellato!";
    esegui()->gestoreFile.rimuovi(nome), messaggio);
}

public void modificaContattoNonEsistente() {
    Contatto contatto = new Contatto("Pluto", "Via dei microfoni 1", "07890");
    String messaggio = "Contatto " + contatto.getNome() + " modificato!";
    esegui()->gestoreFile.modifica(contatto), messaggio);
}

public void rimuoviContattoNonEsistente() {
    String nome = "Ligeia";
    String messaggio = "Contatto " + nome + " cancellato!";
    esegui()->gestoreFile.rimuovi(nome), messaggio);
}

public void recuperaContattoNonEsistente() {
    recuperaContatto("Pippo");
}

public void recuperaContatto(String nomeContatto) {
    esegui()->gestoreFile.recupera(nomeContatto);
}

private void creaContatto(Contatto contatto) {
    String messaggio = "Contatto " + contatto.getNome() + " creato!\n" +
                       contatto;
    esegui()->gestoreFile.inserisci(contatto), messaggio);
}

public <O> O esegui(Retriever<O> retriever) {
    O output = null;
    try {
        output = retriever.esegui();
        System.out.println("Contatto recuperato!\n" + output);
    }
    catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
    return output;
}

public void esegui(Executor executor, String messaggio) {
    try {
        executor.esegui();
        System.out.println(messaggio);
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}
```

```

    }
}

private Contatto[] getContatti() {
    Contatto contatto1 =
        new Contatto("Daniele", "Via delle chitarre 1", "01234560");
    Contatto contatto2 =
        new Contatto("Giovanni", "Via delle scienze 2", "0565432190");
    Contatto contatto3 =
        new Contatto("Ligeia", "Via dei segreti 3", "07899921");
    Contatto[] contatti = {contatto1, contatto2, contatto3};
    return contatti;
}

public static void main(String args[]) {
    Esercizio15K esercizio15K = new Esercizio15K();
    esercizio15K.eseguiTest();
}
}

```

Si noti che abbiamo dovuto creare i due metodi `esegui` di cui uno prende in input un `Retriever` e l'altro un `Executor`. Questi metodi inseriscono la chiamata al metodo `esegui` di `Retriever` o `Executor` all'interno della gestione delle eccezioni che prima veniva replicata in ogni metodo. In questo modo, passando un'espressione lambda contenente il codice da eseguire a questi due nuovi metodi, abbiamo potuto evitare le duplicazioni presenti nella classe `Esercizio15J`.

Segue l'output:

```

TESTIAMO LA CREAZIONE DEI TRE CONTATTI
Contatto Daniele creato!
Nome: Daniele
Indirizzo: Via delle chitarre 1
Telefono: 01234560
Contatto Giovanni creato!
Nome: Giovanni
Indirizzo: Via delle scienze 2
Telefono: 0565432190
Contatto Ligeia creato!
Nome: Ligeia
Indirizzo: Via dei segreti 3
Telefono: 07899921
RECUPERIAMO I TRE CONTATTI
Contatto recuperato!
Nome: Daniele
Indirizzo: Via delle chitarre 1
Telefono: 01234560
Contatto recuperato!
Nome: Giovanni
Indirizzo: Via delle scienze 2
Telefono: 0565432190
Contatto recuperato!
Nome: Ligeia
Indirizzo: Via dei segreti 3
Telefono: 07899921
TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE
Daniele: contatto già esistente!
PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE
Pippo: contatto non trovato!

```

```
MODIFICHIAMO UN CONTATTO ESISTENTE
Contatto Daniele modificato!
Nome: Daniele
Indirizzo: 07890
Telefono: Via dei microfoni 1
RIMUOVIAMO UN CONTATTO ESISTENTE
Contatto Ligeia cancellato!
MODIFICHIAMO UN CONTATTO NON ESISTENTE
Pluto: contatto non trovato!
RIMUOVIAMO UN CONTATTO NON ESISTENTE
Ligeia: contatto non trovato!
```

Soluzione 15.I)

La soluzione potrebbe essere la seguente:

```
package rubrica.integrazione;

import java.util.*;
import java.io.*;
import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.util.*;
import java.nio.file.*;

public class GestoreFileNIO2 implements GestoreSerializzazione<Contatto> {

    @Override
    public void inserisci(Contatto contatto)
        throws ContattoEsistenteException, FileNotFoundException, IOException {
        Path path = Paths.get(FileUtils.getNomeFile(contatto.getNome()));
        if (Files.exists(path)) {
            throw new ContattoEsistenteException(
                contatto.getNome() + ": contatto già esistente!");
        }
        registra(contatto);
    }

    @Override
    public Contatto recupera(String nome)
        throws ContattoInesistenteException, ContattoEsistenteException {
        Contatto contatto = getContatto(FileUtils.getFileName(nome));
        if (contatto == null) {
            throw new ContattoInesistenteException(
                nome + ": contatto non trovato!");
        }
        return contatto;
    }

    @Override
    public void modifica(Contatto contatto)
        throws ContattoInesistenteException, ContattoEsistenteException,
        FileNotFoundException, IOException {
        if (isContattoEsistente(contatto.getNome())) {
            registra(contatto);
        } else {
            throw new ContattoInesistenteException(contatto.getNome());
        }
    }
}
```

```

        + ": contatto non trovato!");
    }
}

@Override
public void rimuovi(String nome) throws ContattoInesistenteException,
    ContattoEsistenteException, FileNotFoundException, IOException {
    Path path = Paths.get(FileUtils.getNomeFile(nome));
    if (Files.exists(path)) {
        Files.delete(path);
    } else {
        throw new ContattoInesistenteException(nome
            + ": contatto non trovato!");
    }
}

private void registra(Contatto contatto)
    throws FileNotFoundException, IOException {
    Path path = Paths.get(FileUtils.getNomeFile(contatto.getNome()));
    Files.write(path, getBytesDaOggetto(contatto));
}

private byte[] getBytesDaOggetto(Object object) throws IOException {
    try (ByteArrayOutputStream bos = new ByteArrayOutputStream();
        ObjectOutputStream out = new ObjectOutputStream(bos)) {
        out.writeObject(object);
        return bos.toByteArray();
    }
}

private Object getOggettoDaByte(byte[] bytes)
    throws IOException, ClassNotFoundException {
    try (ByteArrayInputStream bis = new ByteArrayInputStream(bytes);
        ObjectInput in = new ObjectInputStream(bis)) {
        return in.readObject();
    }
}

private boolean isContattoEsistente(String nome) {
    Path path = Paths.get(FileUtils.getNomeFile(nome));
    return Files.exists(path);
}

private Contatto getContatto(String nome) {
    Path path = Paths.get(nome);
    byte[] bytes = null;
    Contatto contatto = null;
    try {
        bytes = Files.readAllBytes(path);
        contatto = (Contatto) getOggettoDaByte(bytes);
    } catch (Exception exc) {
        return null;
    }
    return contatto;
}
}

```

La classe per testare `Esercizio15N`, si differenzia dalla classe `Esercizio15M` solo dall'applicazione del principio DIP che istanzia la nuova classe `GestorFileNIO2`, usando un reference di tipo `GestoreSerializzazione`. Tutto il resto non cambia.

```
package rubrica.test;

import rubrica.dat.*;
import rubrica.eccezioni.*;
import rubrica.integrazione.*;
import rubrica.util.*;

public class Esercizio15L {
    private GestoreSerializzazione<Contatto> gestoreFile;

    private Contatto[] contatti;

    Esercizio15L() {
        contatti = getContatti();
        gestoreFile = new GestoreFileNIO2();
    }

    private void eseguiTest() {
        System.out.println("TESTIAMO LA CREAZIONE DEI TRE CONTATTI");
        creaContatti();
        System.out.println("RECUPERIAMO I TRE CONTATTI");
        recuperaContatti();
        System.out.println(
            "TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE");
        creaContattoEsistente();
        System.out.println("PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE");
        recuperaContattoNonEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO ESISTENTE");
        modificaContattoEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO ESISTENTE");
        rimuoviContattoEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO NON ESISTENTE");
        modificaContattoNonEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO NON ESISTENTE");
        rimuoviContattoNonEsistente();
    }

    public void creaContatti() {
        for (Contatto contatto : contatti) {
            creaContatto(contatto);
        }
    }

    public void recuperaContatti() {
        for (Contatto contatto : contatti) {
            recuperaContatto(contatto.getNome());
        }
    }

    public void creaContattoEsistente() {
        creaContatto(contatti[0]);
    }

    public void modificaContattoEsistente() {
        Contatto contatto = new Contatto("Daniele", "Via dei microfoni 1",
                                           "07890");
        String messaggio = "Contatto " + contatto.getNome() + " modificato!\n" +
                           contatto;
        esegui(() -> gestoreFile.modifica(contatto), messaggio);
    }
}
```

```
public void rimuoviContattoEsistente() {
    String nome = contatti[2].getNome();
    String messaggio = "Contatto " + nome + " cancellato!";
    esegui()->gestoreFile.rimuovi(nome), messaggio);
}

public void modificaContattoNonEsistente() {
    Contatto contatto = new Contatto("Pluto", "Via dei microfoni 1", "07890");
    String messaggio = "Contatto " + contatto.getNome() + " modificato!";
    esegui()->gestoreFile.modifica(contatto), messaggio);
}

public void rimuoviContattoNonEsistente() {
    String nome = "Ligeia";
    String messaggio = "Contatto " + nome + " cancellato!";
    esegui()->gestoreFile.rimuovi(nome), messaggio);
}

public void recuperaContattoNonEsistente() {
    recuperaContatto("Pippo");
}

public void recuperaContatto(String nomeContatto) {
    esegui()->gestoreFile.recupera(nomeContatto));
}

private void creaContatto(Contatto contatto) {
    String messaggio = "Contatto " + contatto.getNome() + " creato!\n" +
                                                                contatto;
    esegui()->gestoreFile.inserisci(contatto), messaggio);
}

public <O> O esegui(Retriever<O> retriever) {
    O output = null;
    try {
        output = retriever.esegui();
        System.out.println("Contatto recuperato!\n" + output);
    }
    catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
    return output;
}

public void esegui(Executor executor, String messaggio) {
    try {
        executor.esegui();
        System.out.println(messaggio);
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

private Contatto[] getContatti() {
    Contatto contatto1 =
        new Contatto("Daniele", "Via delle chitarre 1", "01234560");
    Contatto contatto2 =
```



```

        new Contatto("Giovanni","Via delle scienze 2","0565432190");
        Contatto contatto3 =
            new Contatto("Ligeia","Via dei segreti 3","07899921");
        Contatto[] contatti = {contatto1, contatto2, contatto3};
        return contatti;
    }

    public static void main(String args[]) {
        Esercizio15L esercizio15L = new Esercizio15L();
        esercizio15L.eseguiTest();
    }
}

```

Possiamo ritornare alla vecchia versione semplicemente sostituendo all'istanza di `GestioneFileNIO2` l'istanza di `GestioneFile`.

Soluzione 15.m)

Abbiamo implementato la classe `GestoreSerializzazioneFactory`, sfruttando anche la reflection. Tuttavia anche una soluzione che faceva uso di una espressione `switch` o qualsiasi costrutto condizionale sarebbe stata valida:

```

package rubrica.integrazione;

import rubrica.dati.Contatto;
import java.util.Properties;
import java.io.*;

public class GestoreSerializzazioneFactory {
    private static final String PACKAGE = "rubrica.integrazione.";
    private static final String SERIALIZATION_METHOD = "rubrica.ser";
    private static Properties properties;
    static {
        properties = new Properties();
        try {
            loadProperties();
        } catch (IOException e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
    public static GestoreSerializzazione<Contatto> getGestoreSerializzazione()
        throws InstantiationException, ClassNotFoundException,
        IllegalAccessException {
        String className = properties.getProperty(SERIALIZATION_METHOD);
        System.out.println("Sto caricando la classe " + className);
        Class<?> classObject = Class.forName(PACKAGE + className);
        return (GestoreSerializzazione<Contatto>)classObject.newInstance();
    }

    private static void loadProperties() throws IOException {
        try (FileInputStream inputStream =
            new FileInputStream("config.properties")) {
            properties.load(inputStream);
        }
    }
}

```

Ovviamente abbiamo supposto che il valore della property coincida con il nome della classe da caricare. Per esempio potremmo salvare il file `config.properties` con il seguente contenuto:

```
rubrica.ser=GestoreFile
```

Infine possiamo testare la nostra implementazione con la seguente classe `Esercizio15M` (come al solito in grassetto ci sono le modifiche rispetto alla versione `Esercizio15L`):

```
package rubrica.test;

import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.integrazione.*;
import rubrica.util.*;

public class Esercizio15M {

    private GestoreSerializzazione<Contatto> gestoreFile;

    private Contatto[] contatti;

    Esercizio15M() {
        contatti = getContatti();
        try {
            gestoreFile =
                GestoreSerializzazioneFactory.getGestoreSerializzazione();
        } catch (Exception exc) {
            exc.printStackTrace();
            System.exit(1);
        }
    }

    private void eseguiTest() {
        System.out.println("TESTIAMO LA CREAZIONE DEI TRE CONTATTI");
        creaContatti();
        System.out.println("RECUPERIAMO I TRE CONTATTI");
        recuperaContatti();
        System.out.println(
            "TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE");
        creaContattoEsistente();
        System.out.println("PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE");
        recuperaContattoNonEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO ESISTENTE");
        modificaContattoEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO ESISTENTE");
        rimuoviContattoEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO NON ESISTENTE");
        modificaContattoNonEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO NON ESISTENTE");
        rimuoviContattoNonEsistente();
    }

    public void creaContatti() {
        for (Contatto contatto : contatti) {
            creaContatto(contatto);
        }
    }

    public void recuperaContatti() {
        for (Contatto contatto : contatti) {
            recuperaContatto(contatto.getNome());
        }
    }
}
```

```
}

public void creaContattoEsistente() {
    creaContatto(contatti[0]);
}

public void modificaContattoEsistente() {
    Contatto contatto = new Contatto("Daniele", "Via dei microfoni 1",
                                     "07890");
    String messaggio = "Contatto " + contatto.getNome() + " modificato!\n" +
                       contatto;
    esegui()->gestoreFile.modifica(contatto, messaggio);
}

public void rimuoviContattoEsistente() {
    String nome = contatti[2].getNome();
    String messaggio = "Contatto " + nome + " cancellato!";
    esegui()->gestoreFile.rimuovi(nome, messaggio);
}

public void modificaContattoNonEsistente() {
    Contatto contatto = new Contatto("Pluto", "Via dei microfoni 1", "07890");
    String messaggio = "Contatto " + contatto.getNome() + " modificato!";
    esegui()->gestoreFile.modifica(contatto, messaggio);
}

public void rimuoviContattoNonEsistente() {
    String nome = "Ligeia";
    String messaggio = "Contatto " + nome + " cancellato!";
    esegui()->gestoreFile.rimuovi(nome, messaggio);
}

public void recuperaContattoNonEsistente() {
    recuperaContatto("Pippo");
}

public void recuperaContatto(String nomeContatto) {
    esegui()->gestoreFile.recupera(nomeContatto);
}

private void creaContatto(Contatto contatto) {
    String messaggio = "Contatto " + contatto.getNome() + " creato!\n" +
                       contatto;
    esegui()->gestoreFile.inserisci(contatto, messaggio);
}

public <O> O esegui(Retriever<O> retriever) {
    O output = null;
    try {
        output = retriever.esegui();
        System.out.println("Contatto recuperato!\n" + output);
    }
    catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
    return output;
}
```

```

    public void esegui(Executor executor, String messaggio) {
        try {
            executor.esegui();
            System.out.println(messaggio);
        } catch (Exception exc) {
            System.out.println(exc.getMessage());
        }
    }

    private Contatto[] getContatti() {
        Contatto contatto1 =
            new Contatto("Daniele", "Via delle chitarre 1", "01234560");
        Contatto contatto2 =
            new Contatto("Giovanni", "Via delle scienze 2", "0565432190");
        Contatto contatto3 =
            new Contatto("Ligeia", "Via dei segreti 3", "07899921");
        Contatto[] contatti = {contatto1, contatto2, contatto3};
        return contatti;
    }

    public static void main(String args[]) {
        Esercizio15M esercizio15M = new Esercizio15M();
        esercizio15M.eseguiTest();
    }
}

```

Soluzione 15.n)

Abbiamo deciso di creare l'eccezione `ComandoNonValidoException` estendendo `IOException` nel seguente modo:

```

package rubrica.eccezioni;

import java.io.IOException;

public class ComandoNonValidoException extends IOException {

    private static final long serialVersionUID = 6742493040156525789L;

    public ComandoNonValidoException(String comando) {
        super("Comando non valido:" + comando);
    }
}

```

Ecco invece l'implementazione della classe `RubricaCLI`:

```

package rubrica.presentation;

import rubrica.dati.*;
import rubrica.eccezioni.*;
import java.util.*;
import java.io.*;

public class RubricaCLI {
    private Scanner scanner;
    private static final String INSERT = "/i";
    private static final String SHOW_CONTACT = "/v";

    public RubricaCLI() {
        scanner = new Scanner(System.in);
    }
}

```

```

    }

    public void visualizzaContatti(List<Contatto> contatti) {
        try {
            int numeroContatti = contatti.size();
            if (numeroContatti == 0) {
                System.out.println("Nessun contatto trovato");
            } else {
                System.out.println("Lista dei contatti in Rubrica:");
                for (Contatto contatto : contatti) {
                    System.out.println(contatto.getNome());
                }
                System.out.printf("Scrivi '%s' seguito dal nome di un contatto,"
                    + " per visualizzarne i dettagli\n", SHOW_CONTACT);
                System.out.printf("Scrivi '%s' per inserire un nuovo" +
                    " contatto\n", INSERT);
                String comando = scanner.nextLine();
                if (comando.equals(INSERT)) {
                    System.out.println("Chiamato metodo inserisci contatto");
                } else if (isComandoValidoPerVisualizzaContatto(contatti,
                                                                    comando)) {
                    System.out.printf("Chiamato metodo visualizza contatto"
                        + " per %s\n", estraiNomeContatto(comando));
                } else {
                    throw new ComandoNonValidoException(comando);
                }
            }
        } catch (Exception exc) {
            System.out.println(exc.getMessage());
        }
    }

    private boolean isComandoValidoPerVisualizzaContatto(
        List<Contatto> contatti, String comando)
        throws ContattoInesistenteException, ComandoNonValidoException {
        boolean result = false;
        if (comando.startsWith(SHOW_CONTACT)) {
            for (Contatto contatto : contatti) {
                if (comando.endsWith(contatto.getNome())) {
                    return true;
                }
            }
        } else {
            throw new ComandoNonValidoException(comando);
        }
        throw new ContattoInesistenteException(String.format(
            "Contatto %s non trovato!", estraiNomeContatto(comando)));
    }

    public String estraiNomeContatto(String comando) {
        return comando.substring(comando.indexOf(" ") + 1, comando.length());
    }
}

```

Infine, segue la classe Esercizio15N:

```

package rubrica.test;

import rubrica.presentation.*;
import rubrica.dati.*;
import rubrica.eccezioni.*;
import java.util.*;

```

```

public class Esercizio15N {

    private void eseguiTest() {
        RubricaCLI cli = new RubricaCLI();
        cli.visualizzaContatti(getContatti());
    }

    private List<Contatto> getContatti() {
        List<Contatto> contatti = new ArrayList<>();
        Contatto contatto1 = new Contatto("Daniele", "01234560",
            "Via delle chitarre 1");
        Contatto contatto2 = new Contatto("Giovanni", "0565432190",
            "Via delle scienze 2");
        Contatto contatto3 = new Contatto("Ligeia", "07899921",
            "Via dei segreti 3");
        contatti.add(contatto1);
        contatti.add(contatto2);
        contatti.add(contatto3);
        return contatti;
    }

    public static void main(String args[]) {
        Esercizio15N esercizio15N = new Esercizio15N();
        esercizio15N.eseguiTest();
    }
}

```

Soluzione 15.o)

Per prima cosa iniziamo a modificare il Model, aggiungendo all'interfaccia

GestoreSerializzazione il nuovo metodo `getContatti`:

```

package rubrica.integrazione;

import java.util.*;
import java.io.*;
import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.util.*;

public interface GestoreSerializzazione<T extends Dato> {

    void inserisci(T dato) throws IOException;

    T recupera(String id) throws IOException, ClassNotFoundException;

    void modifica(T dato) throws IOException;

    void rimuovi(String id) throws IOException;

    List<Contatto> getContatti() throws IOException;
}

```

Poi dobbiamo implementare questo metodo nelle sue due implementazioni `GestoreFile` e `GestoreFileNIO2`. Segue la nuova versione della classe `GestoreFile`:

```

package rubrica.integrazione;

import java.util.*;

```

```
import java.io.*;
import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.util.*;

public class GestoreFile implements GestoreSerializzazione<Contatto> {
    @Override
    public void inserisci(Contatto contatto) throws ContattoEsistenteException,
                                                FileNotFoundException, IOException {
        Contatto contattoEsistente = getContatto(
            FileUtils.getNomeFile(contatto.getNome()));
        if (contattoEsistente != null) {
            throw new ContattoEsistenteException(contatto.getNome() +
                ": contatto già esistente!");
        }
        registra( contatto);
    }

    @Override
    public Contatto recupera(String nome) throws ContattoInesistenteException,
                                                ContattoEsistenteException {
        Contatto contatto = getContatto(FileUtils.getNomeFile(nome));
        if (contatto == null) {
            throw new ContattoInesistenteException(nome +
                ": contatto non trovato!");
        }
        return contatto;
    }

    @Override
    public void modifica(Contatto contatto) throws ContattoInesistenteException,
                                                ContattoEsistenteException, FileNotFoundException, IOException {
        if (isContattoEsistente(contatto.getNome())) {
            registra(contatto);
        } else {
            throw new ContattoInesistenteException(contatto.getNome() +
                ": contatto non trovato!");
        }
    }

    @Override
    public void rimuovi(String nome) throws ContattoInesistenteException,
                                                ContattoEsistenteException, FileNotFoundException, IOException {
        File file = new File(FileUtils.getNomeFile(nome));
        if (file.delete()) {
            System.out.println("Contatto " + nome + " cancellato!");
        } else {
            throw new ContattoInesistenteException(nome +
                ": contatto non trovato!");
        }
    }

    private void registra(Contatto contatto) throws FileNotFoundException,
                                                IOException {
        try (FileOutputStream fos =
            new FileOutputStream (new File(
                FileUtils.getNomeFile(contatto.getNome())));
            ObjectOutputStream s = new ObjectOutputStream (fos);) {
            s.writeObject (contatto);
        }
    }
}
```

```

        System.out.println("Contatto registrato:\n"+ contatto);
    }
}

private boolean isContattoEsistente(String nome) {
    File file = new File(FileUtils.getNomeFile(nome));
    return file.exists();
}

private Contatto getContatto(String nome) {
    try (FileInputStream fis = new FileInputStream (new File(nome));
        ObjectInputStream ois = new ObjectInputStream (fis);) {
        Contatto contatto = (Contatto)ois.readObject();
        System.out.println("Contatto recuperato:\n"+ contatto);
        return contatto;
    } catch (Exception exc) {
        return null;
    }
}

@Override
public List<Contatto> getContatti() throws IOException {
    List<Contatto> contatti = new ArrayList<Contatto>();
    File directory = new File(".");
    for ( File file : directory.listFiles()) {
        if (file.isFile()) {
            String fileName = file.getName();
            if (fileName.endsWith(".con")) {
                contatti.add(getContatto(fileName));
            }
        }
    }
    return contatti;
}
}

```

Mentre la seguente è la nuova versione della classe `GestoreFileNIO2`:

```

package rubrica.integrazione;

import java.util.*;
import java.io.*;
import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.util.*;
import java.util.stream.*;
import java.nio.file.*;

public class GestoreFileNIO2 implements GestoreSerializzazione<Contatto> {
    @Override
    public void inserisci(Contatto contatto) throws ContattoEsistenteException,
        FileNotFoundException, IOException {
        Path path = Paths.get(FileUtils.getNomeFile(contatto.getNome()));
        if (Files.exists(path)) {
            throw new ContattoEsistenteException(contatto.getNome() +
                ": contatto già esistente!");
        }
        registra(contatto);
    }

    @Override
    public Contatto recupera(String nome) throws ContattoInesistenteException,

```



```

        ContattoEsistenteException {
    Contatto contatto = getContatto(FileUtils.getNomeFile(nome));
    if (contatto == null) {
        throw new ContattoInesistenteException(nome +
            ": contatto non trovato!");
    }
    return contatto;
}

@Override
public void modifica(Contatto contatto) throws ContattoInesistenteException,
    ContattoEsistenteException, FileNotFoundException, IOException {
    if (isContattoEsistente(contatto.getNome())) {
        registra(contatto);
    } else {
        throw new ContattoInesistenteException(contatto.getNome() +
            ": contatto non trovato!");
    }
}

@Override
public void rimuovi(String nome) throws ContattoInesistenteException,
    ContattoEsistenteException, FileNotFoundException, IOException {
    Path path = Paths.get(FileUtils.getNomeFile(nome));
    if (Files.exists(path)) {
        Files.delete(path);
        System.out.println("Contatto "+ nome +" cancellato!");
    } else {
        throw new ContattoInesistenteException(nome +
            ": contatto non trovato!");
    }
}

private void registra(Contatto contatto) throws FileNotFoundException,
    IOException {
    Path path = Paths.get(FileUtils.getNomeFile(contatto.getNome()));
    Files.write(path, getBytesDaOggetto(contatto));
    System.out.println("Contatto registrato:\n"+ contatto);
}

private byte[] getBytesDaOggetto(Object object) throws IOException {
    try (ByteArrayOutputStream bos = new ByteArrayOutputStream();
        ObjectOutput out = new ObjectOutput(bos)) {
        out.writeObject(object);
        return bos.toByteArray();
    }
}

private Object getOggettoDaByte(byte[] bytes) throws IOException,
    ClassNotFoundException {
    try (ByteArrayInputStream bis = new ByteArrayInputStream(bytes);
        ObjectInput in = new ObjectInput(bis)) {
        return in.readObject();
    }
}

private boolean isContattoEsistente(String nome) {
    Path path = Paths.get(FileUtils.getNomeFile(nome));
    return Files.exists(path);
}

```

```

    }

    private Contatto getContatto(String nome) {
        Path path = Paths.get(nome);
        byte[] bytes = null;
        Contatto contatto = null;
        try {
            bytes = Files.readAllBytes(path);
            contatto = (Contatto) getOggettoDaByte(bytes);
            System.out.println("Contatto recuperato:\n" + contatto);
        }
        catch (Exception exc) {
            return null;
        }
        return contatto;
    }

    @Override
    public List<Contatto> getContatti() throws IOException {
        List<Contatto> contatti = new ArrayList<>();
        try (Stream<Path> walk = Files.walk(Paths.get("."))) {
            contatti = walk.map(p -> p.toString()).
                filter(f -> f.endsWith(".con")).map(f -> getContatto(f)).
                collect(Collectors.toList());
        }
        return contatti;
    }
}

```

Segue la nuova versione della classe `RubricaCLI`, che contiene la classe innestata

`RubricaController`:

```

package rubrica.presentation;

import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.integrazione.*;
import java.util.*;
import java.io.*;

public class RubricaCLI {
    private Scanner scanner;
    private RubricaController controller;
    private static final String INSERT = "/i";
    private static final String SHOW_CONTACT = "/v";

    public RubricaCLI() {
        scanner = new Scanner(System.in);
        controller = new RubricaController();
    }

    public void start() {
        try {
            controller.gestisciVisualizzaContatti();
        } catch (IOException exc) {
            System.out.println(exc.getMessage());
        }
    }

    public void visualizzaContatti(List<Contatto> contatti) {
        try {

```

```

        int numeroContatti = contatti.size();
        if (numeroContatti == 0) {
            System.out.println("Nessun contatto trovato");
        } else {
            System.out.println("Lista dei contatti in Rubrica:");
            for (Contatto contatto : contatti) {
                System.out.println(contatto.getNome());
            }
            System.out.printf("Scrivi '%s' seguito dal nome di un contatto,"
                + " per visualizzarne i dettagli\n", SHOW_CONTACT);
            System.out.printf("Scrivi '%s' per inserire un nuovo " +
                "contatto\n", INSERT);
            String comando = scanner.nextLine();
            if (comando.equals(INSERT)) {
                System.out.println("Chiamato metodo inserisci contatto");
            } else if (isComandoValidoPerVisualizzaContatto(contatti,
                comando)) {
                System.out.printf("Chiamato metodo visualizza contatto per"
                    + " %s\n", estraiNomeContatto(comando));
            } else {
                throw new ComandoNonValidoException(comando);
            }
        }
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

private boolean isComandoValidoPerVisualizzaContatto(List<Contatto>
    contatti, String comando) throws ContattoInesistenteException,
    ComandoNonValidoException {
    boolean result = false;
    if (comando.startsWith(SHOW_CONTACT)) {
        for (Contatto contatto : contatti) {
            if (comando.endsWith(contatto.getNome())) {
                return true;
            }
        }
    } else {
        throw new ComandoNonValidoException(comando);
    }
    throw new ContattoInesistenteException(String.format(
        "Contatto %s non trovato!", estraiNomeContatto(comando)));
}

public String estraiNomeContatto(String comando) {
    return comando.substring(comando.indexOf(" ")+1, comando.length());
}

class RubricaController {
    GestoreSerializzazione gestoreFile;
    public RubricaController () {
        try {
            gestoreFile =
                GestoreSerializzazioneFactory.getGestoreSerializzazione();
        } catch (Exception exc) {
            exc.printStackTrace();
            System.exit(1);
        }
    }
}

```

```

        public void gestisciVisualizzaContatti() throws IOException {
            List<Contatto> contatti = gestoreFile.getContatti();
            visualizzaContatti(contatti);
        }
    }
}

```

Soluzione 15.p)

Per prima cosa, abbiamo definito l'eccezione `NomeVuotoException` richiesta in questo modo:

```

package rubrica.eccezioni;

public class NomeVuotoException extends Exception {
    private static final long serialVersionUID = 8290498479156525745L;

    public NomeVuotoException() {
        super("Il nome del contatto non può essere vuoto!");
    }
}

```

Ovviamente abbiamo implementato il controllo affinché il nome di un contatto non sia vuoto, all'interno della classe `Contatto` sfruttando l'incapsulamento:

```

package rubrica.dat;

import rubrica.eccezioni.*;

public class Contatto extends Entita {
    protected static final String SCONOSCIUTO = "sconosciuto";
    private String nome;
    private String numeroDiTelefono;
    private String indirizzo;

    public Contatto(String nome, String numeroDiTelefono) throws
        NomeVuotoException {
        this(nome, numeroDiTelefono, SCONOSCIUTO);
    }

    public Contatto(String nome, String numeroDiTelefono, String indirizzo)
        throws NomeVuotoException {
        super();
        this.setNome(nome);
        this.setNumeroDiTelefono(numeroDiTelefono);
        this.setIndirizzo(indirizzo);
    }

    public void setIndirizzo(String indirizzo) {
        this.indirizzo = indirizzo;
    }

    public String getIndirizzo() {
        return indirizzo;
    }

    public void setNumeroDiTelefono(String numeroDiTelefono) {
        this.numeroDiTelefono = numeroDiTelefono;
    }
}

```

```

    public String getNumeroDiTelefono() {
        return numeroDiTelefono;
    }

    public void setName(String nome) throws NomeVuotoException {
        if (nome == null || nome.trim().length() == 0) {
            throw new NomeVuotoException();
        }
        this.nome = nome;
    }

    public String getName() {
        return nome;
    }

    @Override
    public String toString() {
        return "Nome:\t" + nome + "\nIndirizzo:\t" + indirizzo + "\nTelefono:\t" +
            numeroDiTelefono;
    }
}

```

Per quanto riguarda le modifiche delle classi `RubricaCLI` e `RubricaController` richieste, le evidenziamo qui sotto in grassetto:

```

package rubrica.presentation;

import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.integrazione.*;
import java.util.*;
import java.io.*;

public class RubricaCLI {
    private Scanner scanner;
    private RubricaController controller;
    private static final String INSERT = "/i";
    private static final String SHOW_CONTACT = "/v";
    private static final String EXECUTE = "/e";
    private static final String BACK = "/b";
    private static final String END = "/t";

    public RubricaCLI() {
        scanner = new Scanner(System.in);
        controller = new RubricaController();
    }

    public void start() {
        try {
            controller.start();
        } catch (IOException exc) {
            System.out.println(exc.getMessage());
        }
    }

    public void visualizzaContatti(List<Contatto> contatti) {
        try {
            int numeroContatti = contatti.size();
            if (numeroContatti == 0) {
                System.out.println("Nessun contatto trovato");
            } else {

```

```

        System.out.println("Lista dei contatti in Rubrica:");
        int size = contatti.size();
        for (int i = 1; i <= size; ++i) {
            System.out.println(i + "\t" + contatti.get(i-1).getNome());
        }
        System.out.printf("Scrivi '%s' e il nome del contatto per " +
            " visualizzarne i dettagli\n", SHOW_CONTACT);
    }
    System.out.printf("Scrivi '%s' per inserire un nuovo contatto\n",
        INSERT);
    System.out.printf("Scrivi '%s' per terminare il programma\n", END);
    String comando = scanner.nextLine();
    controller.gestisciVisualizzaContatti(comando, contatti);
} catch (Exception exc) {
    System.out.println(exc.getMessage());
}
}

public void inserisciContatto() {
    try {
        String nome = getDatoContatto("nome");
        String numeroDiTelefono = getDatoContatto("numero di telefono");
        String indirizzo = getDatoContatto("indirizzo");
        System.out.printf("Scrivi '%s' per confermare l'inserimento\n",
            EXECUTE);
        System.out.printf("Scrivi '%s' per tornare alla visualizzazione dei "
            + " contatti\n", BACK);
        String comando = scanner.nextLine();
        controller.gestisciInserisciContatto(comando, nome,
            numeroDiTelefono, indirizzo);
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

public void visualizzaConferma(String messaggio) {
    try {
        System.out.println("Operazione confermata!");
        visualizzaMessaggio(messaggio);
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

private void visualizzaMessaggio(String messaggio) throws IOException,
    ComandoNonValidoException{
    System.out.println(messaggio + "\n");
    System.out.printf("Scrivi '%s' per tornare alla visualizzazione dei "
        + "contatti\n", BACK);
    System.out.printf("Scrivi '%s' per terminare il programma\n", END);
    String comando = scanner.nextLine();
    controller.gestisciVisualizzaConferma(comando);
}

private String getDatoContatto(String dato) {
    String messaggio = String.format("Inserisci %s del contatto:", dato);
    return leggiInputUtente(messaggio);
}

private String leggiInputUtente(String messaggio) {

```

```
        System.out.printf(messaggio);
        return scanner.nextLine();
    }

    class RubricaController {
        GestoreSerializzazione gestoreFile;

        public RubricaController () {
            try {
                gestoreFile =
                    GestoreSerializzazioneFactory.getGestoreSerializzazione();
            } catch (Exception exc) {
                exc.printStackTrace();
                System.exit(1);
            }
        }

        public void start() throws IOException {
            List<Contatto> contatti = gestoreFile.getContatti();
            visualizzaContatti(contatti);
        }

        public void gestisciVisualizzaContatti(String comando,
            List<Contatto> contatti) throws IOException {
            if (comando.equals(INSERT)) {
                //System.out.println("Chiamato metodo inserisci contatto");
                inserisciContatto();
            } else if (comando.equals(END)) {
                System.out.println("Programma terminato");
                System.exit(1);
            } else if (isComandoValidoPerVisualizzaContatto(contatti, comando)){
                System.out.printf("Chiamato metodo visualizza contatto per "
                    + "%s\n", estraiNomeContatto(comando));
            } else {
                throw new ComandoNonValidoException(comando);
            }
        }

        public void gestisciInserisciContatto(String comando, String nome,
            String numeroDiTelefono, String indirizzo) throws IOException,
            NomeVuotoException {
            if (comando.equals(EXECUTE)) {
                Contatto contatto = new Contatto(nome, numeroDiTelefono,
                    indirizzo);
                gestoreFile.inserisci(contatto);
                visualizzaConferma("Inserito contatto:\n"+ contatto);
            } else if (comando.equals(BACK)) {
                visualizzaContatti(gestoreFile.getContatti());
            } else {
                throw new ComandoNonValidoException(comando);
            }
        }

        public void gestisciVisualizzaConferma(String comando) throws
            IOException, ComandoNonValidoException {
            if (comando.equals(END)) {
                System.out.println("Programma terminato");
                System.exit(1);
            } else if (comando.equals(BACK)) {
```

```

        visualizzaContatti(gestoreFile.getContatti());
    } else {
        throw new ComandoNonValidoException(comando);
    }
}

private boolean isComandoValidoPerVisualizzaContatto(List<Contatto>
    contatti, String comando) throws ContattoInesistenteException,
    ComandoNonValidoException {
    boolean result = false;
    if (comando.startsWith(SHOW_CONTACT)) {
        for (Contatto contatto : contatti) {
            if (comando.endsWith(contatto.getNome())) {
                return true;
            }
        }
    } else {
        throw new ComandoNonValidoException(comando);
    }
    throw new ContattoInesistenteException(String.format("Contatto %s "
        + "non trovato!", estraiNomeContatto(comando)));
}

public String estraiNomeContatto(String comando) {
    return comando.substring(comando.indexOf(" ")+1, comando.length());
}
}
}

```

Infine segue la classe `Esercizio15P`:

```

package rubrica.test;

import rubrica.presentation.*;
import java.util.*;

public class Esercizio15P {

    private void eseguiTest() {
        RubricaCLI cli = new RubricaCLI();
        cli.start();
    }

    public static void main(String args[]) {
        Esercizio15P esercizio15P = new Esercizio15P();
        esercizio15P.eseguiTest();
    }
}

```

Soluzione 15.q)

Le classi `RubricaCLI` e `RubricaController` sono state modificate nel seguente modo (modifiche in grassetto):

```

package rubrica.presentation;

import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.integrazione.*;
import java.util.*;

```



```
import java.io.*;

public class RubricaCLI {
    private Scanner scanner;
    private RubricaController controller;
    private static final String INSERT = "/i";
    private static final String SHOW_CONTACT = "/v";
    private static final String UPDATE_CONTACT = "/u";
    private static final String REMOVE_CONTACT = "/r";
    private static final String EXECUTE = "/e";
    private static final String BACK = "/b";
    private static final String END = "/t";

    public RubricaCLI() {
        scanner = new Scanner(System.in);
        controller = new RubricaController();
    }

    public void start() {
        try {
            controller.start();
        } catch (IOException exc) {
            visualizzaErrore(exc);
        }
    }

    public void visualizzaContatto(Contacto contatto) {
        try {
            System.out.println("Dettagli del contatto selezionato:\n" +
                contatto);
            System.out.printf("Scrivi '%s' per modificare il contatto\n",
                UPDATE_CONTACT);
            System.out.printf("Scrivi '%s' per rimuovere il contatto\n",
                REMOVE_CONTACT);
            System.out.printf("Scrivi '%s' per tornare alla visualizzazione dei"
                + " contatti\n", BACK);
            String comando = scanner.nextLine();
            controller.gestisciVisualizzaContatto(comando, contatto);
        }
        catch (Exception exc) {
            visualizzaErrore(exc);
        }
    }

    public void modificaContatto(Contacto contatto) {
        try {
            String numeroDiTelefono = getDatoContatto("numero di telefono");
            String indirizzo = getDatoContatto("indirizzo");
            System.out.printf("Scrivi '%s' per confermare la modifica\n",
                EXECUTE);
            System.out.printf("Scrivi '%s' per tornare alla visualizzazione dei"
                + " contatti\n", BACK);
            String comando = scanner.nextLine();
            controller.gestisciModificaContatto(comando, contatto.getNome(),
                numeroDiTelefono, indirizzo);
        }
        catch (Exception exc) {
            visualizzaErrore(exc);
        }
    }
}
```

```
public void rimuoviContatto(Contatto contatto) {
    try {
        System.out.printf("Scrivi '%s' per confermare la rimozione\n",
            EXECUTE);
        System.out.printf("Scrivi '%s' per tornare alla visualizzazione dei"
            + " contatti\n", BACK);
        String comando = scanner.nextLine();
        controller.gestisciRimuoviContatto(comando, contatto.getNome());
    }
    catch (Exception exc) {
        visualizzaErrore(exc);
    }
}

public void visualizzaContatti(List<Contatto> contatti) {
    try {
        int numeroContatti = contatti.size();
        if (numeroContatti == 0) {
            System.out.println("Nessun contatto trovato");
        } else {
            System.out.println("Lista dei contatti in Rubrica:");
            int size = contatti.size();
            for (int i = 1; i <= size; ++i) {
                System.out.println(i + "\t" + contatti.get(i-1).getNome());
            }
            System.out.printf("Scrivi '%s' e il nome del contatto per " +
                "visualizzarne i dettagli\n", SHOW_CONTACT);
        }
        System.out.printf("Scrivi '%s' per inserire un nuovo contatto\n",
            INSERT);
        System.out.printf("Scrivi '%s' per terminare il programma\n", END);
        String comando = scanner.nextLine();
        controller.gestisciVisualizzaContatti(comando, contatti);
    } catch (Exception exc) {
        visualizzaErrore(exc);
    }
}

public void inserisciContatto() {
    try {
        String nome = getDatoContatto("nome");
        String numeroDiTelefono = getDatoContatto("numero di telefono");
        String indirizzo = getDatoContatto("indirizzo");
        System.out.printf("Scrivi '%s' per confermare l'inserimento\n",
            EXECUTE);
        System.out.printf("Scrivi '%s' per tornare alla visualizzazione dei"
            + " contatti\n", BACK);
        String comando = scanner.nextLine();
        controller.gestisciInserisciContatto(comando, nome,
            numeroDiTelefono, indirizzo);
    } catch (Exception exc) {
        visualizzaErrore(exc);
    }
}

public void visualizzaConferma(String messaggio) {
    System.out.println("Operazione confermata!");
    visualizzaMessaggio(messaggio);
}
```

```
}

public void visualizzaErrore(Exception exc) {
    System.out.println("C'è un problema!");
    visualizzaMessaggio(exc.toString());
}

private void visualizzaMessaggio(String messaggio) {
    try {
        System.out.println(messaggio + "\n");
        System.out.printf("Scrivi '%s' per tornare alla visualizzazione dei"
            + " contatti\n", BACK);
        System.out.printf("Scrivi '%s' per terminare il programma\n", END);
        String comando = scanner.nextLine();
        controller.gestisciVisualizzaMessaggio(comando);
    }
    catch (Exception exc) {
        visualizzaMessaggio(messaggio);
    }
}

private String getDataContatto(String dato) {
    String messaggio = String.format("Inserisci %s del contatto:", dato);
    return leggiInputUtente(messaggio);
}

private String leggiInputUtente(String messaggio) {
    System.out.printf(messaggio);
    return scanner.nextLine();
}

class RubricaController {
    GestoreSerializzazione<Contatto> gestoreFile;

    public RubricaController() {
        try {
            gestoreFile =
                GestoreSerializzazioneFactory.getGestoreSerializzazione();
        } catch (Exception exc) {
            exc.printStackTrace();
            System.exit(1);
        }
    }

    public void start() throws IOException {
        List<Contatto> contatti = gestoreFile.getContatti();
        visualizzaContatti(contatti);
    }

    public void gestisciVisualizzaContatti(String comando, List<Contatto>
        contatti) throws IOException, ClassNotFoundException {
        if (comando.equals(INSERT)) {
            //System.out.println("Chiamato metodo inserisci contatto");
            inserisciContatto();
        } else if (comando.equals(END)) {
            System.out.println("Programma terminato");
            System.exit(1);
        } else if (isComandoValidoPerVisualizzaContatto(contatti, comando)){
            String nomeContatto = estraiNomeContatto(comando);
        }
    }
}
```

```
        System.out.printf("Chiamato metodo visualizza contatto per " +
            "%s\n", nomeContatto);
        Contatto contatto = gestoreFile.recupera(nomeContatto);
        visualizzaContatto(contatto);
    } else {
        throw new ComandoNonValidoException(comando);
    }
}

public void gestisciVisualizzaContatto(String comando, Contatto contatto)
    throws IOException {
    if (comando.equals(UPDATE_CONTACT)) {
        modificaContatto(contatto);
    } else if (comando.equals(REMOVE_CONTACT)) {
        rimuoviContatto(contatto);
    } else if (comando.equals(BACK)) {
        visualizzaContatti(gestoreFile.getContatti());
    } else {
        throw new ComandoNonValidoException(comando);
    }
}

public void gestisciInserisciContatto(String comando, String nome,
    String numeroDiTelefono, String indirizzo) throws IOException {
    if (comando.equals(EXECUTE)) {
        Contatto contatto = new Contatto(nome, numeroDiTelefono,
            indirizzo);
        gestoreFile.inserisci(contatto);
        visualizzaConferma("Inserimento confermato:\n" + contatto);
    } else if (comando.equals(BACK)) {
        visualizzaContatti(gestoreFile.getContatti());
    } else {
        throw new ComandoNonValidoException(comando);
    }
}

public void gestisciModificaContatto(String comando, String nome, String
    numeroDiTelefono, String indirizzo) throws IOException {
    if (comando.equals(EXECUTE)) {
        Contatto contatto = new Contatto(nome, numeroDiTelefono,
            indirizzo);
        gestoreFile.modifica(contatto);
        visualizzaConferma("Modifica confermata:\n" + contatto);
    } else if (comando.equals(BACK)) {
        visualizzaContatti(gestoreFile.getContatti());
    } else {
        throw new ComandoNonValidoException(comando);
    }
}

public void gestisciRimuoviContatto(String comando, String nome) throws
    IOException {
    if (comando.equals(EXECUTE)) {
        gestoreFile.rimuovi(nome);
        visualizzaConferma("Rimozione confermata:\n" + nome);
    } else if (comando.equals(BACK)) {
        visualizzaContatti(gestoreFile.getContatti());
    } else {
        throw new ComandoNonValidoException(comando);
    }
}
```

```

    }

    private boolean isComandoValidoPerVisualizzaContatto(List<Contatto>
        contatti, String comando) throws ContattoInesistenteException,
        ComandoNonValidoException {

        boolean result = false;
        if (comando.startsWith(SHOW_CONTACT)) {
            for (Contatto contatto : contatti) {
                if (comando.endsWith(contatto.getNome())) {
                    return true;
                }
            }
        } else {
            throw new ComandoNonValidoException(comando);
        }
        throw new ContattoInesistenteException(String.format("Contatto %s "
            + "non trovato!", estraiNomeContatto(comando)));
    }

    public void gestisciVisualizzaMessaggio(String comando) throws
        IOException{

        if (comando.equals(BACK)) {
            visualizzaContatti(gestoreFile.getContatti());
        } else if (comando.equals(BACK)) {
            visualizzaContatti(gestoreFile.getContatti());
        } else {
            throw new ComandoNonValidoException(comando);
        }
    }

    public String estraiNomeContatto(String comando) {
        return comando.substring(comando.indexOf(" ")+1, comando.length());
    }
}

```

Ovviamente la classe `Esercizio15Q` è la seguente:

```

package rubrica.test;

import rubrica.presentation.*;
import java.util.*;

public class Esercizio15Q {

    private void eseguiTest() {
        RubricaCLI cli = new RubricaCLI();
        cli.start();
    }

    public static void main(String args[]) {
        Esercizio15Q esercizio15Q = new Esercizio15Q();
        esercizio15Q.eseguiTest();
    }
}

```

Soluzione 15.r)

Le affermazioni corrette sono la prima e la quarta. L'affermazione numero 2 è scorretta in quanto `getParent` restituisce una stringa che rappresenta una directory. L'affermazione numero 3 è scorretta in quanto `pathSeparator` è una variabile statica della classe `File`, che rappresenta il separatore di path dipendente dal sistema operativo. Lo si poteva evincere sia dal nome che è scritto in minuscolo (le costanti invece per convenzione sono scritte in maiuscolo), sia dal fatto che non può assumere un valore a priori visto che esso dipende dal sistema operativo.

Soluzione 15.s)

L'unica affermazione corretta è la prima. L'affermazione numero 2 è scorretta in quanto `Reader` e `Writer` sono interfacce e non classi. L'affermazione numero 3 è scorretta in quanto il metodo `readLine` è definito all'interno della classe `BufferedReader`. L'affermazione numero 4 è scorretta in quanto il costruttore di un `ObjectInputStream` può prendere in input un qualsiasi oggetto di tipo `InputStream`.

Soluzione 15.t)

L'unica affermazione scorretta è la numero 1. L'affermazione numero 2 è corretta anche se l'efficienza e la comodità della lettura verrebbero indubbiamente migliorate "agganciando" un `BufferedReader` al `FileReader`. Per esempio potremmo leggere un file con un codice simile al seguente:

```
Reader fileReader = new FileReader("c:/miofile.txt");
int data = fileReader.read();
while (data != -1) {
    System.out.print(data);
    data = fileReader.read();
}
fileReader.close();
```

L'affermazione numero 3 è corretta, perché la classe `File` definisce il metodo `mkdir`, ed anche la 4 è corretta in quanto la classe `File` definisce il metodo `list`.

Soluzione 15.u)

La risposta corretta è la numero 2, ovvero 12 byte. Infatti vengono scritti all'interno del file un tipo `int` (32 bit = 4 byte) e un `double` (64 bit = 8 byte).

Soluzione 15.v)

La soluzione potrebbe essere la seguente:

```
import java.io.*;

public class Soluzione15S {
    public static void main(String args[]) throws Exception {
        try (FileInputStream fis = new FileInputStream("nuovo file.txt");
```

```
        DataInputStream dis = new DataInputStream(fis);) {  
        for (int i = 0; i < 50; i++) {  
            System.out.print(dis.readInt());  
        }  
    }  
}
```

Il cui output è:

```
0  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46
```

```
47
48
49
```

Soluzione 15.w)

Le affermazioni corrette sono le numero 2 e 3. La 1 non è corretta perché esistono implementazioni di `Reader`, come `StringReader`, che non dichiarano clausole `throws a IOException`. L'affermazione 4 non è corretta in quanto devono gestire tali implementazioni solo le implementazioni di `Writer` che hanno a che fare con i file come `FileWriter`.

Soluzione 15.x)

Le affermazioni corrette sono le numero 1, 3 e 4. L'affermazione 2 non è corretta perché abbiamo visto, anche nel paragrafo 15.4.1, che è possibile ottenere lo stesso risultato concatenando un `BufferedReader` ad uno `InputStreamReader` la cui fonte di lettura è l'oggetto `System.in` (ovvero il dispositivo di input di default, probabilmente la tastiera). Per comodità riportiamo di seguito la classe definita nel paragrafo 15.4.1.1 `KeyboardInput`:

```
import java.io.*;

public class KeyboardInput {

    public static void main (String args[]) throws IOException {
        String stringa = null;
        System.out.println("Digita qualcosa e premi invio...\n"
            + "Per terminare il programma digitare \"fine\"");
        try (InputStreamReader ir = new InputStreamReader(System.in);
            BufferedReader in = new BufferedReader(ir)) {
            stringa = in.readLine();
            while ( stringa != null ) {
                if (stringa.equals("fine")) {
                    System.out.println("Programma terminato");
                    break;
                }
                System.out.println("Hai scritto: " + stringa);
                stringa = in.readLine();
            }
        }
    }
}
```

Soluzione 15.y)

Solo le affermazioni 1 e 3 non sono corrette. In particolare l'affermazione 2 è corretta alla luce dell'esistenza del metodo `getParent`. La 3 invece non è corretta visto che il metodo `getOwner` è definito dalla classe `Files`.

Soluzione 15.z)

Solo le affermazioni 2 e 5 sono corrette. L'affermazione 1 non è corretta perché, il metodo `list` di `Files`, restituisce uno `Stream` di oggetti `Path`. La 3 non è corretta perché i tipi `Files` e `Path` appartengono al package `java.nio.file`. Mentre la 4 non è corretta perché `Files` è una classe astratta.

Soluzione 15.aa) Networking, Vero o Falso:

1. **Vero.**
2. **Vero.**
3. **Vero.**
4. **Vero.**
5. **Vero.**

Soluzione I.a) Date-Time API, Vero o Falso:

1. **Vero.**
2. **Falso**, anche la stessa classe `Instant` definisce un metodo `parse`.
3. **Falso**, `Duration` non è connessa alla timeline visto che rappresenta un intervallo di tempo compreso tra due `Instant`.
4. **Vero.**
5. **Falso.**
6. **Falso**, la classe `ZonedDateTime` semplicemente non esiste.
7. **Falso.**
8. **Vero.**
9. **Falso.**
10. **Vero.**

Soluzione I.b)

Il listato potrebbe essere il seguente:

```
import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;

public class DateUtils {

    public static long getIntervallo(LocalDateTime ldt1, LocalDateTime ldt2,
                                     ChronoUnit chronoUnit) {
        return chronoUnit.between(ldt1, ldt2);
    }
}
```

Ma ci sono tante alternative a questa soluzione.

Soluzione I.c)

Il listato del metodo richiesto è semplice:

```
public static long getTempoPassato(Instant instant1, ChronoUnit chronoUnit) {  
    return getIntervallo(instant1, Instant.now(), chronoUnit);  
}
```

Infatti abbiamo riutilizzato il metodo scritto nell'esercizio precedente.

Soluzione I.d)

Il listato del metodo richiesto è semplice:

```
public static String oraEsatta() {  
    LocalTime ora = LocalTime.now();  
    String oraEsatta = (ora.getHour() + ":" + ora.getMinute() + " "  
        + ora.getSecond());  
    return oraEsatta;  
}
```

Soluzione I.e)

Il listato del metodo richiesto è anch'esso semplice:

```
public static String formattaData(LocalDateTime localDateTime, String pattern)  
    throws DateTimeException {  
    String formattedDate = null;  
    try {  
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern(pattern);  
        formattedDate = formatter.format(localDateTime);  
    } catch (DateTimeException dateTimeException) {  
        dateTimeException.printStackTrace();  
        throw dateTimeException;  
    }  
    return formattedDate;  
}
```

Si noti che abbiamo gestito l'eccezione solo per stamparne lo stack trace, per poi rilanciarla.

Abbiamo anche inserito la clausola `throws` accanto alla definizione del metodo. In questo modo chi userà questo metodo saprà che potrebbe lanciare l'eccezione nel caso si specifichi un pattern errato.

Soluzione I.f)

Il listato del metodo richiesto potrebbe essere il seguente:

```
public static LocalDate analizzaData(String data, String pattern)  
    throws DateTimeParseException {  
    LocalDate localDate = null;  
    try {  
        localDate = LocalDate.parse(data, DateTimeFormatter.ofPattern(pattern));  
    } catch (DateTimeParseException dateTimeParseException) {  
        dateTimeParseException.printStackTrace();  
        throw dateTimeParseException;  
    }  
    return localDate;  
}
```

Si noti che anche in questo caso abbiamo gestito l'eccezione solo per stamparne lo stack trace, per poi rilanciarla. Abbiamo anche inserito la clausola `throws` accanto alla definizione del metodo. In questo modo chi userà questo metodo saprà che potrebbe lanciare l'eccezione nel caso si specifichi un pattern errato. L'eccezione questa volta è però di tipo `DateTimeParseException`.

Anche in questo caso la soluzione è molto semplice. Si noti che è possibile anche usare altre classi e altri metodi per ottenere gli stessi risultati ottenuti negli ultimi quattro esercizi. La libreria per la gestione delle date e del tempo è davvero semplice e potente.

Soluzione I.g)

Il listato richiesto potrebbe essere il seguente:

```
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;

public class TestDateUtils {

    private static final String FORMATO_DATA = "MM/dd/yy  hh:mm a";

    public static void main(String args[]) {
        final String oraEsatta = DateUtils.oraEsatta();
        System.out.println("Sono le: " + oraEsatta);
        Instant duemila = Instant.parse("2000-01-01T00:00:00.00Z");
        Instant duemiladieci = Instant.parse("2010-01-01T00:00:00.00Z");
        long intervalloInGiorni = DateUtils.getIntervallo(
            duemila, duemiladieci, ChronoUnit.DAYS);
        System.out.println("Dal primo gennaio 2000 al primo gennaio 2010 "
            + "sono passati " + intervalloInGiorni + " giorni");
        final long tempoPassatoInMinuti =
            DateUtils.getTempoPassato(duemila, ChronoUnit.MINUTES);
        System.out.println("Dal primo gennaio 2010 ad oggi sono passati "
            + tempoPassatoInMinuti
            + " minuti");
        LocalDateTime localDateTime = LocalDateTime.now();
        final String dataFormattata =
            DateUtils.formattaData(localDateTime, FORMATO_DATA);
        System.out.println("Data formattata: " + dataFormattata);
        LocalDate localDate =
            DateUtils.analizzaData(dataFormattata, FORMATO_DATA);
        System.out.println(localDate);
        System.out.println("Lanciamo un'eccezione!");
        localDate = DateUtils.analizzaData(dataFormattata, "ABC");
    }
}
```

Soluzione I.h)

Tutte le affermazioni sono vere tranne la numero 1. Infatti i metodi `from` hanno un parametro sempre più completo rispetto al tipo che devono ritornare.

Soluzione I.i)

Tutte le affermazioni sono vere tranne la numero 3. Infatti la classe `ZoneDateTime` rappresenta la versione localizzata della classe `LocalDateTime`, non di `LocalDate`.

Soluzione I.j)

Tutte le affermazioni sono vere.

Soluzioni degli esercizi del capitolo 16

Moduli

Soluzione 16.a)

Le affermazioni corrette sono le numero 1, 3 e 5. L'affermazione numero 2 è scorretta perché il forte incapsulamento permette di rendere accessibile un certo package solo ai moduli specificati (non ai package specificati). L'affermazione numero 4 è scorretta perché come affermato nel paragrafo 16.1.1, le tecniche di ottimizzazione della JVM sono più efficaci nel caso siano noti a priori i tipi che saranno utilizzati nell'applicazione.

Soluzione 16.b)

Le uniche affermazioni corrette sono le numero 2 e 5. La numero 1 non è corretta perché un JAR modulare ha un'estensione .jar. Un file con estensione .jmod invece è caratterizzato dal fatto di contenere anche risorse native. L'affermazione numero 3 è palesemente falsa: ad essere un unico file è il descrittore del modulo, non il modulo stesso. Infine l'affermazione 4 è anch'essa falsa perché i package in questione non sono stati eliminati, bensì nascosti all'utilizzo mediante il forte incapsulamento.

Soluzione 16.c)

Tutte le affermazioni sono corrette.

Soluzione 16.d)

Con il seguente comando:

```
javac -d mods/com.domain.mymodule src/com.domain.mymodule/com/domain/*  
src/com.domain.mymodule/module-info.java
```

stiamo compilando un modulo di nome com.domain.mymodule.

In particolare con l'opzione:

```
-d mods/com.domain.mymodule
```

stiamo ordinando al comando javac che il risultato della compilazione dovrà essere posizionato all'interno della sottocartella com.domain.mymodule della cartella mods (cartella che deve essere presente nella stessa posizione da dove stiamo eseguendo il comando di compilazione).

Con l'argomento:

```
src/com.domain.mymodule/com/domain/*
```

stiamo invece specificando che devono essere compilati tutti i file presenti nel percorso src/com.domain.mymodule/com/domain.

Infine con l'argomento:

```
src/com.domain.mymodule/module-info.java
```

specifichiamo che deve essere compilato anche il descrittore del modulo `module-info.java` presente nella cartella `src/com.domain.mymodule`.

Soluzione 16.e)

Con il seguente comando:

```
java --module-path mods  
-m com.domain/com.domain.mymodule.HelloModularWorld
```

stiamo eseguendo la classe contenente il metodo `main()`

`com.domain.mymodule.HelloModularWorld` del modulo di nome `com.domain.mymodule`, specificando come `modulepath` la cartella `mods`.

Soluzione 16.f)

Un **componente software** è un insieme di classi con un'interfaccia ben definita, che mette a disposizione funzionalità eseguibili indipendentemente dal contesto. Un componente software è quindi un sottosistema eseguibile e riutilizzabile.

In un **partizionamento verticale** l'applicazione viene divisa per funzionalità della stessa importanza, e ognuna di queste può essere sviluppata indipendentemente dall'altra.

Un **partizionamento orizzontale** è un partizionamento basato soprattutto sull'ereditarietà. Un tipico esempio è quello di un **framework**, ovvero una micro-architettura che mette a disposizione un modello estendibile per realizzare applicazioni nell'ambito di uno specifico dominio.

La **coesione** è la misura di quanto un certo elemento (classe, metodo, package, sottosistema etc.) contribuisce a realizzare un certo scopo all'interno del sistema.

L'**accoppiamento** è la metrica che misura la dipendenza tra classi, tra package, tra metodi e così via.

Un **sottosistema** è un insieme di classi relazionate da associazioni, eventi e vincoli, e per il quale è possibile uno sviluppo indipendente dagli altri sottosistemi. I sottosistemi sono altamente coesi (ovvero dichiara solo tipi che collaborano tra loro per realizzare un determinato scopo), internamente sono presenti tipi altamente accoppiati, esternamente sono bassamente accoppiati con tipi di altre classi.

Il **principio di inversione della dipendenza** afferma che le classi devono dipendere dalle astrazioni e non dalle implementazioni.

Soluzione 16.g)

Una soluzione potrebbe essere la seguente:

```
module negozio.interfacciagrafica {  
    requires negozio.fatturazione;  
}
```

e

```
module negozio.fatturazione {  
    exports negozio.fatturazione.documenti;  
    exports negozio.fatturazione.funzionidisponibili;  
    //exports negozio.fatturazione.algoritmiinterni;  
}
```

Ovvero il modulo `negozio.interfacciagrafica` legge il modulo `negozio.fatturazione`, che a sua volta gli espone solo due dei tre package (si noti che la terza direttiva è commentata). Infatti il package `negozio.fatturazione.algoritmiinterni`, considerato il nome, con tutta probabilità potrebbe non essere utilizzato direttamente dall'esterno.

Soluzione 16.h)

Muovendoci in un contesto tanto astratto, è possibile ipotizzare diverse soluzioni. Partendo dal presupposto che il descrittore del modulo `negozio.fatturazione` non sarà modificato, potremmo semplicemente esportare i giusti package dal modulo `negozio.vendita`:

```
module negozio.vendita {  
    exports negozio.vendita.articoli;  
    exports negozio.vendita.funzionidisponibili;  
    //exports negozio.vendita.algoritmiinterni;  
}
```

e far in modo che il modulo `negozio.interfacciagrafica`, legga anche `negozio.vendita`:

```
module negozio.interfacciagrafica {  
    requires negozio.fatturazione;  
    requires negozio.vendita;  
}
```

delegando così all'interfaccia grafica l'onere di definire una funzione che vada a contestualizzare la vendita e la fatturazione di un articolo in un'unica richiesta da parte dell'utente. Siccome non è una buona pratica assegnare delle regole di business ad un'interfaccia grafica (che deve già definire le regole di presentazione dell'interfaccia stessa), decidiamo di rendere la situazione più flessibile, in modo tale da poter prendere delle decisioni più tardi. Allora utilizzando la direttiva `requires transitive`, andiamo a modificare il descrittore del modulo `negozio.vendita`, in modo tale che possa leggere (e far leggere) il modulo `negozio.fatturazione`:

```
module negozio.vendita {  
    exports negozio.vendita.articoli;  
    exports negozio.vendita.funzionidisponibili;  
    //exports negozio.vendita.algoritmiinterni;  
    requires transitive negozio.fatturazione;  
}
```

A questo punto il modulo `negozio.interfacciagrafica` può leggere solo `negozio.vendita`, visto che transitivamente leggerà anche `negozio.fatturazione`:

```
module negozio.interfacciagrafica {  
    //requires negozio.fatturazione;  
    requires negozio.vendita;  
}
```

Soluzione 16.i)

In realtà non si può fare nulla a meno che non si vogliano rivalutare tutte le dipendenze già specificate. Se `negozio.fatturazione` volesse leggere `negozio.interfacciagrafica`, otterremmo un errore di dipendenza ciclica.

Soluzione 16.l)

La soluzione consiste nel rivedere il descrittore del modulo `negozio.fatturazione` esportando il package `negozio.vendita.algoritmiinterni`, e contemporaneamente aprirlo al modulo `negozio.interfacciagrafica`:

```
module negozio.fatturazione {  
    exports negozio.fatturazione.articoli;  
    exports negozio.fatturazione.funzionidisponibili;  
    exports negozio.fatturazione.algoritmiinterni;  
    opens negozio.fatturazione.algoritmiinterni to negozio.interfacciagrafica;  
}
```

Soluzione 16.m)

L'unica soluzione è eseguire il programma specificando da riga di comando l'apertura del modulo `negozio.fatturazione` al modulo `negozio.vendita` con la seguente sintassi:

```
-add-opens  
negozio.fatturazione/negozio.fatturazione.algoritmiinterni=negozio.vendita
```

Soluzione 16.n)

Le affermazioni corrette sono le numero 1, e 5. L'affermazione numero 2 è falsa, perché sono le implementazioni a dipendere dal service provider interface. L'affermazione numero 4 è falsa in quanto non è necessario esportare le implementazioni del service provider interface, bensì bisogna usare la direttiva `provides to`.

Soluzione 16.o)

Una possibile soluzione è quella di creare un modulo che chiameremo `com.claudiodesio.rubrica.dati`. All'interno inseriremo le classi `Contatto` e `Dato`, a loro volta modificati per appartenere ad un package che coincide con il nome del modulo:

```
package com.claudiodesio.rubrica.dati;
```

```
import java.io.Serializable;
```

```
public interface Dato extends Serializable {  
}
```

e

```
package com.claudiodesio.rubrica.dati;
```

```
import java.io.Serializable;
```

```
public class Contatto implements Dato {
```

```
    private static final long serialVersionUID = 8942402240056525661L;
```

```
    private String nome;
```

```
    private String indirizzo;
```



```
private String numeroDiTelefono;

public Contatto (String nome, String indirizzo, String numeroDiTelefono) {
    this.nome = nome;
    this.indirizzo = indirizzo;
    this.numeroDiTelefono = numeroDiTelefono;
}

public void setNumeroDiTelefono(String numeroDiTelefono) {
    this.numeroDiTelefono = numeroDiTelefono;
}

public String getNumeroDiTelefono() {
    return numeroDiTelefono;
}

public void setIndirizzo(String indirizzo) {
    this.indirizzo = indirizzo;
}

public String getIndirizzo() {
    return indirizzo;
}

public void setName(String nome) {
    this.nome = nome;
}

public String getName() {
    return nome;
}

@Override
public String toString() {
    return "Nome:\t" + nome + "\nIndirizzo:\t" + indirizzo +
        "\nTelefono:\t" + numeroDiTelefono;
}
}
```

I package delle altre classi li potremo vedere nelle soluzioni dei prossimi esercizi.

Il descrittore del modulo sarà il seguente:

```
module com.claudiodesio.rubrica.dati {
    exports com.claudiodesio.rubrica.dati;
}
```

Per compilare il modulo utilizzeremo il seguente comando utilizzando la solita struttura di cartelle che abbiamo utilizzato sinora (l'esercizio completo è nella cartella Codice\capitolo_16\esercizi\16.0 del file contenente gli esempi di codice che avete probabilmente scaricato insieme al file che state leggendo, all'indirizzo <https://www.nuovojava.it>):

```
javac -d mods/com.claudiodesio.rubrica.dati
src/com.claudiodesio.rubrica.dati/module-info.java
src/com.claudiodesio.rubrica.dati/com/claudiodesio/rubrica/dati/*.java
```

Soluzione 16.p)

Una possibile soluzione è quella di creare un modulo che chiameremo `com.claudiodesio.rubrica.eccezioni`. All'interno inseriremo le classi

ContattoInesistenteException e ContattoEsistenteException, a loro volta modificate per appartenere ad un package che coincide con il nome del modulo:

```
package com.claudiodesio.rubrica.eccezioni;
```

```
import java.io.IOException;
```

```
public class ContattoInesistenteException extends IOException {  
    private static final long serialVersionUID = 8942402240056525663L;  
    public ContattoInesistenteException(String message) {  
        super(message);  
    }  
}
```

e

```
package com.claudiodesio.rubrica.eccezioni;
```

```
import java.io.IOException;
```

```
public class ContattoEsistenteException extends IOException {  
    private static final long serialVersionUID = 8942402240056525662L;  
    public ContattoEsistenteException (String message) {  
        super(message);  
    }  
}
```

Il descrittore del modulo sarà il seguente:

```
module com.claudiodesio.rubrica.eccezioni {  
    exports com.claudiodesio.rubrica.eccezioni;  
}
```

Per compilare entrambi i moduli definiti, utilizzeremo il seguente comando utilizzando la solita struttura di cartelle che abbiamo utilizzato sinora (l'esercizio completo è nella cartella Codice\capitolo_16\esercizi\16.p del file contenente gli esempi di codice):

```
javac -d mods --module-source-path src  
    src/com.claudiodesio.rubrica.dati/module-info.java  
    src/com.claudiodesio.rubrica.dati/com/claudiodesio/rubrica/dati/*.java  
    src/com.claudiodesio.rubrica.eccezioni/module-info.java  
    src/com.claudiodesio.rubrica.eccezioni/com/claudiodesio/  
rubrica/eccezioni/*.java
```

Soluzione 16.q)

Una possibile soluzione è quella di creare un modulo che chiameremo

com.claudiodesio.rubrica.util. All'interno abbiamo deciso di inserire le classi Retriever, Executor e FileUtils, a loro volta modificate per appartenere ad un package che coincide con il nome del modulo. Seguono le dichiarazioni delle tre classi:

```
package com.claudiodesio.rubrica.util;
```

```
@FunctionalInterface  
public interface Retriever<O> {  
    O esegui() throws Exception;  
}
```

```
e
package com.claudiodesio.rubrica.util;

@FunctionalInterface
public interface Executor {

    void esegui() throws Exception;
}
```

```
e
package com.claudiodesio.rubrica.util;

public class FileUtils {
    public static final String SUFFIX = ".con";

    public static String getNomeFile(String nome) {
        return nome + SUFFIX;
    }
}
```

Il descrittore del modulo sarà il seguente:

```
module com.claudiodesio.rubrica.util {
    exports com.claudiodesio.rubrica.util;
}
```

Per compilare i tre moduli definiti, utilizzeremo il seguente comando utilizzando la solita struttura di cartelle che abbiamo utilizzato sinora (l'esercizio completo è nella cartella Codice\capitolo_16\esercizi\16.q del file contenente gli esempi di codice):

```
javac -d mods --module-source-path src
src/com.claudiodesio.rubrica.dati/module-info.java
src/com.claudiodesio.rubrica.dati/com.claudiodesio.rubrica.dati/*.java
src/com.claudiodesio.rubrica.eccezioni/module-info.java
src/com.claudiodesio.rubrica.eccezioni/com.claudiodesio.rubrica/
eccezioni/*.java src/com.claudiodesio.rubrica.util/module-info.java
src/com.claudiodesio.rubrica.util/com.claudiodesio.rubrica.util/*.java
```

Soluzione 16.r)

La classe che farà da service provider interface è GestioneSerializzazione, che faremo appartenere al modulo com.claudiodesio.rubrica.spi. Abbiamo modificato la classe per appartenere al package com.claudiodesio.rubrica.spi:

```
package com.claudiodesio.rubrica.spi;

import com.claudiodesio.rubrica.dati.Dato;
import java.io.*;
import java.util.*;

public interface GestioneSerializzazione<T extends Dato> {

    void inserisci(T dato) throws IOException;

    T recupera(String id) throws IOException, ClassNotFoundException;

    void modifica(T dato) throws IOException;

    void rimuovi(String id) throws IOException;
}
```

Il descrittore del modulo sarà il seguente:

```
module com.claudiodesio.rubrica.spi {
    exports com.claudiodesio.rubrica.spi;
    requires com.claudiodesio.rubrica.dati;
}
```

In questo caso c'è stato bisogno di leggere il modulo `com.claudiodesio.rubrica.dati`, visto che la classe `GestoreSerializzazione` utilizza l'interfaccia `Dato`.

Per compilare i quattro moduli definiti sinora utilizzeremo il seguente comando utilizzando la solita struttura di cartelle (l'esercizio completo è nella cartella `Codice\capitolo_16\esercizi\16.r` del file contenente gli esempi di codice):

```
javac -d mods --module-source-path src
src/com.claudiodesio.rubrica.dati/module-info.java
src/com.claudiodesio.rubrica.dati/com/claudiodesio/rubrica/dati/*.java
src/com.claudiodesio.rubrica.eccezioni/module-info.java
src/com.claudiodesio.rubrica.eccezioni/com/claudiodesio/rubrica/
eccezioni/*.java src/com.claudiodesio.rubrica.util/module-info.java
src/com.claudiodesio.rubrica.util/com/claudiodesio/rubrica/util/*.java
src/com.claudiodesio.rubrica.spi/module-info.java
src/com.claudiodesio.rubrica.spi/com/claudiodesio/rubrica/spi/*.java
```

Soluzione 16.s)

Per prima cosa abbiamo dovuto aggiungere alle classi `GestoreFile` e `GestoreFileNIO2`, oltre alla dichiarazione dei package, diversi import:

```
package com.claudiodesio.rubrica.io;

import com.claudiodesio.rubrica.spi.GestoreSerializzazione;
import com.claudiodesio.rubrica.dati.Contatto;
import com.claudiodesio.rubrica.eccezioni.*;
import com.claudiodesio.rubrica.util.*;
import java.util.*;
import java.io.*;

public class GestoreFile implements GestoreSerializzazione<Contatto> {

    @Override
    public void inserisci(Contatto contatto)
        throws ContattoEsistenteException, FileNotFoundException, IOException {
        Contatto contattoEsistente = getContatto(contatto.getNome());
        if (contattoEsistente != null) {
            throw new ContattoEsistenteException(
                contatto.getNome() + ": contatto già esistente!");
        }
        registra( contatto);
    }

    @Override
    public Contatto recupera(String nome)
        throws ContattoInesistenteException, ContattoEsistenteException {
        Contatto contatto = getContatto(nome);
        if (contatto == null) {
            throw new ContattoInesistenteException(
                nome + ": contatto non trovato!");
        }
        return contatto;
    }
}
```

```

@Override
public void modifica(Contatto contatto) throws ContattoInesistenteException,
    ContattoEsistenteException, FileNotFoundException, IOException {
    if (isContattoEsistente(contatto.getNome())) {
        registra(contatto);
    } else {
        throw new ContattoInesistenteException(
            contatto.getNome() + ": contatto non trovato!");
    }
}

@Override
public void rimuovi(String nome) throws ContattoInesistenteException,
    ContattoEsistenteException, FileNotFoundException, IOException {
    File file = new File(FileUtils.getNomeFile(nome));
    if (file.delete()) {
        System.out.println("Contatto " + nome + " cancellato!");
    } else {
        throw new ContattoInesistenteException(
            nome + ": contatto non trovato!");
    }
}

private void registra(Contatto contatto)
    throws FileNotFoundException, IOException {
    try (FileOutputStream fos =
        new FileOutputStream (new File(
            FileUtils.getNomeFile(contatto.getNome())));
        ObjectOutputStream s = new ObjectOutputStream (fos);) {
        s.writeObject (contatto);
        System.out.println("Contatto registrato:\n" + contatto);
    }
}

private boolean isContattoEsistente(String nome) {
    File file = new File(FileUtils.getNomeFile(nome));
    return file.exists();
}

private Contatto getContatto(String nome) {
    try (FileInputStream fis = new FileInputStream (
        new File(FileUtils.getNomeFile(nome)));
        ObjectInputStream ois = new ObjectInputStream (fis);) {
        Contatto contatto = (Contatto)ois.readObject();
        System.out.println("Contatto recuperato:\n"+ contatto);
        return contatto;
    } catch (Exception exc) {
        return null;
    }
}
}

```

e

```

package com.claudiodesio.rubrica.nio;

import com.claudiodesio.rubrica.spi.GestoreSerializzazione;
import com.claudiodesio.rubrica.dati.Contatto;
import com.claudiodesio.rubrica.eccezioni.*;
import com.claudiodesio.rubrica.util.*;
import java.util.*;

```

```
import java.io.*;
import java.util.*;
import java.io.*;
import java.nio.file.*;

public class GestoreFileNIO2 implements GestoreSerializzazione<Contatto> {

    @Override
    public void inserisci(Contatto contatto)
        throws ContattoEsistenteException, FileNotFoundException, IOException {
        Path path = Paths.get(FileUtils.getNomeFile(contatto.getNome()));
        if (Files.exists(path)) {
            throw new ContattoEsistenteException(
                contatto.getNome() + ": contatto già esistente!");
        }
        registra(contatto);
    }

    @Override
    public Contatto recupera(String nome)
        throws ContattoInesistenteException, ContattoEsistenteException {
        Contatto contatto = getContatto(nome);
        if (contatto == null) {
            throw new ContattoInesistenteException(
                nome + ": contatto non trovato!");
        }
        return contatto;
    }

    @Override
    public void modifica(Contatto contatto) throws ContattoInesistenteException,
        ContattoEsistenteException, FileNotFoundException, IOException {
        if (isContattoEsistente(contatto.getNome())) {
            registra(contatto);
        } else {
            throw new ContattoInesistenteException(
                contatto.getNome() + ": contatto non trovato!");
        }
    }

    @Override
    public void rimuovi(String nome) throws ContattoInesistenteException,
        ContattoEsistenteException, FileNotFoundException, IOException {
        Path path = Paths.get(FileUtils.getNomeFile(nome));
        if (Files.exists(path)) {
            Files.delete(path);
            System.out.println("Contatto " + nome + " cancellato!");
        } else {
            throw new ContattoInesistenteException(
                nome + ": contatto non trovato!");
        }
    }

    private void registra(Contatto contatto)
        throws FileNotFoundException, IOException {
        Path path = Paths.get(FileUtils.getNomeFile(contatto.getNome()));
        Files.write(path, getBytesDaOggetto(contatto));
        System.out.println("Contatto registrato:\n" + contatto);
    }
}
```

```

private byte[] getBytesDaOggetto(Object object) throws IOException {
    try (ByteArrayOutputStream bos = new ByteArrayOutputStream();
        ObjectOutput out = new ObjectOutputStream(bos)) {
        out.writeObject(object);
        return bos.toByteArray();
    }
}

private Object getOggettoDaByte(byte[] bytes)
    throws IOException, ClassNotFoundException {
    try (ByteArrayInputStream bis = new ByteArrayInputStream(bytes);
        ObjectInput in = new ObjectInputStream(bis)) {
        return in.readObject();
    }
}

private boolean isContattoEsistente(String nome) {
    Path path = Paths.get(FileUtils.getNomeFile(nome));
    return Files.exists(path);
}

private Contatto getContatto(String nome) {
    Path path = Paths.get(FileUtils.getNomeFile(nome));
    byte[] bytes = null;
    Contatto contatto = null;
    try {
        bytes = Files.readAllBytes(path);
        contatto = (Contatto) getOggettoDaByte(bytes);
        System.out.println("Contatto recuperato:\n" + contatto);
    }
    catch (Exception exc) {
        return null;
    }
    return contatto;
}
}

```

Il primo modulo conterrà `GestoreFile` e sarà chiamato `com.claudiodesio.rubrica.io`, mentre `GestoreFileNIO2` sarà contenuto nel modulo `com.claudiodesio.rubrica.nio`.

Per il primo modulo possiamo creare il seguente descrittore:

```

module com.claudiodesio.rubrica.io {
    //exports com.claudiodesio.rubrica.io;
    provides com.claudiodesio.rubrica.spi.GestoreSerializzazione
        with com.claudiodesio.rubrica.io.GestoreFile;
    requires com.claudiodesio.rubrica.dati;
    requires com.claudiodesio.rubrica.eccezioni;
    requires com.claudiodesio.rubrica.spi;
    requires com.claudiodesio.rubrica.util;
}

```

Mentre per il secondo possiamo creare quest'altro:

```

module com.claudiodesio.rubrica.nio {
    //exports com.claudiodesio.rubrica.nio;
    provides com.claudiodesio.rubrica.spi.GestoreSerializzazione
        with com.claudiodesio.rubrica.nio.GestoreFileNIO2;
    requires com.claudiodesio.rubrica.dati;
    requires com.claudiodesio.rubrica.eccezioni;
    requires com.claudiodesio.rubrica.spi;
    requires com.claudiodesio.rubrica.util;
}

```

Per compilare i sei moduli definiti sinora, utilizzeremo il seguente comando utilizzando la solita struttura di cartelle (l'esercizio completo è nella cartella Codice\capitolo_16\esercizi\16.s del file contenente gli esempi di codice):

```
javac -d mods --module-source-path src
src/com.claudiodesio.rubrica.dati/module-info.java
src/com.claudiodesio.rubrica.dati/com/claudiodesio/rubrica/dati/*.java
src/com.claudiodesio.rubrica.eccezioni/module-info.java
src/com.claudiodesio.rubrica.eccezioni/com/claudiodesio/rubrica/
eccezioni/*.java src/com.claudiodesio.rubrica.util/module-info.java
src/com.claudiodesio.rubrica.util/com/claudiodesio/rubrica/util/*.java
src/com.claudiodesio.rubrica.spi/module-info.java
src/com.claudiodesio.rubrica.spi/com/claudiodesio/rubrica/spi/*.java
src/com.claudiodesio.rubrica.io/module-info.java
src/com.claudiodesio.rubrica.io/com/claudiodesio/rubrica/io/*.java
src/com.claudiodesio.rubrica.nio/module-info.java
src/com.claudiodesio.rubrica.nio/com/claudiodesio/rubrica/nio/*.java
```

Soluzione 16.t)

Modifichiamo la classe `Esercizio15N` in `Esercizio16T` per sfruttare il `ServiceLoader` e caricare i servizi dichiarati nell'esercizio precedente. Inoltre abbiamo rielaborato i diversi import e dichiarato il package di appartenenza.

```
package com.claudiodesio.rubrica.test;

import java.util.function.*;
import com.claudiodesio.rubrica.spi.GestoreSerializzazione;
import com.claudiodesio.rubrica.dati.Contatto;
import com.claudiodesio.rubrica.util.*;
import java.util.Iterator;
import java.util.ServiceLoader;

public class Esercizio16T {

    private GestoreSerializzazione<Contatto> gestoreFile;

    private Contatto[] contatti;

    public Esercizio16T(String className) {
        contatti = getContatti();
        gestoreFile = getGestoreSerializzazione(className);
    }

    public GestoreSerializzazione<Contatto>
    getGestoreSerializzazione(String className) {
        ServiceLoader<GestoreSerializzazione> serviceLoader =
            ServiceLoader.load(
                com.claudiodesio.rubrica.spi.GestoreSerializzazione.class);
        for (GestoreSerializzazione<Contatto> gestoreSerializzazione :
            serviceLoader) {
            if (gestoreSerializzazione.getClass().getSimpleName().
                equals(className)) {
                return gestoreSerializzazione;
            }
        }
        throw new IllegalArgumentException(
```



```
        "Nessun gestore di serializzazione trovato per classe =" + className);
    }

    private void eseguiTest() {
        System.out.println("TESTIAMO LA CREAZIONE DEI TRE CONTATTI");
        creaContatti();
        System.out.println("RECUPERIAMO I TRE CONTATTI");
        recuperaContatti();
        System.out.println(
            "TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE");
        creaContattoEsistente();
        System.out.println("PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE");
        recuperaContattoNonEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO ESISTENTE");
        modificaContattoEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO ESISTENTE");
        rimuoviContattoEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO NON ESISTENTE");
        modificaContattoNonEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO NON ESISTENTE");
        rimuoviContattoNonEsistente();
    }

    public void creaContattoEsistente() {
        esegui(()->gestoreFile.inserisci(contatti[0]));
    }

    public void modificaContattoEsistente() {
        esegui(()->gestoreFile.modifica(
            new Contatto("Daniele", "Via dei microfoni 1", "07890")));
    }

    public void rimuoviContattoEsistente() {
        esegui(()->gestoreFile.rimuovi(contatti[2].getNome()));
    }

    public void modificaContattoNonEsistente() {
        esegui(()->gestoreFile.modifica(
            new Contatto("Pluto", "Via dei microfoni 1", "07890")));
    }

    public void rimuoviContattoNonEsistente() {
        esegui(()->gestoreFile.rimuovi("Ligeia"));
    }

    public void recuperaContattoNonEsistente() {
        esegui(()->gestoreFile.recupera("Pippo"));
    }

    public void creaContatti() {
        for (Contatto contatto : contatti) {
            System.out.println("Creazione contatto:\n" + contatto);
            creaContatto(contatto);
        }
    }

    public void recuperaContatti() {
        for (Contatto contatto : contatti) {
            System.out.println("Recupero contatto: " + contatto.getNome());
            recuperaContatto(contatto.getNome());
        }
    }
}
```

```

    }
}

public void recuperaContatto(String nomeContatto) {
    esegui(()->gestoreFile.recupera(nomeContatto));
}

private void creaContatto(Contatto contatto) {
    esegui(()->gestoreFile.inserisci(contatto));
}

private Contatto[] getContatti() {
    Contatto contatto1 =
        new Contatto("Daniele","Via delle chitarre 1","01234560");
    Contatto contatto2 =
        new Contatto("Giovanni","Via delle scienze 2","0565432190");
    Contatto contatto3 =
        new Contatto("Ligeia","Via dei segreti 3","07899921");
    Contatto[] contatti = {contatto1, contatto2, contatto3};
    return contatti;
}

public <O> O esegui(Retriever<O> retriever) {
    O output = null;
    try {
        output = retriever.esegui();
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
    return output;
}

public void esegui(Executor executor) {
    try {
        executor.esegui();
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

public static void main(String args[]) {
    Esercizio16T esercizio16T = new Esercizio16T(args[0]);
    esercizio16T.eseguiTest();
}
}

```

In grassetto abbiamo evidenziato il metodo `factory` `getGestoreSerializzazione` che gestisce quale servizio utilizzare (si basa sulla specifica del nome della classe). Poi abbiamo definito il relativo modulo di cui riportiamo il descrittore di seguito, che può accedere tramite reflection a `GestoreSerializzazione`:

```

module com.claudiodesio.rubrica.test {
    uses com.claudiodesio.rubrica.spi.GestoreSerializzazione;
    requires com.claudiodesio.rubrica.spi;
    requires com.claudiodesio.rubrica.dati;
    requires com.claudiodesio.rubrica.util;
}

```

Possiamo adesso compilare i sette moduli definiti sinora con il seguente comando (come sempre l'esercizio completo è nella cartella `Codice\capitolo_16\esercizi\16.t` del file contenente gli esempi di codice):

```
javac -d mods --module-source-path src
src/com.claudiodesio.rubrica.dati/module-info.java
src/com.claudiodesio.rubrica.dati/com/claudiodesio/rubrica/dati/*.java
src/com.claudiodesio.rubrica.eccezioni/module-info.java
src/com.claudiodesio.rubrica.eccezioni/com/claudiodesio/rubrica/
eccezioni/*.java src/com.claudiodesio.rubrica.util/module-info.java
src/com.claudiodesio.rubrica.util/com/claudiodesio/rubrica/util/*.java
src/com.claudiodesio.rubrica.spi/module-info.java
src/com.claudiodesio.rubrica.spi/com/claudiodesio/rubrica/spi/*.java
src/com.claudiodesio.rubrica.io/module-info.java
src/com.claudiodesio.rubrica.io/com/claudiodesio/rubrica/io/*.java
src/com.claudiodesio.rubrica.nio/module-info.java
src/com.claudiodesio.rubrica.nio/com/claudiodesio/rubrica/nio/*.java
src/com.claudiodesio.rubrica.test/module-info.java
src/com.claudiodesio.rubrica.test/com/claudiodesio/rubrica/test/*.java
```

Per eseguire l'applicazione possiamo utilizzare il seguente comando:

```
java --module-path mods
-m com.claudiodesio.rubrica.test/
com.claudiodesio.rubrica.test.Esercizio16T GestoreFile
```

Che stamperà lo stesso output visto nella soluzione dell'esercizio 15.n e che riportiamo di seguito per comodità:

TESTIAMO LA CREAZIONE DEI TRE CONTATTI

Creazione contatto:

Nome: Daniele

Indirizzo: Via delle chitarre 1

Telefono: 01234560

Daniele: contatto già esistente!

Creazione contatto:

Nome: Giovanni

Indirizzo: Via delle scienze 2

Telefono: 0565432190

Giovanni: contatto già esistente!

Creazione contatto:

Nome: Ligeia

Indirizzo: Via dei segreti 3

Telefono: 07899921

Contatto registrato:

Nome: Ligeia

Indirizzo: Via dei segreti 3

Telefono: 07899921

RECUPERIAMO I TRE CONTATTI

Recupero contatto: Daniele

Contatto recuperato:

Nome: Daniele

Indirizzo: Via dei microfoni 1

Telefono: 07890

Recupero contatto: Giovanni

Contatto recuperato:

Nome: Giovanni

Indirizzo: Via delle scienze 2

Telefono: 0565432190

Recupero contatto: Ligeia

Contatto recuperato:

Nome: Ligeia

```
Indirizzo:      Via dei segreti 3
Telefono:       07899921
TESTIAMO LA CREAIONE DI UN CONTATTO GIÀ ESISTENTE
Daniele: contatto già esistente!
PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE
Pippo: contatto non trovato!
MODIFICHIAMO UN CONTATTO ESISTENTE
Contatto registrato:
Nome:   Daniele
Indirizzo:      Via dei microfoni 1
Telefono:       07890
RIMUOVIAMO UN CONTATTO ESISTENTE
Contatto Ligeia cancellato!
MODIFICHIAMO UN CONTATTO NON ESISTENTE
Pluto: contatto non trovato!
RIMUOVIAMO UN CONTATTO NON ESISTENTE
Ligeia: contatto non trovato!
```

Per utilizzare la classe `GestoreFileNIO2` basterà specificarla come argomento da riga di comando al posto di `GestoreFile`.

Soluzione 16.u)

Riscriviamo così il costruttore della classe `Esercizio16U`:

```
public Esercizio16U(String className) {
    contatti = getContatti();
    gestoreFile =
        GestoreSerializzazioneFactory.getGestoreSerializzazione(className);
}
```

nel quale si invoca il metodo (reso statico) `getGestoreSerializzazione` della classe

`GestoreSerializzazioneFactory`. Infatti ecco la classe `GestoreSerializzazioneFactory`:

```
package com.claudiodesio.rubrica.factory;

import com.claudiodesio.rubrica.spi.GestoreSerializzazione;
import com.claudiodesio.rubrica.dati.Contatto;
import java.util.Iterator;
import java.util.ServiceLoader;

public class GestoreSerializzazioneFactory {
    public static GestoreSerializzazione<Contatto>
        getGestoreSerializzazione(String className) {
        ServiceLoader<GestoreSerializzazione> serviceLoader =
            ServiceLoader.load(
                com.claudiodesio.rubrica.spi.GestoreSerializzazione.class);
        for (GestoreSerializzazione gestoreSerializzazione : serviceLoader) {
            if (gestoreSerializzazione.getClass().getSimpleName()
                .equals(className)) {
                return gestoreSerializzazione;
            }
        }
        throw new IllegalArgumentException(
            "Nessun gestore di serializzazione trovato per classe =" + className);
    }
}
```

Il descrittore del nuovo modulo `com.claudiodesio.rubrica.factory` è il seguente:

```
module com.claudiodesio.rubrica.factory {
    exports com.claudiodesio.rubrica.factory to com.claudiodesio.rubrica.test;
    requires com.claudiodesio.rubrica.spi;
    requires com.claudiodesio.rubrica.dati;
    uses com.claudiodesio.rubrica.spi.GestoreSerializzazione;
}
```

Mentre il descrittore `com.claudiodesio.rubrica.test` si può semplificare in questo modo:

```
module com.claudiodesio.rubrica.test {
    requires com.claudiodesio.rubrica.spi;
    requires com.claudiodesio.rubrica.factory;
    requires com.claudiodesio.rubrica.dati;
    requires com.claudiodesio.rubrica.util;
}
```

Il comando per compilare tutto sarà:

```
javac -d mods --module-source-path src
src/com.claudiodesio.rubrica.dati/module-info.java
src/com.claudiodesio.rubrica.dati/com/claudiodesio/rubrica/dati/*.java
src/com.claudiodesio.rubrica.eccezioni/module-info.java
src/com.claudiodesio.rubrica.eccezioni/com/claudiodesio/rubrica/
eccezioni/*.java src/com.claudiodesio.rubrica.util/module-info.java
src/com.claudiodesio.rubrica.util/com/claudiodesio/rubrica/util/*.java
src/com.claudiodesio.rubrica.spi/module-info.java
src/com.claudiodesio.rubrica.spi/com/claudiodesio/rubrica/spi/*.java
src/com.claudiodesio.rubrica.io/module-info.java
src/com.claudiodesio.rubrica.io/com/claudiodesio/rubrica/io/*.java
src/com.claudiodesio.rubrica.nio/module-info.java
src/com.claudiodesio.rubrica.nio/com/claudiodesio/rubrica/nio/*.java
src/com.claudiodesio.rubrica.test/module-info.java
src/com.claudiodesio.rubrica.test/com/claudiodesio/rubrica/test/*.java
src/com.claudiodesio.rubrica.factory/com/claudiodesio/rubrica/
factory/*.java src/com.claudiodesio.rubrica.factory/module-info.java
```

Il comando di esecuzione non cambia rispetto a quell'usato nell'esercizio precedente, così come l'output.

Soluzione 16.v)

Intanto abbiamo rinominato la classe `Esercizio16U` in `Esercizio16V` e ricompilato i moduli.

Abbiamo poi creato gli otto file modulari con questo comando:

```
jar --create --file=lib/com.claudiodesio.rubrica.dati.jar
--module-version=1.0 -C mods/com.claudiodesio.rubrica.dati .
jar --create --file=lib/com.claudiodesio.rubrica.eccezioni.jar
--module-version=1.0 -C mods/com.claudiodesio.rubrica.eccezioni .
jar --create --file=lib/com.claudiodesio.rubrica.util.jar
--module-version=1.0 -C mods/com.claudiodesio.rubrica.util .
jar --create --file=lib/com.claudiodesio.rubrica.spi.jar
--module-version=1.0 -C mods/com.claudiodesio.rubrica.spi .
jar --create --file=lib/com.claudiodesio.rubrica.io.jar
--module-version=1.0 -C mods/com.claudiodesio.rubrica.io .
jar --create --file=lib/com.claudiodesio.rubrica.nio.jar
--module-version=1.0 -C mods/com.claudiodesio.rubrica.nio .
jar --create --file=lib/com.claudiodesio.rubrica.factory.jar
--module-version=1.0 -C mods/com.claudiodesio.rubrica.factory .
jar --create --file=lib/com.claudiodesio.rubrica.test.jar
```

```
--module-version=1.0
--main-class=com.claudiodesio.rubrica.test.Esercizio16V
-C mods/com.claudiodesio.rubrica.test .
```

Poi abbiamo eseguito il modulo con il comando:

```
java -p lib -m com.claudiodesio.rubrica.test GestoreFile
```

o alternativamente con:

```
java -p lib -m com.claudiodesio.rubrica.test GestoreFileNIO2
```

Soluzione 16.z)

Con il seguente comando creiamo il runtime richiesto:

```
jlink --module-path "C:/Program Files/Java/jdk-9.0.1/jmods"
--add-modules java.base --output javabasert
```

Mentre con il seguente comando eseguiamo il nostro JAR modulare

com.claudiodesio.rubrica.test con il runtime creato:

```
javabasert\bin\java -p lib -m com.claudiodesio.rubrica.test GestoreFile
```

Come sempre basta sostituire GestoreFile con GestoreFileNIO2 per ottenere la serializzazione nel modo alternativo:

```
javabasert\bin\java -p lib
-m com.claudiodesio.rubrica.test GestoreFileNIO2
```

Soluzioni degli esercizi del capitolo 17 e dell'appendice G

Java Database Connectivity e package java.util

Soluzione 17.a) JDBC, Vero o Falso:

1. **Vero.**
2. **Vero.**
3. **Vero.**
4. **Falso**, per eliminare un record bisogna utilizzare il metodo `executeUpdate`.
5. **Vero.**
6. **Vero.**
7. **Vero.**
8. **Vero.**
9. **Vero.**
10. **Falso**, il tipo `CHAR` di SQL si mappa al tipo `String` di Java.

Soluzione 17.b)

La classe `DBGenerator` potrebbe essere la seguente:

```
import java.sql.*;
import java.io.InputStream;
import java.io.FileInputStream;
import java.io.IOException;
import java.util.Properties;
import java.util.stream.*;
import java.nio.file.*;

public class DBGenerator {
    private Properties p;
    private static final String CONFIGURATION_FILE = "DBConfig.txt";
    private static final String SCRIPT_FILE = "DBScript.txt";

    public DBGenerator ( ) throws Exception {
        p = new Properties();
        try (InputStream fis = new FileInputStream(CONFIGURATION_FILE);) {
            p.load(fis);
            createDB();
        } catch (IOException exc) {
            exc.printStackTrace();
        }
    }

    public void createDB() throws Exception {
        try (Connection conn =
            DriverManager.getConnection(p.getProperty("db.url"));
            Stream<String> stream = Files.lines(Paths.get(SCRIPT_FILE));
            Statement stmt = conn.createStatement()) {
```

```

        stream.forEach(line-> executeUpdate(stmt, line));
        System.out.println("Database created!");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

private void executeUpdate(Statement stmt, String cmd) {
    try {
        stmt.executeUpdate(cmd);
    } catch (SQLException exc) {
        exc.printStackTrace();
    }
}

public static void main(String args[]) throws Exception {
    new DBGenerator();
}
}

```

Soluzione 17.c)

La classe DBExplorer potrebbe essere la seguente:

```

import java.sql.*;
import java.io.InputStream;
import java.io.FileInputStream;
import java.io.IOException;
import java.util.Properties;
import java.util.stream.*;
import java.nio.file.*;

public class DBExplorer {
    private Properties p;
    private static final String CONFIGURATION_FILE = "DBConfig.txt";

    public DBExplorer ( ) throws Exception {
        p = new Properties();
        try (InputStream fis = new FileInputStream(CONFIGURATION_FILE);) {
            p.load(fis);
        } catch (IOException exc) {
            exc.printStackTrace();
        }
        printContacts();
    }

    private void printContacts() {
        try (Connection conn =
            DriverManager.getConnection(p.getProperty("db.url"));
            Statement stmt = conn.createStatement();
            ResultSet res = stmt.executeQuery("SELECT * FROM Contact")) {
            while (res.next()) {
                System.out.printf("%s\nPhone: %s\nAddress: (%s)\n\n",
                    res.getString("Name"),
                    res.getString("Phone"),
                    res.getString("Address"));
            }
        } catch (SQLException e) {

```



```

        e.printStackTrace();
    }
}

public static void main(String args[]) throws Exception {
    new DBExplorer();
}
}

```



Soluzione 17.d)

Modifichiamo l'interfaccia `GestoreSerializzazione` per gestire una eccezione generica (soluzione più veloce):

```

package rubrica.integrazione;

import java.util.*;
import rubrica.dati.*;
import rubrica.util.*;

public interface GestoreSerializzazione<T extends Dato> {

    void inserisci(T dato) throws Exception;

    T recupera(String id) throws Exception;

    void modifica(T dato) throws Exception;

    void rimuovi(String id) throws Exception;

    List<T> getContatti() throws Exception;
}

```

Possiamo implementare la classe `GestioneDB` nel seguente modo:

```

package rubrica.integrazione;

import java.util.*;
import rubrica.dati.*;
import rubrica.eccezioni.*;
import rubrica.util.*;
import java.sql.*;
import java.io.*;

public class GestioneDB implements GestoreSerializzazione<Contatto> {
    private Properties p;
    private static final String CONFIGURATION_FILE = "DBConfig.txt";

    public GestioneDB() {
        p = new Properties();
        try (InputStream fis = new FileInputStream(CONFIGURATION_FILE);) {
            p.load(fis);
        } catch (IOException exc) {
            exc.printStackTrace();
        }
    }
}

```

```

@Override
public void inserisci(Contatto contatto) throws ContattoEsistenteException,
    SQLException {
    try (Connection connection =
        DriverManager.getConnection(p.getProperty("db.url"));
        Statement cmd = connection.createStatement();) {
        String insert = "INSERT INTO CONTATTO (NOME, TELEFONO, INDIRIZZO) "+
            "VALUES ('"+ contatto.getNome() + "', '"+
            contatto.getNumeroDiTelefono() +
            "', '"+ contatto.getIndirizzo()+ "')";
        int result = cmd.executeUpdate(insert);
    }
}

@Override
public Contatto recupera(String nome) throws ContattoInesistenteException,
    ContattoEsistenteException {
    return null;
}

@Override
public void modifica(Contatto contatto) throws ContattoInesistenteException,
    ContattoEsistenteException, SQLException {
}

@Override
public void rimuovi(String nome) throws ContattoInesistenteException,
    ContattoEsistenteException, SQLException {
}

@Override
public List<Contatto> getContatti() throws Exception {
    List<Contatto> contatti = new ArrayList<Contatto>();
    try (Connection conn =
        DriverManager.getConnection(p.getProperty("db.url"));
        Statement stmt = conn.createStatement();
        ResultSet res = stmt.executeQuery("SELECT * FROM CONTATTO")) {
        while (res.next()) {
            contatti.add(new Contatto(res.getString("NOME"),
                res.getString("TELEFONO"), res.getString("INDIRIZZO")));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return contatti;
}
}

```

Abbiamo anche modificato il file `config.properties` per far sì che venga caricata la classe `GestoreDB` come gestore di serializzazione:

```
rubrica.ser=GestoreDB
```

L'output attuale dell'applicazione sarà:

```

Sto caricando la classe GestoreDB
Lista dei contatti in Rubrica:
1    Claudio De Sio Cesari
2    Stevie Wonder

```

```

3      Gennaro Capuozzo
Scrivi '/v' e il nome del contatto per visualizzarne i dettagli
Scrivi '/i' per inserire un nuovo contatto
Scrivi '/t' per terminare il programma

```

Ovviamente per ora l'applicazione lancerà eccezioni se proviamo ad usare le altre funzionalità.

Soluzione 17.e)

Potremmo implementare il metodo `recupera` nel seguente modo:

```

@Override
public Contatto recupera(String nome) throws ContattoInesistenteException {
    Contatto contatto = null;
    try (Connection conn = DriverManager.getConnection(p.getProperty("db.url"));
        Statement stmt = conn.createStatement();
        ResultSet res = stmt.executeQuery(
            "SELECT * FROM CONTATTO WHERE NOME = '" + nome + "'")) {
        if (res.next()) {
            contatto = new Contatto(res.getString("NOME"),
                res.getString("TELEFONO"), res.getString("INDIRIZZO"));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    } catch (NomeVuotoException e) {
        throw new ContattoInesistenteException(
            "Non esistono contatti senza nome");
    }
    return contatto;
}

```

Soluzione 17.f)

Potremmo implementare il metodo `inserisci` nel seguente modo:

```

@Override
public void inserisci(Contatto contatto) throws ContattoEsistenteException,
    SQLException {
    try (Connection connection =
        DriverManager.getConnection(p.getProperty("db.url"));
        Statement cmd = connection.createStatement();) {
        String insert = "INSERT INTO CONTATTO (NOME, TELEFONO, INDIRIZZO) "+
            "VALUES ('"+ contatto.getNome() + "', '"+
            contatto.getNumeroDiTelefono() +
            "', '"+ contatto.getIndirizzo() + "')";
        int result = cmd.executeUpdate(insert);
    } catch (SQLException e) {
        throw new ContattoEsistenteException("Contatto già presente");
    }
}

```

Soluzione 17.g)

Potremmo implementare il metodo `modifica` nel seguente modo:

```

@Override
public void modifica(Contatto contatto) throws ContattoInesistenteException,
    SQLException {
    try (Connection conn = DriverManager.getConnection(p.getProperty("db.url"));
        Statement cmd = conn.createStatement();) {
        String update = String.format(
            "UPDATE CONTATTO SET TELEFONO = '%s', INDIRIZZO = '%s' " +
            "WHERE NOME = '%s'",
            contatto.getNumeroDiTelefono(), contatto.getIndirizzo(),
            contatto.getNome());
        int result = cmd.executeUpdate(update);
        if (result == 0) {
            throw new ContattoInesistenteException(
                "Non esiste un contatto con nome " + contatto.getNome());
        }
    }
}

```

Soluzione 17.h)

Potremmo implementare il metodo `rimuovi` nel seguente modo:

```

@Override
public void rimuovi(String nome) throws ContattoInesistenteException,
    SQLException {
    try (Connection conn = DriverManager.getConnection(p.getProperty("db.url"));
        Statement cmd = conn.createStatement();) {
        String delete = String.format("DELETE FROM CONTATTO WHERE NOME = '%s'",
            nome);
        int result = cmd.executeUpdate(delete);
        if (result == 0) {
            throw new ContattoInesistenteException(
                "Non esiste un contatto con nome " + nome);
        }
    }
}

```

Soluzione G.a) *Package java.util, Vero o Falso:*

1. **Vero.**
2. **Vero.**
3. **Vero.**
4. **Falso**, tutte le “t” non dovrebbero esserci.
5. **Falso**, è valido ma stamperà false. Affinché stampi true l’espressione si deve modificare in “\\bb”.
6. **Falso**, il modo in cui vengono immagazzinati i dati persistenti è dipendente dalla piattaforma. Un oggetto `Properties` invece può usare anche file di configurazione in formato XML.
7. **Falso**, la classe `PrintStream` definisce il metodo `printf`, la classe `Formatter` definisce il metodo `format`.
8. **Falso**, è un gruppo di caratteri.

9. **Falso**, la classe `Date` non ha un metodo `now`. La prima riga da sola avrebbe adempiuto al compito richiesto.
10. **Falso**, la classe `SimpleNumberFormat` semplicemente non esiste. Ma esiste la classe `SimpleDateFormat`.

Soluzione G.b)

Il listato della classe `Traduttore` potrebbe essere come il seguente:

```
import java.util.Locale;
import java.util.ResourceBundle;

public class Traduttore {

    private LinguaEnum lingua;

    private ResourceBundle resources;

    public Traduttore (LinguaEnum lingua) {
        this.lingua = lingua;
        String chiaveLingua = lingua.getChiave();
        Locale locale = new Locale(chiaveLingua);
        resources = ResourceBundle.getBundle("risorse.vocabolario", locale);
    }

    public String traduci(ParoleEnum testo) {
        String traduzione = resources.getString(testo.getChiave());
        return traduzione;
    }

    public void setLingua(LinguaEnum lingua) {
        this.lingua = lingua;
    }

    public LinguaEnum getLingua() {
        return lingua;
    }
}
```

Si noti che per semplificare il nostro esercizio abbiamo deciso di limitare il numero di parole da tradurre mediante l'enumerazione `ParoleEnum`:

```
public enum ParoleEnum {

    LIBRO("libro"), TEMPO("tempo"), CASA("casa");

    private String chiave;

    private ParoleEnum(String chiave) {
        this.chiave = chiave;
    }

    public String getChiave() {
        return chiave;
    }
}
```

Abbiamo anche limitato il numero di lingue supportate tramite l'enumerazione `LingueEnum`:

```
public enum LinguaEnum {
```

```

    ITALIANO("it", "Italiano"), INGLESE("en", "Inglese");

    String chiave;

    String descrizione;

    LinguaEnum(String chiave, String descrizione) {
        this.chiave = chiave;
        this.descrizione = descrizione;
    }

    public String getChiave() {
        return chiave;
    }

    public String toString() {
        return descrizione;
    }
}

```

Inoltre abbiamo creato nella cartella risorse (inserita nella cartella dei sorgenti scaricabili: <https://www.nuovojava.it>) i file di properties che ci servivano: vocabolario_it.properties:

```

libro=libro
casa=casa
tempo=tempo

```

e vocabolario_en.properties:

```

libro=book
casa=home
tempo=time

```

Soluzione G.c)

Il listato potrebbe essere il seguente:

```

public class TestTraduttore {

    public static void main(String args[]) {
        Traduttore traduttore = new Traduttore(LinguaEnum.INGLESE);
        String parolaTradotta = traduttore.traduci(ParoleEnum.LIBRO);
        System.out.println(parolaTradotta);
    }
}

```

L'output sarà:

```
book
```

potremmo anche usare la classe Scanner e rendere interattivo il programma.

Soluzione G.d)

Il listato potrebbe essere il seguente:

```

import java.util.ArrayList;
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class StringUtils {

```

```
public static List<String> ricerca(String testo, char iniziale) {
    List<String> list = new ArrayList<>();
    Pattern pattern = Pattern.compile("\\b"+iniziale+"[a-zA-Z]+\\b");
    Matcher matcher = pattern.matcher(testo);
    while (matcher.find()) {
        list.add(matcher.group());
    }
    return list;
}
}
```

Soluzione G.e)

Il listato della classe `TestStringUtils` potrebbe essere il seguente:

```
import java.util.List;

public class TestStringUtils {

    public static void main(String args[]) {
        List<String> list = StringUtils.ricerca(
            "The smile of dawn arrived early May "
            + "she carried a gift from her home "
            + "the night shed a tear to tell her of fear "
            + "and of sorrow and pain she'll never outgrow ", 't');
        for (String string : list) {
            System.out.println(string);
        }
    }
}
```

L'output del codice precedente è:

```
the
tear
to
tell
```

Soluzione G.f)

La classe giusta da utilizzare è la classe `java.util.Random`, e il suo metodo `nextInt` che prende in input un limite superiore (escluso), e che ha come limite inferiore 0 (incluso). Basta sommargli il valore 1 e il gioco è fatto. Il listato potrebbe essere il seguente:

```
import java.util.Random;

public class LancioDeiDadi {

    public static void main(String args[]) {
        System.out.println("Lancio il dado... "
            + (1 + new Random().nextInt(6)) + "!");
    }
}
```

Soluzione G.g)

Riutilizziamo le classi Utente:

```
package com.claudiodesio.autenticazione;

public class Utente {

    private String nome;
    private String username;
    private String password;

    public Utente(String n, String u, String p) {
        this.nome = n;
        this.username = u;
        this.password = p;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }
}
```

La classe Amministratore:

```
package com.claudiodesio.autenticazione;

public class Amministratore extends Utente {
    public Amministratore (String nome, String username, String password) {
        super(nome, username, password);
    }
}
```

La classe Commesso:

```
package com.claudiodesio.autenticazione;

public class Commesso extends Utente{
    public Commesso(String nome, String username, String password) {
        super(nome, username, password);
    }
}
```

Poi modifichiamo la classe ProfiliUtenti in modo tale da gestire i profili degli utenti:

```
package com.claudiodesio.autenticazione;
```



```

import java.util.*;
import java.io.*;

public class ProfiliUtenti {

    private static ProfiliUtenti instance;

    private Properties properties;

    private ProfiliUtenti() {
        properties = new Properties();
        try {
            loadProperties();
        } catch (IOException e) {
            e.printStackTrace();
            System.exit(1);
        }
    }

    public void loadProperties() throws IOException {
        try (FileInputStream inputStream =
            new FileInputStream("config.properties");) {
            properties.load(inputStream);
        }
    }

    public static ProfiliUtenti getInstance() {
        if (instance == null) {
            instance = new ProfiliUtenti();
        }
        return instance;
    }

    public void inserisciUtente(String[] args) throws IOException {
        String ruolo = args[0];
        String nome = args[1];
        String username = args[2];
        String password = args[3];
        // Se non si specifica il ruolo Amministratore
        // verrà inserito sempre un Commesso
        Utente utente = (ruolo.equals("Amministratore") ?
            new Amministratore(nome, username, password):
            new Commesso(nome, username, password));
        String valore = nome + "," + ruolo + "," + password;
        properties.setProperty(username, valore);
        try (FileOutputStream fos = new FileOutputStream("config.properties")) {
            properties.store(fos, "File di configurazione");
        }
        System.out.println("Inserita la proprietà: " + username + "=" + valore);
    }
}

```

In particolare abbiamo aggiunto un metodo per aggiungere profili utente, che ci permette di aggiungere nel file di properties una riga la cui chiave è lo username, e il cui valore è una lista con le altre informazioni dell'utente separate da virgole. Infine con la seguente classe che prende da riga di comando dei valori da inserire, viene eseguita l'applicazione:

```

package com.claudiodesio.autenticazione;

import java.io.*;

```

```
public class EsercizioGG {

    public static void main(String args[]) throws IOException {
        if (args.length != 4) {
            System.out.println("Specificare ruolo, nome, username, password");
            System.exit(1);
        }
        ProfiliUtenti.getInstance().inserisciUtente(args);
    }
}
```

Eseguendo l'applicazione, per esempio senza specificare argomenti di riga di comando, otterremo il seguente output:

```
java com.claudiodesio.autenticazione.EsercizioGG
Specificare ruolo, nome, username, password
```

invece specificando i seguenti argomenti:

```
java com.claudiodesio.autenticazione.EsercizioGG Amministratore
Claudio desio xxxxxxxx
Inserita la proprietà: desio=Claudio,Amministratore,xxxxxxx
```

otterremo che nel file di properties troveremo il seguente contenuto:

```
#File di configurazione
#Sun Mar 25 19:10:31 CEST 2018
desio=Claudio,Amministratore,xxxxxxx
```

Soluzione G.h)

Una possibile soluzione risiede nella modifica della classe `ProfiliUtenti` nel seguente modo:

```
package com.claudiodesio.autenticazione;

import java.util.*;
import java.io.*;

public class ProfiliUtenti {

    private static ProfiliUtenti instance;

    private Properties properties;

    private ProfiliUtenti() {
        properties = new Properties();
        try {
            loadProperties();
        } catch (IOException e) {
            e.printStackTrace();
            System.exit(1);
        }
    }

    public static ProfiliUtenti getInstance() {
        if (instance == null) {
            instance = new ProfiliUtenti();
        }
        return instance;
    }
}
```

```

public void loadProperties() throws IOException {
    try (FileInputStream inputStream =
        new FileInputStream("config.properties");) {
        properties.load(inputStream);
    }
}

public void inserisciUtente(String[] args) throws IOException {
    String ruolo = args[0];
    String nome = args[1];
    String username = args[2];
    String password = args[3];
    // Se non si specifica il ruolo Amministratore
    // verrà inserito sempre un Commesso
    Utente utente = (ruolo.equals("Amministratore") ?
        new Amministratore(nome, username, password):
        new Commesso(nome, username, password));
    String valore = nome + "," + ruolo + "," + password;
    properties.setProperty(username, valore);
    try (FileOutputStream fos = new FileOutputStream("config.properties")) {
        properties.store(fos, "File di configurazione");
    }
    System.out.println("Inserita la proprietà: " + username + "=" + valore);
}

public Utente getUtente(String username) {
    String valore = (String)properties.getProperty(username);
    if (valore == null) {
        return null;
    }
    String [] campi = valore.split(",");
    Utente utente = (campi[1].equals("Amministratore") ?
        new Amministratore(campi[0], username, campi[2]):
        new Commesso(campi[0], username, campi[2]));
    return utente;
}
}

```

In particolare abbiamo creato il metodo `getUtente` che andiamo a richiamare dalla classe Autenticazione opportunamente modificata:

```

package com.claudiodesio.autenticazione;

import java.util.Scanner;

public class Autenticazione {

    public void login() {
        boolean autorizzato = false;
        Scanner scanner = new Scanner(System.in);
        do {
            Stampa.richiediUsername();
            String username = scanner.nextLine();
            Utente utente = ProfiliUtenti.getInstance().getUtente(username);
            if (utente != null) {
                Stampa.richiediPassword();
                String password = scanner.nextLine();
                if (verificaPassword(utente, password)) {
                    Stampa.auguraBenvenuto(utente.getNome());
                    autorizzato = true;
                } else {

```

```

        Stampa.autenticazioneFallita();
    }
    } else {
        Stampa.usernameInesistente();
    }
} while (!autorizzato);
}

private boolean verificaPassword(Utente utente, String password) {
    boolean trovato = false;
    if (password != null) {
        if (password.equals(utente.getPassword())) {
            trovato = true;
        }
    }
    return trovato;
}
}

```

Infine il metodo main lo abbiamo spostato nella classe `EsercizioGH`:

```

package com.claudiodesio.autenticazione;

public class EsercizioGH {
    public static void main(String args[]) {
        Autenticazione autenticazione = new Autenticazione();
        autenticazione.login();
    }
}

```

Eseguendo l'applicazione tutto funziona come prima:

```

Inserisci username.
Pippo
Utente non trovato!
Inserisci username.
desio
Inserisci password.
YYYYYYYY
Autenticazione fallita
Inserisci username.
desio
Inserisci password.
xxxxxxx
Benvenuto Claudio

```

Soluzione G.i)

Tutte le affermazioni sono vere.

Soluzione G.j)

Tutte le affermazioni sono vere tranne la numero 3. Infatti la classe `Preferences` non estende `Hashtable`, ma è una classe astratta che estende `Object`. È sufficiente leggere la documentazione delle API Java.

Soluzione G.k)

Le affermazioni corrette sono le numero 2, 4 e 5. La numero 1 è falsa perché il metodo `find` non ritorna una stringa ma un booleano (il metodo `find` viene in fatti usato solitamente come condizione di un ciclo).

La numero 3 invece è falsa perché i metodi `start` e `end` non appartengono alla classe `Pattern` ma vengono dichiarati dalla classe `Matcher`.

Soluzioni degli esercizi del capitolo 18 e dell'appendice M

Java e il mondo XML e JAXB

Soluzione 18.a) JAXP, Vero o Falso:

1. **Vero.**
2. **Falso**, l'interfaccia `Text` implementa `Node`.
3. **Vero.**
4. **Falso**, si raggiunge il primo nodo “fratello” di `node`.
5. **Falso.**
6. **Vero.**
7. **Falso**, il testo viene aggiunto prima del tag con il metodo `insertBefore`. Sarebbe invece opportuno utilizzare la seguente istruzione per aggiungere il testo all'interno del tag nuovo:
`element.appendChild(text);`
8. **Falso.**
9. **Vero.**
10. **Vero.**

Soluzioni degli esercizi del capitolo 19

Introduzione alle Interfacce grafiche (GUI)

Soluzione 19.a) GUI, AWT e Layout Manager, Vero o Falso:

1. **Vero.**
2. **Falso**, in particolare il Model rappresenta l'intera applicazione composta da dati e funzionalità.
3. **Vero.**
4. **Vero.**
5. **Falso**, è basata sul pattern Composite che, nonostante abbia alcuni punti di contatto con il Decorator, è completamente diverso.
6. **Vero.**
7. **Falso**, ma perderemmo la robustezza e la consistenza della GUI.
8. **Vero.**
9. **Falso.**
10. **Falso.**

Soluzione 19.b) Gestione degli eventi, Applet e Swing, Vero o Falso:

1. **Vero.**
2. **Vero.**
3. **Falso.**
4. **Falso.**
5. **Vero.**
6. **Falso**, solo di tipo `ActionListener`.
7. **Vero**, può sfruttare il metodo `System.exit(0)`, ma non è pertinente con gli eventi di tipo `WindowEvent`.
8. **Vero.**
9. **Vero.**
10. **Vero.**

Soluzioni degli esercizi del capitolo 20

Introduzione a JavaFX

Soluzione 20.a) *JavaFX, Vero o Falso:*

1. **Vero.**
2. **Falso**, è esattamente il contrario.
3. **Vero.**
4. **Falso**, tali caratteristiche sono del `TilePane`
5. **Vero.**
6. **Falso.**
7. **Falso.**
8. **Vero.**
9. **Falso**, abbiamo visto un esempio di sfocatura tramite la classe `BlurMotion` che in realtà non implementa movimenti ed estende la classe astratta `Effect`. Invece le classi che implementano un movimento estendono `Transition`, e di conseguenza hanno terminano con la parola `Transition`.
10. **Falso**: una volta era possibile, ma ormai la tecnologia Applet è stata messa da parte.